

Pengantar Deep Learning untuk Meteorologi

Kanada Kurniawan

PENGANTAR DEEP LEARNING UNTUK METEOROLOGI

Dari Praktisi untuk Praktisi di Indonesia

Kanada Kurniawan

Meteorology Officer — BMKG Stasiun Meteorologi Kelas III Maritim Dwikora,
Pontianak Indonesia

Edisi 2.0 — Lengkap, 10 bab

DOI: 10.5281/zenodo.00000000 (*placeholder — diperbarui saat rilis*) **Lisensi:**
CC BY-SA 4.0 (isi) · CC BY 4.0 (kode) · lisensi data mengikuti penyedia

Pontianak, Indonesia — September 2026

Buku ini ditujukan sebagai **materi pengenalan** (*introductory material*), bukan hasil riset baru. Seluruh isi merupakan ringkasan dan penyusunan ulang dari literatur klasik *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia. Kutip silang dan definisi lengkap istilah tersedia di bagian “Glosarium dan Notasi”; sumber data yang dipakai dicantumkan di bagian “Daftar Dataset dan Sumber” pada bagian akhir buku.

Versi dokumen ini: **v2.0** · Manuskrip sumber & kode pendamping tersedia secara terbuka.

Hak Cipta dan Lisensi

© 2026 Kanada Kurniawan

Pengantar Deep Learning untuk Meteorologi: Dari Praktisi untuk Praktisi di Indonesia Edisi 2.0

ISBN: (*placeholder — daftarkan saat rilis*) **DOI:** 10.5281/zenodo.0000000 (*placeholder — daftarkan saat rilis*) **Penerbit:** Diterbitkan secara mandiri oleh penulis, Pontianak, Indonesia.

Riwayat versi:

Versi	Tanggal	Keterangan
v1.0	2026-09-09	Snapshot internal pembangunan (Bab 1–5+6–10 dalam proses)
v2.0	2026-09-12	Edisi lengkap 10 bab — rilis publik pertama

Lisensi

Buku *Pengantar Deep Learning untuk Meteorologi* dilisensikan di bawah **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

Anda bebas untuk:

- **Membagikan** — menyalin dan menyebarluaskan kembali materi ini dalam bentuk apa pun atau media apa pun.
- **Mengadaptasi** — mengubah, mengubah, dan membuat turunan dari materi ini untuk tujuan apa pun, termasuk komersial, selama Anda mengikuti ketentuan lisensi.

Dengan ketentuan:

- **Atribusi** — Anda harus memberikan kredit yang sesuai, mencantumkan tautan ke lisensi, dan menunjukkan jika ada perubahan yang dilakukan. Anda dapat melakukannya dengan wajar, tetapi tidak boleh menyarankan bahwa pemberi lisensi mendukung Anda atau penggunaan Anda.
- **Berbagi Serupa** — Jika Anda mengubah, mengubah, atau membuat turunan dari materi ini, Anda harus mendistribusikan kontribusi Anda di bawah lisensi yang sama dengan materi asli.
- **Tidak ada pembatasan tambahan** — Anda tidak boleh menerapkan ketentuan hukum atau tindakan teknis yang secara hukum membatasi orang lain untuk melakukan hal yang diizinkan lisensi.

Lisensi lengkap: <https://creativecommons.org/licenses/by-sa/4.0/legalcode>

Kode, notebook, dan figur yang menyertai buku ini dilisensikan di bawah **Creative Commons Attribution 4.0 International (CC BY 4.0)**. Lisensi lengkap: <https://creativecommons.org/licenses/by/4.0/legalcode>

Data yang digunakan dalam studi kasus (mis. pasang surut IOC/UHSLC/PSMSL, grid hujan CHIRPS, ERA5) tetap tunduk pada lisensi masing-masing penyedia. Lihat bagian “Daftar Dataset dan Sumber” di bagian belakang buku untuk detail lisensi dan atribusi yang diminta setiap penyedia.

Sitasi yang disarankan

Jika Anda mengutip buku ini, gunakan format IEEE berikut:

K. Kurniawan, *Pengantar Deep Learning untuk Meteorologi: Dari Praktisi untuk Praktisi di Indonesia*, ed. 2.0, Pontianak, Indonesia, Sep. 2026. doi: 10.5281/zenodo.00000000.

Jika Anda mengutip salah satu bab, cukup tambahkan nomor bab pada sitasi di atas dan rujuk DOI yang sama (seluruh buku memiliki satu DOI konsep).

Pernyataan

- **Non-endorsement:** Nama BMKG dan afiliasi penulis dipertahankan untuk menunjukkan sumber keahlian penulis. Pandangan yang diungkapkan dalam buku ini adalah pandangan pribadi penulis dan **tidak** mewakili posisi resmi BMKG.
- **Materi pengenalan:** seluruh isi adalah materi pembelajaran dan bukan nasihat operasional resmi; angka performa model di dalamnya bersifat ilustratif.
- **Merek dagang:** merek dagang yang disebutkan (TensorFlow, Keras, Google Colab, NumPy, Pandas, dll.) adalah milik pemiliknya masing-masing dan tidak berafiliasi dengan buku ini.

Daftar Isi

Front Matter

- Halaman Judul
- Hak Cipta dan Lisensi
- Daftar Isi
- Daftar Gambar dan Daftar Tabel
- Prakata
- Cara Memakai Buku Ini
- Glosarium dan Notasi

Bagian I — Fondasi (Bab 1–5)

- **Bab 1. Pengantar: Deep Learning untuk Meteorologi**
 - Posisi AI, *machine learning*, dan *deep learning*
 - Peta aplikasi meteorologi yang dibahas buku ini
 - Kapan *deep learning* layak dipakai (vs *baseline* statistik)
 - Menyiapkan Google Colab + TensorFlow, dan tensor pertama
- **Bab 2. Regresi: Perceptron dan Jaringan Saraf untuk Prediksi Besaran**
 - Anatomi neuron: bobot, bias, fungsi aktivasi
 - Dari regresi linear ke MLP; kapan non-linearitas diperlukan
 - Mini-kasus pasang surut: *windowing*, *baseline persistence*, MAE vs MSE
 - Split data deret waktu yang mencegah *leakage*
- **Bab 3. Klasifikasi: Mengenali Kategori Fenomena Cuaca**
 - Sigmoid/softmax dan cross-entropy
 - *Class imbalance* dan mengapa akurasi menipu
 - Precision/recall/F1, *confusion matrix*, pengenalan CSI/FAR
 - Trade-off *threshold* ala praktisi peramalan
- **Bab 4. Backpropagation, Optimasi dan Pelatihan**
 - Intuisi *gradient descent* dan *backpropagation* (aturan rantai)
 - Fungsi aktivasi ditinjau dari gradien; *vanishing gradient*
 - *Learning rate*, *batch size*, *epoch*; SGD vs Adam
 - Callback: *early stopping*, ModelCheckpoint, ReduceLROnPlateau
- **Bab 5. Overfitting, Regularisasi dan Evaluasi untuk Data Iklim**
 - Bias-variance, underfit/overfit, *learning curve*
 - L2, *dropout*, *early stopping*
 - Metrik domain: MAE/RMSE/R²/Willmott/KGE dan CSI/FAR/POD/TS
 - Cross-validation deret waktu: *walk-forward/blocked* anti-*leakage*

Bagian II — Data dan Model Sekuensial (Bab 6–7)

- **Bab 6. Data Meteorologi: Sumber, Kualitas dan Persiapan**
 - Sumber data: stasiun & grid terbuka, ERA5, CMIP6, pasang surut,

- satelit; lisensi
- Format CSV/NetCDF/GRIB; nilai hilang, *outlier*, imputasi dasar
- Eksplorasi: dekomposisi musiman, distribusi, korelasi silang
- Feature engineering: lag, musiman, ENSO/MJO
- Normalisasi (fit hanya pada *train*) dan split berbasis waktu
- **Bab 7. Deret Waktu dan Model Sekuensial: RNN, LSTM, GRU**
 - *Windowing* dan *horizon*: prediksi satu dan multi-langkah
 - *Baseline* dulu: *persistence*, mean, AR — DL harus mengalahkannya
 - Intuisi RNN → LSTM → GRU (gate ingatan/lupa) dan keterbatasannya
 - Arsitektur praktis univariate/multivariate; strategi multi-step

Bagian III — Studi Kasus (Bab 8–9)

- **Bab 8. Studi Kasus: Prediksi Pasang Surut di Perairan Indonesia (Contoh Cilacap)**
 - Konteks banjir rob pesisir dan tipe pasang surut Indonesia
 - Memilih stasiun: Cilacap (GLOSS #291) dan sumber data terbuka
 - Alur kerja (*pipeline*): *baseline* vs MLP vs LSTM/GRU, *walk-forward* 4 blok
 - Evaluasi MAE/RMSE vs toleransi tinggi pasang; prediksi 1–7 hari
 - Framing jujur: analisis harmonik untuk penjelasan, ML untuk prediksi cepat
- **Bab 9. Studi Kasus: Prediksi Curah Hujan dengan Data Terbuka**
 - Regresi jumlah hujan + klasifikasi kategori intensitas
 - Verifikasi operasional CSI/FAR/POD dengan *trade-off threshold*
 - *Walk-forward* vs *baseline* (*persistence*, klimatologi, ARIMA)
 - Interpretasi awal (permutation importance/SHAP) dan verifikasi per kategori

Bagian IV — Operasional dan Arah Riset (Bab 10)

- **Bab 10. Dari Riset ke Praktik: Operasional, Interpretasi, dan Arah ke Depan**
 - Monitoring *drift* dan strategi retraining/kalibrasi
 - Ketidakpastian prediksi: *quantile/interval*, ensembel multi-*seed*
 - Interpretasi SHAP dan tautannya ke pengetahuan atmosfer
 - Keterbatasan dan etika; peta arah riset (CNN, *nowcasting*, *downscaling*, *generative*)

Back Matter

- Lampiran A — Reprodusibilitas dan Lingkungan Komputasi
- Lampiran B — Konvensi Penulisan dan Penomoran Aset
- Daftar Pustaka

- Indeks
- Daftar Dataset dan Sumber
- Daftar Notebook dan DOI
- Tentang Penulis
- Kolofon

Catatan: nomor halaman final (untuk edisi cetak/PDF) dihasilkan otomatis oleh mesin pengolah dokumen (Pandoc/LaTeX) saat *build*; daftar di atas merupakan sumber kebenaran struktur buku.

Daftar Gambar dan Daftar Tabel

Daftar ini merekam seluruh aset visual (gambar) dan tabulasi (tabel) yang dipakai di seluruh bab. Penomoran mengikuti konvensi **bab.jenis-urutan** (mis. “Gambar 2.1”, “Tabel 3.2”) dan **reset di tiap bab**, sesuai register aset di REGISTER.md. Pada edisi cetak/PDF, daftar bernomor halaman dapat dihasilkan otomatis oleh LaTeX (`\listoffigures`, `\listoftables`).

Daftar Gambar

Nomor	Keterangan	Bab
Gambar 1.1	Keterkaitan AI, <i>machine learning</i> , dan <i>deep learning</i>	1
Gambar 2.1	Struktur neuron buatan ($x \rightarrow z \rightarrow \text{aktivasi } a$)	2
Gambar 2.2	Arsitektur MLP contoh ($1 \rightarrow 8 \text{ ReLU} \rightarrow 8 \text{ ReLU} \rightarrow 1$)	2
Gambar 3.1	Kurva sigmoid memetakan z ke $(0, 1)$	3
Gambar 3.2	<i>Confusion matrix</i> contoh data tidak seimbang	3
Gambar 4.1	Contoh <i>learning curve</i> (<i>train</i> turun, <i>val</i> naik $\rightarrow$ <i>overfit</i>)	4
Gambar 5.1	<i>Learning curve overfit</i>	5
Gambar 6.1	Distribusi curah hujan harian (ekor panjang)	6
Gambar 7.1	Ilustrasi RNN <i>unrolled</i> (state h)	7
Gambar 8.1	Spektrum frekuensi pasang surut (M2/K1)	8
Gambar 8.2	Prediksi vs aktual 7 hari (data sample Cilacap)	8
Gambar 8.3	Residu per amplitudo dan fase pasang M2	8
Gambar 9.1	Precision–recall untuk hujan lebat	9
Gambar 9.2	Verifikasi per kategori intensitas	9
Gambar 10.1	Grafik kendali MAE (deteksi <i>drift</i>)	10

Daftar Tabel

Nomor	Keterangan	Bab
Tabel 1.1	Peta aplikasi <i>deep learning</i> dalam meteorologi	1
Tabel 1.2	Contoh data cuaca mini → shape tensor	1
Tabel 1.3	Glosarium mini bab 1	1
Tabel 2.1	Contoh target regresi meteorologi (satuan & sifat data)	2
Tabel 2.2	Contoh deret raw pasang surut (nilai ilustratif)	2
Tabel 2.3	Contoh <i>windowing</i> (dua langkah) pasang surut	2
Tabel 2.4	Perbandingan MAE vs MSE	2
Tabel 3.1	Perbedaan regresi vs klasifikasi	3
Tabel 3.2	Perbandingan sigmoid vs softmax	3
Tabel 3.3	Contoh data tidak seimbang	3
Tabel 3.4	Struktur <i>confusion matrix</i> biner	3
Tabel 4.1	Perbandingan fungsi aktivasi dari sisi gradien	4
Tabel 4.2	SGD vs Adam	4
Tabel 5.1	Underfit / fit / overfit	5
Tabel 5.2	Panduan memilih metrik regresi	5
Tabel 5.3	Kuartet verifikasi WMO (POD/FAR/CSI/TS)	5
Tabel 5.4	Dua model, cerita metrik berbeda	5
Tabel 5.5	Skema <i>walk-forward</i> (5 <i>fold</i>)	5
Tabel 6.1	Sumber data utama (GHCND/CHIRPS, ERA5, CMIP6, PSMSL, satelit)	6

Nomor	Keterangan	Bab
Tabel 6.2	Perbandingan format berkas (CSV/NetCDF/GRIB)	6
Tabel 7.1	Contoh <i>windowing</i> ($w = 3, h = 1$)	7
Tabel 7.2	Pilihan panjang <i>window</i>	7
Tabel 7.3	<i>Baseline</i> deret waktu	7
Tabel 7.4	LSTM vs GRU	7
Tabel 7.5	Strategi multi-langkah	7
Tabel 8.1	Tipe pasang surut Indonesia	8
Tabel 8.2	Harmonik vs <i>machine learning</i>	8
Tabel 8.3	Stasiun Indonesia di sumber terbuka (IOC/UHSLC/PSMSL)	8
Tabel 8.4	Ringkasan dataset Cilacap yang dibangun (sintetik deterministik)	8
Tabel 8.5	Pilihan <i>window</i> (jam-an)	8
Tabel 8.6	Contoh hasil MAE per <i>horizon</i>	8
Tabel 8.7	<i>Skill score</i> relatif vs <i>persistence</i>	8
Tabel 9.1	Fitur yang dibangun untuk stasiun	9
Tabel 9.2	Kategori intensitas hujan	9
Tabel 9.3	Verifikasi <i>threshold</i> (POD/FAR/CSI)	9
Tabel 9.4	Rancangan eksperimen	9
Tabel 9.5	Verifikasi per kategori	9
Tabel 10.1	Prosedur <i>retraining</i> bertahap	10
Tabel 10.2	Alur keputusan <i>retraining</i>	10
Tabel 10.3	Etika penggunaan <i>deep learning</i>	10

Nomor tabel Bab 10 pada daftar ini menyesuaikan urutan sekuensial yang konsisten; periksa silang dengan REGISTER.md saat *review* final.

Prakata (Edisi 2.0)

Buku ini lahir dari sebuah pertanyaan sederhana yang sering saya dengar dari teman-teman di BMKG dan dari mahasiswa kebumian: “*Saya ingin belajar deep learning, tapi dari mana mulainya?*”

Jawaban yang jujur adalah: bahan belajarnya banyak, tetapi hampir semuanya ditulis dalam bahasa Inggris dan dengan contoh dari belahan dunia lain. Mahasiswa kebumian di Indonesia yang ingin belajar *deep learning* biasanya harus menerjemahkan dua hal sekaligus: bahasa dan konteks. Buku ini ditulis dalam bahasa Indonesia dan dengan contoh yang dekat dengan dunia meteorologi Indonesia, sehingga mengatasi kedua hambatan itu sekaligus.

Saya menulis buku ini dengan keyakinan bahwa **deep learning bukan monopoli ilmuwan komputer**: belajar deep learning bukan hanya memahami arsitektur *neural network*, tetapi juga memakai metode itu untuk menghadapi persoalan nyata. Alat terbaik untuk praktisi meteorologi adalah yang dijelaskan dengan data meteorologi. Karena itu, di setiap bab Anda akan menemukan:

1. **Konsep** — dijelaskan dari nol, dengan analogi dunia nyata.
2. **Kode** — notebook yang dapat dijalankan langsung di Google Colab.
3. **Data Indonesia** — pasang surut, curah hujan, data terbuka (ERA5/CHIRPS).
4. **Evaluasi yang jujur** — model selalu dibandingkan dengan *baseline*; tidak ada klaim berlebihan.

Mengapa buku ini ada

Tiga alasan utama:

1. **Celah konten**: materi *deep learning* berbahasa Indonesia dengan konteks meteorologi masih jarang.
2. **Kebutuhan praktisi**: banyak tugas harian (prediksi tinggi pasang, prediksi hujan, verifikasi peringatan dini) bisa dibantu model *neural network*, tetapi praktisi butuh panduan yang relevan dengan pekerjaannya.
3. **Prinsip keterbukaan**: buku ini gratis, *living* (bisa diperbarui), dan setiap hasil dilaporkan apa adanya.

Struktur buku

Buku ini terdiri atas empat bagian:

- **Bagian I — Fondasi (Bab 1–5)**: posisi *deep learning*, regresi, klasifikasi, optimasi/backpropagation, serta evaluasi dan regularisasi untuk data iklim.
- **Bagian II — Data dan Model Sekuensial (Bab 6–7)**: pengelolaan data meteorologi dan model untuk deret waktu (RNN, LSTM, GRU).

- **Bagian III — Studi Kasus (Bab 8–9):** dua proyek *end-to-end* — prediksi pasang surut (Cilacap) dan curah hujan harian dengan data terbuka.
- **Bagian IV — Operasional dan Arah Riset (Bab 10):** dari riset ke praktik, interpretasi model, keterbatasan, etika, dan peta riset lanjutan.

Setiap bab berdiri sendiri dan dilengkapi *sidebar* “Prasyarat”, satu notebook Colab, latihan, serta daftar referensi dalam format IEEE. Bagian “Cara Memakai Buku” memetakan urutan baca yang disarankan.

Siapa yang sebaiknya membaca

- **Mahasiswa S1 kebumian** (meteorologi, oseanografi, geofisika) yang ingin mulai belajar *deep learning* dari nol.
- **Praktisi meteorologi dan lembaga riset** yang ingin menilai kapan *deep learning* layak dipakai dan bagaimana membangun model sederhana.
- **Siapa pun** dengan latar belakang non-informatika yang ingin memahami cara kerja jaringan saraf melalui contoh data nyata.

Keterbatasan yang jujur

Buku ini adalah **pengantar**, bukan pengganti buku teks seperti *Deep Learning* (Goodfellow et al., 2016) atau kursus yang lebih dalam. Buku ini bukan hasil riset baru, melainkan jembatan menuju literatur primer. Angka dan hasil yang ditampilkan harus dipahami sebagai ilustrasi alur, bukan sebagai klaim performa untuk semua kondisi. Bab 10 membahas keterbatasan ini lebih lanjut.

Ucapan terima kasih

Buku ini tidak akan ada tanpa:

- Teman-teman di BMKG yang selalu bertanya “bagaimana caranya?” dan memotivasi penjelasan yang sederhana.
- Mahasiswa kebumian yang bersedia menjadi pembaca awal dan memberi masukan.
- Komunitas open-source TensorFlow, Pandas, NumPy, dan xarray yang membuat alat ini tersedia gratis.
- Penyedia data terbuka yang dipakai studi kasus — CHC UCSB (CHIRPS), NOAA, Copernicus/ERA5, UNESCO/IOC, UHSLC, dan PSMSL.
- Para penelaah anonim yang memberi umpan balik pada versi draf buku ini.

Cara menggunakan buku ini

Baca Bab 1–5 berurutan untuk fondasi, lalu Bab 6–10 sesuai kebutuhan. Selalu jalankan notebook saat membaca — jangan hanya membaca. Panduan lengkap ada di bagian “Cara Memakai Buku”.

Selamat belajar. Semoga buku ini bermanfaat untuk pekerjaan dan riset Anda.

Kanada Kurniawan

Pontianak, September 2026

Cara Memakai Buku Ini

Bagian depan buku (*front matter*) ini bukan bagian dari bab, sehingga **tidak dirilis ke Zenodo sebagai bab ber-DOI**. Dokumen ini menjadi satu sumber untuk panduan penggunaan versi buku (halaman depan) dan halaman panduan di blog.

Struktur dan urutan baca

- **Baca berurutan** untuk pengalaman mengalir: Bab 1–5 fondasi, Bab 6–7 data dan model sekuensial, Bab 8–9 studi kasus, Bab 10 operasional.
- **Atau baca per bab**: setiap bab berdiri sendiri dengan *sidebar* “Prasyarat: Bab ...” yang memetakan urutan minimal yang perlu dikuasai sebelumnya.

Rute baca	Bab	Tujuan
Jalur fondasi	1 → 2 → 3 → 4 → 5	Menguasai dasar regresi, klasifikasi, optimasi, dan evaluasi
Jalur data & deret waktu	6 → 7	Menguasai pengelolaan data meteo dan model LSTM/GRU
Jalur studi kasus	8 → 9	Praktik <i>end-to-end</i> dengan data Indonesia
Jalur operasional	10	Interpretasi, etika, dan arah riset

Konvensi yang dipakai di seluruh buku

- **Penomoran aset**: Gambar `bab.nomor`, Tabel `bab.nomor`, Persamaan (`bab.nomor`), Kode `bab.nomor` — deret nomor masing-masing jenis re-set di tiap bab.
- **Notasi matematis**: notasi terpusat di bagian “Glosarium dan Notasi”; istilah Indonesia + Inggris ditulis seragam di seluruh buku.
- **Istilah baru** ditulis dalam bahasa Indonesia dengan istilah Inggris dalam tanda kurung pada pemunculan pertama (mis. “fungsi aktivasi (*activation function*)”) dan dikumpulkan di glosarium.
- **Kode**: blok kode ditulis verbatim; nama identifier tidak diubah menjadi miring.

Praktikkan dengan notebook

Setiap bab disertai notebook Colab yang dapat dieksekusi serta data yang dirilis di Zenodo (DOI tercantum di metadata bab dan di bagian “Daftar Notebook dan DOI”). Cara menjalankan:

1. Buka Google Colab (gratis, tanpa instalasi) dan impor notebook GitHub, atau
2. Jalankan secara lokal: `pip install tensorflow numpy pandas matplotlib` (persyaratan per-bab tercantum di notebook dan Lampiran A).

Reproduksibilitas

- **Seed tetap** (`np.random.seed(42)`, `tf.random.set_seed(42)`) dipakai agar hasil dapat diulang.
- **Versi TensorFlow** tercatat di notebook dan Lampiran A.
- Data contoh sintetik dipakai agar notebook dapat berjalan tanpa internet; hasilnya **bukan** klaim data nyata.

Glosarium dan Notasi

Glosarium ini memuat istilah-istilah yang dipakai di seluruh buku. Pada pemunculan pertama di tiap bab, istilah ditulis dengan pola “padanan Indonesia (*istilah Inggris*).” Setelahnya, buku konsisten memakai salah satu bentuk saja. Konvensi penulisan mengikuti aturan di `outline.md` (PUEBI §18: istilah asing yang belum diserap dicetak miring).

Glosarium Istilah

Istilah	Definisi (ringkas)	Bab terkait
<i>Artificial intelligence</i> (AI)	Kecerdasan buatan: bidang studi membuat mesin meniru kemampuan kognitif manusia	1
<i>Machine learning</i> (ML)	Pembelajaran mesin: cabang AI yang belajar pola dari data tanpa diprogram eksplisit per aturan	1
<i>Deep learning</i> (DL)	Pembelajaran mendalam: cabang ML yang memakai jaringan saraf berlapis	1
<i>Neural network</i>	Jaringan saraf: model matematis berlapis yang terinspirasi neuron biologis	1–10
Neuron / perceptron	Unit komputasi dasar jaringan: $\mathbf{z} = \sum \mathbf{w} \mathbf{x} + \mathbf{b}$, lalu aktivasi	2
Fungsi aktivasi (<i>activation function</i>)	Fungsi non-linear yang memberi kapasitas model (mis. ReLU, sigmoid, tanh)	2, 4
Bobot (<i>weight</i>), bias	Parameter model yang dipelajari saat pelatihan	2, 4
Tolok ukur (<i>baseline</i>)	Model sederhana sebagai pembanding minimal (persistence, mean, ARIMA)	5, 7, 8, 9

Istilah	Definisi (ringkas)	Bab terkait
<i>Persistence</i>	<i>Baseline</i> yang memprediksi nilai sekarang untuk waktu depan ($\hat{y}(t+h) = y(t)$)	7, 8
Regresi	Memprediksi nilai kontinu	2, 5, 7–9
Klasifikasi	Memprediksi label/kategori	3, 5, 9
<i>Loss function</i>	Fungsi kerugian yang diminimalkan saat pelatihan (MSE, MAE, cross-entropy)	2–4
<i>Gradient descent</i>	Algoritma optimasi dengan turunan dari <i>loss</i>	4
<i>Backpropagation</i>	Cara menghitung gradien dengan aturan rantai	4
<i>Learning rate</i>	Langkah ukuran pembaruan bobot	4
<i>Epoch, batch size</i>	Satu putaran penuh dataset; jumlah sampel per pembaruan	4, 5
<i>Overfitting / underfitting</i>	Model terlalu menghafal data / terlalu sederhana	5
<i>Regularisasi</i> (L2, <i>dropout</i>)	Teknik mencegah <i>overfitting</i>	5
<i>Early stopping</i>	Menghentikan pelatihan saat validasi berhenti membaik	5
<i>Walk-forward / blocked</i>	Validasi silang deret waktu dengan urutan waktu dijaga	5, 8, 9
<i>Leakage</i>	Kebocoran informasi masa depan ke data latih	5, 8
<i>Reanalysis</i>	Data cuaca historis gabungan model + observasi (mis. ERA5)	6, 9
Nilai hilang / <i>gap</i>	Data yang tidak terekam; ditangani dengan imputasi/pemotongan	6, 8

Istilah	Definisi (ringkas)	Bab terkait
<i>Outlier</i> / pencilan	Nilai ekstrem yang menyimpang dari pola; potensi galat pengukuran	6
Imputasi	Mengisi nilai hilang dengan estimasi	6, 8, 10
Normalisasi	Menskala fitur (mis. z-score) agar pelatihan stabil	6
Fitur lag / <i>window</i>	Konstruksi masukan deret waktu dari masa lalu	7, 8
RNN / LSTM / GRU	Jaringan sekuensial; LSTM/GRU punya <i>gate</i> ingatan	7
Multi-step forecast	Prediksi beberapa langkah ke depan (recursive/direct/seq2seq)	7
Analisis harmonik	Pemodelan pasang surut dengan konstituen astronomis (M2, S2, K1, O1)	8
Formzahl (F)	Rasio komponen untuk menentukan tipe pasang	8
<i>Tide gauge</i> / pasang surut	Alat/deret tinggi muka air	8
MAE / RMSE	Metrik galat regresi: <i>mean absolute error</i> ; <i>root mean square error</i>	2, 5, 8, 9
R ² , Willmott, KGE	Metrik kesesuaian regresi	5
CSI / FAR / POD / TS	Metrik verifikasi kejadian ekstrem (kategori)	3, 5, 9
<i>Confusion matrix</i>	Tabel tabulasi prediksi vs aktual untuk klasifikasi	3, 5
<i>Class imbalance</i>	Ketidakseimbangan jumlah sampel antar-kelas	3, 5, 9
<i>Threshold</i>	Ambang keputusan (mis. hujan jika probabilitas > 0,5)	3, 9

Istilah	Definisi (ringkas)	Bab terkait
<i>Drift</i>	Perubahan distribusi data seiring waktu (atmosfer non-stasioner)	10
<i>Quantile / interval</i> prediksi	Rentang ketidakpastian prediksi	10
SHAP	Metode interpretasi model (kontribusi fitur)	10
<i>Downscaling</i>	Dari skala reanalysis/global ke skala lokal	10
<i>Nowcasting</i>	Prediksi sangat jangka pendek (kini–6 jam)	10
<i>Generative</i>	Model yang menghasilkan data baru (imputasi, skenario)	10
<i>Seed</i>	Nilai acak tetap untuk reproduksibilitas	5, 6, 8

Notasi Matematis

Konvensi notasi di seluruh buku:

Simbol	Makna
$\mathbf{x}$	Fitur / input ke- i
$\mathbf{w}$	Bobot (<i>weight</i>) ke- i
$\mathbf{b}$	Bias
$\mathbf{z}$	Masukan sebelum aktivasi: $\mathbf{z} = \Sigma \mathbf{w} \mathbf{x} + \mathbf{b}$
$\mathbf{a} / \hat{\mathbf{y}}$	Output (aktivasi / prediksi)
$f(\cdot), (\cdot)$	Fungsi aktivasi; khususnya sigmoid: $(z) = 1/(1+e^{-z})$
$\text{ReLU}(\mathbf{x})$	$\max(0, \mathbf{x})$
$\mathbf{y}(\mathbf{t})$	Nilai aktual deret waktu pada waktu $\mathbf{t}$
$\hat{\mathbf{y}}(\mathbf{t}+\mathbf{h})$	Prediksi pada <i>horizon</i> $\mathbf{h}$ ke depan
$\mathbf{w}$ (<i>window</i>), $\mathbf{h}$ (<i>horizon</i>)	Panjang jendela input; jarak prediksi
$\mathbf{X}, \mathbf{y}$	Matriks fitur; vektor target
η	Parameter model; <i>learning rate</i>
$\mathcal{L}, J$	<i>Loss function</i> (fungsi kerugian)
MAE	$(1/n) \sum y - \hat{y} $
RMSE	$\sqrt{(1/n) \sum (y - \hat{y})^2}$
POD / FAR / CSI	Peluang deteksi; <i>false alarm ratio</i> ; <i>critical success index</i>

Simbol	Makna
SS	<i>Skill score</i> relatif terhadap <i>baseline</i> : $1 - \text{MAE_model} / \text{MAE_baseline}$
F (Formzahl)	$(K1 + O1) / (M2 + S2)$ untuk tipe pasang

Singkatan yang Sering Dipakai

Singkatan	Kepanjangan
AI	<i>Artificial Intelligence</i> (kecerdasan buatan)
ML	<i>Machine Learning</i> (pembelajaran mesin)
DL	<i>Deep Learning</i> (pembelajaran mendalam)
RNN / LSTM / GRU	<i>Recurrent Neural Network / Long Short-Term Memory / Gated Recurrent Unit</i>
MLP	<i>Multilayer Perceptron</i>
CNN	<i>Convolutional Neural Network</i>
MAE / MSE / RMSE	<i>Mean Absolute Error / Mean Squared Error / Root Mean Square Error</i>
R ²	Koefisien determinasi
KGE	<i>Kling-Gupta Efficiency</i>
POD / FAR / CSI / TS	<i>Probability of Detection / False Alarm Ratio / Critical Success Index / Threat Score</i>
SS	<i>Skill Score</i>
GHCND	<i>Global Historical Climatology Network - Daily</i> (NOAA)
CHIRPS	<i>Climate Hazards Group InfraRed Precipitation with Station data</i>
BIG	Badan Informasi Geospasial
WMO	<i>World Meteorological Organization</i>
ERA5	Reanalysis global generasi kelima ECMWF
CHIRPS	<i>Climate Hazards Group InfraRed Precipitation with Station data</i>
ENSO / MEI	<i>El Niño-Southern Oscillation / Multivariate ENSO Index</i>
MJO / RMM	<i>Madden-Julian Oscillation / Real-time Multivariate MJO index</i>

Singkatan	Kepanjangan
PSMSL / IOC / UHSLC	<i>Permanent Service for Mean Sea Level / Intergovernmental Oceanographic Commission / University of Hawaii Sea Level Center</i>
SHAP	<i>SHapley Additive exPlanations</i>

Persamaan bernomor mengikuti format **Persamaan (bab.nomor)**, mis. **Persamaan (2.1)**. Aset lain: **Gambar bab.nomor**, **Tabel bab.nomor**, **Kode bab.nomor**.

Buku ini ditujukan sebagai **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan dan penyusunan ulang dari literatur klasik *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Prasyarat bab ini: tidak ada, ini titik awal. Bab berikutnya mengasumsikan Bab 1 dikuasai. Jika Anda sudah terbiasa dengan dasar TensorFlow, Anda boleh melompat ke Bab 2, tetapi baca Bagian 1.6-1.8 untuk memahami notasi yang dipakai buku ini.

1.0 Mengapa Buku Ini, dan Mengapa Sekarang untuk Pembaca Indonesia

Belajar *deep learning* bukan hanya memahami arsitektur *neural network*, tetapi juga memakai metode itu untuk menghadapi persoalan nyata. Buku ini ditujukan untuk mahasiswa kebumihan dan praktisi yang mulai dari nol, dengan **meteorologi Indonesia** sebagai konteks utama.

Sebelum masuk ke definisi, ada baiknya kita berhenti sejenak: **mengapa sebuah buku *deep learning* berbahasa Indonesia, dengan konteks meteorologi lokal, layak dibaca di tengah membanjirnya kursus dan tutorial daring?**

Ada dua alasan praktis.

1. Materi DL berbahasa Indonesia masih jarang, dan yang ada jarang yang kontekstual. Sebagian besar referensi fundamental *deep learning* (kursus daring, buku teks, makalah) ditulis dalam bahasa Inggris dan dengan contoh dari belahan dunia lain [1]. Mahasiswa S1 kebumihan di Indonesia yang ingin belajar DL biasanya harus menerjemahkan dua hal sekaligus: bahasa dan konteks. Buku ini ditulis dalam bahasa Indonesia dan dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia, sehingga mengatasi kedua hambatan itu sekaligus. Kami tidak menggantikan sumber primer; kami menyediakan **jembatan** ke sumber tersebut.

2. Data lokal, notebook yang dapat dijalankan, dan evaluasi yang jujur. Kami sengaja tidak menulis bab ini sebagai esai filosofis: konsep dijelaskan dengan intuisi + kode. Tiap bab membawa notebook Colab yang dapat dijalankan langsung dan dataset kecil, yaitu data sintetis atau data publik terbuka seperti ERA5, CHIRPS, atau pasang surut dengan sumber yang dicantumkan. Evaluasi disajikan jujur: model selalu dibandingkan dengan *baseline*. Anda tidak hanya membaca, tetapi juga menjalankan ulang.

Ringkasnya: buku ini bukan satu-satunya sumber yang Anda butuhkan, melainkan **teman belajar yang membawa Anda dari nol hingga mampu membangun model DL pertama untuk data meteorologi Indonesia**, dengan rujukan ke literatur primer untuk pendalaman.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Membedakan** *artificial intelligence*, *machine learning*, dan *deep learning* beserta contoh aplikasinya di meteorologi.
2. **Memetakan** aplikasi *deep learning* meteorologi ke bab yang relevan dan membedakan mana yang dibahas buku ini vs literatur lanjut.
3. **Menilai** secara kritis kapan *deep learning* layak dipakai dibanding *baseline* statistik (ukuran data, non-linearitas, konteks operasional).
4. **Menyiapkan** lingkungan kerja Google Colab + TensorFlow/Keras dan membuat tensor pertama dari contoh data cuaca mini.

1.1 *Artificial Intelligence, Machine Learning, dan Deep Learning*

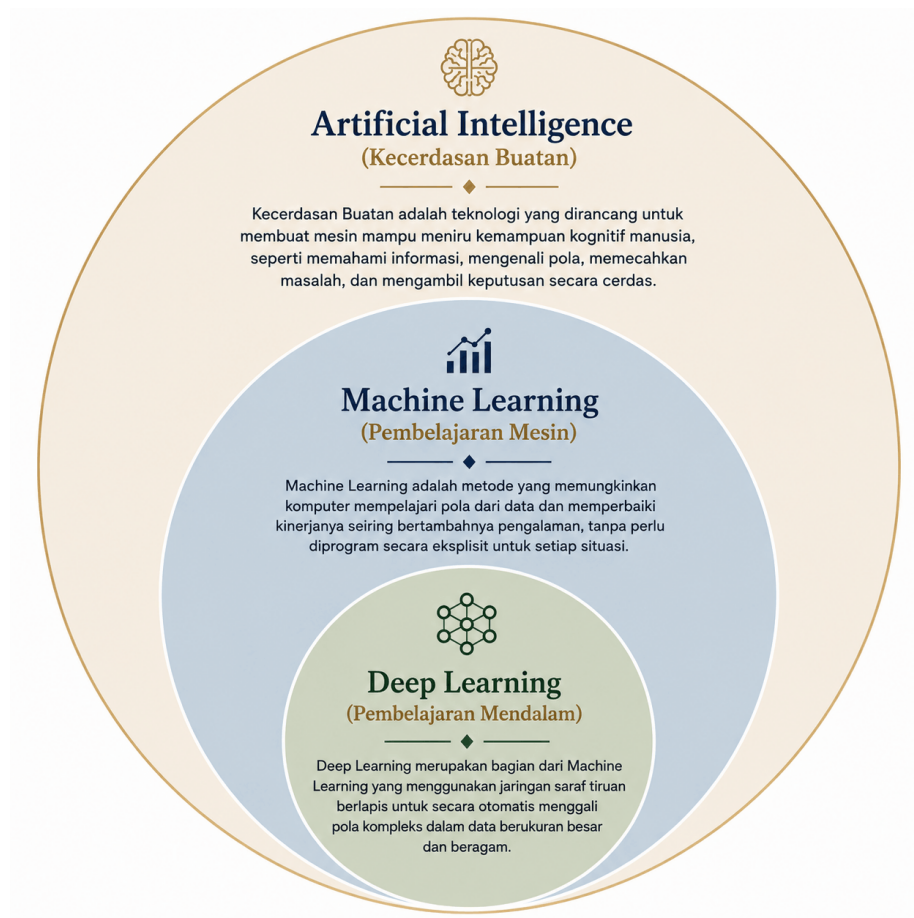
Artificial intelligence (AI, kecerdasan buatan) adalah bidang yang mempelajari cara membuat mesin meniru kemampuan kognitif manusia, seperti memahami bahasa, mengenali pola, dan mengambil keputusan [2]. Istilah ini sudah ada sejak 1956 ketika para ilmuwan mulai menyelidiki apakah mesin dapat “berpikir”. Seiring waktu, AI berkembang menjadi banyak sub-bidang: sistem berbasis aturan, pengenalan pola, robotika, pemrosesan bahasa alami, dan *machine learning* (pembelajaran mesin).

Machine learning (ML) adalah cabang AI yang membuat komputer belajar pola dari data tanpa diprogram secara eksplisit untuk setiap aturan [3]. Alih-alih menulis aturan manual seperti “jika hari ini hujan dan kelembapan tinggi, maka besok akan hujan”, kita memberi model ribuan contoh dan membiarkannya **menemukan sendiri** pola yang berguna, misalnya variabel mana yang paling berpengaruh terhadap hujan. Contoh sederhana: jika kita memberi model data riwayat hujan, suhu, dan kelembapan selama bertahun-tahun, model dapat menemukan kombinasi yang paling menjelaskan kapan hujan turun dan kapan tidak. Proses belajar itu akan dijelaskan di Bab 4.

Deep learning (DL) adalah salah satu cabang *machine learning* yang menggunakan **jaringan saraf berlapis (*neural network*)**, model matematis yang terinspirasi dari susunan neuron biologis [4]. “Dalam” (*deep*) merujuk pada banyaknya lapisan: model tersusun dari banyak lapisan unit sederhana, dan semakin banyak lapisannya, model mampu menangkap pola yang semakin rumit, seperti hubungan rumit antara banyak variabel meteorologi [4]. Kekuatan DL justru datang dari *kedalaman* ini: lapisan-lapisan awal mempelajari pola sederhana, lalu lapisan-lapisan berikutnya menggabungkannya menjadi representasi yang semakin abstrak [1].

Gambar 1.1: Keterkaitan AI, ML, dan DL.

Sebagaimana dilihat pada Gambar 1.1, *artificial intelligence* adalah payung terluas, *machine learning* adalah cabangnya yang belajar dari data, dan *deep*



Gambar 1: Gambar 1.1 - Keterkaitan AI, ML, dan DL

learning adalah bagian dari *machine learning* yang menggunakan jaringan saraf berlapis untuk belajar dari data. Model generatif (*generative*) modern seperti *chatbot* juga termasuk *deep learning*: model tidak hanya mengklasifikasikan data, tetapi belajar pola dari data lalu **menghasilkan konten baru** (teks, gambar) yang mengikuti pola itu. Definisi dan keterbatasannya dibahas lebih lanjut di Bab 10.

Secara praktis, perbedaan utamanya adalah cara model memproses pola data. Model *machine learning* konvensional seperti regresi linear, *decision tree*, atau SVM bekerja dengan fungsi atau logika pemisahan data yang spesifik. Pendekatan ini efektif untuk pola dasar, tetapi sering kesulitan menghadapi dinamika atmosfer yang kompleks. Sebaliknya, *deep learning* dirancang untuk mengenali hubungan data yang rumit menggunakan struktur jaringan saraf. Komponen dasar jaringan ini meliputi neuron, *perceptron*, dan fungsi aktivasi, yang akan dibahas lebih rinci pada Bab 2.

Kapan Anda memakai yang mana? Ringkasnya: untuk data kecil dan pola yang relatif sederhana, model ML klasik biasanya lebih stabil dan lebih mudah dijelaskan; *deep learning* baru layak dicoba saat data besar atau polanya kompleks dan non-linear. Perbandingan ini bukan hitam-putih, dan aturan praktis yang lebih rinci (termasuk *baseline*, model berbasis pohon, serta biaya dan pemeliharaan) kita bahas dalam Bagian 1.4.

1.2 Mengapa Deep Learning Relevan Sekarang

Deep learning bukan teknologi baru dalam konsep (cikal bakalnya, *perceptron*, lahir tahun 1958 [5]), tetapi baru praktis digunakan secara luas dalam dekade terakhir karena tiga hal bertemu sekaligus:

1. **Data besar**, sensor otomatis, *reanalysis* seperti ERA5 (data cuaca historis dari gabungan model dan observasi), dan arsip klimatologi menyediakan data meteorologi dalam jumlah besar (Bab 6 membahas sumber datanya). Model DL baru bersinar saat volume datanya besar.
2. **Komputasi murah**, GPU (kartu grafis untuk komputasi paralel) dan *cloud notebook* gratis seperti **Google Colab** membuat pelatihan jaringan saraf dapat dilakukan tanpa server mahal. Sebuah GPU modern bisa berisi ribuan inti yang memproses data secara paralel, mempercepat matematika jaringan saraf puluhan hingga ratusan kali.
3. **Tooling matang**, TensorFlow/Keras dan PyTorch menyediakan API yang relatif mudah dipelajari [6]. Tidak perlu lagi menulis kode matematika dari nol untuk tiap proyek; Anda tinggal merakit blok yang sudah tersedia.

Kombinasi ini membuat *deep learning* dapat diadopsi oleh mahasiswa dan praktisi kebumih, bukan hanya peneliti ilmu komputer [7]. Ini kunci filosofi buku ini: Anda tidak perlu gelar di bidang komputer untuk mulai menggunakan DL, asalkan punya data, perangkat, dan kemauan belajar.

Sejarah singkat (agar konteksnya jelas)

- 1958, *Perceptron* oleh Frank Rosenblatt: cikal bakal jaringan saraf, namun masih **satu lapis** (bukan *deep learning* seperti yang kita kenal sekarang) [5].
- 1986, *Backpropagation* dipopulerkan: cara melatih jaringan bertingkat banyak [8].
- 2012, **AlexNet** memenangkan kompetisi ImageNet dengan CNN [9].
- 2015, Ulasan “Deep learning” di *Nature* menempatkan DL sebagai teknik inti AI [4].
- 2019, Ulasan Reichstein et al. menegaskan bahwa DL tidak hanya berpeluang besar untuk prediksi, tetapi juga membantu pemahaman proses dalam sistem kebumihan [7].

Anda tidak perlu hafal tahun-tahun ini, tetapi memahami bahwa DL bukan “keajaiban” yang baru lahir kemarin membantu Anda menilai klaim-klaim besar di berita.

1.3 Peta Aplikasi Deep Learning dalam Meteorologi

Deep learning telah diterapkan di berbagai permasalahan meteorologi. Agar pembaca tidak tersesat, buku ini memetakan aplikasi tersebut dan menandai mana yang **dibahas di sini** secara mendalam dan mana yang hanya diarahkan ke literatur lanjut:

Tabel 1.1: Peta aplikasi *deep learning* dalam meteorologi.

Aplikasi	Contoh pertanyaan	Dibahas di buku ini
Prediksi deret waktu (<i>time series</i>)	Berapa tinggi pasang surut besok? Berapa hujan minggu depan?	Bab 2, 7-9
Klasifikasi kejadian	Hujan lebat atau tidak? Level bahaya apa?	Bab 3, 5, 9
Imputasi (pengisian) data hilang	Bagaimana mengisi gap data stasiun?	Bab 6 (dasar), Bab 8 (isi gap), Bab 10 (generatif)
<i>Nowcasting</i> (prediksi kini-6 jam)	Apa yang terjadi kini hingga 6 jam ke depan (radar/satelit)?	Bab 10 (arah riset)
<i>Downscaling</i> / data spasial	Dari skala <i>reanalysis</i> ke skala lokal	Bab 10 (arah riset)
Model generatif (<i>generative</i>)	Membuat skenario iklim, imputasi realistis, super-resolusi	Bab 10 (arah riset)

Fokus buku ini sengaja dibatasi pada **dua aplikasi inti** (lihat baris pertama dan kedua Tabel 1.1): prediksi besaran (regresi) dan klasifikasi kejadian, pada data deret waktu meteorologi Indonesia. Ada tiga alasan pembatasan ini:

1. **Kedalaman > keluasan.** Daripada membahas sepuluh topik secara setengah-setengah, buku ini mengupas dua topik sampai tuntas, dari data hingga evaluasi operasional.
2. **Aplikasi paling dekat dengan praktisi.** Regresi dan klasifikasi adalah dua hal pertama yang Anda butuhkan untuk tugas prediksi dan pengambilan keputusan harian.
3. **Bekal yang portabel.** Penguasaan analisis regresi, klasifikasi, dan deret waktu berbasis TensorFlow mempermudah pemahaman arsitektur tingkat lanjut, seperti CNN untuk citra radar maupun model *diffusion* untuk imputasi data.

Tiga aplikasi berikutnya dalam Tabel 1.1 — *nowcasting*, *downscaling*, dan model generatif adalah arah yang kami promosikan di Bab 10 sebagai peta jalan riset. Buku ini tidak mengupasnya secara mendalam karena (a) membutuhkan data spasial/grid yang volumenya jauh lebih besar dan (b) arsitekturnya (U-Net, Transformer, difusi) berada di luar cakupan buku pengantar ini. Khusus verifikasi & *post-processing*, buku ini hanya menyebutnya singkat di Bab 10.

Apa yang TIDAK dibahas buku ini (secara sengaja)

Untuk kejelasan, kami juga mendaftar yang **tidak** akan Anda temui di sini, agar Anda memiliki ekspektasi yang tepat:

- **Bahasa pemrograman lain selain Python.** TensorFlow dan ekosistemnya (Pandas, NumPy, xarray) sudah merupakan pilihan dominan untuk DL saintifik.
- **Arsitektur di luar MLP, LSTM/GRU.** CNN, Transformer, dan model generatif disentuh hanya di Bab 10 sebagai peta jalan.
- **Pengembangan model untuk skala *big data*.** Buku ini fokus pada kasus yang dapat dieksekusi di Colab gratis dengan data berukuran puluhan hingga ratusan MB.
- **Topik ML klasik murni** (regresi linear, ARIMA, SVM) hanya dibahas singkat sebagai *baseline*, bukan sebagai topik utama.

1.4 Kapan Deep Learning Layak, Kapan Tidak

Deep learning bukan solusi untuk semua masalah. Aturan praktis yang akan dipakai di seluruh buku:

- **Gunakan model statistik klasik dulu sebagai pembanding (*baseline*).** Regresi linear, ARIMA (model statistik untuk deret waktu), atau *persistence* (“keadaan besok = keadaan hari ini”) sering kali lebih dari cukup untuk data pendek atau pola sederhana. *Deep learning* hanya layak jika **mengalahkan *baseline*** dengan data yang cukup. Prinsip ini menjadi tulang punggung Bab 7-9. Jika model sederhana sudah cukup, tidak perlu menambah kompleksitas.
- **Ingat *baseline* non-linear: model berbasis pohon.** Untuk data tabular dan deret waktu, algoritma seperti **Random Forest** atau **XGBoost** sering mengalahkan jaringan saraf dari segi kecepatan, akurasi, dan keterbacaan. Buku ini fokus pada jaringan saraf, tetapi praktisi wajib tahu bahwa model-model itu bisa menjadi pembanding yang sangat kuat sebelum beralih ke *deep learning*, yang benar-benar unggul saat data masif atau melibatkan data spasial (citra radar/satelit, grid).
- **Perhatikan ukuran data.** Jaringan saraf besar membutuhkan banyak data untuk belajar. Untuk deret waktu stasiun dengan puluhan ribu pengamatan, model sekuensial (*sequence*) seperti LSTM (Bab 7) adalah pilihan yang masuk akal, tetapi jangan langsung melompat ke arsitektur masif yang dirancang untuk miliaran parameter (angka-angka yang dipelajari model dari data; butuh data jauh lebih besar). Catatan penting: sejak beberapa tahun terakhir, ***transfer learning*** dan model terlatih (*pre-trained model*) memungkinkan *deep learning* tetap berguna meskipun data kita terbatas. Misalnya mengambil model yang dilatih pada data reanalisis global lalu menyesuaikannya (*fine-tuning*) dengan data stasiun lokal. Poin “data besar” di atas tidak lagi mutlak; Bab 10 membahas *transfer learning* singkat sebagai salah satu arah riset.
- **Utamakan keterbacaan dan kepercayaan di konteks operasional.** Di lingkungan layanan meteorologi, model sederhana yang dapat dijelaskan kadang lebih diterima daripada model “kotak hitam”. Bab 10 membahas interpretasi dan keterbatasan. Tidak semua pengguna akhir (kepala stasiun, pengguna data, pengambil keputusan) nyaman dengan hasil yang tidak bisa dijelaskan.
- **Perhatikan biaya dan pemeliharaan.** Model DL perlu dijalankan, dimonitor, dan dilatih ulang secara berkala. Jika model klasik yang sederhana bisa bertahan lama tanpa perawatan, itu bisa jadi pilihan yang lebih cerdas di lingkungan dengan sumber daya terbatas.

Kesimpulannya: anggap *deep learning* sebagai **satu alat di dalam kotak peralatan**, bukan pengganti semua metode. Cara pandang ini menjaga pembaca dari *overhype*, kecenderungan percaya bahwa model bisa menyelesaikan semua masalah. Sebagai uji cepat: jika data Anda sedikit, hasilnya harus bisa dijelaskan kepada pengguna non-teknis, atau hubungan yang perlu dimodelkan ternyata sederhana, sangat mungkin jawabannya ada pada model statistik, bukan *deep learning*.

Karakteristik Data Meteorologi Indonesia (dan Mengapa Ini Penting untuk DL)

Sebelum masuk ke kode, pahami **ciri khas data meteorologi tropis** yang akan Anda temui di sepanjang buku. Ciri-ciri ini bukan sekadar latar belakang; mereka menentukan kapan *deep learning* benar-benar dibutuhkan dan kapan model sederhana sudah cukup.

1. Variabilitas tinggi dan rezim ganda. Curah hujan di Indonesia dipengaruhi monsun Australia-Asia, *Madden-Julian Oscillation* (MJO) [10], *El Niño-Southern Oscillation* (ENSO), dan siklus diurnal laut-darat. Pola yang sama bisa muncul dengan amplitudo sangat berbeda antara musim kemarau dan musim hujan. Model yang belajar dari satu rezim saja akan gagal saat rezim berganti. Bab 6 membahas bagaimana membagi dan menyeimbangkan data sehingga model tidak “lupa” pada satu musim.

2. Ekor kanan (*right tail*) yang berat pada curah hujan. Distribusi hujan harian di sebagian besar wilayah Indonesia memiliki banyak hari tanpa hujan (nol) dan sedikit hari dengan hujan ekstrem (>50 mm/hari), dengan ekor distribusi yang lebih berat daripada distribusi Gaussian [11]. Ini membuat metrik rata-rata seperti RMSE tidak cukup; Bab 5 memperkenalkan metrik kejadian (CSI, FAR, POD) untuk menilai performa pada hari ekstrem.

3. Data hilang, *outlier*, dan inhomogenitas. Jaringan pengamatan meteorologi berkembang bertahap: beberapa stasiun memiliki catatan puluhan tahun, sebagian lain baru beberapa tahun. Sensor dapat diganti, dikalibrasi ulang, atau catatan digital dihitung ulang; inhomogenitas seperti ini adalah salah satu tantangan utama yang disoroti dalam literatur pembelajaran mesin untuk sains kebumihan [7]. Bab 6 membahas imputasi dasar dan eksplorasi data yang hati-hati.

4. Sinyal pasang surut yang kuat tetapi nonstasioner. Di stasiun pesisir seperti Cilacap (Bab 8), sinyal pasang surut memiliki komponen harmonik yang kuat (semi-diurnal, diurnal, dan campuran) tetapi amplitudo dan fase dipengaruhi faktor non-astronomis: cuaca (angin, tekanan), variabilitas laut regional, dan debit sungai (di muara sungai seperti Kapuas). Ini menjadikannya kasus menarik untuk model sekuensial: pola periodik yang bisa dipelajari, dengan komponen residual yang menantang.

5. Keterbatasan data latih untuk kejadian ekstrem. Hujan ekstrem (basis peringatan dini) dan pasang surut rob adalah **ekor distribusi**, persis bagian yang paling ingin kita prediksi dengan baik, tetapi paling jarang ada datanya [7], [11]. Bab 3, 5, dan 9 membahas cara menghadapi *class imbalance* dan verifikasi operasional untuk kejadian langka.

Implikasi untuk *deep learning*:

- **Ukuran data meteorologi Indonesia cukup untuk LSTM/GRU** (Bab 7), tetapi umumnya tidak cukup untuk melatih arsitektur masif dari

nol. Kita akan selalu membandingkan dengan *baseline* sederhana, *persistence*, rata-rata klimatologis, atau ARIMA singkat. Untuk data yang sangat terbatas, *transfer learning* dari model yang dilatih pada data re-analisis global adalah pintu masuk alternatif yang sedang berkembang (Bab 10).

- **Pemilihan fitur tetap penting.** Walau DL dapat belajar representasi, fitur meteorologi yang baik (lag, musiman, indeks ENSO) membuat model jauh lebih efisien.
- **Cross-validation harus menghormati waktu.** Bab 5 memperkenalkan *walk-forward validation*, *k-fold* acak tidak berlaku untuk deret waktu.

1.5 Alur Kerja Proyek Machine Learning

Sebelum masuk ke kode, pahami siklus hidup proyek ML. Hampir semua proyek dalam buku ini mengikuti alur berikut:

Masalah → Data → Persiapan → Model → Evaluasi → (Putuskan: cukup / perbaiki)

1. **Definisikan masalah.** Prediksi apa? Regresi (angka) atau klasifikasi (kategori)? Apa yang menjadi target keberhasilan?
2. **Kumpulkan data.** Dari sumber mana? Bagaimana lisensinya? Berapa panjang dan resolusinya? (Bab 6).
3. **Persiapkan data.** Bersihkan nilai hilang, normalisasi, buat fitur, bagi train/val/test dengan benar (Bab 2, 5, 6).
4. **Bangun model.** Mulai dari *baseline* sederhana, lalu tingkatkan (Bab 2-4, 7).
5. **Evaluasi.** Gunakan metrik yang sesuai dengan tujuan operasional (Bab 5).
6. **Putuskan.** Gunakan model jika hasilnya memuaskan. Jika belum, lakukan iterasi: perbaiki data, sesuaikan parameter, atau ganti arsitektur model.
7. **Pantau dan rawat.** Kondisi atmosfer dan sensor terus berubah. Melatih ulang model secara berkala adalah kewajiban dalam siklus hidup model, bukan sekadar bonus.

Konsep *baseline* akan menjadi teman sepanjang buku. Sebelum menantang dengan *deep learning*, Anda harus punya *baseline* sederhana yang bisa Anda bandingkan. Ini menjauhkan Anda dari klaim berlebihan dan membantu menilai apakah DL benar-benar diperlukan.

Catatan tentang iterasi

Gambaran di atas adalah penyederhanaan: dalam praktik, Anda akan bolak-balik antar langkah. Model Anda *underfit*? Kembali ke persiapan data atau tambah kapasitas model. Data ternyata memiliki *outlier* yang merusak pelatihan? Kembali ke langkah 3. Evaluasi menunjukkan model *overfit*? Kembali

ke regularisasi (Bab 5) atau pengumpulan data tambahan. Iterasi bukan tanda kegagalan; ini adalah proses normal dalam setiap proyek ML.

Tentang versi dan dokumentasi

Setiap percobaan, perubahan *hyperparameter*, normalisasi, atau fitur, hendaknya disimpan dan diberi catatan singkat. Cara paling sederhana: gunakan notebook yang berbeda untuk setiap eksperimen, atau *commit* ke Git dengan pesan yang jelas. Tanpa pencatatan, Anda akan sulit menjawab pertanyaan enam bulan kemudian: “mengapa model ini bekerja lebih baik dari yang sebelumnya?” Bab 10 membahas *versioning* model dan pelatihan ulang secara lebih sistematis.

1.6 Menyiapkan Lingkungan Kerja

Buku ini menggunakan **Google Colab** (gratis) dengan **TensorFlow/Keras** [6]. Colab berjalan di *browser* tanpa instalasi lokal dan menyediakan GPU untuk latihan. Ini ideal untuk mahasiswa dan praktisi yang tidak memiliki server sendiri.

Keunggulan Colab untuk buku ini:

- Gratis dan tanpa instalasi (cukup akun Google).
- GPU tersedia untuk latihan jaringan saraf.
- Berbagi notebook semudah membagikan tautan.
- Terintegrasi dengan Google Drive untuk menyimpan data dan hasil.

Untuk memulai, buka colab.research.google.com, buat notebook baru, lalu jalankan:

Kode 1.1 - Memeriksa versi TensorFlow dan ketersediaan GPU di Colab.

```
import tensorflow as tf

print(tf.__version__)
print("GPU tersedia:", tf.config.list_physical_devices("GPU"))
```

Jika kolom “GPU tersedia” kosong, pilih menu *Runtime > Change runtime type > Hardware accelerator > GPU*. Jika TensorFlow belum terpasang, instal dengan `pip install tensorflow`.

Notebook pendamping bab ini, `ch-01-00_fondasi_tensorflow.ipynb` (folder `notebooks/` di repo), berisi langkah verifikasi lingkungan dan pengenalan tensor dengan contoh data cuaca mini. Pastikan versi TensorFlow yang terinstal sesuai dengan versi yang tercantum pada metadata bab, agar hasil dapat direproduksi.

Catatan versi & reproduksibilitas (aturan konsisten sepanjang buku):

- Selalu catat versi library utama (mis. `print(tf.__version__)`).

- Gunakan nilai *seed* tetap (`np.random.seed`, `tf.random.set_seed`) agar hasil dapat direproduksi.
- Simpan data dengan hash/versi agar tidak berubah diam-diam.

Instalasi pustaka tambahan

Untuk buku ini, selain TensorFlow, Anda akan memerlukan beberapa pustaka standar:

```
!pip install numpy pandas matplotlib scikit-learn xarray netCDF4
```

- **NumPy**, operasi array numerik (tidak perlu diinstal terpisah di Colab, sudah tersedia).
- **Pandas**, pembacaan CSV dan tabular.
- **Matplotlib**, visualisasi.
- **scikit-learn**, metrik dan utilitas ML klasik (*baseline*).
- **xarray + netCDF4**, membaca data NetCDF untuk ERA5 dan produk grid (Bab 6).

Jalankan baris di atas satu kali per sesi Colab, atau simpan di sel pertama notebook Anda.

Troubleshooting lingkungan Colab (sesi terputus, memori habis, versi library berubah) dicantumkan dalam Lampiran A, tepat sebelum Anda melatih model besar di Bab 7-10.

1.7 Tensor Pertama dengan Contoh Data Cuaca Mini

Semua data yang masuk ke jaringan saraf direpresentasikan sebagai **tensor**, generalisasi matriks ke dimensi berapa pun. Anda bisa membayangkan tensor sebagai “kotak angka” dengan beberapa sumbu. Dengan contoh data cuaca:

Tabel 1.2: Contoh data cuaca mini dan bentuk tensornya.

Contoh	Bentuk	Dimensi (rank)
Suhu satu pengamatan, 26.5	skalar (tensor 0D)	0
Suhu min & max satu hari, [26.5, 31.0]	vektor (tensor 1D)	1
Suhu min & max 3 hari, [[26.5, 31.0], [26.8, 30.5], [27.2, 32.1]]	matriks (tensor 2D)	2
Suhu min & max di 3 stasiun selama 3 hari	tensor 3D	3

Tabel 1.2 merangkum contoh-contoh yang akan kita gunakan di notebook. Dalam TensorFlow, tensor dibuat dengan `tf.constant` atau `tf.Variable`:

Kode 1.2 - Membuat tensor suhu harian satu dimensi.

```
import tensorflow as tf

suhu_hari = tf.constant([26.5, 26.8, 27.2])      # 1D: 3 pengamatan
print(suhu_hari.shape)                          # (3,)
```

Bentuk (*shape*) tensor inilah yang nantinya menentukan bentuk masukan (*input shape*) arsitektur model di Bab 2 dan Bab 7. Pembaca tidak perlu menguasai aljabar tensor secara mendalam, yang penting adalah memahami (a) urutan dimensi, dan (b) bahwa data waktu akan disusun menjadi langkah-langkah waktu (*timesteps*) saat masuk ke model.

Mini-challenge: Prediksi *Persistence* vs. Rata-rata Klimatologis

Sebelum kita bicara tentang model yang *sophisticated*, mari kita lihat dua *baseline* yang akan menjadi “lawan tanding” *deep learning* di sepanjang buku. Keduanya dipakai luas dalam verifikasi prediksi cuaca sebagai rujukan keterampilan model [12]:

- ***Persistence*** (prediksi-beku): $\hat{y}_{t+1} = y_t$. Prediksi besok sama dengan pengamatan terakhir yang kita punya. Secara intuitif: “asumsinya tidak berubah”. Untuk data yang berubah pelan (pasang surut, suhu harian), *persistence* sering kali sudah cukup baik; untuk data yang berfluktuasi cepat, *persistence* kalah. Istilah *persistence* dipakai di seluruh literatur verifikasi prediksi internasional (WMO WWRP/WGNE, [12]).
- **Rata-rata klimatologis**: $\hat{y}_{t+1} = \bar{y}_{\text{bulan, stasiun}}$, prediksi besok = rata-rata historis untuk hari yang sama di bulan dan lokasi tersebut. Cocok untuk pola musiman yang kuat; gagal saat rezim menyimpang dari klimatologis (mis. El Niño kuat).

Mari kita uji pada data sintetis sederhana (variasi harian menyerupai suhu):

Kode 1.3 - Mini-challenge: membandingkan *persistence* dan klimatologis.

```
import numpy as np
import matplotlib.pyplot as plt

# Simulasi 60 hari suhu dengan pola sinus + sedikit noise
np.random.seed(42)
hari = np.arange(60)
suhu = 27 + 2 * np.sin(2 * np.pi * hari / 30) + np.random.normal(0, 0.5, 60)

# Baseline persistence: besok = hari ini
pred_persistence = suhu[:-1]
aktual = suhu[1:]

# Baseline klimatologis: rata-rata untuk "hari ke-N dalam siklus"
```

```

siklus = 30
klimatologi = np.array([suhu[i::siklus].mean() for i in range(siklus)])
pred_klimatologi = klimatologi[(np.arange(1, 60)) % siklus]

mae_persistence = np.mean(np.abs(aktual - pred_persistence))
mae_klimatologi = np.mean(np.abs(aktual - pred_klimatologi))

print(f"MAE persistence      : {mae_persistence:.3f} °C")
print(f"MAE klimatologis     : {mae_klimatologi:.3f} °C")

plt.figure(figsize=(10, 4))
plt.plot(aktual, label="Aktual", marker="o")
plt.plot(pred_persistence, label="Persistence", linestyle="--")
plt.plot(pred_klimatologi, label="Klimatologis", linestyle=":")
plt.xlabel("Hari"); plt.ylabel("Suhu (°C)")
plt.legend(); plt.grid(True)
plt.title("Dua baseline pada data sintetis")
plt.show()

```

Pada data seperti ini, **klimatologis biasanya mengalahkan *persistence*** karena kita sengaja membuat pola yang periodik. Untuk fenomena dengan *persistence* tinggi (pasang surut, suhu harian), *persistence* sering menang. Untuk fenomena periodik (curah hujan musiman), klimatologis sering lebih baik. *Deep learning* baru layak jika bisa mengalahkan keduanya secara konsisten; kita akan kembali ke prinsip ini di setiap studi kasus (Bab 8-9).

1.8 Latihan Mini: Mengenali Jenis Masalah

Sebelum Bab 2, mari latih naluri memetakan masalah ke jenis model. Ingat pedoman dari Bagian 1.5: **regresi** menjawab “berapa?” dengan angka (mis. tinggi pasang besok), sedangkan **klasifikasi** menjawab “yang mana?” dengan kategori (mis. hujan lebat atau tidak, level bahaya apa). Untuk masing-masing pertanyaan berikut, tentukan (a) regresi atau klasifikasi, dan (b) apakah *deep learning* layak dicoba (asumsikan data tersedia cukup):

1. Prediksi suhu minimum besok di Pontianak.
2. Deteksi apakah hari ini akan hujan deras (>50 mm/24 jam) ya atau tidak.
3. Menentukan level siaga banjir rob: waspada / siaga / awas.
4. Prediksi jumlah hari hujan pada bulan depannya (satu angka per bulan).

Jawaban singkat:

1. **Regresi** (nilai kontinu: suhu minimum). DL bisa; mulai dari *baseline*.
2. **Klasifikasi biner** (dua kelas: hujan deras / tidak). DL bisa; perhatikan data tidak seimbang (jarang hujan deras), Bab 3, 5.
3. **Klasifikasi multi-kelas** (tiga level). DL bisa; pastikan metrik sesuai kejadian ekstrem, Bab 3, 5.

4. **Regresi** (satu angka), tapi mungkin lebih baik menggunakan rata-rata klimatologis sebagai *baseline* dulu. DL tidak selalu jawaban, Bagian 1.4.

Latihan semacam ini (yang muncul di setiap bab) melatih Anda berpikir seperti praktisi: definisikan masalah dulu, baru pilih alat.

1.9 Berapa Banyak Matematika yang Anda Butuhkan?

Banyak calon pembaca khawatir buku ini berisi matematika berat. Kabar baiknya: untuk **menggunakan** DL secara bertanggung jawab, Anda cukup menguasai tiga hal:

1. **Aljabar linear dasar**, vektor dan matriks (Perkalian matriks adalah inti neuron). Bab ini sudah mengenalkan tensor; kita hanya akan memakai sedikit notasi di Bab 2-4.
2. **Kalkulus dasar**, konsep turunan untuk memahami *gradient descent* (Bab 4). Anda tidak perlu menurunkan rumus, cukup paham intuisi kemiringan.
3. **Statistika deskriptif**, rata-rata, varians, korelasi, dan sedikit probabilitas (untuk klasifikasi di Bab 3).

Jika Anda sudah lama tidak berurusan dengan matematika, jangan khawatir. Buku ini selalu mendekati rumus dengan **intuisi + kode**, bukan derivasi formal. Setiap rumus yang muncul dijelaskan dengan bahasa sehari-hari dan contoh konkret meteorologi, sehingga Anda tetap bisa mengikuti.

Kapan harus berhenti dan belajar lebih dalam? Jika Anda berencana meneliti atau mengembangkan arsitektur baru, Anda perlu matematika lebih dalam; untuk itu kami sarankan [1]. Tetapi untuk penggunaan praktis (membangun model, mengevaluasi, menerapkan), bekal di atas cukup.

1.10 Apa Arti Buku Ini bagi Anda

Buku ini menyebut dirinya **materi pengenalan**, bukan hasil riset baru. Bagi Anda, artinya tiga hal yang berkaitan langsung dengan cara Anda memakai buku ini:

1. **Anda mendapat kurasi yang rapi.** Daripada menyaring sendiri banyak sumber primer, Anda membaca literatur dan praktik yang sudah mapan, disusun ulang dalam satu narasi dengan contoh data meteorologi Indonesia.
2. **Anda tidak akan disesatkan angka.** Tidak ada klaim performa “metode baru yang lebih baik”; setiap hasil dibandingkan dengan *baseline* dan disajikan apa adanya, sebagai bahan belajar, bukan klaim untuk keputusan operasional.
3. **Ukuran keberhasilannya adalah Anda.** Buku ini berhasil jika Anda mampu membangun dan mengevaluasi model *deep learning* sendiri untuk data Anda.

Jadi, saat Anda membaca “studi kasus” di Bab 8-9, perlakukan sebagai latihan *end-to-end* yang dapat diulang, bukan sebagai makalah penelitian. Ini adalah sikap yang juga Anda pegang sebagai praktisi: selalu tanya “apakah ini mengalahkan *baseline*?”

1.11 Ekosistem dan Sumber Belajar Lanjutan

Anda mungkin bertanya: “bukankah banyak buku *deep learning* yang sudah ada?” Buku ini menempati celah yang sudah dijelaskan di Bagian 1.0: ditulis dalam bahasa Indonesia dengan konteks meteorologi lokal. Untuk **penggunaan umum DL**, referensi utama komunitas adalah *Deep Learning* dari Goodfellow, Bengio, dan Courville [1], sangat mendalam tetapi cenderung teoretis. Untuk **praktik dengan Keras/TensorFlow**, buku dari François Chollet dan tutorial situs resmi TensorFlow sangat membantu.

Di samping buku ini, berikut ekosistem yang bermanfaat:

- **PyTorch**, alternatif populer di riset; konsep yang Anda pelajari di sini mudah dipindahkan.
- **Kaggle / GitHub**, sumber dataset dan contoh notebook untuk latihan mandiri.
- **Pandas & xarray**, penanganan data tabular dan data NetCDF (rincian & instalasi di Bagian 1.6).

Pertanyaan yang Sering Muncul (FAQ)

Apakah saya harus bisa Python dulu? Tidak wajib mahir, tetapi disarankan pernah menulis Python dasar (variabel, loop, fungsi). Kode dalam buku selalu diberikan lengkap, dan setiap baris penting dijelaskan.

Apakah butuh paham meteorologi untuk Bab 2-5? Tidak. Bab 2-5 berfokus pada konsep DL dengan contoh meteorologi sebagai ilustrasi. Bab 6-9 akan memerlukan pemahaman konteks data, tetapi tetap dijelaskan.

Apakah GPU wajib? Tidak. Notebook dapat berjalan di CPU, hanya lebih lambat. Untuk studi kasus di Bab 8-9, GPU mempermudah dan Colab menyediakannya gratis.

1.12 Glosarium Mini Bab 1

Daftar istilah singkat yang muncul di bab ini. Glosarium lengkap buku ada di *front matter*; di sini hanya yang esensial untuk bab ini.

Istilah	Indonesia	Inggris	Penjelasan singkat
AI	Kecerdasan buatan	<i>Artificial Intelligence</i>	Bidang luas: mesin yang meniru kemampuan kognitif.
ML	Pembelajaran mesin	<i>Machine Learning</i>	Sub-bidang AI: belajar pola dari data.
DL	Pembelajaran mendalam	<i>Deep Learning</i>	ML dengan jaringan saraf berlapis.
NN	Jaringan saraf	<i>Neural Network</i>	Model matematis berlapis unit sederhana.
Baseline	Tolok ukur	<i>Baseline</i>	Model sederhana acuan
Persistence	-	<i>Persistence</i>	(<i>persistence</i> , klimatologis). $\hat{y}_{t+1} = y_t$; besok = hari ini.
Tensor	-	<i>Tensor</i>	“Kotak angka” generalisasi matriks ke banyak dimensi.
GPU	-	<i>Graphics Processing Unit</i>	Kartu grafis untuk komputasi paralel; mempercepat DL.
Notebook	Buku catatan	<i>Notebook</i>	Dokumen interaktif kode + narasi (Colab, Jupyter).
Overhype	Berlebih-lebihan	<i>Overhype</i>	Klaim model lebih bagus dari kenyataan; kita hindari.
Seed	Benih	<i>Seed</i>	Nilai awal generator acak; kunci reproduksibilitas.

Istilah	Indonesia	Inggris	Penjelasan singkat
ENSO	-	<i>El Niño-Southern Oscillation</i>	Mode variabilitas iklim Pasifik yang memengaruhi hujan Indonesia.
MJO	-	<i>Madden-Julian Oscillation</i>	Osilasi intramusiman tropis (30-60 hari) [10].
Rob	-	<i>Rob</i>	Banjir pesisir akibat pasang tinggi (penting di Semarang, Pontianak).

1.13 Koneksi ke Bab Berikutnya

Untuk membantu Anda memutuskan ke mana melompat setelah bab ini, berikut peta singkat.

Jika Anda ingin langsung menulis kode regresi pertama: lanjut ke Bab 2. Anda akan membangun perceptron dan MLP pada data pasang surut mini, lalu membandingkannya dengan *baseline persistence*.

Jika Anda ingin memahami cara kerja *backpropagation* sebelum kode: Bab 4 membahas *gradient descent*, aturan rantai, dan perannya dalam melatih jaringan, sangat bermanfaat sebelum masuk ke model sekuensial.

Jika Anda sudah familiar dengan ML klasik (regresi, pohon keputusan): Bab 1 cukup; Anda bisa langsung ke Bab 3 untuk klasifikasi dan Bab 5 untuk metrik domain.

Jika Anda ingin konteks data dulu: Bab 6 (data terbuka/ERA5/pasang surut) menjelaskan sumber, lisensi, dan kualitas, bekal penting untuk Bab 8-9.

Ringkasan

- *Deep learning* adalah cabang *machine learning* berbasis jaringan saraf berlapis; relevan untuk meteorologi karena data besar, komputasi murah, dan tooling matang.
- Fokus buku: prediksi deret waktu dan klasifikasi kejadian untuk data Indonesia; aplikasi lain (*nowcasting*, *downscaling*, model generatif) diarahkan ke Bab 10.
- *Deep learning* layak digunakan jika mengalahkan *baseline* yang sederhana; jangan pernah memulai tanpa pembandingan.

- Lingkungan kerja: Google Colab + TensorFlow/Keras; semua data dibawa sebagai tensor.
- Alur kerja proyek ML (masalah → data → persiapan → model → evaluasi) menjadi pola acuan semua bab.
- Matematika yang dibutuhkan terbatas pada aljabar linear, kalkulus, dan statistika dasar, cukup untuk penggunaan praktis.
- Buku ini “bukan riset baru”; buku ini merupakan hasil kurasi literatur untuk konteks meteorologi Indonesia, dengan referensi lanjutan untuk pendalaman.

References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
2. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Harlow, UK: Pearson, 2021. ISBN: 978-0134610993.
3. T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997. ISBN: 978-0070428072.
4. Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436-444, May 2015, doi: 10.1038/nature14539.
5. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386-408, 1958, doi: 10.1037/h0042519.
6. M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016. [Online]. Available: <https://arxiv.org/abs/1603.04467> (diakses: 20 September 2026)
7. M. Reichstein et al., “Deep learning and process understanding for data-driven Earth system science,” *Nature*, vol. 566, no. 7743, pp. 195-204, Feb. 2019, doi: 10.1038/s41586-019-0912-1.
8. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533-536, Oct. 1986, doi: 10.1038/323533a0.
9. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, Dec. 2012, pp. 1097-1105; reissued in *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, 2017, doi: 10.1145/3065386.
10. M. C. Wheeler and H. H. Hendon, “An all-season real-time multivariate MJO index: Development of an index for monitoring and prediction,” *Monthly Weather Review*, vol. 132, no. 8, pp. 1917-1932, Aug. 2004, doi: 10.1175/1520-0493(2004)132<1917:AARMMI>2.0.CO;2.
11. S. Lestari, A. King, C. Vincent, D. Karoly, and A. Protat, “Seasonal dependence of rainfall extremes in and around Jakarta, Indonesia,” *Weather and Climate Extremes*, vol. 24, p. 100202, Jun. 2019, doi: 10.1016/j.wace.2019.100202.
12. I. T. Jolliffe and D. B. Stephenson, eds., *Forecast Verification: A Prac-*

titioner's Guide in Atmospheric Science, 2nd ed. Chichester, UK: Wiley, 2011, doi: 10.1002/9781119960003.

— title: “Regresi: Perceptron dan Jaringan Saraf untuk Prediksi Besaran” description: “Bab 2 - membangun model regresi pertama untuk prediksi besaran meteorologi: anatomi neuron (bobot, bias, fungsi aktivasi), regresi linear sebagai kasus khusus, kebutuhan non-linearitas (ReLU), mini-kasus pasang surut, perbandingan MAE vs MSE, dan alasan split berbasis waktu.” pubDate: 2026-09-01 categories: [“*Deep Learning*”, “*Meteorologi*”] tags: [“regresi”, “*neural network*”, “perceptron”, “fungsi aktivasi”, “ReLU”, “*time series*”] version: “1.1.5” bookDOI: “10.5281/zenodo.0000000” status: published chapter: 2 book: “Pengantar *Deep Learning* untuk Meteorologi” —

Prasyarat: Bab 1 (tensor, konsep ML/DL). Kode memakai TensorFlow/Keras di Google Colab (Bab 1: setup lingkungan). Jika Anda sudah paham regresi linear dan pernah menulis *Dense layer*, Anda boleh lompat cepat ke Bagian 2.5, tetapi pastikan Anda tahu istilah bobot, bias, dan fungsi aktivasi.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Membangun** model regresi *neural network* (*perceptron*/MLP) untuk prediksi besaran meteorologi dengan TensorFlow/Keras.
2. **Menjelaskan** peran bobot, bias, dan fungsi aktivasi (termasuk ReLU) serta kapan non-linearitas diperlukan.
3. **Menerapkan** mini-kasus pasang surut: *windowing*, *baseline persistence*, dan perbandingan MAE antara model *neural* vs *baseline*.
4. **Memilih** antara MAE dan MSE berdasarkan sifat data dan tujuan, serta membagi data deret waktu secara kronologis yang mencegah *leakage*.

2.1 Prediksi Besaran Sebagai Masalah Regresi

Banyak pertanyaan meteorologi yang jawabannya berupa **angka**:

- Berapa suhu maksimum besok di Pontianak?
- Berapa milimeter hujan pada hari Jumat mendatang?
- Berapa tinggi pasang surut pada pukul 18.00 nanti?
- Berapa kecepatan angin maksimum saat badai lewat?

Masalah seperti ini disebut **regresi**, memprediksi nilai kontinu dari pola di data. Kata “kontinu” penting: keluarannya adalah bilangan nyata (mis. 26.5°C , 34 mm), bukan kategori. Ini kontras dengan **klasifikasi** (Bab 3) yang memprediksi label, misalnya “hujan” atau “tidak hujan”.

Mengapa regresi sering menjadi tempat pertama belajar *neural network*? Karena cara kerja sel sarafnya sama persis dengan klasifikasi; perbedaannya hanya di bagian ujung: lapisan keluaran dan fungsi *loss* yang dipakai. Jika Anda menguasai regresi, setengah jalan menuju klasifikasi sudah terlewati.

Ada jenis pertanyaan yang terlihat seperti angka tetapi sebenarnya bukan regresi. Test sederhana: bertanya pada diri sendiri, “apakah setiap bilangan nyata masih bermakna sebagai jawaban?” Untuk “berapa hari hujan bulan depan?” jawabannya adalah bilangan bulat (0, 1, 2, ...); -1 hari tidak bermakna. Model regresi dilatih untuk memprediksi nilai kontinu, sehingga tanpa restriksi tambahan ia dapat mengeluarkan jawaban yang tidak bermakna seperti (-3 hari atau 3,7 hari). Karena itu, pertanyaan seperti ini lebih baik diperlakukan sebagai masalah *count* atau dibentuk ulang sebagai deret waktu bulanan (*windowing*: Bab 7).

Contoh yang jelas bukan regresi: “apakah akan berpotensi banjir?”. Pertanyaan ini tampak berhubungan dengan angka, seperti tinggi air, tetapi jawaban yang dimaksud adalah **kategori**: ya atau tidak. Regresi memprediksi nilai (misal tinggi air 2.3 m); klasifikasi memprediksi label (banjir atau tidak). Di klasifikasi, galat diukur bukan dengan jarak angka, tetapi apakah label tepat atau keliru; dan fungsi *loss* yang dipakai (*cross-entropy*) berbeda dari *loss* regresi (Bab 3). Bab 1 Bagian 1.8 sudah melatih Anda membedakan keduanya; di Bab 2 kita fokus pada regresi sungguhan.

Contoh target regresi dalam meteorologi

Tabel 2.1: Contoh target regresi meteorologi, satuan, dan sifat datanya.

Target	Satuan	Sifat data
Suhu udara (maks/min)	°C	Relatif mulus, distribusi simetris
Curah hujan	mm	Ekor kanan, banyak nol, jarang sangat besar
Tinggi pasang surut	m	Periodik, deterministik kuat
Kecepatan angin	m/s	Berubah, tergantung musim/lokal

Tabel 2.1 menunjukkan bahwa “regresi meteo” bukan satu jenis data yang sama: hujan berbeda jauh dari suhu. Tiap jenis data memiliki sifat yang berbeda, perhatikan kolom sifat data pada tabel di atas. Sifat yang berbeda membutuhkan pilihan pengaturan yang berbeda. Fungsi *loss*, metrik, dan arsitektur model tergantung pada sifat tersebut (*loss* dan metrik dibahas di Bagian 2.6; arsitektur di Bab 4 saat *tuning* dibahas).

2.2 Anatomi Neuron: Bobot, Bias, dan Fungsi Aktivasi

Neuron buatan (*artificial neuron*) adalah unit dasar jaringan saraf. Ia menerima beberapa masukan **x**, mengalikan tiap masukan dengan **bobot w**, menjum-

lahkannya, lalu menambahkan **bias** b :

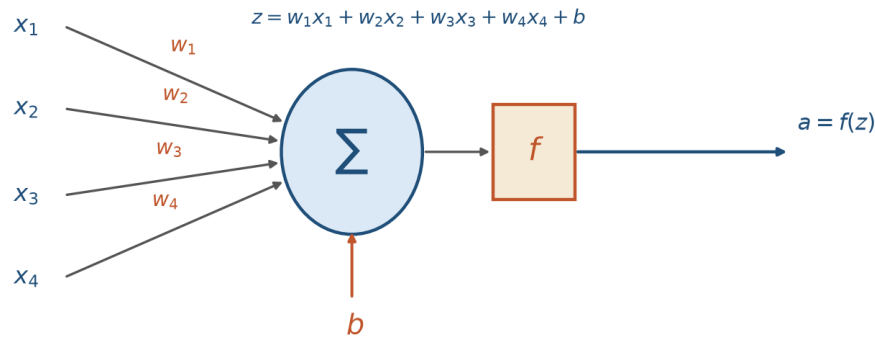
$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad (2.1)$$

Hasilnya “diaktifkan” oleh **fungsi aktivasi** f , menghasilkan keluaran:

$$a = f(z) \quad (2.2)$$

Bobot w mencerminkan **seberapa penting** tiap masukan; bias b adalah “ambang” yang memungkinkan neuron aktif bahkan ketika semua masukan nol. Keduanya adalah **parameter**: istilah untuk seluruh angka dalam model yang nilainya dipelajari dari data selama pelatihan, bukan ditetapkan manusia. Saat awal, nilainya acak kecil; melalui ribuan contoh, model menyesuaikan bobot agar prediksinya semakin akurat.

Contoh paling sederhana adalah *perceptron* yang diperkenalkan Rosenblatt pada 1958 [1]: neuron dengan fungsi aktivasi berbentuk aturan, misalnya $a = 1$ jika $z > 0$, dan $a = 0$ sebaliknya. Perceptron penting karena menunjukkan bahwa mesin bisa belajar, tetapi terbatas pada masalah yang *linearly separable*. Dalam bab ini kita menggunakan versi modern: neuron **tanpa aktivasi di lapisan keluaran** untuk regresi (nilai bebas, bukan 0/1).



Gambar 2: Gambar 2.1 - Struktur neuron buatan

Gambar 2.1: Struktur neuron buatan.

Persamaan (2.1) dan (2.2) diilustrasikan pada Gambar 2.1: setiap panah masukan membawa satu komponen x_i yang dikalikan w_i ; semua hasil dijumlahkan bersama bias menjadi z (nilai sebelum aktivasi); lalu f menghasilkan keluaran a .

Contoh numerik sederhana

Misalkan kita memprediksi suhu minimum besok (y) dari dua variabel: suhu hari ini ($x = 26$) dan kelembapan ($x = 90\%$). Jika bobot $w = 0.5$, $w = -0.05$, dan bias $b = 10$, maka:

$$z = (0.5 \times 26) + (-0.05 \times 90) + 10 = 13 - 4.5 + 10 = 18.5$$

Tanpa fungsi aktivasi (identitas), prediksi $\hat{y} = 18.5^\circ\text{C}$. Pembaca bisa melihat intuisi: suhu hari ini menaikkan prediksi (bobot positif), kelembapan tinggi menurunkannya (bobot negatif). Tugas pelatihan justru menemukan w dan b yang “paling masuk akal” ini dari data, bukan menentukannya manual.

2.3 Satu Neuron Linear = Regresi Linear

Sebuah neuron **tanpa fungsi aktivasi** (identitas) untuk satu masukan:

$$\hat{y} = wx + b \quad (2.3)$$

Persamaan (2.3) identik dengan **regresi linear** yang biasa Anda pelajari di statistika. Bedanya hanya di jalur penemuan parameter:

- Statistika klasik: w dan b dihitung dengan rumus kuadrat terkecil (*closed-form*).
- *Neural network*: w dan b ditemukan lewat proses berulang (*gradient descent*, Bab 4).

Hasil akhirnya sama dalam kondisi dasar: fungsi *loss* MSE, tanpa regularisasi, dan pelatihan yang selesai dengan baik. Jika Anda ganti *loss* ke MAE, menambahkan regularisasi atau menghentikan pelatihan lebih awal (*early stopping*), hasilnya akan berbeda dari solusi *closed-form*.

Catatan ini penting: “sama” berlaku hanya untuk kasus dasar di Bagian 2.3. Karena itu, satu neuron linear sebaiknya dianggap sebagai **baseline**. Prinsip di Bab 1 tetap berlaku: mulai dari model paling sederhana, ukur kinerjanya, lalu tingkatkan jika perlu.

Mengapa kita tetap mempelajari regresi linear di buku *deep learning*? Tiga alasan:

1. **Interpretasi**: w memiliki arti langsung (“kenaikan satu unit x menaikkan y sebesar w ”). Ini sangat berharga di meteorologi, di mana rekan kerja bertanya “kenapa model bilang begini?”.
2. **Baseline wajib**: hampir semua Bab 8-9 membandingkan LSTM/MLP dengan *baseline* linear. Tanpa memahami *baseline*, kita tidak bisa menilai “apakah DL benar-benar menambah nilai?”.

3. **Blok bangunan:** regresi linear adalah neuron tunggal tanpa fungsi aktivasi; jaringan saraf adalah kumpulan neuron yang dihubungkan. Semua konsep (bobot, bias, loss) muncul di sini.

2.4 Kebutuhan Non-linearitas: Perkenalan ReLU

Data meteorologi jarang linear sempurna. Hubungan antara variabel seperti kelembapan, suhu, dan curah hujan tidak bisa diwakili hanya oleh garis lurus: hujan tidak meningkat terus-menerus seiring kelembapan; ada ambang, jenuh, dan interaksi. Contoh sederhana: hubungan suhu dan laju penguapan mungkin kurva, bukan garis.

Masalahnya: jika kita menyusun beberapa neuron **linear** berlapis, seluruh jaringan tetap linear, karena jumlah fungsi linear adalah fungsi linear. Komposisi `linear(linear(x))` tidak menghasilkan kemampuan baru: seberapa dalam pun, model tetap “garis lurus” dalam ruang berdimensi banyak.

Agar model mampu menangkap pola non-linear, setiap lapisan menyisipkan **fungsi aktivasi non-linear**. Fungsi yang paling umum sekarang adalah **ReLU** (*rectified linear unit*):

$$\text{ReLU}(x) = \max(0, x) \quad (2.4)$$

ReLU mengeluarkan nilai masukan jika positif, dan nol jika negatif (persamaan (2.4)). Sederhana, murah dihitung (satu perbandingan), dan menjadi komponen dasar banyak jaringan modern [2]. Referensi ini merujuk pada AlexNet (2012), jaringan yang mempopulerkan ReLU dalam kompetisi ImageNet. ReLU juga menghindari beberapa masalah *gradien* yang dimiliki sigmoid/tanh (Bab 4 membahas topik ini lebih dalam).

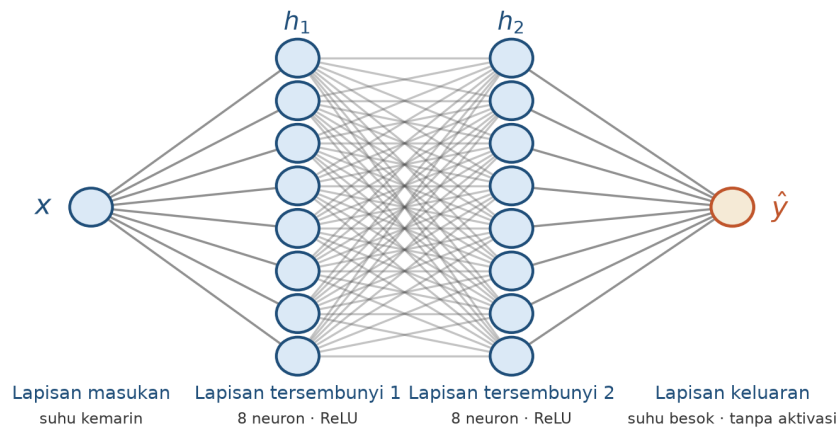
Dari neuron ke MLP

MLP (*multi-layer perceptron*) adalah jaringan dengan satu lapisan masukan, satu atau lebih lapisan tersembunyi (masing-masing dengan fungsi aktivasi non-linear), dan lapisan keluaran. Untuk regresi, lapisan keluaran **tidak memakai aktivasi**, kita ingin nilai bebas, bukan dibatasi ke rentang tertentu. Contoh arsitektur MLP untuk memprediksi suhu besok dari suhu kemarin diilustrasikan pada Gambar 2.2:

Gambar 2.2: Arsitektur MLP contoh.

Cara membacanya dari kiri ke kanan: masukan $\mathbf{x}$ (suhu kemarin), dua kolom tengah adalah lapisan tersembunyi masing-masing 8 neuron dengan ReLU, dan satu neuron di ujung kanan adalah keluaran $\hat{y}$ (suhu besok) tanpa aktivasi. Sambungan antar lapisan menggambarkan sambungan *dense* (*fully connected*).

Kode 2.1 - Definisi arsitektur MLP regresi dengan Keras.



Gambar 3: Gambar 2.2 - Arsitektur MLP contoh

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(1,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1),
])
model.summary()
```

Kode 2.1 terlihat pendek, tetapi di dalamnya ada tiga jenis lapisan dengan tugas berbeda. Mari kita bedah satu per satu, karena pembagian ini berlaku untuk hampir semua jaringan di bab-bab selanjutnya.

Lapisan masukan (*input layer*). Tugasnya hanya menerima data, bukan menghitung: lapisan ini tidak punya bobot dan tidak belajar apa pun. Ia hanya mendefinisikan bentuk masukan lewat `input_shape=(1,)`, satu fitur per sampel, yaitu suhu kemarin. Di *library* Keras lapisan ini implisit: Anda tidak menuliskannya di `Sequential`, karena Keras membuatnya otomatis dari argumen tersebut. Jika fitur bertambah (misalnya suhu, kelembapan, dan tekanan udara), cukup diubah menjadi `input_shape=(3,)`.

Lapisan tersembunyi (*hidden layer*). Di sinilah komputasi terjadi: dua baris `Dense(8, activation="relu")`. `Dense` berarti setiap neuron terhubung ke seluruh keluaran lapisan sebelumnya (*fully connected*), dan angka 8 adalah jumlah neuron. Setiap neuron adalah neuron di Bagian 2.2 (persamaan (2.1)-(2.2)): jumlah berbobot masukan ditambah bias, lalu lewat fungsi aktivasi dalam hal ini ReLU, yang mengubah hasil negatif menjadi nol. ReLU inilah yang mencegah dua lapisan berturut-turut jatuh kembali menjadi satu fungsi linear (Bagian 2.4). Karena masukannya hanya satu, tiap neuron di lapisan per-

tama bekerja seperti sakelar pada ambang tertentu (efek ReLU); lapisan kedua menggabungkan sakelar-sakelar itu menjadi pola yang lebih fleksibel. Jumlah 8 dan banyaknya lapisan (2) bukan angka ajaib, melainkan keputusan desain; tuningnya dibahas di Bab 4.

Lapisan keluaran (*output layer*). Baris terakhir, `Dense(1)`, ditulis tanpa argumen `activation`. Tanpa aktivasi berarti identitas: keluaran neuron terakhir langsung menjadi prediksi $\hat{y}$. Ini disengaja untuk regresi, karena prediksi suhu boleh bernilai bebas (misalnya 22.7°C atau -1.5°C); ReLU akan membuang semua nilai negatif, sedangkan sigmoid/tanh menjepit keluaran ke rentang tetap. Satu neuron berarti satu angka keluaran: suhu besok.

Alur seluruh jaringan bisa ditulis satu baris (persamaan (2.5)); tiap $\mathbf{h}$ adalah vektor 8 angka):

$$x \rightarrow h_1 = \text{ReLU}(W_1x + b_1) \rightarrow h_2 = \text{ReLU}(W_2h_1 + b_2) \rightarrow \hat{y} = W_3h_2 + b_3 \quad (2.5)$$

Karena tiap lapisan punya parameter yang dipelajari, `model.summary()` menghitungnya secara eksplisit:

Layer (type)	Output Shape	Param #	
dense (Dense)	(None, 8)	16	# W (1×8) + b (8)
dense_1 (Dense)	(None, 8)	72	# W (8×8) + b (8)
dense_2 (Dense)	(None, 1)	9	# W (8×1) + b (1)

Cara membacanya sederhana. Kolom Param # menghitung berapa banyak parameter yang harus dipelajari tiap lapisan: lapisan tersembunyi pertama butuh 16 (8 bobot + 8 bias), lapisan kedua 72 (64 bobot + 8 bias), dan lapisan keluaran 9 (8 bobot + 1 bias) = total 97. Bukan kita yang menentukan angka 97 itu; pelatihan yang menemukannya dari data lewat *gradient descent* (Bab 4). Lalu, apa arti `None` di kolom Output Shape? Itu adalah jumlah data yang diproses sekaligus dalam satu kelompok (*batch*); angkanya baru ditentukan saat pelatihan, jadi untuk sekarang pustaka Keras menuliskannya kosong. Versi Keras yang lebih baru kadang juga menampilkan satu baris `InputLayer` bernilai 0 di bagian atas, itulah lapisan masukan implisit tadi, yang memang tidak punya parameter untuk dipelajari.

Kesimpulan arsitekturnya: 1 lapisan masukan + 2 lapisan tersembunyi + 1 lapisan keluaran, bukan $1 + 1 + 1$. Tiga baris `Dense` di Kode 2.1 adalah dua lapisan tersembunyi plus satu lapisan keluaran; lapisan masukan tak tampak karena ia hanya bentuk data, bukan baris kode. Resep yang sama dipakai lagi di Kode 2.3, hanya `input_shape=(2,)` karena fiturnya dua: tinggi pasang surut dua hari dan satu hari sebelumnya.

Berapa banyak lapisan yang diperlukan?

Aturan praktis soal “berapa banyak lapisan”: mulailah dari yang kecil, tambah kompleksitas hanya bila diperlukan. Untuk regresi deret waktu, stasiun dengan

data jam-jaman (ratusan ribu hingga jutaan baris atau data harian seratus tahun pun hanya sekitar 36 ribu baris), 1-3 lapisan tersembunyi umumnya sudah cukup. Model yang terlalu besar akan *overfit* (dijelaskan pada Bab 5) menghafal data latih dan gagal di data baru.

ReLU dan variannya

ReLU sederhana dan efektif, tetapi punya satu kelemahan: untuk masukan negatif, keluarannya persis nol dan gradiennya juga nol. Neuron yang “mati” (selalu memberi nol) tidak ikut belajar lagi; fenomena ini disebut *dying ReLU*. Dalam praktik ringan (regresi sederhana, jaringan kecil) masalah ini jarang berpengaruh besar, tetapi di jaringan yang dalam (banyak lapisannya) bisa muncul masalah fatal.

Beberapa varian yang sering dipakai sebagai pengganti:

- **Leaky ReLU**: $\max(0.01x, x)$ - memberi kemiringan kecil untuk nilai negatif sehingga neuron tidak pernah sepenuhnya mati.
- **ELU**: versi mulus dengan perilaku asimtotik untuk nilai sangat negatif.
- **tanh** dan **sigmoid**: fungsi *S*-kurva yang lebih tua; relevan untuk klasifikasi (Bab 3) dan beberapa kasus lain.

Untuk Bab 2-9, ReLU (atau Leaky ReLU) adalah pilihan *default* yang aman untuk lapisan tersembunyi. Anda tidak perlu menghafal semua varian sekarang; yang penting memahami *mengapa* non-linearitas dibutuhkan dan bahwa ada beberapa pilihan dengan *trade-off*.

2.5 Memprediksi Tinggi Pasang Surut Sederhana

Pasang surut bersifat periodik dan di sebagian besar perairan Indonesia bertipe semi-diurnal atau diurnal campuran [3] sehingga cocok untuk regresi sederhana: ada pola yang bisa dipelajari, cukup deterministik.

Pendekatan paling dasar: memprediksi tinggi air **besok** berdasarkan tinggi air hari ini dan kemarin (dua fitur). Ini contoh *autoregressive*: target (besok) dijelaskan oleh nilai-nilai sebelumnya. Data nyata pasang surut akan dibahas penuh di Bab 8; di sini kita mengenalkan alurnya saja:

```
input: [tinggi(t-2), tinggi(t-1)] → Dense+ReLU → output: tinggi(t)
```

Langkah 1 - Bentuk data (*windowing*)

Model neural network tidak menerima deret waktu sebagai satu blok; ia menerima **pasangan fitur-target**, seperti di semua contoh sebelumnya. Pertanyaan desain pertama: apa yang jadi fitur dan apa yang jadi target? Untuk prediksi tinggi besok, fitur yang paling natural adalah **nilai beberapa langkah sebelumnya**. Kami menyebut bentuk data ini *windowing*.

Lihat dulu datanya (nilai ilustratif, jam demi jam):

Tabel 2.2: Contoh data deret waktu `tinggi(t)` pasang surut.

t (indeks)	tinggi(t) (m)
1	0.95
2	1.02
3	0.98
4	1.05
5	1.12

Metode *windowing* dua langkah memakai data berisi dua nilai yang bergeser **satu langkah demi langkah**. Untuk tiap posisi: **fiturnya** adalah dua nilai `tinggi(t-2)` dan `tinggi(t-1)`. **Targetnya** adalah nilai baru `tinggi(t)`. Catatan: angka *raw* yang sama tampil di beberapa *window*; hanya posisinya yang berubah.

Urutan fiturnya sengaja dicocokkan dengan Kode 2.2: kolom pertama `[t-2]`, kolom kedua `[t-1]`. Setiap baris contoh:

contoh	fitur <code>[x1, x2] = [t-2, t-1]</code>	target y (t)
1	<code>[0.95, 1.02]</code>	0.98
2	<code>[1.02, 0.98]</code>	1.05
3	<code>[0.98, 1.05]</code>	1.12

Tabel 2.3: Contoh *windowing* dua langkah pasang surut.

Cara membangun *windowing* ini dijelaskan di Bab 7 secara mendalam; di Bab 2 kita cukup memakai bentuk tabel di atas sebagai ilustrasi konsep.

Langkah 2 - *Baseline*

Sebelum menantang neural network, ukur *baseline* sederhana. Untuk deret periodik seperti pasang surut, *baseline* paling natural adalah *persistence*: “prediksi tinggi besok = tinggi hari ini” ($\hat{y}(t) = y(t-1)$). Tabel 2.3 memberi kita *baseline*: berapa MAE yang dihasilkan model yang selalu menebak nilai kemarin?

Prinsip di Bab 1 menuntut: *neural network* layak dipakai hanya jika **menalahkan *persistence***. Jika tidak, lebih baik kita memakai *persistence*: sederhana, tanpa pelatihan, tanpa pemeliharaan.

Langkah 3 - Latih jaringan

Kode 2.2 - Membangun windowing, split berbasis waktu, dan data sintetik pasang surut.

```

import numpy as np
import tensorflow as tf

np.random.seed(42)
tf.random.set_seed(42)

# data sintetik semi-diurnal (~12.42 jam) + sedikit noise
t = np.arange(0, 500)
tinggi = np.sin(2 * np.pi * t / 12.42) + 0.05 * np.random.randn(len(t))

X = np.column_stack([tinggi[:-2], tinggi[1:-1]]) # fitur: [t-2, t-1]
y = tinggi[2:] # target: t

n = len(X)
n_train = int(n * 0.7)
n_val = int(n * 0.15)
X_train, y_train = X[:n_train], y[:n_train]
X_val, y_val = X[n_train:n_train+n_val], y[n_train:n_train+n_val]
X_test, y_test = X[n_train+n_val:], y[n_train+n_val:]
print(X_train.shape, X_val.shape, X_test.shape)

```

Kode 2.2 memperlihatkan split yang **berurutan waktu**: 70% pertama untuk latih, 15% berikutnya validasi, 15% terakhir uji. (Pembahasan mengapa tidak acak ada di Bagian 2.7.)

Kode 2.3 - Compile, latih, dan evaluasi model terhadap *baseline persistence*.

```

model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(2,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=60, batch_size=32, verbose=0,
)

# evaluasi vs baseline persistence
pred = model.predict(X_test, verbose=0).ravel()
base_pred = X_test[:, 1] # persistence: pakai nilai t-1 (kolom fitur ke-2)

def mae(a, b):
    return float(np.mean(np.abs(a - b)))

```

```
print("MAE persistence:", round(mae(y_test, base_pred), 4))
print("MAE neural network:", round(mae(y_test, pred), 4))
```

Jalankan Kode 2.3 di notebook `ch-02-01_regresi_pasang_surut.ipynb`. Anda akan melihat dua angka MAE. Jika MAE jaringan **lebih kecil** daripada *persistence*, model bekerja; jika tidak, tinjau ulang (Bab 7).

Interpretasi hasil, bukan hanya angka

Dua angka itu, misalnya:

Model	MAE (m)
<i>Persistence</i> (tebak nilai kemarin)	0.030
Jaringan (MLP)	0.020

Nilai MAE *Persistence* lebih besar dari MLP. Tetapi walaupun MLP “lebih kecil”, belum berarti “lebih berguna”: angka MAE tidak menginterpretasi sendiri. Tiga lensa membantu menilai:

1. **Lensa operasional: apakah selisih mengubah keputusan?** Selisih $= 0.030 - 0.020 = 0.010 \text{ m} = 1 \text{ cm}$. Jika toleransi operasional tinggi pasang $\pm 0.10 \text{ m}$ (10 cm), perbaikan 1 cm tidak mengubah keputusan navigasi atau peringatan: model “lebih akurat”, tetapi dampak operasionalnya minimal. Pertanyaan praktisnya seperti “apakah selisih ini mengubah keputusan pelabuhan?” - jika MAE jauh di bawah toleransi yang disyaratkan, model sudah cukup.
2. **Lensa statistik: menang atau hanya kebetulan?** Selisih kecil belum tentu berarti suatu model lebih baik; bisa saja perbedaan itu muncul karena *noise* pada data uji. Uji statistik seperti **Diebold-Mariano** menentukan apakah selisih tersebut signifikan atau hanya kebetulan. Selain itu, periksa konsistensi antar-periode: model yang menang minggu ini bisa kalah minggu depan, jadi evaluasi yang *robust* memakai beberapa periode uji (Bab 5 membahas metode; Bab 8-9 menjabarkan konsep).
3. **Lensa pragmatis: apakah model sederhana sudah cukup?** Apakah memakai *neural network* dapat menambah nilai yang signifikan hingga mampu mengubah keputusan? Jika Anda tidak yakin, maka *baseline* sederhana seperti ARIMA atau *persistence* bisa menjadi pilihan yang lebih baik (kembali ke prinsip Bab 1).

Kesimpulan: angka MAE saja belum cukup untuk menentukan kualitas model. Kita perlu melihat juga **baseline operasional, uji statistik, konsistensi antar-periode, serta biaya untuk menjalankan dan memelihara model**. Tidak ada model “sempurna”; yang dicari adalah model yang **cukup baik untuk tujuan** dan **lebih baik dari baseline sederhana**. Sikap ini yang membedakan praktisi dari sekadar pengguna *library*.

2.6 Mengukur Galat: MAE vs MSE

Setelah model menghasilkan prediksi, kita perlu mengukur **seberapa besar galat prediksinya**. Dua fungsi *loss* regresi yang paling umum:

Tabel 2.4: Perbandingan MAE dan MSE sebagai fungsi galat regresi.

Loss	Definisi	Sifat
MAE (<i>mean absolute error</i>)	rata-rata selisih **MSE** (*mean squared error*) rata-rata selisih ²	Memberi hukuman besar pada galat besar (kuadrat)

Keduanya dipakai (Tabel 2.4); data seperti curah hujan memiliki distribusi dengan ekor kanan (kadang nilai sangat besar), sehingga pilihan *loss* bisa memengaruhi perilaku model. Prinsip: pilih sesuai skala & tujuan, dan selalu bandingkan dengan *baseline* (Bab 7).

Contoh numerik sederhana

Misalkan tiga data uji memiliki aktual $y = [10, 12, 8]$ dan model memprediksi $\hat{y} = [9, 15, 8]$. Selisihnya $[-1, +3, 0]$, sehingga:

$$\text{MAE} = \frac{|-1| + |3| + |0|}{3} = \frac{4}{3} \approx 1.33 \quad (2.6)$$

$$\text{MSE} = \frac{(-1)^2 + (3)^2 + (0)^2}{3} = \frac{10}{3} \approx 3.33 \quad (2.7)$$

Perhatikan: pada MSE satu galat 3 “menyumbang” nilai 9 karena 3 kuadrat. Hal ini jauh lebih besar daripada kontribusinya di MAE (3). Itulah inti perbedaan: **MSE lebih sensitif pada galat besar**.

Kapan memilih yang mana?

- Jika galat besar sangat merugikan (misal prediksi pasang surut yang meleset jauh pada jam kritis), MSE/RMSE memberi penalti lebih besar, layak dipertimbangkan.
- Jika data mengandung pencilan (sensor rusak, hujan ekstrem sesekali) dan Anda tidak ingin model “dibuat sibuk” oleh satu nilai besar, MAE lebih tenang.
- Dalam praktik, umumnya MAE dan RMSE dilaporkan bersama, karena keduanya memberi gambaran berbeda: MAE untuk galat khas, RMSE untuk bobot galat ekstrem.

Di Bab 5 kita tambah metrik domain (KGE, CSI, dsb.), tetapi MAE/RMSE tetap fondasi untuk regresi.

Memahami loss sebagai “jarak” dan “hukuman”

Pandangan yang membantu: **loss** adalah ukuran seberapa buruk prediksi **si model** dalam satu angka, yang diminimalkan selama pelatihan. Ketika Anda menulis `loss="mse"`, *optimizer* menyesuaikan bobot dengan tujuan memperkecil rata-rata kuadrat selisih antara aktual dan prediksi. Bayangkan data suhu: jika model menebak 28.0°C padahal aktual 30.0°C, selisih 2.0. MSE mengkuadratkannya menjadi 4.0, “hukuman” ini lebih besar dari gabungan dua galat 1.0 (yang hanya 2.0). Sifat ini membuat model yang dilatih dengan MSE cenderung menghindari galat besar, kadang mengorbankan presisi pada galat kecil.

MAE tidak mengkuadratkan, sehingga semua galat diberi bobot sama. Untuk data dengan pencilan (misal satu hari hujan ekstrem 150 mm di tengah ratusan hari 0–20 mm), model MSE bisa “terganggu” oleh satu nilai besar itu dan mempengaruhi semuanya; model MAE lebih tahan terhadap nilai ekstrem. Namun MAE memiliki *gradien* konstan (biasanya ± 1) untuk hampir semua nilai galat, yang membuat optimasi sedikit berbeda: karena *gradien*-nya tidak mengecil saat model mendekati minimum, konvergensi bisa menjadi lambat atau berosilasi di sekitar solusi. Berbeda dengan MSE, yang gradiennya bervariasi dengan galat (gradien kecil saat galat kecil), sehingga konvergensi lebih halus di sekitar minimum. Detail optimasi ini dibahas di Bab 4.

Metrik *versus* loss

Penting membedakan dua peran:

- **Loss** - dipakai selama pelatihan (*optimizer* meminimalkannya).
- **Metrik** - dilaporkan kepada pembaca untuk menilai kualitas (bisa sama atau berbeda).

Di Kode 2.3 kita menggunakan `loss="mse"` tetapi `metrics=["mae"]`. Ini wajar: melatih dengan MSE (agar galat besar dihukum) sambil melaporkan MAE (lebih mudah diinterpretasi dalam satuan °C/m). Praktik seperti ini umum; yang penting sadar bahwa keduanya tidak harus identik.

2.7 Adam, *Learning Rate*, dan *Split Waktu*

Setelah model dibentuk dan *loss* dipilih, kita melatihnya dengan *optimizer*. Dua istilah yang perlu dikenal sekarang:

- **Adam** adalah algoritma optimasi modern (turunan dari *gradient descent*) yang mengatur besar langkah penyesuaian bobot per iterasi secara adaptif. Kode 2.3 memakainya cukup dengan satu baris `optimizer="adam"`. Detail cara kerja & alternatifnya dibahas di Bab 4.

- **Learning rate** mengontrol seberapa besar tiap langkah: terlalu besar → model melompat dan tidak konvergen; terlalu kecil → belajar sangat lambat. Adam sudah “cerdas” memilih ukuran langkah adaptif, tetapi nilai awal *learning rate* tetap perlu masuk akal (*default* Keras biasanya aman untuk pemula).

Kenapa *split* tidak boleh acak untuk data waktu?

Untuk data **deret waktu meteorologi**, pembagian data menjadi latih/validasi/uji harus **berdasarkan waktu**, bukan acak:

train (2015–2021) | validation (2022) | test (2023)

Alasannya adalah **leakage** (kebocoran informasi). Bagi acak berarti sampel uji bisa berasal dari tanggal yang berdekatan atau di antara data latih; informasi dari masa depan “bocor” ke masa latih, sehingga performa terlihat terlalu bagus. Di dunia nyata, saat model dipakai operasional, ia hanya punya data hingga hari ini: menguji dengan data masa lalu yang dicampur acak tidak merepresentasikan kondisi itu.

Bayangkan memprediksi pasang surut besok. Jika data uji mencakup tanggal yang juga ada di data latih (hanya beda beberapa hari), model bisa “meniru”. Itu bukan keterampilan prediksi masa depan, itu menyontek. *Split* berbasis waktu memaksa model memprediksi periode yang benar-benar belum pernah dilihat, sama seperti kondisi nyata.

Validasi untuk *tuning*

Data **validasi** digunakan selama pelatihan untuk memantau kinerja (misalnya lewat `validation_data` di Kode 2.3) dan bisa dipakai memilih *hyperparameter* (jumlah neuron, *learning rate*). Setelah semua keputusan selesai, model diuji **sekali** di data **test** supaya tidak curang: memakai tes berkali-kali untuk menyetel model sama saja dengan “membocorkan” tes ke latihan. Bab 5 dan Bab 7 akan mengulang dan memperdalam aturan ini.

Ringkas alur mini-kasus

1. Bentuk *windowing* (Tabel 2.3, Kode 2.2).
2. Ukur *baseline persistence*.
3. Bangun & latih MLP (Kode 2.1-2.3).
4. Bandingkan MAE dengan *baseline*.
5. Jika *neural networks* tidak kalah, pakai; jika sebanding, gunakan *baseline*.

2.8 Latihan

Soal konsep

1. Jelaskan perbedaan bobot dan bias pada neuron, dengan contoh variabel meteorologi.

2. Mengapa MLP perlu fungsi aktivasi non-linear? Apa yang terjadi jika semua lapisan linear?
3. Apa keunggulan ReLU dibanding perceptron langkah (*step*) untuk pelatihan?
4. Data curah hujan memiliki banyak nol dan beberapa nilai sangat besar. *Loss* mana yang Anda pilih, MAE atau MSE, dan mengapa?

Latihan praktik (notebook `ch-02-01_regresi_pasang_surut.ipynb`)

1. Ubah jumlah neuron pada dua lapisan `Dense` (misal 4 dan 32). Catat MAE `train/val/test`. Kapan model mulai *overfit* (*train* jauh lebih baik daripada *val*)?
2. Tambahkan fitur ketiga `tinggi(t-3)`. Apakah MAE membaik? Apa alasan Anda?
3. Ganti `loss="mse"` dengan `loss="mae"`. Bandingkan akhir MAE test. Diskusikan perbedaan.
4. Buat *baseline* kedua: rata-rata klimatologis (nilai **rata-rata** keseluruhan *train*) sebagai prediksi tetap untuk seluruh test. Bandingkan dengan *persistence* dan jaringan.
5. **Proyek mini:** ambil data suhu harian stasiun lokal (misal dari GHCN-Daily - Bab 6), lakukan *windowing* 2 langkah, dan bandingkan MLP 2 lapisan vs *persistence*. Laporkan MAE.
6. (**Opsional, statistik**) Jika punya library `statsmodels` atau `sklearn`, ulangi perbandingan MAE model vs *persistence* di beberapa blok waktu dan jalankan skema uji **Diebold-Mariano** untuk menyajikan apakah selisih signifikan. Bila *library* tidak tersedia, cukup catat MAE per blok dan diskusikan konsistensinya.

Jawaban latihan tidak harus “menang”: yang penting adalah Anda terbiasa membandingkan model dengan *baseline* dan membaca angka MAE secara kritis.

2.9 Galat Umum Pemula

Beberapa jebakan yang sering muncul saat pertama kali membangun model regresi, beserta cara menghindarinya:

1. **Tidak memakai *baseline*.** Membangun MLP lalu menyimpulkan “bekerja” tanpa pernah mengukurnya terhadap *persistence*/linear. Solusi: selalu ukur *baseline* dulu (Bagian 2.5).
2. ***Split* acak untuk data waktu.** Memakai `train_test_split default` (acak) pada deret waktu menyebabkan *leakage*. Solusi: potong berurutan berdasarkan waktu (Bagian 2.7).
3. **Melihat data tes berulang kali.** Menyetel model terhadap tes sampai “jadi” adalah bentuk bocor. Solusi: simpan tes untuk evaluasi akhir; pakai validasi untuk *tuning* (Bagian 2.7).
4. **Mengubah skala tanpa menyimpan statistiknya.** Jika Anda menor-

malisasi fitur, Anda harus menyimpan rata-rata/deviasi dari *train* dan menerapkannya pada *tes* dan produksi. Menerapkan statistik dari seluruh data (termasuk tes) adalah *leakage*. (Normalisasi dibahas di Bab 6.)

5. Melaporkan hanya *loss*, tanpa konteks satuan. MAE 0.02 m berarti apa? Selalu nyatakan dengan satuan dan bandingkan dengan toleransi operasional.

6. Langsung ke arsitektur besar. Menambah banyak lapisan/neuron sebelum mencoba model kecil. Solusi: mulai kecil, tingkatkan bertahap (Bab 4).

7. Mengabaikan bentuk tensor. `input_shape` yang salah adalah sumber galat umum. Periksa `model.summary()` untuk memastikan bentuk output tiap lapisan sesuai (Kode 2.1).

Menghindari galat di atas menghemat banyak waktu dan, lebih penting, menjauhkan Anda dari kesimpulan yang keliru tentang kinerja model.

Ringkasan

- Regresi = memprediksi besaran kontinu; neuron = bobot + bias + fungsi aktivasi.
- 1 neuron linear identik regresi linear; non-linearitas (ReLU) diperlukan untuk pola yang tidak lurus.
- Mini-kasus pasang surut memperkenalkan penggunaan langsung pada data laut Indonesia; *baseline persistence* selalu dipakai sebagai pembanding.
- MAE vs MSE: pilih sesuai skala & tujuan; Adam & learning rate; split berbasis waktu untuk melawan *leakage*.
- Proyek ML selalu: bentuk data $\rightarrow$ *baseline* $\rightarrow$ model $\rightarrow$ bandingkan $\rightarrow$ putuskan.

References

1. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386-408, 1958, doi: 10.1037/h0042519.
2. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, Dec. 2012, pp. 1097-1105; reissued in *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, 2017, doi: 10.1145/3065386.
3. Badan Informasi Geospasial (BIG), “Peta pasang surut dan pola pasut perairan Indonesia,” [Online]. Available: <https://tides.big.go.id/> (diakses: September 2026)

Prasyarat: Bab 1 (konsep ML/DL, lingkungan Colab) dan Bab 2 (neuron, fungsi aktivasi, MAE/MSE, split waktu). TensorFlow/Keras siap di lingkungan Anda.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Membangun** model klasifikasi biner dan multi-kelas (hujan/tidak hujan, level bahaya) dengan TensorFlow/Keras.
2. **Menjelaskan** peran sigmoid dan softmax serta *binary/categorical cross-entropy*.
3. **Mendiagnosis** *class imbalance* dan memilih metrik yang tepat (*precision*, *recall*, F1, pengenalan CSI/FAR) - bukan hanya akurasi.
4. **Menerapkan** *trade-off threshold* untuk prediksi fenomena langka.

3.1 Mengapa Klasifikasi Berbeda

Bab 2 membahas regresi yaitu memprediksi besaran kontinu (suhu, tinggi pasang). Di dunia nyata, banyak keputusan meteorologi bukan “berapa?”, melainkan “apa?”:

- Apakah besok akan hujan atau tidak?
- Apakah intensitas hujan masuk kategori lebat, sedang, atau ringan?
- Apakah perlu peringatan dini banjir untuk level siaga?

Masalah ini disebut **klasifikasi**, memprediksi **kategori** (label) dari fitur. Ada dua varian utama:

- **Biner (dua kelas):** misal 0 = tidak hujan, 1 = hujan.
- **Multi-kelas:** misal ringan, sedang, lebat - atau level bahaya waspada, siaga, awas.

Perbedaan inti dari regresi adalah keluaran bukan bilangan kontinu melainkan **probabilitas atas kategori**. Di balik layar, kita tetap memakai neuron tetapi lapisan keluaran memakai fungsi aktivasi khusus: **sigmoid** untuk biner, **softmax** untuk multi-kelas. Prinsip umum pelatihan *supervised* dijelaskan dalam literatur *deep learning* [1]. Akar historisnya adalah **perceptron** (Rosenblatt, 1958): neuron tunggal yang mengklasifikasikan *input* ke dua kelas berdasarkan ambang [2].

Mengapa kita perlu probabilitas, bukan sekadar label? Karena informasi **seberapa yakin** model sangat berharga di operasional. Dua model yang sama-sama memprediksi “hujan” tidaklah setara jika yang satu yakin 90% dan yang lain

51%. Probabilitas memberi kita ruang untuk menetapkan ambang keputusan yang sesuai risiko, topik Bagian 3.7.

Regresi vs klasifikasi: tabel perbandingan

Tabel 3.1: Perbedaan utama regresi dan klasifikasi.

Aspek	Regresi (Bab 2)	Klasifikasi (bab ini)
Keluaran	Bilangan kontinu	Kategori/label
Aktivasi	Tanpa aktivasi	Sigmoid (biner) / softmax
keluaran	(linear)	(multi-kelas)
<i>Loss</i> utama	MAE atau MSE	<i>Binary/categorical cross-entropy</i>
Metrik	MAE, RMSE, R^2	Akurasi, <i>precision</i> , <i>recall</i> , F1, CSI/FAR
Contoh meteo	Suhu besok, tinggi pasang	Hujan/tidak, level bahaya

Tabel 3.1 merangkum perbedaan yang akan kita bahas satu per satu. Perhatikan: struktur model (lapisan **Dense** + ReLU di tengah) sama dengan Bab 2; yang berubah hanyalah ujung jaringan dan cara mengukurnya.

3.2 Sigmoid: Aktivasi Keluaran untuk Dua Kelas

Untuk klasifikasi biner, lapisan terakhir memakai **sigmoid**, yang memampatkan nilai z ke rentang 0-1:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (3.1)$$

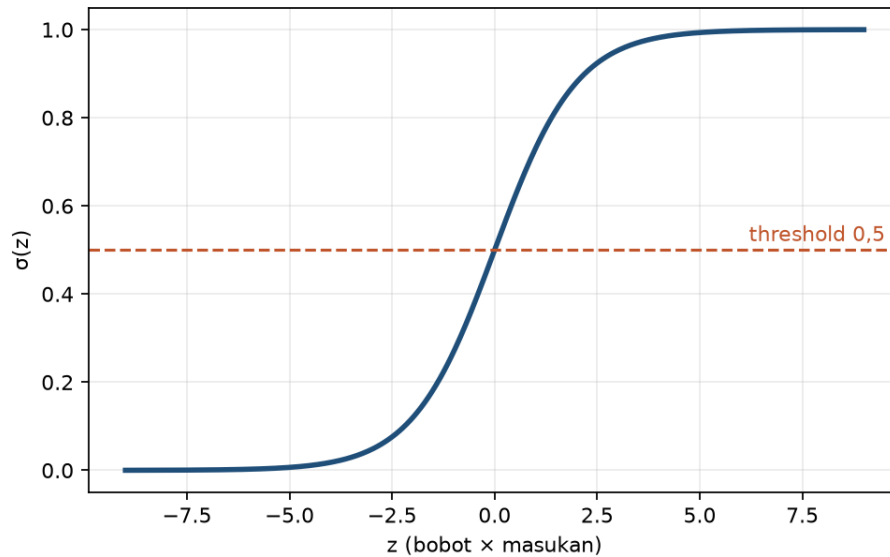
Sigmoid pada Persamaan (3.1) memberi interpretasi probabilistik: keluaran 0.85 berarti keyakinan 85% bahwa sampel masuk kelas 1 (misal hujan). Sifat sigmoid yang penting:

- Nilai sangat positif $\rightarrow$ mendekati 1.
- Nilai sangat negatif $\rightarrow$ mendekati 0.
- Nilai nol $\rightarrow$ tepat 0.5 (titik paling “ragu”).

Gambar 3.1: Kurva sigmoid.

Gambar 3.1 memperlihatkan kurva S khas sigmoid: mulus, monoton naik, dan termampatkan; nilai z yang boleh dari $-\infty$ sampai $+\infty$ selalu dipetakan ke rentang (0, 1).

Aturan ambang (*threshold*) standar adalah 0.5: jika $\sigma(z) \geq 0.5$, prediksi kelas 1; selain itu kelas 0. Namun *threshold* ini **tidak wajib**: untuk fenomena jarang seperti hujan lebat, kita sering menaikkan/menurunkan *threshold* (dibahas Bagian 3.7).



Gambar 4: Gambar 3.1 - Kurva sigmoid

Contoh numerik sigmoid

Misalkan model memberi $z = 1.2$. Maka:

$$\sigma(1.2) = \frac{1}{1 + e^{-1.2}} = \frac{1}{1 + 0.301} \approx 0.77$$

Dengan *threshold* 0.5, sampel masuk kelas 1. Jika kita menaikkan *threshold* ke 0.8, sampel ini menjadi kelas 0; keputusan berubah hanya karena ambang, bukan model.

3.3 Softmax: Aktivasi Keluaran untuk Banyak Kelas

Untuk klasifikasi multi-kelas, kita memakai **softmax**, yang mengubah vektor nilai z menjadi distribusi probabilitas yang **jumlahnya 1**:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (3.2)$$

Softmax pada Persamaan (3.2) memberi probabilitas untuk tiap kelas i di antara K kelas. Contoh: $[0.70, 0.20, 0.10]$ untuk kelas [**ringan**, **sedang**, **lebat**], model paling yakin “ringan”.

Penting: softmax bersifat relatif, ia membandingkan semua kelas. Jika kita menambahkan satu kelas lagi, probabilitas semua kelas bisa berubah meski data

untuk kelas lama sama. Ini berbeda dari sigmoid yang “mandiri” per kelas (untuk biner, hanya satu).

Perbandingan sigmoid vs softmax

Tabel 3.2: Perbandingan sigmoid dan softmax.

	Sigmoid	Softmax
Jumlah kelas	1 neuron, 2 kelas (komplementer)	K neuron, K kelas
Jumlah probabilitas	p tunggal; komplemen 1-p implisit	Vektor K nilai, jumlah 1
Fungsi	$\frac{1}{1+e^{-z}}$	$\frac{e^{z_i}}{\sum_j e^{z_j}}$
Kapan dipakai	Masalah biner	Masalah multi-kelas

3.4 *Cross-Entropy*: Fungsi *Loss* Klasifikasi

Seperti MAE/MSE untuk regresi, klasifikasi memakai fungsi *loss* khusus:

- **Binary cross-entropy** (biner): menghukum galat antara probabilitas sigmoid dan label biner.
- **Categorical cross-entropy** (multi-kelas): menghukum galat antara distribusi softmax dan label *one-hot*.

Catatan singkat: *one-hot* berarti label diubah menjadi vektor dengan panjang sesuai jumlah kelas, berisi 1 di posisi kelas yang benar dan 0 di posisi lain; contoh dan `sparse_categorical_crossentropy` ada di Bagian 3.5.

Rumus untuk *binary cross-entropy* (per sampel):

$$\mathcal{L} = -[y \log(p) + (1 - y) \log(1 - p)] \quad (3.3)$$

di mana y label (0 atau 1) dan p probabilitas prediksi. Intuisi: jika $y=1$ dan p mendekati 1, $\log(p)$ mendekati 0 $\rightarrow$ *loss* kecil. Jika $y=1$ tetapi p mendekati 0, *loss* sangat besar, model dihukum karena yakin salah.

Prinsip intuisi: *loss* **kecil** jika model yakin benar, **besar** jika model yakin salah. Berbeda dari MSE yang menghukum selisih linier/kuadrat, *cross-entropy* langsung menargetkan ketidaktepatan keyakinan, inilah mengapa akurasi bisa tetap tinggi meski keyakinannya rendah.

Kenapa bukan MSE untuk klasifikasi?

Ada dua alasan utama:

1. **Interpretasi probabilitas.** MSE dioptimalkan untuk nilai kontinu; ia tidak memberi “hukuman” sesuai makna probabilitas. *Cross-entropy* lahir dari teori informasi dan cocok dengan keluaran 0-1.
2. **Pelatihan.** Dengan sigmoid + MSE, gradien bisa sangat kecil ketika kurva sigmoid datar, terutama saat model yakin **tetapi salah** (misal label 1, prediksi $p \approx 0$). Karena turunan sigmoid mendekati nol dan gradien MSE mengecil, proses belajar melambat [1]. Sebaliknya, saat model yakin dan benar ($y=1, p \approx 1$), gradien *cross-entropy* juga kecil, itulah yang kita ingin. *Cross-entropy* + sigmoid/softmax menghasilkan gradien yang lebih sehat, karena proporsional terhadap $(p - y)$ dan tetap besar ketika model yakin salah. Detail di Bab 4.

3.5 Kode: Model Klasifikasi Pertama

Kode 3.1 - Kode model klasifikasi biner hujan/tidak hujan dengan Keras.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(n_features,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"), # biner: 1 neuron, sigmoid
])
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy", tf.keras.metrics.Precision(), tf.keras.metrics.Recall()],
)
model.summary()
```

Kode 3.1 memakai API Keras di atas TensorFlow [3]. Untuk multi-kelas, ganti lapisan keluaran menjadi `Dense(K, activation="softmax")` dan `loss categorical_crossentropy`.

Kode multi-kelas

Kode 3.2 - Model klasifikasi multi-kelas intensitas hujan dengan Keras.

```
import tensorflow as tf

K = 3 # ringan, sedang, lebat

model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(n_features,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(K, activation="softmax"), # multi-kelas: K neuron, softmax
])
```

```

])
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy", # label integer, tanpa one-hot manual
    metrics=["accuracy"],
)
model.summary()

```

Perhatikan Kode 3.2: lapisan keluaran berisi K neuron dengan softmax, dan memakai `sparse_categorical_crossentropy` bila label diberikan sebagai integer (0, 1, 2). Ini variasi praktis dari `categorical_crossentropy` yang menuntut *one-hot*.

Latihan dengan bobot kelas (*imbalance*)

Kode 3.2 tidak menangani ketidakseimbangan. Untuk menekankan kelas minoritas, gunakan `class_weight` saat fit:

Kode 3.3 - Fit dengan bobot kelas untuk menangani *imbalance*.

```

import numpy as np

# contoh: kelas mayoritas (0) berbobot 1, kelas minoritas (1) berbobot lebih besar
class_weight = {0: 1.0, 1: 10.0}

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    class_weight=class_weight,
    epochs=50, batch_size=32, verbose=0,
)

```

Bobot 10.0 pada kelas 1 membuat galat pada kejadian langka dihukum 10 kali lipat. Rasio ketidakseimbangan (mis. bila 5% kejadian, bobot ~19) adalah **titik awal** yang membantu, bukan aturan baku: bobot optimal tidak selalu dilihat dari frekuensi invers, dan nilai terlalu ekstrem bisa menaikkan *overfit* pada kelas minoritas. Sebaiknya *tuning* bobot empiris (mis. coba 2, 5, 10, 19) dan pilih yang memberi F1/CSI terbaik pada validasi. Bandingkan hasil Kode 3.3 dengan tanpa bobot di notebook.

Memahami keluaran *one-hot*

Untuk multi-kelas, label biasanya di-encode sebagai **one-hot**: vektor panjang K dengan satu pada posisi kelas yang benar.

```

ringan → [1, 0, 0]
sedang → [0, 1, 0]
lebat  → [0, 0, 1]

```

Keras menyediakan `to_categorical` untuk konversi, dan label integer lugas juga bisa dipakai dengan `loss sparse_categorical_crossentropy`, variasi yang sama, tanpa perlu *encode* manual.

3.6 *Class Imbalance*: Mengapa Akurasi Dapat Menipu

Data meteorologi sering **tidak seimbang**: hujan 50 mm mungkin terjadi hanya beberapa hari dalam setahun di banyak stasiun tropis. Sebagai contoh nyata, di stasiun Cilacap (data harian GHCN-Daily, 1960-2024) hujan 50 mm tercatat hanya 166 dari 5904 hari pengamatan, sekitar 2,8% (rata-rata ± 5 hari per tahun) [6]. Jika model selalu memprediksi “tidak hujan deras”, akurasinya 97,2% — tampak hebat, padahal model **gagal total** pada kejadian yang justru paling penting.

Mengapa ini sangat relevan untuk meteorologi? Karena banyak fenomena berisiko justru langka: hujan ekstrem, angin kencang, banjir rob, atau cuaca buruk penerbangan. Sebagian besar hari adalah “biasa”; kejadian berbahaya adalah sebagian kecil. Model yang dioptimalkan hanya untuk akurasi global akan “belajar” memprediksi kelas mayoritas dan praktis buta terhadap kelas langka; ironisnya, kelas langka itu adalah kelas yang paling kita perlukan.

Tabel berikut menggambarkan jebakan ini:

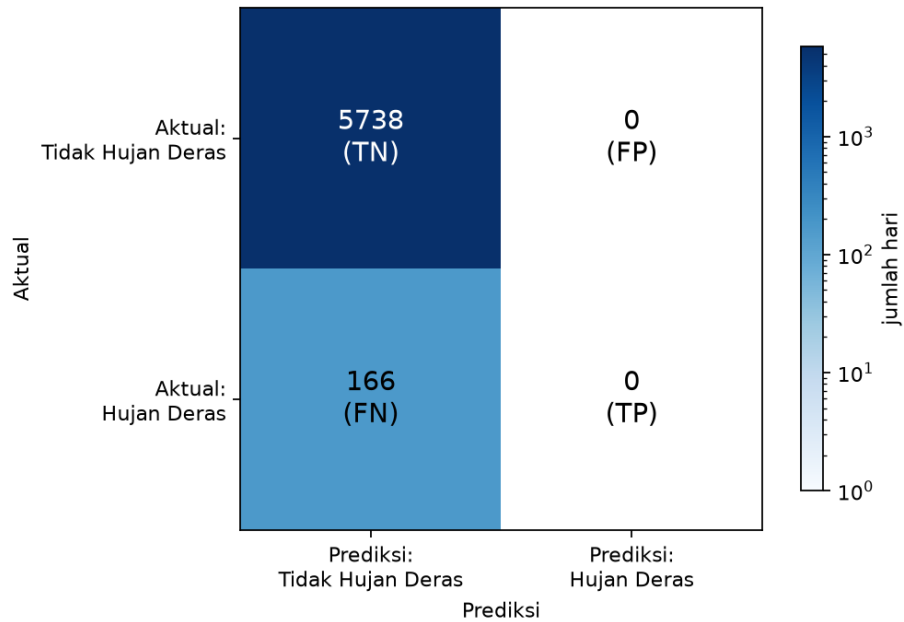
Tabel 3.3: Contoh data tidak seimbang (stasiun Cilacap, GHCN-Daily 1960-2024).

	Prediksi: Tidak Hujan Deras	Prediksi: Hujan Deras
Aktual: Tidak Hujan Deras	5738 (TN)	0 (FP)
Aktual: Hujan Deras	166 (FN)	0 (TP)

Catatan: TP/FP/FN/TN adalah empat kemungkinan hasil klasifikasi (definisi lengkap di Bagian 3.8): TP = kejadian hujan yang benar tertangkap, FP = peringatan hujan yang keliru, FN = hujan yang terlewat, TN = hari tidak hujan yang benar dinyatakan tidak hujan.

Gambar 3.2: *Confusion matrix* contoh data tidak seimbang.

Gambar 3.2 memvisualkan Tabel 3.3 untuk model yang selalu memprediksi “tidak hujan deras” pada seluruh rekaman Cilacap 1960-2024: TN=5738 (hari tidak hujan yang benar), FP=0 (tidak ada peringatan hujan keliru), FN=166 (semua hujan deras terlewat), TP=0 (tidak ada hujan deras yang tertangkap). Akurasi = $(TP+TN)/(TP+FP+FN+TN) = (0+5738)/5904 = 97.2\%$. Model ini “tampak hebat” di akurasi, padahal dari 166 hari hujan deras tidak satu pun dapat diprediksi (*recall* 0%). Untuk peringatan dini, model seperti ini sama sekali tidak berguna.



Gambar 5: Gambar 3.2 - *Confusion matrix* data tidak seimbang

Karena itu, metrik utama yang dipakai:

- **Precision** - dari semua yang diprediksi “hujan deras”, berapa yang benar? $TP/(TP+FP)$. Untuk model ini 0/0 (tidak ada prediksi hujan), sehingga precision tidak terdefinisi.
- **Recall** - dari semua yang benar-benar hujan deras, berapa yang diprediksi benar? $TP/(TP+FN) = 0/166 = 0\%$.
- **F1** - rata-rata harmonik *precision-recall* (seimbang):

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \quad (3.4)$$

Untuk kejadian langka dalam meteorologi operasional, kuartet yang lebih terpercaya adalah **CSI, POD, FAR, TS**; akan dibahas penuh di Bab 5. Pedoman resmi verifikasi perkiraan operasional dikeluarkan WMO [4]. Di bab ini kita cukup paham mengapa akurasi tidak cukup.

Empat cara mengatasi *imbalance* (pratinjau)

1. **Gunakan metrik yang tepat** - *precision/recall/F1*, bukan akurasi.
2. **Atur *threshold***, turunkan ambang agar kejadian langka lebih sering terangkap (Bagian 3.7).

3. **Pemberian bobot kelas** - `class_weight` di Keras memberi penalti lebih besar untuk galat pada kelas minoritas (contoh dalam notebook).
4. **Resampling** - *undersampling* kelas mayoritas atau *oversampling* minoritas (konsekuensi: mengubah distribusi; diskusi di Bab 5). **Hati-hati pada deret waktu:** *oversampling* acak (mis. SMOTE) merusak urutan temporal - lebih aman pakai `class_weight` (Bab 9).

Contoh numerik lengkap *precision/recall*

Ambil kembali Tabel 3.3: TP=0, FP=0, FN=166. Perhitungannya:

- $Precision = 0/(0+0) = 0/0$ - tidak terdefinisi, karena model tidak pernah memprediksi “hujan deras”.
- $Recall = 0/(0+166) = 0/166 = 0$ - tidak ada kejadian yang tertangkap.
- $F1 = 2 \cdot TP / (2 \cdot TP + FP + FN) = 0/(0+0+166) = 0$ (bentuk ekuivalen yang terdefinisi).

Nilai F1 = 0 menandakan model sama sekali tidak berguna untuk kejadian langka, meski akurasi 97,2%. Ini ilustrasi kuat mengapa laporan model untuk meteorologi sebaiknya **selalu menyertakan *confusion matrix* dan metrik langka**, bukan hanya akurasi.

3.7 *Trade-off Threshold* untuk Prediksi

Model memberi probabilitas (kekuatan sigmoid/softmax). Pertanyaan praktisnya: **di ambang berapakah kita bertindak?**

- *Threshold* rendah (mis. 0.2) → lebih banyak hujan deras terdeteksi (*recall* naik), tetapi juga lebih banyak *false alarm* (*precision* turun).
- *Threshold* tinggi (mis. 0.8) → lebih hati-hati; *false alarm* turun, tetapi banyak kejadian terlewat (*recall* turun).

Tidak ada jawaban universal: tergantung **biaya galat**. Untuk peringatan dini bencana, *false alarm* mungkin lebih diterima daripada kejadian terlewat, maka pilih *recall* tinggi. Untuk keputusan yang mahal (misal evakuasi), mungkin *precision* lebih penting.

Kurva **precision-recall** dan **ROC** membantu memilih: kita mengevaluasi model di banyak *threshold* sekaligus, bukan hanya 0.5. Di Bab 9, *trade-off* ini diterapkan pada prediksi hujan harian.

Bagaimana memilih *threshold* secara sistematis? Salah satu cara sederhana: hitung *precision* dan *recall* untuk rentang *threshold* (mis. 0.1, 0.2, ..., 0.9), lalu pilih titik yang paling sesuai kebutuhan. Cara lain: gunakan *cost matrix*: tetapkan berapa “harga” sebuah *miss* vs *false alarm* (misal 5:1), lalu pilih *threshold* yang meminimalkan total biaya pada validasi. Tidak ada jawaban tunggal, tetapi prosesnya **harus eksplisit dan terdokumentasi**.

Contoh keputusan *threshold* dalam konteks peringatan dini

Bayangkan sistem peringatan dini banjir rob. Jika *threshold* terlalu tinggi (konservatif), kita jarang mengeluarkan peringatan salah, tetapi ada risiko kejadian terlewat dan warga tidak sempat bersiap. Jika *threshold* terlalu rendah, peringatan terlalu sering salah sehingga masyarakat lama-kelamaan jenuh dan mengabaikannya; fenomena ini dikenal sebagai *alarm fatigue* (kelelahan peringatan). Pilihan *threshold* karena itu adalah **keputusan kebijakan** yang melibatkan biaya sosial, bukan sekadar statistik.

Threshold mana yang “paling baik”?

Jika tidak ada preferensi biaya eksplisit, praktisi sering memilih *threshold* yang memaksimalkan **F1**, karena F1 menyeimbangkan *precision* dan *recall* dalam satu angka. Peringatan: F1 adalah rata-rata harmonik dengan bobot yang **sama** untuk *precision* dan *recall*; ia implisit mengasumsikan bahwa biaya *false alarm* = biaya *miss*. Jika biaya *miss* jauh lebih besar (sering dalam peringatan dini bencana), *threshold* yang memaksimalkan F1 bisa bukan pilihan optimal; dalam hal itu gunakan *cost matrix* dengan biaya yang jelas. Selain itu, dua model dengan F1 sama bisa memiliki perilaku berbeda di lapangan; karena itu jangan pernah hanya melihat F1, tapi periksa juga angka *precision* & *recall*-nya dan, jika memungkinkan, *curve*-nya (ROC/*precision-recall*).

3.8 *Confusion Matrix*: Membaca yang Terlewat dan Keliru

Confusion matrix adalah tabel yang merangkum empat kemungkinan hasil klasifikasi:

- **TP** (*true positive*): aktual positif, diprediksi positif → benar.
- **FP** (*false positive* / *false alarm*): aktual negatif, diprediksi positif → peringatan yang keliru.
- **FN** (*false negative* / *miss*): aktual positif, diprediksi negatif → kejadian terlewat.
- **TN** (*true negative*): aktual negatif, diprediksi negatif → benar.

Tabel 3.4: Struktur *confusion matrix* untuk masalah biner.

	Prediksi: Positif	Prediksi: Negatif
Aktual: Positif	TP	FN (<i>miss</i>)
Aktual: Negatif	FP (<i>false alarm</i>)	TN

Di meteorologi operasional, dua sel yang paling diperhatikan adalah **FP** (*false alarm*) dan **FN** (*miss*), karena membawa konsekuensi langsung: peringatan keliru menggerus kepercayaan, kejadian terlewat membawa risiko keselamatan.

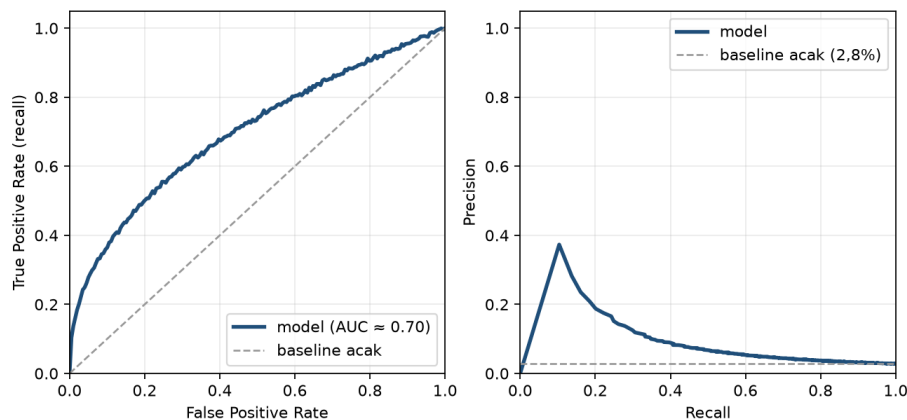
Dari *confusion matrix* ini, semua metrik di atas diturunkan:

- Akurasi = $(TP+TN)/(TP+FP+FN+TN)$
- $Precision = TP/(TP+FP)$
- $Recall = TP/(TP+FN)$
- $F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$

3.9 ROC dan *Precision-Recall Curve*

Karena *threshold* bisa digeser, kinerja model lebih baik dinilai dengan **kurva** daripada satu titik:

- **ROC curve**: plot *true positive rate (recall)* terhadap *false positive rate* ($FP/(FP+TN)$) untuk semua *threshold*. Luas di bawahnya disebut **AUC** - semakin mendekati 1 semakin baik (Gambar 3.3, kiri).
- ***Precision-recall curve***: plot *precision* terhadap *recall*; lebih informatif untuk data sangat tidak seimbang, karena tidak terpengaruh oleh TN yang melimpah (Gambar 3.3, kanan).



Gambar 6: Gambar 3.3 - Kurva ROC dan *precision-recall* untuk data tidak seimbang

Gambar 3.3: Kurva ROC (kiri) dan *precision-recall* (kanan) untuk data dengan proporsi kelas positif 2,8% (proporsi Cilacap, Tabel 3.3; model ilustrasi). ROC tampak cukup baik (AUC $\approx 0,70$), tetapi kurva *precision-recall* menyingkap *precision* yang rendah: kebanyakan peringatan yang keluar ternyata salah.

Kapan pakai yang mana?

- Jika kelas seimbang, ROC/AUC umum dipakai.
- Jika kelas sangat langka (hujan deras, banjir), ***precision-recall curve*** lebih jujur - ROC bisa tampak “bagus” padahal model praktis tak berguna karena FN/FP penting.

Bab 9 akan memakai *precision-recall* untuk verifikasi hujan harian.

3.10 FAQ Singkat

Apakah akurasi selalu buruk? Tidak. Untuk masalah seimbang (misal membedakan dua jenis awan yang frekuensinya setara), akurasi adalah ringkasan yang masuk akal. Ia menjadi menyesatkan hanya pada data sangat tidak seimbang.

Apakah saya perlu menyeimbangkan data dulu? Tidak selalu. Mengubah distribusi kelas (*undersampling/oversampling*) mengubah masalah itu sendiri. Sering lebih baik: gunakan metrik yang tepat + bobot kelas, lalu evaluasi dengan CSI/FAR. Nanti di Bab 5.

Mengapa memakai softmax, bukan beberapa sigmoid untuk multi-kelas? Softmax memaksa total probabilitas = 1 dan “bersaing” antar kelas, sesuai asumsi label saling eksklusif [1]. Beberapa sigmoid (*multi-label*) cocok jika sebuah sampel bisa punya lebih dari satu label sekaligus (misal “hujan” DAN “angin kencang” bersamaan). Untuk *multi-label*, lapisan keluaran berisi beberapa neuron sigmoid dan *loss* yang dipakai adalah `binary_crossentropy` per neuron. Bukan `categorical_crossentropy`, yang menuntut distribusi softmax. Varian ini dilaporkan dalam literatur klasifikasi [1].

3.11 Alur Kerja Model Klasifikasi

Berdasarkan seluruh bab, alur kerja praktis untuk setiap masalah klasifikasi:

1. **Definisikan masalah** - biner atau multi-kelas? Apa “kelas positif” (yang paling penting menangkapnya)? Apa biaya FP vs FN?
2. **Bangun *baseline***, untuk klasifikasi meteo, *baseline* yang wajar adalah klimatologi (selalu prediksi kelas yang paling sering) atau *persistence*. Ukur metrik langka (*recall*, F1) dari *baseline* dulu.
3. **Siapkan data** - split berbasis waktu (Bab 2), tidak ada *leakage*.
4. **Bangun model** - MLP + ReLU, keluaran sigmoid/softmax, *cross-entropy* (Kode 3.1-3.2).
5. **Evaluasi dengan metrik yang tepat** - *confusion matrix*, *precision/recall*/F1; untuk kejadian langka juga CSI/FAR (Bab 5).
6. **Atur *threshold*** sesuai biaya (Bagian 3.7) dan tampilkan kurva PR/ROC (Bagian 3.9).

Baseline klimatologi untuk klasifikasi mengingatkan kita pada prinsip Bab 1: jangan impresif dengan akurasi tinggi jika kelas langka sama sekali tidak dapat diprediksi. Kerangka di atas akan dipakai berulang di Bab 5 dan Bab 9.

3.12 Galat Umum pada Klasifikasi

1. Melaporkan hanya akurasi. Pada data tidak seimbang, akurasi hampir tak bermakna. Selalu sertakan *confusion matrix* + *precision/recall*/F1 (dan akhirnya CSI/FAR).

2. Mengatur *threshold* tetapi tidak melaporkannya. Hasil *threshold* 0.5 tidak otomatis “standar”; jika Anda menggesernya, tulislah *threshold* yang dipakai agar dapat ditiru.

3. Menggunakan akurasi untuk tuning pada data langka. Optimasi model pada data tidak seimbang sebaiknya memakai metrik yang sesuai (F1/CSI), bukan akurasi.

4. Normalisasi/statistik dari seluruh data. Sama seperti Bab 2 - jangan sampai informasi/statistik dari data test bocor ke dalam proses pelatihan (data *leakage*).

5. Menganggap softmax sebagai “probabilitas sejati”. Softmax hanya peringkat relatif, bukan kalibrasi probabilistik sesungguhnya (model bisa terlalu yakin). Misal, dengan logit [10, 5, 5] softmax memberikan [0.99, 0.005, 0.005], “yakin” ~99% pada kelas pertama, padahal data di lapangan tidak jelas. Ini fenomena yang dikenal sebagai *overconfidence* pada *neural network* [5]. Kalibrasi dibahas singkat di Bab 10.

Dengan menghindari galat ini, laporan klasifikasi Anda jujur dan berguna: nilai kepercayaan yang mahal di dunia operasional.

Ringkasan

- Klasifikasi = prediksi kategori; biner memakai sigmoid, multi-kelas memakai softmax.
- *Cross-entropy* adalah *loss* utama; akurasi menyesatkan pada data tidak seimbang.
- *Precision/recall*/F1 dan CSI/FAR/POD adalah metrik yang lebih sesuai untuk fenomena langka.
- *Threshold* bukan selalu 0.5 - atur sesuai biaya galat (*false alarm* vs *miss*).
- *Confusion matrix* adalah titik awal membaca kinerja; ROC/PR membantu memilih *threshold*.
- Praktik yang benar: *baseline* dulu, split waktu, metrik langka, *threshold* terdokumentasi.

3.13 Latihan

Soal konsep

1. Jelaskan perbedaan keluaran sigmoid vs softmax, dan kapan masing-masing dipakai?
2. Mengapa *cross-entropy* lebih cocok untuk klasifikasi daripada MSE (kuadrat)?
3. Data hujan deras hanya $\pm 3\%$ dari hari (stasiun Cilacap, GHCN-Daily). Mengapa akurasi 97% bisa menyesatkan?
4. Jika biaya *false alarm* rendah tetapi biaya *miss* tinggi, ambang *threshold* apa yang Anda pilih? Jelaskan.

Latihan praktik (notebook `ch-03-02_klasifikasi_hujan.ipynb`)

1. Bangun model biner hujan/tidak hujan; hitung *precision*, *recall*, F1 pada beberapa *threshold* (0.2, 0.5, 0.8) dan buat tabelnya.
2. Latih model multi-kelas intensitas (ringan/sedang/lebat). Catat *confusion matrix*.
3. Bandingkan akurasi vs F1 pada data tidak seimbang; diskusikan mana yang lebih informatif.
4. (Proyek mini) Gunakan data suhu/kelembapan stasiun lokal untuk prediksi hujan besok; laporkan CSI/POD/FAR untuk *threshold* terbaik Anda.

References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
2. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386-408, 1958, doi: 10.1037/h0042519.
3. M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016. [Online]. Available: <https://arxiv.org/abs/1603.04467> (diakses: September 2026).
4. World Meteorological Organization, “WMO guidelines on the verification of operational forecasts,” WMO, Geneva, Switzerland, 2018. [Online]. Available: <https://library.wmo.int> (diakses: September 2026).
5. C. Guo, G. Pleiss, Y. Sun, and K. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. on Machine Learning (ICML)*, PMLR, vol. 70, 2017, pp. 1321-1330. [Online]. Available: <https://arxiv.org/abs/1706.04596> (diakses: September 2026).
6. M. J. Menne et al., “An overview of the Global Historical Climatology Network-Daily database,” *Journal of Atmospheric and Oceanic Technology*, vol. 29, no. 7, pp. 897-910, 2012, doi: 10.1175/JTECH-D-11-00103.1. Data stasiun Cilacap (ID000096805) diunduh dari <https://www.ncei.noaa.gov/pub/data/ghcn/daily/> (diakses: September 2026).

Bab 4 - Backpropagation, Optimasi dan Pelatihan

Prasyarat: Bab 2 (neuron, regresi, MAE/MSE) dan Bab 3 (klasifikasi, cross-entropy). Diperlukan kalkulus dasar: turunan (derivatif); diingatkan ulang di §4.1.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Menjelaskan** mekanisme *gradient descent* dan *backpropagation* (aturan rantai) secara intuitif.
2. **Menganalisis** peran fungsi aktivasi dari sisi gradien (ReLU vs sigmoid/tanh) dan mengenali *vanishing gradient*.
3. **Menerapkan** tuning *hyperparameter* (*learning rate*, *batch size*, *epochs*) dan *callback* (*early stopping*, ModelCheckpoint, ReduceLROnPlateau).
4. **Membaca** learning curve untuk mendiagnosis *underfit/overfit* sebagai transisi ke Bab 5.

4.1 Gradien: Kemiringan sebagai Petunjuk

Sebelum masuk *backpropagation*, kita perlu satu konsep kalkulus: **turunan** (derivatif) adalah kemiringan (slope) suatu fungsi pada satu titik. Jika kita punya fungsi loss $L(\mathbf{w})$ terhadap bobot $\mathbf{w}$, maka:

- Jika $dL/d\mathbf{w}$ positif $\rightarrow$ menaikkan $\mathbf{w}$ akan menaikkan loss $\rightarrow$ kita harus **menurunkan $\mathbf{w}$** .
- Jika $dL/d\mathbf{w}$ negatif $\rightarrow$ menaikkan $\mathbf{w}$ akan menurunkan loss $\rightarrow$ kita harus **menaikkan $\mathbf{w}$** .

Aturan umum: bergerak **berlawanan arah gradien**. Inilah *gradient descent*.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad (4.1)$$

Persamaan (4.1) menyatakan: bobot baru = bobot lama dikurangi (*learning rate*) dikali gradien. *Learning rate* mengontrol besar langkah: terlalu besar $\rightarrow$ melompat melewati minimum; terlalu kecil $\rightarrow$ lambat.

Analogi

Bayangkan Anda berdiri di atas gunung berkabut (ruang bobot) dan ingin turun ke lembah (loss minimum). Anda tidak bisa melihat jauh; yang Anda rasakan

hanya kemiringan di kaki. *Gradient descent*: rasakan arah turun paling curam, ambil langkah, ulangi. Semakin dekat ke lembah, semakin kecil kemiringan, sehingga langkah mengecil, sampai Anda berhenti.

Tiga varian gradient descent

Berdasarkan berapa banyak data yang dipakai untuk satu langkah update:

- **Batch gradient descent** - gunakan *seluruh training set* per langkah. Akurat, tapi sangat lambat untuk data besar.
- **Stochastic gradient descent (SGD)** - gunakan *satu sampel* per langkah. Cepat, tapi sangat berisik (tiap sampel bisa “menarik” ke arah berbeda).
- **Mini-batch gradient descent** - gunakan *subkelompok* kecil (misal 32 sampel) per langkah. **Kompromi terbaik dan yang standar dipakai** - termasuk oleh Keras.

Istilah “SGD” di Keras/PyTorch sebenarnya merujuk pada varian mini-batch: kerangka yang sama, batch kecil. Konsistensi di seluruh buku: kita menyebutnya **batch size** untuk jumlah sampel per langkah.

Loss landscape: bayangan bentuk “lembah”

Untuk satu bobot, loss $L(\mathbf{w})$ berupa kurva; untuk dua bobot, berupa permukaan bergelombang (*loss landscape*). *Gradient descent* menuruni permukaan ini. Dua sifat yang perlu dikenal:

- **Minimum lokal** - lembah kecil tempat kurva turun lalu naik lagi; *gradient descent* bisa “terjebak” di sana padahal ada lembah lebih dalam di tempat lain.
- **Saddle point** - titik datar yang bukan minimum; di sini gradient = 0 sehingga langkah nyaris berhenti.

Kabar baiknya: di ruang berdimensi tinggi (model besar, banyak parameter), minimum lokal “tidak selalu buruk”; banyak di antaranya memberi galat yang hampir sama dengan minimum global sebenarnya [1]. Catatan: temuan ini terutama berlaku untuk model besar; pada model kecil (dimensi rendah), minimum lokal yang buruk lebih sering muncul. Untuk masalah-masalah di buku ini (model kecil, data cukup), algoritma hampir selalu menemukan solusi yang layak. Yang lebih menentukan adalah kualitas data dan fitur, bukan terjebak di satu minimum.

4.2 Backpropagation: Aturan Rantai untuk Seluruh Jaringan

Jaringan saraf punya banyak lapisan; loss berada di ujung (lapisan keluaran), tetapi bobot berada di semua lapisan. Bagaimana cara menghitung pengaruh bobot lapisan dalam terhadap loss? Jawabannya: **aturan rantai** (*chain rule*).

Untuk jaringan 2 lapisan, pengaruh bobot w di lapisan tersembunyi terhadap loss L merantai melalui nilai aktivasi a :

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (4.2)$$

Persamaan (4.2) adalah inti *backpropagation*: galat di lapisan keluaran “dipropagasikan mundur” (*backward*) melalui turunan berantai, memberitahu tiap lapisan seberapa besar sumbangannya terhadap galat akhir. Nama *backpropagation* merujuk persis pada alur ini: hitung galat di depan, lalu **kirim mundur** untuk memperbarui bobot.

Kode 4.1 - Langkah pelatihan manual dengan GradientTape; dalam praktik, *backpropagation* ditangani `model.fit`.

```
import tensorflow as tf

# contoh langkah pelatihan manual dengan GradientTape (untuk pemahaman)
w = tf.Variable(0.5)
b = tf.Variable(0.1)
opt = tf.keras.optimizers.Adam(learning_rate=0.1)

x = tf.constant([1.0, 2.0, 3.0])
y = tf.constant([2.0, 4.0, 6.0])

for step in range(50):
    with tf.GradientTape() as tape:
        pred = w * x + b
        loss = tf.reduce_mean(tf.square(pred - y)) # MSE
        grads = tape.gradient(loss, [w, b])
        opt.apply_gradients(zip(grads, [w, b]))

print("w:", w.numpy(), "| b:", b.numpy(), "| loss:", loss.numpy())
```

Kode 4.1 memperlihatkan otot-otot *backpropagation* secara manual: `GradientTape` mencatat operasi, menghitung gradien ke `[w, b]`, lalu optimizer menggeser nilai. Dalam produk, `model.fit` melakukan semua ini di balik layar, tetapi memahami langkah ini menjelaskan apa yang sebenarnya terjadi.

Kenapa “propagasi” penting untuk jaringan dalam

Tanpa aturan rantai, kita harus menghitung turunan numerik *brute-force* untuk setiap bobot (ribuan hingga jutaan), dan ini sangat mahal. *Backpropagation* menghitung semuanya dalam satu *pass* maju dan satu *pass* mundur, dengan efisiensi yang membuat pelatihan jaringan besar menjadi mungkin. Kertas asli yang memperkenalkan teknik ini adalah Rumelhart, Hinton, dan Williams (1986) [2], dan fondasinya diuraikan lebih lengkap dalam [3].

Catatan terminologi: *backpropagation* adalah penerapan aturan rantai yang khusus pada *neural network*; kerangka modern (TensorFlow/PyTorch) memakai *automatic differentiation* (autodiff), yang lebih umum dan fleksibel, tetapi efeknya bagi Anda sama saja: `model.fit` menghitung semua gradien secara efisien.

Perjalanan sinyal dalam *backpropagation* (ringkasan 2 lapisan)

Untuk memahami alur, ikuti “perjalanan” satu sampel melalui jaringan 2 lapisan:

1. **Forward pass:** $x \rightarrow z^1 \rightarrow a^1$ (ReLU) $\rightarrow z^2 \rightarrow \text{pred} \rightarrow \text{loss}$. Nilai disimpan di tiap lapisan (dibutuhkan nanti).
2. **Backward pass:** $d\text{Loss}/d\text{pred} \rightarrow dz^2 \rightarrow da^1 \rightarrow dz^1 \rightarrow dw^1$. Galat mengalir mundur, dan aturan rantai menggabungkan turunan antar lapisan.
3. **Update:** tiap bobot w dikurangi $\times$ gradien (Pers. 4.1).

Intuisi penting: lapisan tersembunyi “belajar” bukan dari data langsung, melainkan dari **sinyal galat yang dikirim mundur** dari lapisan setelahnya. Inilah sebabnya kedalaman bisa bermakna: setiap lapisan menyesuaikan representasinya agar galat total berkurang.

Contoh numerik sederhana *backpropagation*

Bayangkan jaringan dengan satu neuron $y = w \cdot x$, satu sampel $x=2$, $y_{\text{target}}=4$, dan loss MSE $L = (wx - y_t)^2$ (di sini $n = 1$, sehingga pembagian oleh n gugur). Misal $w = 1$. Maka:

1. **Forward:** $\text{pred} = 1 \cdot 2 = 2$, $L = (2-4)^2 = 4$.
2. **Turunan:** $dL/d\text{pred} = 2(\text{pred} - y_t) = 2(2-4) = -4$; $d\text{pred}/dw = x = 2$.
3. **Chain rule:** $dL/dw = (-4) \cdot 2 = -8$.
4. **Update** ($=0.1$): $w \leftarrow 1 - 0.1 \cdot (-8) = 1 + 0.8 = 1.8$.

Perhatikan: gradien negatif (-8) membuat bobot **naik** ke arah 2, persis “berlawanan arah” prinsip §4.1. Setelah beberapa iterasi w mendekati 2 (mengingat $y_t = 4/2$). Inilah inti loop pelatihan, dan `model.fit` mengulanginya jutaan kali dengan vektor-matriks pada semua bobot sekaligus.

4.3 Fungsi Aktivasi dari Sisi Gradien: Vanishing Gradient

Di Bab 2-3 kita memakai ReLU, sigmoid, dan softmax. Sekarang lihat dari sisi **gradien**: nilai turunan fungsi ini menentukan seberapa cepat bobot di belakangnya berubah.

Untuk sigmoid, turunannya adalah:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) \quad (4.3)$$

Persamaan (4.3) punya sifat penting: $\sigma'(z)$ **selalu** < 1 , dan mendekati 0 ketika z jauh dari 0. Akibatnya, dalam jaringan yang dalam, perkalian berantai dari gradien kecil “menguap”, sehingga lapisan dekat masukan nyaris tidak belajar. Ini disebut *vanishing gradient*.

Ilustrasinya: jika tiap lapisan mengalikan gradien dengan (misal) 0.2, setelah 10 lapisan faktornya $0.2^{10} \approx 1e-7$, praktis nol.

Perbandingan fungsi aktivasi:

Tabel 4.1: Perbandingan fungsi aktivasi dari sisi gradien.

Aktivasi	Rentang	Turunan	Masalah
$\text{sigmoid}(z)$	$(0, 1)$	0.25	<i>Vanishing gradient</i>
$\tanh(z)$	$(-1, 1)$	1	<i>Vanishing</i> tapi lebih baik
$\text{ReLU}(z)$	$[0, \infty)$	1 untuk $z > 0$, 0 untuk $z < 0$; pada $z=0$ tidak terdefinisi tegas (subgradien; praktiknya dipakai 0)	<i>Dying ReLU</i> (neuron mati)
<i>leaky ReLU</i>	$(-\infty, \infty)$	kecil tapi tidak 0	Menghindari <i>dying ReLU</i>

ReLU unggul karena turunannya 1 untuk $z > 0$, sehingga gradien tidak menyusut di bagian positif, dan jaringan dalam dapat belajar. Kelemahannya: untuk $z < 0$ turunan 0, bisa “membunuh” neuron. Leaky ReLU menambal ini.

Kapan *vanishing gradient* relevan di buku ini? Untuk model kecil 1-3 lapisan (Bab 2-3), jarang fatal. Tetapi ketika sampai ke LSTM (Bab 7) dan jaringan yang lebih dalam, pemahaman ini penting, karena LSTM dirancang sebagian untuk mengatasi masalah gradien pada data sekuensial.

Exploding gradient adalah kebalikannya: gradien membesar secara eksponensial (sering pada RNN/LSTM), sehingga bobot melompat jauh dan loss menjadi NaN. Solusi standarnya adalah *gradient clipping*: batasi besarnya gradien (`tf.clip_by_value`, atau `clipvalue/clipnorm` pada optimizer) agar pelatihan tetap stabil. Pada model-model di buku ini (Bab 2-4) kasus ini jarang muncul, tetapi ini sisi lain dari koin yang sama dan akan muncul kembali di Bab 7.

Inisialisasi bobot (Xavier/He) dan vanishing gradient

Satu pertahanan praktis terhadap vanishing/exploding gradient (selain ReLU) adalah **inisialisasi** bobot yang baik. Dua metode yang paling umum:

- **Xavier/Glorot** - menskalakan nilai awal berdasarkan jumlah neuron yang masuk dan keluar dari lapisan, sehingga varians sinyal tetap terjaga. Direkomendasikan untuk sigmoid/tanh. Di Keras: `kernel_initializer="glorot_uniform"`.
- **He** - disesuaikan untuk ReLU, agar varians tetap terjaga di paruh positif. Di Keras: `kernel_initializer="he_normal"`.

Di Keras, inisialisasi yang baik sudah menjadi bawaan **Dense** (itulah sebabnya terasa “langsung jalan”), tetapi Anda bisa memilihnya secara eksplisit lewat `kernel_initializer`. Inisialisasi yang buruk (misal semua bobot nol atau sangat besar) memperparah vanishing/exploding gradient. Ini relevan untuk model yang lebih dalam dan LSTM (Bab 7).

Tanh vs sigmoid: mengapa tanh “lebih baik” di lapisan tengah?

Turunan **tanh** maksimal 1 (tepat di $z=0$, yaitu $\tanh'(0)=1$) (vs sigmoid 0.25), sehingga gradien tidak menyusut secepat sigmoid. Namun maksimal 1 hanya berlaku di $z=0$; untuk $|z|$ lebih besar, turunan mendekati 0 (saturasi). Jadi di jaringan dalam “penguapan” tetap terjadi, hanya lebih lambat. Inilah mengapa ReLU (turunan 1) menjadi pilihan bawaan untuk lapisan tersembunyi di jaringan modern. Titik ini penting saat kita membandingkan arsitektur & fungsi aktivasi di Bab 7.

Kenapa sigmoid masih dipakai di keluaran?

Di Bab 3, lapisan keluaran memakai sigmoid (biner) / softmax (multi-kelas). Mengapa tidak ReLU? Karena kita **ingin** keluaran berupa probabilitas di rentang (0,1), dan cross-entropy + sigmoid “cocok” dari sisi gradien (lihat Bab 3 §3.4). Kesimpulannya: ReLU di **lapisan tersembunyi** (biarkan gradien mengalir), sigmoid/softmax di **lapisan keluaran** (untuk probabilitas). Dua peran berbeda, dan kini Anda paham alasannya.

4.4 Optimizer: SGD vs Adam

SGD (*stochastic gradient descent*) adalah *gradient descent* dasar: perbarui bobot menggunakan gradien dari satu *batch* kecil data (stochastic).

Adam (Adaptive Moment Estimation) adalah versi cerdas yang:

- Menghitung rata-rata gradien (momentum) sehingga langkah lebih halus.
- Menyesuaikan *learning rate* per-parameter berdasarkan riwayat gradien.

Adam diperkenalkan oleh Kingma & Ba (2015) [4]. Praktik umum: **mulai dengan Adam**, karena bekerja baik pada banyak masalah tanpa tuning yang banyak. SGD kadang memberi hasil akhir sedikit lebih baik bila disetel dengan hati-hati [5], tetapi menuntut usaha lebih. Catatan: di sebagian domain (terutama *computer vision*), SGD yang disetel baik (+ momentum) bisa menggeneralisasi lebih baik dari Adam; pada data tabular/meteorologi, selisihnya biasanya

kecil. Untuk buku ini, Adam adalah pilihan bawaan (seperti yang sudah dipakai di Bab 2-3).

Tabel 4.2: Perbandingan SGD dan Adam untuk pemula.

Aspek	SGD	Adam
<i>Learning rate</i>	Perlu di-tuning hati-hati	Lebih toleran (default ~0.001)
Momentum	Tidak otomatis (varian SGD+Momentum)	Otomatis
Kecepatan konvergensi	Lambat	Cepat
Kapan dipakai	Sesudah berpengalaman / model sederhana	Default semua bab

4.5 Hyperparameter Pelatihan

Tiga *hyperparameter* utama yang menentukan perilaku pelatihan:

- ***Learning rate*** () - besar langkah. Terlalu besar: loss melonjak / tidak konvergen. Terlalu kecil: berjalan sangat lambat, mungkin terjebak.
- ***Batch size*** - jumlah sampel per langkah update. Kecil (mis. 16-32): update sering, lebih “berisik”. Besar (mis. 256+): update halus tetapi butuh memori. Catatan: riset menunjukkan batch yang sangat besar sering mengarah ke *sharp minima* (generalisasi lebih buruk), sedangkan batch kecil cenderung menghasilkan *flat minima* (generalisasi lebih baik) [6], sebuah *trade-off* yang relevan untuk meteorologi. Untuk dataset stasiun, batch 32-128 lazim.
- ***Epoch*** - berapa kali model melihat seluruh data latih. Terlalu banyak *epochs* + model besar = *overfit* (Bab 5); terlalu sedikit = *underfit*.

Bagaimana ketiganya berinteraksi?

Learning rate dan *batch size* saling memengaruhi. Batch besar memberi gradien “lebih tenang” sehingga *learning rate* (LR) lebih besar bisa dipakai; batch kecil lebih berisik sehingga LR kecil lebih aman. *Epoch* yang “cukup” tidak bisa diketahui langsung, tetapi dipantau lewat learning curve (val loss), bukan dipatok angka. Inilah mengapa ***callback early stopping*** hampir selalu direkomendasikan: ia menjawab “berapa *epochs*?” dengan melihat data, bukan tebakan.

Aturan jempol untuk memulai

Parameter	Nilai awal	Kapan naik	Kapan turun
<i>Learning rate</i>	0.001 (Adam)	<i>Underfit</i> & mulus	Loss melonjak/fluktuasi keras
<i>Batch size</i>	32	Data sangat besar, mau cepat	Memori kurang / mau stabilitas
<i>Epochs</i>	100 + EarlyStopping	Kurva masih turun tajam	Val naik (<i>overfit</i>)

Efisiensi: kapan mempertimbangkan GPU vs CPU

Di Colab, GPU mempercepat terutama untuk **matriks besar** (model besar, batch besar, banyak data). Untuk masalah kecil di Bab 2-4, selisihnya kecil. Anda tidak perlu berinvestasi pada GPU untuk mengikuti buku ini; Colab menyediakannya gratis (Bab 1). Tips praktis:

- Mulai di **CPU** untuk prototipe cepat & kecil; pindah **GPU** untuk eksperimen panjang (Bab 8-9).
- Kurangi waktu eksperimen: latih pada *subset* kecil dulu, baru data penuh setelah yakin.
- Pantau waktu per *epoch* (`model.fit(..., verbose=1)`) untuk memperkirakan total waktu.

Contoh tuning singkat (kasus Bab 2)

Bayangkan model regresi pasang surut Bab 2 underfit (train & val MAE tinggi). Langkah sesuai panduan: pindah dari LR 0.001 ke 0.01 → amati kurva; tambah neuron 8→32 → amati; aktifkan ReduceLROnPlateau agar LR menurun otomatis. Tiap perubahan **satu-satu** agar Anda tahu apa yang bekerja.

Kode 4.2 - Mengatur *learning rate* dan *batch size* di Keras.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(2,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(1),
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"],
)
```

```

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100, batch_size=32, verbose=0,
)

```

Kode 4.2 memakai Adam(learning_rate=0.001) eksplisit (nilai default); Anda bisa bereksperimen menaikkan/menurunkan untuk melihat efek. Semua contoh di bab ini berjalan di atas TensorFlow [7].

4.6 Callback: Mengotomatiskan Keputusan

Callbacks adalah “fungsi yang dipanggil selama pelatihan” oleh Keras. Tiga yang paling berguna:

- **EarlyStopping** - hentikan pelatihan jika metrik validasi tidak membaik selama *patience epoch* (mencegah *overfit* & menghemat waktu).
- **ModelCheckpoint** - simpan bobot terbaik (misal berdasarkan *val_loss*) ke file, agar tidak kehilangan model terbaik saat *overfit*.
- **ReduceLROnPlateau** - turunkan *learning rate* otomatis jika metrik validasi “stuck”, membantu keluar dari pelatihan yang macet.

Mengapa tiga sekaligus, bukan salah satu? Mereka melengkapi: **EarlyStopping** menentukan *kapan berhenti* , **Checkpoint** memastikan *bobot mana yang disimpan* (yang terbaik, bukan yang terakhir), dan **ReduceLROnPlateau** *menggali lebih dalam* saat sudah dekat minimum. Dipakai bersama, mereka membuat pelatihan hampir *set-and-forget* untuk masalah sederhana.

Satu peringatan: **jangan** memilih arsitektur/*hyperparameter* terbaik berdasarkan nilai terbaik yang pernah terjadi di validasi selama pencarian, lalu melaporkannya sebagai hasil akhir tanpa menguji di test. Melihat validasi berkali-kali membuat validasi “bocor”; itu sebabnya data test dipakai sekali di akhir (Bab 2 §2.7, diulang di Bab 5).

Kode 4.3 - Callback untuk pelatihan yang sehat.

```

import tensorflow as tf

callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True),
    tf.keras.callbacks.ModelCheckpoint("best_weights.keras", monitor="val_loss", save_best_only=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=5),
]

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=200, batch_size=32, callbacks=callbacks, verbose=0,
)

```

```
)
print("Terlatih pada epoch:", len(history.history["loss"]))
```

Kode 4.3: `restore_best_weights=True` memastikan setelah `EarlyStopping`, bobot kembali ke versi terbaik (bukan yang terakhir yang mungkin sudah *overfit*).

Learning rate scheduler

Selain `ReduceLROnPlateau` (otomatis), Keras juga menyediakan scheduler manual:

Kode 4.4 - *Learning rate scheduler* manual.

```
def lr_schedule(epoch):
    return 0.001 * (0.5 ** (epoch // 30)) # halving tiap 30 epoch

callbacks.append(tf.keras.callbacks.LearningRateScheduler(lr_schedule))
```

Scheduler “keras kepala” seperti ini berguna saat Anda sudah tahu bentuk peluruhan yang diinginkan; `ReduceLROnPlateau` lebih adaptif ketika tidak tahu. Keduanya wajar dipakai di dunia nyata.

4.7 Membaca Learning Curve

Setelah pelatihan, `history` berisi loss/metrik per *epoch* untuk train dan validasi. Keduanya di-plot untuk melihat:

- **Train loss turun, val loss juga turun** → model masih belajar (baik).
- **Train loss terus turun, val loss mulai naik** → *overfit*: model menghafal train dan kehilangan generalisasi. Titik di mana val loss mulai naik adalah sinyal berhenti.
- **Keduanya datar/tinggi** → *underfit*: model terlalu sederhana / LR terlalu kecil / data kurang.

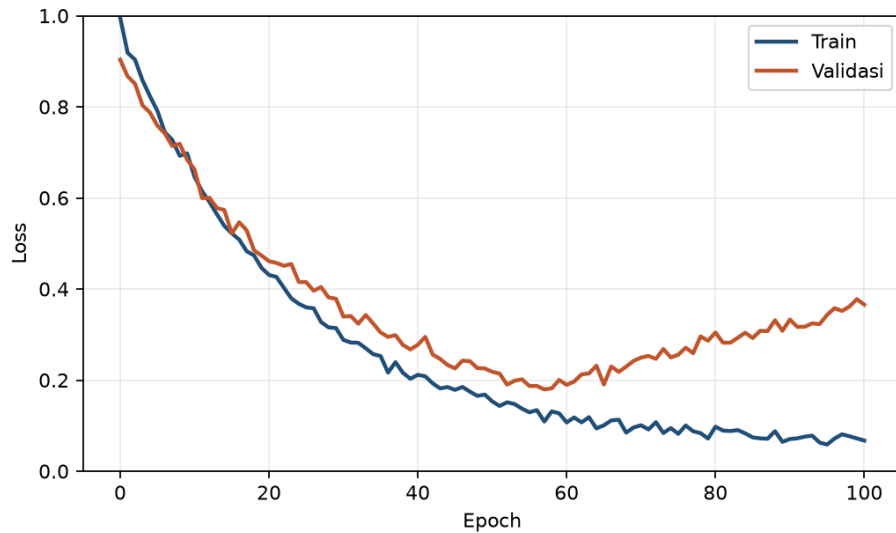
Gambar 4.1: Contoh *learning curve* overfit.

Gambar 4.1 menunjukkan pola *overfit* khas: train terus menurun, validation mulai naik sekitar epoch 60 membentuk “U” terbalik. Di sinilah callback (*early stopping*) berguna, dan Bab 5 memperdalam diagnosis serta pencegahannya (regularisasi).

Apa yang harus dilakukan jika learning curve “aneh”?

Empat pola umum dan tindakannya:

1. **Train turun, val naik** → *overfit*. Coba: *early stopping*, regularisasi (Bab 5), data lebih banyak.
2. **Train & val datar/tinggi** → *underfit*. Coba: *learning rate* lebih besar, arsitektur lebih besar, fitur lebih informatif.



Gambar 7: Gambar 4.1 - Contoh learning curve overfit

3. **Val berfluktuasi keras** → batch terlalu kecil atau LR terlalu besar. Coba kecilkan LR, naikkan batch, atau pastikan data validasi cukup.
4. **Train tinggi, val sangat tinggi** → kemungkinan *leakage* atau skala salah. Periksa *preprocessing* dan *split* (Bab 2, 5, 6).

Kurva adalah alat diagnosis cepat; Bab 5 memberikan alat untuk mengatasinya.

Kapan kurva validasi “tidak wajar”?

Selain empat pola umum di atas, dua hal yang sering membingungkan:

- **Val loss NA/NaN di awal** - biasanya *learning rate* terlalu besar atau skala data rusak (nilai sangat besar/NaN). Periksa *preprocessing*.
- **Val loss lebih rendah daripada train loss** - bisa terjadi karena *dropout* hanya aktif saat pelatihan (Bab 5): train loss dihitung pada model yang “sebagian disesuaikan” (dengan *dropout*), sedangkan val loss dihitung pada model penuh (tanpa *dropout*). Akibatnya val loss bisa tampak lebih rendah. Biasanya bukan masalah.

Membaca kurva adalah keterampilan yang tumbuh lewat eksperimen: makin sering Anda melihat pola, makin cepat Anda tahu perbaikannya.

4.8 Panduan Praktis Memilih Hyperparameter

Tidak ada rumus tunggal, tetapi urutan kerja berikut terbukti berjalan:

1. **Mulai dengan default** - Adam LR 0.001, batch 32, ReLU, 1-3 lapisan

- kecil. Ukur *baseline* (Bab 2) dan kurva awal.
2. **Cek skala data** - pastikan fitur tidak bervariasi liar (normalisasi, Bab 6); skala yang buruk membuat pelatihan lambat/macet.
 3. **Uji *learning rate*** - buat *learning rate finder* sederhana: latih 1 *epoch* per LR, plot loss. Pilih LR sekitar titik penurunan paling curam.
 4. **Naikkan kapasitas hanya jika perlu** - jika *underfit*, tambah neuron/lapisan; jika *overfit*, tambah regularisasi (Bab 5), bukan sekadar mengecilkan model.
 5. **Selalu aktifkan ketiga callback** - EarlyStopping, Checkpoint, ReduceLROnPlateau tetap aktif selama mencoba-coba agar perbandingan adil.
 6. **Cepat bereksperimen di sebagian data** - untuk pengembangan, cukup *subset* data; baru data penuh saat sudah yakin.

Pendekatan “mulai kecil + naikkan bertahap + pantau kurva” jauh lebih efisien daripada mencoba semua kombinasi acak.

Siklus pengembangan model (bukan sekali jalan)

Bab 2-4 memberi Anda satu siklus penuh: bentuk data → pelatihan → evaluasi. Dalam praktik, Anda akan mengulang siklus ini berkali-kali:

masalah → data → baseline → model → evaluasi → (analisis galat) → perbaikan → ...

Analisis galat adalah bagian yang sering dilupakan pemula: setelah model pertama jalan, lihat **di mana yang salah**: misalnya, apakah galat besar terjadi pada hari hujan ekstrem? Fitur apa yang kurang? Pola apa yang model belum tangkap? Perbaikan bisa datang dari data (fitur baru, Bab 6), arsitektur (Bab 7), atau metrik atau *threshold* (Bab 5). Pemodelan sekarang bukan lagi “menulis model sekali”, melainkan iterasi yang cepat dan jujur.

4.9 FAQ

Apakah saya perlu memilih optimizer selain Adam? Untuk buku ini, tidak. Adam cukup untuk Bab 2-10. SGD lebih sederhana dan bisa di-tune lebih baik, tetapi itu topik lanjut.

Mengapa loss kadang “naik turun”? Karena SGD/Adam memakai batch acak; setiap langkah sedikit berisik. Tren menurun dalam jangka panjang, bukan kurva mulus.

Berapa *epochs* “cukup”? Gunakan EarlyStopping + patience (mulai dari 10-20, sesuaikan dengan derau val loss Anda). Jangan menebak.

Apakah *vanishing gradient* sudah mustahil terjadi? Tidak; pada jaringan sangat dalam atau data berurutan panjang (Bab 7), tetap bisa muncul. Arsitektur modern menanganinya lewat residual dan *skip connection*, serta normalisasi, yang dibahas singkat di Bab 10.

Apa beda *val_loss* dan *loss*? *loss* dihitung pada data latih (yang sedang

dilihat); `val_loss` pada data validasi (tidak dilatih). `val_loss` adalah perkiraan generalisasi; di sinilah *overfit* terlihat.

Kapan saya tahu model “cukup dilatih”? Kombinasi: (1) val loss tidak lagi menurun secara berarti, (2) train dan val tidak berbeda jauh (*overfit* kecil), (3) model mengalahkan *baseline* Bab 2. Tidak perlu mengejar loss mendekati nol; itu biasanya tanda *overfit*.

Apakah *backpropagation* perlu saya implementasikan manual? Untuk memakai buku ini, tidak; `model.fit` menanganinya. Memahami mekanismenya (Kode 4.1) membantu saat *debugging* dan membaca literatur, misalnya memahami mengapa *gradient flow* penting di Bab 7.

Yang paling sering salah pemula? Mengubah banyak hal sekaligus tanpa memantau kurva. Ubah satu variabel, amati efeknya, catat. Itu disiplin yang akan dipakai ulang di Bab 5 (tuning regularisasi) dan Bab 8-9 (eksperimen kasus).

4.10 Latihan

Soal konsep

1. Turunkan secara intuitif: mengapa kita bergerak *berlawanan* arah gradien saat *gradient descent*?
2. Apa itu *vanishing gradient* dan bagaimana ReLU membantu?
3. Apa beda *learning rate*, *batch size*, dan *epochs* - dan bagaimana masing-masing memengaruhi pelatihan?
4. Mengapa *early stopping* tidak bisa menggantikan evaluasi yang jujur di test?

Latihan praktik (notebook `ch-04-03_optimasi_callbacks.ipynb`)

5. Pada kasus regresi Bab 2, coba *learning rate* [0.01, 0.001, 0.0001] dan catat kurva loss-nya. Yang mana konvergen? Yang mana macet/melonjak?
6. Bandingkan `batch_size` 16 vs 256; amati perbedaan “berisik” kurva train.
7. Terapkan `EarlyStopping` dan `ModelCheckpoint`; bandingkan *epoch* berhenti dengan tanpa *callback*.
8. Plot learning curve train vs val dan tandai titik *overfit* (jika ada).
9. Ganti aktivasi lapisan tersembunyi menjadi `tanh`; bandingkan konvergensinya dengan ReLU.

Ringkasan

- *Gradient descent* memperbarui bobot berlawanan arah gradien (Pers. 4.1).
- *Backpropagation* memakai aturan rantai (Pers. 4.2) untuk mengirim galat mundur.
- *Vanishing gradient* dihindari dengan ReLU (turunan 1 untuk $z > 0$).

- Adam adalah optimizer default; *learning rate*, *batch size*, *epochs* menentukan keseimbangan.
- *Callback* (EarlyStopping, Checkpoint, ReduceLROnPlateau) mengotomasi pelatihan.
- Learning curve mendiagnosis *underfit/overfit* - jembatan ke Bab 5.
- Praktik baik: *baseline* dulu, ubah satu variabel, pantau kurva, test sekali di akhir.

References

1. H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, “Visualizing the loss landscape of neural nets,” *NeurIPS*, 2018. [Online]. Available: <https://arxiv.org/abs/1712.09913> (diakses: September 2026).
2. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533-536, Oct. 1986, doi: 10.1038/323533a0.
3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
4. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
5. A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, “The marginal value of adaptive gradient methods in machine learning,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, PMLR, 2017. [Online]. Available: <https://arxiv.org/abs/1705.08292> (diakses: September 2026).
6. N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, “On large-batch training for deep learning: Generalization gap and sharp minima,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.04836> (diakses: September 2026).
7. M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016. [Online]. Available: <https://arxiv.org/abs/1603.04467> (diakses: September 2026)

Bab 5 - Overfitting, Regularisasi dan Evaluasi untuk Data Iklim

Prasyarat: Bab 2 (regresi, MAE/MSE), Bab 3 (klasifikasi, metrik), Bab 4 (pelatihan, learning curve, callback). Bab 5 adalah “jembatan” antara kemampuan membangun model dan evaluasi yang jujur untuk operasional.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Mendiagnosa** underfit/overfit melalui learning curve dan konsep bias-variance.
2. **Menerapkan** regularisasi (L2, dropout, early stopping) untuk mencegah overfit.
3. **Memilih** metrik operasional yang tepat (MAE/RMSE/R²/Willmott/KGE dan CSI/FAR/POD/TS) sesuai tujuan.
4. **Menerapkan** cross-validation deret waktu yang benar (walk-forward/blocked) dan mencegah leakage.

5.1 Bias-Variance: Dua Sumber Galat

Setiap model memiliki dua jenis galat struktural. Kerangka ini juga yang dipakai literatur verifikasi operasional, misalnya bagaimana WMO meninjau keandalan metrik perkiraan [1]; rujukan standar verifikasi perkiraan adalah Jolliffe & Stephenson [2], dan untuk deep learning umum (termasuk aspek bias-variance) lihat Goodfellow et al. [3]:

- **Bias tinggi** - model terlalu sederhana, tidak menangkap pola data (underfit). Misal: memakai garis lurus untuk data yang jelas tidak linear.
- **Varians tinggi** - model terlalu sensitif pada data latih; sedikit perubahan data mengubah prediksi besar (overfit). Misal: jaringan sangat besar yang “menghafal” noise.

Ini bukan dua “tipe” yang terpisah, melainkan **trade-off**: saat kapasitas model naik, bias turun tetapi varian naik. Titik keseimbangan terbaik adalah di mana galat total (bias + varian + noise) minimal; dicapai di “titik manis” antara underfit dan overfit.

Analoginya di meteorologi: peramal yang “selalu memprediksi rata-rata klimatologi” punya **bias tinggi namun varian nol** (tidak pernah meleset besar, tapi selalu kurang tajam). Peramal yang “selalu memprediksi kondisi tahun

lalu persis” bisa sangat akurat untuk data yang dihafalnya, tetapi **variannya tinggi**, sehingga memburuk drastis saat kondisi berubah. Kita ingin peramal yang berada di tengah: mengikuti pola nyata tetapi tidak menghafal kebetulan tahun lalu.

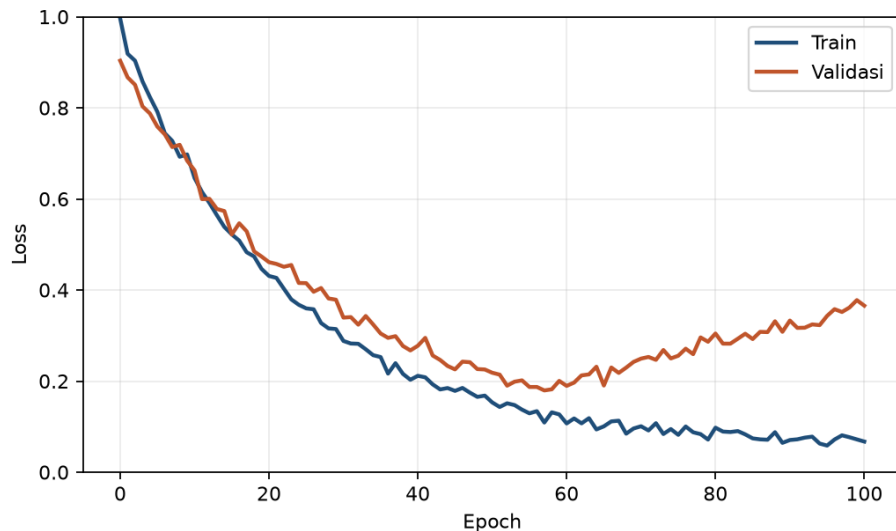
Tabel 5.1: Perbandingan pola underfit, fit baik, dan overfit.

Aspek	Underfit	Fit baik	Overfit
Kapasitas model	Terlalu kecil	Cukup	Terlalu besar
Train galat	Tinggi	Rendah	Sangat rendah (mendekati 0)
Val/Test galat	Tinggi	Rendah	Naik kembali
Penyebab umum	Model/fitur/algoritma sederhana	Tuning tepat	Terlalu banyak parameter/epoch

5.2 Mendiagnosa via Learning Curve

Cara paling langsung melihat underfit/overfit: **plot train galat vs validation galat** terhadap epoch (sudah dikenalkan Bab 4 §4.7):

- **Underfit:** kedua kurva tinggi & datar - model tidak mampu belajar. Solusi: kapasitas lebih besar, fitur lebih baik, LR lebih besar.
- **Overfit:** train terus turun, validation naik setelah titik tertentu. Solusi: regularisasi (§5.3), lebih banyak data, lebih sedikit parameter, early stopping.



Gambar 8: Gambar 5.1 - Learning curve klasik overfit

Gambar 5.1: *Learning curve* klasik overfit.

Gambar 5.1 adalah pola overfit paling umum. Perhatikan titik di mana val mulai naik; itu sinyal bahwa model mulai menghafal.

Untuk **klasifikasi**, kurva yang sama bisa dipakai dengan loss (cross-entropy); untuk regresi, dengan MAE/MSE.

Berapa “jauh” overfit dikatakan buruk?

Tidak ada angka pasti, tetapi indikator umum:

- Jika val galat naik **secara konsisten** (bukan fluktuasi kecil) setelah titik tertentu → sudah overfit.
- Jika train galat mendekati nol sementara val galat jauh lebih besar → overfit jelas.
- Bandingkan dengan **baseline** (Bab 2): jika model overfit masih kalah dari baseline sederhana di validasi, model itu tidak berguna - buang atau perbaiki.

Kunci: kita menilai pada **validasi** (dan akhirnya test), bukan train. Nilai train yang luar biasa rendah adalah jebakan umum.

5.3 Regularisasi: Mencegah Overfit

Regularisasi adalah teknik yang “menahan” model agar tidak terlalu bebas menghafal data. Empat teknik utama:

1. Early stopping (sudah dikenal)

Hentikan pelatihan saat validation mulai tidak membaik (Bab 4 §4.6). Paling mudah dan hampir selalu membantu.

Kode 5.1 - Early stopping dengan restore best weights.

```
import tensorflow as tf

callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=15, restore_best_weights=True
    )
]
```

Nilai `patience=15` di Kode 5.1 hanyalah titik mulai. Nilai optimal bergantung pada seberapa berisik (noisy) kurva validasi Anda: pada data cuaca/hujan yang fluktuasinya kuat, patience yang terlalu kecil bisa menghentikan pelatihan karena fluktuasi, bukan konvergensi sejati; sebaliknya, pada dataset besar dengan loss halus, patience besar hanya membuang waktu. Mulai dengan 10-20, amati kurva, lalu sesuaikan.

2. Regularisasi L2 (weight decay)

Menambahkan penalti ke loss sebanding dengan kuadrat besarnya bobot:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_j w_j^2 \quad (5.1)$$

Persamaan (5.1): λ (lambda) mengontrol kekuatan penalti. Model “didorong” memakai bobot kecil, sehingga tidak terlalu bergantung pada satu fitur. Di Keras:

```
tf.keras.layers.Dense(8, activation="relu", kernel_regularizer=tf.keras.regularizers.l2(1e-4))
```

Catatan untuk pelanjut: istilah “L2” dan “weight decay” baru menjadi benar-benar identik pada SGD vanila. Pada optimizer adaptif seperti Adam, menambahkan penalti L2 ke dalam loss tidak sama dengan *decoupled weight decay* (penalti terpisah pada parameter `weight_decay`, dikenal sebagai AdamW); versi terpisah ini sering lebih stabil [4]. Nuansa ini tidak diperlukan untuk latihan bab ini; cukup pakai `kernel_regularizer`.

3. Dropout

Selama **pelatihan**, dropout mematikan sebagian neuron secara acak (mis. 20-50%). Ini memaksa jaringan belajar dengan “banyak versi” model sehingga lebih robust. Saat **evaluasi**, neuron aktif semua.

```
tf.keras.layers.Dropout(0.3)
```

Kode 5.2 - Contoh arsitektur dengan regularisasi (kernel regularizer L2 + dropout).

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="relu",
                           kernel_regularizer=tf.keras.regularizers.l2(1e-4), input_shape=(n,)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(16, activation="relu",
                           kernel_regularizer=tf.keras.regularizers.l2(1e-4)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(1),
])
```

4. Batch Normalization (sekilas)

Menormalkan aktivasi di dalam jaringan agar pelatihan lebih stabil; sering dipakai bersamaan. Dibahas singkat; paling berguna di jaringan yang lebih dalam (Bab 7).

Kapan memakai yang mana?

- Data kecil + overfit ringan → early stopping + L2 ringan.

- Overfit menengah-berat → tambah dropout.
- Underfit → **jangan** tambah regularisasi; perbaiki kapasitas/fitur dulu.

Aturan penting: **jangan** menambah regularisasi hanya karena “model tidak jalan” tanpa melihat learning curve, yang bisa memperburuk underfit.

Bagaimana regularisasi bekerja secara intuitif?

- **L2** “mengecilkan” bobot: model tidak boleh bertumpu besar pada satu fitur, sehingga lebih tahan terhadap noise pada fitur tertentu (Persamaan 5.1).
- **Dropout** membuat jaringan belajar dengan “ansambel efektif” dari banyak sub-jaringan (neuron acak dimatikan), sehingga tidak ada neuron yang menjadi titik tunggal kegagalan; konsep ini diperkenalkan oleh Srivastava et al. (2014) [5].
- **Early stopping** membatasi *berapa lama* model boleh belajar, mencegah model “menghafal” cukup dalam untuk menangkap noise.

Ketiganya menyerang arah berbeda dari masalah yang sama: terlalu bebasnya model menyesuaikan data latih.

Contoh sederhana efek L2

Bayangkan fitur kelembapan penting untuk hujan. Tanpa L2, model bisa memberi bobot besar pada kelembapan dan mengabaikan yang lain; dengan L2, bobot besar “dikenai biaya” (penalti kuadrat) sehingga model lebih menyebar bobotnya. Efeknya: prediksi lebih stabil saat ada sedikit noise pada pengukuran kelembapan, yang relevan karena data lapangan selalu ber-noise.

Memilih kekuatan regularisasi (lambda & dropout rate)

- **L2 lambda**: mulai dari $1e-4$ – $1e-3$. Terlalu kecil → tidak berefek; terlalu besar → model “terlalu tumpul” (underfit). Lihat kurva.
- **Dropout rate**: mulai 0.2–0.5 untuk lapisan tersembunyi. Terlalu tinggi → model sulit belajar di data kecil; pada data sangat kecil (di bawah ribuan sampel), mulai lebih rendah (mis. 0.1–0.2).
- Aturan: ubah **satu demi satu**, pantau val - seperti tuning hyperparameter di Bab 4.

Perbandingan kode lengkap (Kode 5.2 & 5.3 dipakai bersama)

Kode 5.1-5.2 menunjukkan pola; dalam notebook `ch-05-04_metrik_walkforward.ipynb`, `build_model(reg=True, drop=0.3)` menggabungkan L2 + dropout dan dibandingkan dengan versi tanpa regularisasi pada learning curve serta metrik test; latihan benarnya di §5.8.

5.4 Memilih Metrik Operasional yang Tepat

Sekarang kita masuk bagian yang paling membedakan buku ini dengan buku ML umum: **metrik yang benar untuk meteorologi** bergantung pada fenomena dan tujuan.

Untuk regresi (besaran kontinu)

- **MAE** - galat rata-rata, mudah dipahami (satuan sama). Lebih tahan terhadap pencilan daripada RMSE (tidak mengkuadratkan galat) - tetapi bukan kebal: satu galat ekstrem 100 memberi kontribusi 100, sama seperti 100 galat yang bernilai 1.
- **RMSE** - akar MSE; menghukum galat besar lebih. Selalu MAE; selisihnya menunjukkan bobot galat ekstrem.
- **R²** - proporsi varians yang dijelaskan ($1 - SSE/SST$); bisa negatif jika model lebih buruk daripada sekadar memprediksi rata-rata target. R² negatif pada data uji adalah tanda bahaya (model gagal total), bukan hanya “kurang informatif”; di luar range pelatihan R² umumnya turun tajam karena model belum pernah lihat skala tersebut.
- **Willmott index (d)** - antara 0-1; lebih “ramah” pada data berskala (sering dipakai hidrologi/meteo). Versi original merujuk Willmott (1981) [6]; ada versi modifikasi (Willmott et al., 2012) [7] yang rumusnya sedikit berbeda - catat versi mana yang dipakai saat mengimplementasikan.
- **KGE** (Kling-Gupta efficiency) - menggabungkan korelasi, bias, dan variabilitas; populer di hidrologi; dirinci oleh Gupta et al. (2009) [8].

Sebagai referensi dasar evaluasi & praktik model pada umumnya, lihat pula [3]; untuk verifikasi perkiraan yang lebih mendalam, lihat Jolliffe & Stephenson [2].

Tabel 5.2: Panduan memilih metrik regresi.

Situasi	Metrik yang disarankan
Melaporkan galat khas ke pengguna umum	MAE
Memberi bobot pada galat besar (ekstrem)	RMSE
Menilai kesesuaian pola secara luas	R ² / KGE
Data dengan skala & bias sering dibahas (hidrologi)	KGE / Willmott
Standar laporan operasional meteo	MAE + RMSE (keduanya)

Bahasa setiap metrik dalam satu kalimat

- **MAE**: “rata-rata, sejauh apa prediksi meleset?” (dalam satuan asli).
- **RMSE**: “seberapa besar galat ekstrem ikut berbobot?” (akar kuadrat rata-rata).
- **R²**: “berapa persen variasi data bisa dijelaskan model?” (0-1; bisa negatif).

- **Willmott d**: “seberapa dekat prediksi ke aktual dalam skala 0-1 yang ‘ramah’ terhadap pencilan” - sering digunakan pada laporan hidrologi Indonesia.
- **KGE**: “seberapa baik model menangkap korelasi, tanpa bias, dan variabilitas sekaligus?” (terurai oleh Gupta et al., 2009 [8]).

KGE dirumuskan lewat tiga komponen: korelasi (r), bias rasio, dan rasio variabilitas; nilai 1 = sempurna. Untuk model yang selalu memprediksi rata-rata klimatologi, korelasi = 0 dan rasio variabilitas = 0, sehingga $KGE = 1 - \sqrt{2} - 0.41$, bukan 0 seperti yang kadang keliru disebut. Nilai di bawah baseline itu berarti lebih buruk daripada memakai rata-rata. Konteks ini membuat rekomendasi pemilihan di Tabel 5.2 menjadi masuk akal.

Contoh ukuran cepat untuk memahami skala metrik

Untuk data uji dengan $y = [10, 20, 30]$ dan prediksi $\hat{y} = [12, 19, 29]$:

- $MAE = (2+1+1)/3 = 1.33$
- $RMSE = \sqrt{(4+1+1)/3} = 1.41$
- $R^2 = 1 - (6/200) = 0.97$ (contoh ini hanya untuk memahami skala; dengan 3 data saja, angka R^2 sangat tidak stabil dan tidak mencerminkan performa nyata - selalu uji pada data yang cukup banyak, §5.7)
- Willmott d = 0.99
- (KGE perlu varian pred/obs; dihitung di notebook)

Bandingkan cerita: MAE/RMSE memberi ukuran fisik, R^2/d memberi kualitas pola; keduanya dilaporkan bergantian sesuai tujuan.

Untuk klasifikasi / kejadian langka

Bab 3 sudah mengenalkan precision/recall/F1. Di operasional, kuartet klasik peramal:

- **POD** (*probability of detection*) = $\text{recall} = TP/(TP+FN)$ - “berapa kejadian yang tertangkap?”.
- **FAR** (*false alarm ratio*) = $FP/(TP+FP)$ - “dari yang diumumkan, berapa yang meleset?”.
- **CSI** (*critical success index*) = $TP/(TP+FP+FN)$ - “skor sukses”, menghukum baik miss maupun false alarm.
- **TS** mengacu pada *threat score* - sama dengan CSI.

Tabel 5.3: Kuartet verifikasi klasik WMO.

Metrik	Formula	Pertanyaan	Target
POD	$TP/(TP+FN)$	Kejadian tertangkap?	tinggi
FAR	$FP/(TP+FP)$	Banyak false alarm?	rendah

Metrik	Formula	Pertanyaan	Target
CSI/TS	$TP/(TP+FP+FN)$	Skor sukses keseluruhan	tinggi
(Bias score)	$(TP+FP)/(TP+FN)$	Model terlalu sering/malas prediksi?	~ 1

Pedoman resmi: WMO *Guidelines on the Verification of Operational Forecasts* [1].

Bias score (terlalu sering atau terlalu jarang memprediksi?): Bias score = $(TP+FP)/(TP+FN)$ menjawab: “berapa jumlah kejadian yang diumumkan dibandingkan yang benar terjadi?” Nilai > 1 berarti model *over-forecast*: terlalu sering mengumumkan kejadian, kemungkinan banyak peringatan kosong; nilai < 1 berarti *under-forecast*: terlalu jarang, kemungkinan banyak kejadian terlewat. Perlu diingat: bias score hanya menghitung kecenderungan **jumlah**, bukan ketepatan posisi kejadian (waktu/wilayah); dua model dengan bias score sama bisa jauh berbeda kualitasnya. Karena itu ia selalu dibaca bersama POD/FAR/CSI, bukan berdiri sendiri.

Mengapa ini penting? Data kejadian langka (hujan deras, gelombang tinggi, badai) membuat akurasi menyesatkan (Bab 3). CSI/POD/FAR memberi gambaran yang jujur tentang **nilai operasional** model, bukan sekadar “benar berapa persen”.

Numerik singkat: beda cerita antar metrik

Ambil kasus: dari 100 hari, 10 hari benar-benar hujan deras. Model A dan B:

	TP	FP	FN	Akurasi	POD	FAR	CSI
Model A	8	2	2	96%	0.80	0.20	0.67
Model B	3	0	7	93%	0.30	0.00	0.30

Tabel 5.4: Dua model, cerita yang sangat berbeda.

Model A berakurasi 96% dan POD 0.80, tangkapannya bagus, meski 20% peringatan salah (FAR). Model B tidak pernah salah beri peringatan (FAR 0), tetapi melewatkan 7 dari 10 kejadian (POD 0.30). CSI mengungkap yang sebenarnya: keduanya tidak hebat, dan **tujuan operasional** yang menentukan mana yang lebih diterima (peringatan dini $\rightarrow$ Model A; biaya evakuasi mahal $\rightarrow$ Model B). Ini persis alasan kita tidak bisa berhenti di akurasi.

5.5 Cross-Validation untuk Deret Waktu: Walk-Forward

Cross-validation (bagi-train-val-test berulang) untuk data **independen identik** memakai k-fold acak. Data **deret waktu** tidak independen: nilai berdekatan berkorelasi, dan penggunaan masa depan saat latih = leakage (Bab 2 §2.7).

Solusinya: **walk-forward validation** (juga disebut *forward chaining* atau *expanding window*), yang mensimulasikan penggunaan operasional:

1. Mulai dengan windows latih di awal deret.
2. Prediksi window berikutnya (untuk validasi).
3. Geser batas latih maju; ulangi.

Tabel 5.5: Skema walk-forward (ilustrasi 5 fold; horizon = 20 langkah).

Fold	Train	Validate
1	t1-t100	t101-t120
2	t1-t120	t121-t140
3	t1-t140	t141-t160
4	t1-t160	t161-t180
5	t1-t180	t181-t200

Pada Tabel 5.5, latih selalu **hanya masa lalu**; validasi selalu **di depan** batas latih. Ini mereplikasi kondisi nyata: saat model dipakai, ia hanya tahu data hingga hari ini.

Tabel 5.5 memakai **expanding window**: latih selalu dimulai dari awal deret (t1), hanya batas akhirnya yang bergeser. Alternatifnya: **sliding window**, di mana latih *juga* bergeser (mis. t1-t100, t21-t120, dst). Expanding window memakai semua data sejarah; cocok untuk deret dengan tren jangka panjang, tetapi mulai berat saat deret sangat panjang; sliding window lebih cocok ketika perilaku lama tidak lagi relevan karena distribusi berubah seiring waktu. Pilih sesuai karakter data Anda, dan sebutkan pilihan itu saat melaporkan hasil.

Kode 5.3 - Contoh walk-forward sederhana (pseudo; lengkap di notebook).

```
horizon = 20
results = []
for start in range(0, len(X) - horizon, 20):
    i_end = start + 100          # contoh: latih 100 langkah dari posisi awal
    Xtr_fold, ytr_fold = X[start:i_end], y[start:i_end]
    Xva_fold, yva_fold = X[i_end:i_end+horizon], y[i_end:i_end+horizon]
    model = build_model()       # model baru tiap fold (jujur)
    model.fit(Xtr_fold, ytr_fold, epochs=60, batch_size=32, verbose=0)
    results.append(mae(yva_fold, model.predict(Xva_fold, verbose=0)))
print("Rata-rata MAE walk-forward:", round(sum(results)/len(results), 4))
```

Penting: **latih model baru di tiap fold**; jika Anda melatih sekali & memprediksi semua fold, informasi masa depan bocor.

Catatan: Kode 5.3 (dan contoh di notebook) menggambarkan variant **sliding window** — batas *kiri* latih (**start**) juga bergeser tiap fold. Variant ini valid untuk data tanpa tren jangka panjang; untuk deret dengan tren kuat, versi **expanding window** (latih selalu mulai dari awal deret, hanya batas kanan bergeser, seperti Tabel 5.5) lebih cocok. Kode hampir sama — cukup mantenir **start** = 0 dan hanya berubah **i_end**, misalnya: `for i_end in range(100, len(X) - horizon, 20): Xtr_fold, ytr_fold = X[:i_end], y[:i_end]`.

Kenapa bukan “k-fold acak” untuk data iklim?

Data cuaca menunjukkan **autokorelasi** (nilai hari ini mirip hari kemarin). K-fold acak menempatkan sampel berdekatan di train dan test, sehingga evaluasi “menyontek” dari ketetangaan. Studi kasus Bab 8-9 akan memperlihatkan betapa besar perbedaannya: model yang tampak hebat pada k-fold acak bisa gagal total pada walk-forward, persis perilaku yang akan dialami di produksi.

k-fold (blocked) sebagai alternatif

Jika dataset panjang dan Anda butuh lebih banyak fold, pakai **blocked/rolling k-fold**: potong deret menjadi blok berurutan, lalu untuk tiap fold latih blok-blok sebelum fold validasi tersebut (tanpa melihat masa depan). Ini “saudara” walk-forward dengan jumlah fold tetap. Intinya tetap sama: **validasi selalu mengikuti waktu**, bukan kebalikannya.

Kapan memakai k-fold acak masih ok?

Hanya jika data Anda benar-benar *i.i.d.* (misal koleksi gambar cuaca independen). Untuk deret waktu stasiun, pasang surut, atau hujan; **selalu** walk-forward/blocked. Ini aturan yang akan dipakai keras di Bab 8-9.

Galat umum saat walk-forward

1. **Berbagi preprocessing statistik antar fold** - jika Anda menghitung normalisasi (min/max/z-score) dari seluruh data sebelum membagi fold, masa depan “bocor” ke latih. Solusi: hitung statistik **hanya dari fold latih**, lalu terapkan ke validasi-fold (Bab 6 memperdalam).
2. **Memakai model yang di-fit sekali untuk semua fold** - setiap fold harus punya model sendiri yang dilatih hanya dengan data sebelum fold tersebut (Kode 5.3).
3. **Menghitung metrik pada campuran semua fold** - laporkan per-fold atau rata-rata; jangan mengumpulkan prediksi semua fold menjadi satu set (karena fold awal dan akhir tidak sebanding).

5.6 Menerapkan Evaluasi: Alur Lengkap

Untuk menutup Bab 5, rangkum evaluasi yang jujur dalam urutan:

1. **Split** (bab 2): train / val / test berbasis waktu; test hanya dipakai sekali.
2. **Baseline** (bab 1-2): ukur model sederhana (persistence, klimatologi, linear) dulu.
3. **Latih & pantau** (bab 4): learning curve; early stopping saat val berhenti membaik.
4. **Diagnosa** (bab 5): overfit? $\rightarrow$ regularisasi; underfit? $\rightarrow$ kapasitas/fitur.
5. **Evaluasi** (bab 5): regresi $\rightarrow$ MAE/RMSE/ R^2 /KGE (dan Willmott); kejadian $\rightarrow$ CSI/POD/FAR + threshold.
6. **Validasi temporal** (bab 5): walk-forward untuk mengukur generalisasi seiring waktu.
7. **Lapor** (bab 10): angka + satuan + threshold + keterbatasan, banding dengan baseline.

Langkah 5-6 adalah “izin keluar” sebelum klaim apa pun: jangan pernah melaporkan “prediksi akurat 97%” tanpa metrik yang sesuai konteks (KGE/CSI, bukan akurasi dangkal).

Analisis residu: melihat di mana model gagal

Metrik memberi satu angka; **residu** (selisih aktual-prediksi) menunjukkan *di mana*. Cara diagnostik sederhana yang sering mengungkap masalah:

- **Plot residu terhadap waktu** - apakah ada musim atau bulan yang selalu meleset? (mis. model buruk di puncak hujan karena data tidak seimbang.)
- **Plot residu terhadap fitur penting** - misal, apakah galat membesar ketika kelembapan sangat tinggi?
- **Distribusi residu** - residu yang condong kuat (skew) menandakan model bias sistematis (misal selalu memprediksi terlalu rendah pada hujan besar).

Cara membacanya: residu yang berayun mengikuti musim (mis. selalu positif di musim hujan, selalu negatif di musim kemarau) mengisyaratkan model gagal menangkap pola musiman; tambahkan fitur musiman (bulan, hari dalam tahun) atau interaksi. Residu yang membesar saat kelembapan sangat tinggi mengisyaratkan hubungan non-linear; coba transformasi target atau fitur interaksi. Pola sistematis yang masih terlihat di residu = sinyal bahwa ada struktur yang belum dipelajari model.

Analisis residu hampir selalu menghasilkan ide perbaikan: fitur baru, transformasi target (mis. $\log(y+1)$ untuk hujan), atau threshold yang lebih sesuai. Inilah “analisis galat” yang disebut di Bab 4, dan merupakan keterampilan paling berharga seorang praktisi model cuaca.

Kapan sebuah model “lulus”?

Tidak ada formulasi tunggal; kerangka penilaian:

1. **Lulus teknis** - metrik sesuai target & mengalahkan baseline di walk-forward.
2. **Lulus operasional** - metrik memenuhi tolok ukur layanan (mis. toleransi pasang ± 0.10 m, CSI > 0.4 untuk peringatan dini).
3. **Lulus praktis** - model dapat dipelihara (data tersedia, retraining terjadwal, interpretasi cukup bagi pengguna, Bab 10).

Hanya ketika ketiganya terpenuhi model layak diusulkan untuk produksi. Bab 8-9 mempraktikkan ketiga mata uji ini pada kasus nyata.

Memilih rangkaian eksperimen yang efisien

Ketika mencari perbaikan (regularisasi, kapasitas, fitur), rancang eksperimen agar jawabannya jelas:

- **Satu perubahan per eksperimen** - ubah dropout saja, bandingkan; lalu L2, dst. Mengubah banyak variabel sekaligus membuat Anda tak tahu penyebab perbedaan.
- **Tetapkan metrik keputusan tunggal** - misal: “val MAE” untuk regresi, “CSI pada periode validasi” untuk kejadian - dipakai konsisten di semua eksperimen.
- **Catat hasil di tabel** - kolom: konfigurasi, train/val metrik, parameter. Ini menjadi dokumen jejak keputusan (berguna saat laporan operasional, Bab 10).
- **Gunakan seed tetap** (Bab 1) agar perbedaan antar eksperimen bukan karena acak.

Tabel eksperimen bisa dimulai sesederhana menyalin output notebook ke satu lembar catatan; disiplin ini menghemat banyak waktu di Bab 8-9 ketika studi kasus menuntut banyak percobaan.

5.7 Membaca Metrik Secara Kritis

Angka metrik tidak berdiri sendiri. Sebelum melaporkan, tanyakan:

1. **Satuan & konteks** - MAE 0.05 m untuk pasang surut baik atau buruk? Bandingkan dengan toleransi operasional (mis. ± 0.10 m).
2. **Dibanding baseline?** - metrik hanya bermakna relatif terhadap persistence/klimatologi.
3. **Metrik apa yang diukur?** - akurasi untuk fenomena langka menyensatkan; gunakan CSI/POD/FAR (regresi: KGE/MAE/RMSE sesuai tujuan).
4. **Apakah angka stabil?** - satu run bisa beruntung; gunakan beberapa seed atau walk-forward untuk melihat varians.

5. **Threshold mana?** - POD/FAR bergantung threshold; selalu laporkan threshold yang dipakai.

Disiplin ini, bukan sekadar “akurasi tinggi”, yang membedakan laporan yang bisa dipercaya di dunia operasional meteorologi.

FAQ singkat

Kapan KGE lebih baik daripada R^2 ? Jika Anda peduli pada bias/skala (khas hidrologi & peramalan), KGE memisah korelasi, bias, dan variabilitas. R^2 hanya korelasi-pola; model dengan bias besar tapi pola bagus bisa R^2 tinggi padahal tidak akurat.

Apakah dropout membuat model selalu lebih baik? Tidak. Dropout menambah regularisasi; bagus untuk overfit, tetapi bisa memperburuk underfit. Terapkan sesuai diagnosis.

Apakah walk-forward yang lebih lambat itu wajib? Untuk klaim evaluasi pada data iklim, ya. Kecepatan bisa ditingkatkan dengan model kecil / subset; kejujuran tidak bisa dikompromi.

Apa beda CSI dan TS? Dalam praktik peramalan, keduanya merujuk rumus yang sama; TS (*threat score*) adalah nama lama untuk CSI.

Mengapa metrik “nilai rata-rata” tidak cukup? Rata-rata menyembunyikan distribusi: MAE 2 mm bisa berarti “selalu meleset 2 mm” atau “sempurna kecuali beberapa hari ekstrem”. Laporkan distribusi galat (kuantil, plot) bila penting.

Apakah saya perlu melaporkan semua metrik? Tidak, pilih yang informatif untuk tujuan Anda (Tabel 5.2) dan sertakan baseline + threshold. Lebih baik sedikit angka yang bermakna daripada banyak angka yang membingungkan.

5.8 Latihan

Soal konsep

1. Jelaskan bias-variance trade-off dengan analogi meteorologi.
2. Bagaimana learning curve membantu memutuskan apakah menambah dropout atau mengurangi kapasitas?
3. Mengapa RMSE MAE selalu? Apa implikasi untuk data hujan ekstrem?
4. Kapan memakai R^2 vs KGE? Jelaskan kelemahan R^2 pada data di luar range pelatihan.

Latihan praktik (notebook `ch-05-04_metrik_walkforward.ipynb`)

5. Ambil model klasifikasi Bab 3; evaluasi dengan POD/FAR/CSI di beberapa threshold. Buat tabel ringkasnya.

6. Bangun model regresi Bab 2; hitung MAE, RMSE, R^2 , Willmott d, dan KGE (gunakan fungsi sederhana). Diskusikan perbedaan cerita tiap metrik.
7. Terapkan regularisasi (L2 + dropout) pada model yang overfit; bandingkan learning curve sebelum/sesudah.
8. Terapkan walk-forward (fold) pada data pasang surut Bab 2 dan bandingkan MAE rata-rata dengan split tunggal.
9. (Proyek mini) Tulis fungsi evaluasi reusable: input (y_true, y_pred) → output MAE/RMSE/ R^2 /KGE atau POD/FAR/CSI. Simpan untuk Bab 8-9.

Ringkasan

- Overfit = varian tinggi (menghafal data latih); underfit = bias tinggi (terlalu sederhana); keduanya saling trade-off (bias-variance).
- Learning curve adalah alat diagnosis utama: train terus turun + val naik → overfit; keduanya tinggi & datar → underfit.
- Regularisasi: early stopping (kapan berhenti), L2 (kecilkan bobot, Persamaan 5.1), dropout (latih “ansambel” neuron acak) - terapkan berdasarkan diagnosis, bukan asal.
- Metrik regresi: MAE (galat khas), RMSE (bobot galat ekstrem), R^2 (pola), Willmott (skala ramah), KGE (korelasi+bias+variabilitas).
- Metrik kejadian langka: POD (kejadian tertangkap), FAR (alarm palsu), CSI/TS (skor sukses); akurasi menyesatkan.
- Cross-validation deret waktu: walk-forward/blocked (validasi selalu setelah latih) - bukan k-fold acak; jaga preprocessing per-fold agar tidak bocor.
- Evaluasi jujur = metrik + satuan + threshold + baseline + walk-forward + analisis residu.

References

1. World Meteorological Organization, “WMO guidelines on the verification of operational forecasts,” WMO, Geneva, Switzerland, 2018.
2. I. T. Jolliffe and D. B. Stephenson, *Forecast Verification: A Practitioner’s Guide in Atmospheric Science*, 2nd ed. Chichester, UK: Wiley, 2011, doi: 10.1002/9781119960003.
3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
4. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.
5. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929-1958, 2014.
6. C. J. Willmott, “On the validation of models,” *Phys. Geogr.*, vol. 2, no. 2, pp. 184-194, 1981.

7. C. J. Willmott, S. M. Robeson, and K. Matsuura, "A refined index of model performance," *Int. J. Climatol.*, vol. 32, no. 6, pp. 573-580, 2012.
8. H. V. Gupta, H. Kling, K. K. Yilmaz, and G. F. Martinez, "Decomposition of the mean squared error and NSE performance criteria: Implications for improving hydrological modelling," *J. Hydrol.*, vol. 377, no. 1-2, pp. 80-91, 2009, doi: 10.1016/j.jhydrol.2009.08.003.

Bab 6 - Data Meteorologi: Sumber, Kualitas dan Persiapan

Prasyarat: Bab 1 (lingkungan Colab), Bab 2 (split berbasis waktu, MAE/MSE), Bab 5 (walk-forward, anti-*leakage*). Bab ini bersifat praktis: banyak kode, sedikit teori.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

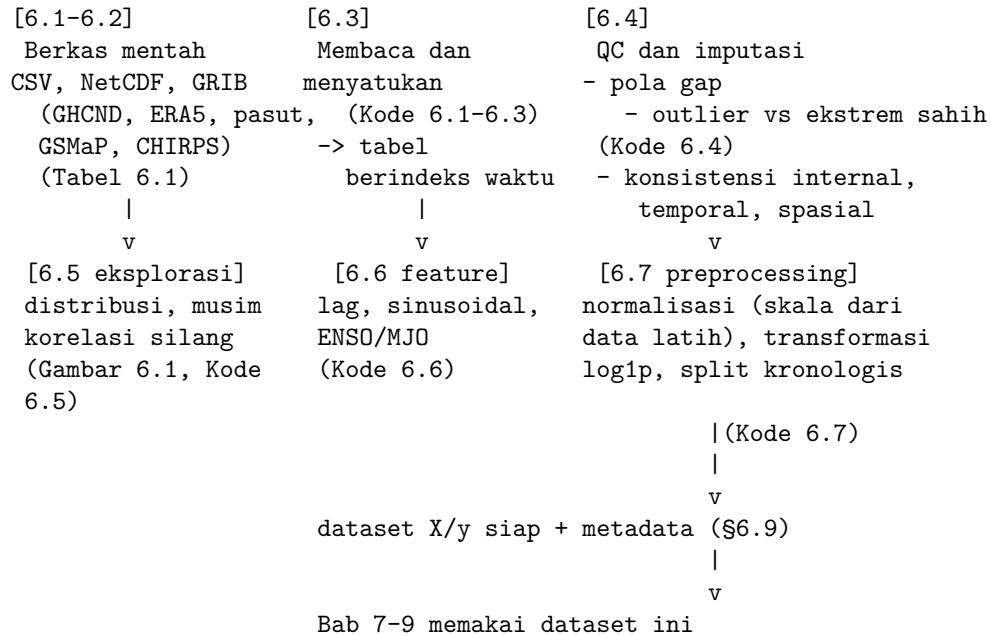
1. **Mengambil dan menghubungkan** data meteorologi Indonesia (stasiun dan grid terbuka, reanalysis ERA5, pasang surut, satelit) beserta lisensi dan batasannya.
2. **Membaca** format berkas CSV, NetCDF, dan GRIB; menangani nilai hilang, pencilan, dan imputasi dasar.
3. **Melakukan eksplorasi** (dekomposisi musiman, distribusi, korelasi silang) dan *feature engineering* (deret tunda, indikator musiman, ENSO/MJO).
4. **Menerapkan** normalisasi (dipasang hanya pada data latih) dan split berbasis waktu yang mencegah *leakage*.

6.1 Mengapa Data Menentukan Segalanya

Bab 2-5 membangun model; bab ini kembali ke fondasi: **data**. Di dunia meteorologi, ungkapan *garbage in, garbage out* terasa sangat nyata; model sehebat apa pun tidak berguna jika masukannya salah. Urutan ini sengaja: menunda bab data sampai setelah pembaca mengenal konsep *leakage*, split berbasis waktu, dan metrik evaluasi (Bab 2 dan 5) membuat alasan mengapa data harus bersih menjadi konkret, bukan abstrak. Tiga kenyataan yang perlu dipahami sejak awal:

1. **Data meteorologi tidaklah bersih.** Sensor rusak, nilai hilang, stasiun pindah lokasi, dan pencilan (ingat distribusi hujan yang berekor panjang di Bab 2 dan 5) adalah hal biasa.
2. **Data adalah deret waktu.** Urutan waktu bermakna; kita tidak boleh mengacak, memotong sembarangan, atau membiarkan informasi masa depan “bocor” ke masa lalu (Bab 2 §2.7, Bab 5 §5.5).
3. **Sumber data punya aturan.** Lisensi, batasan penggunaan, dan cara kutip berbeda antar lembaga. Memahami aturan ini bagian dari etika riset (Kriteria Sitasi, bagian 3).

Bab ini memberi peta sumber data + keterampilan teknis mengubahnya menjadi *dataset* yang siap dilatih, persis yang akan dipakai di Bab 7-9. Peta alurnya:



Alur 6.1 - Pipeline data dari berkas mentah ke *dataset* siap dilatih (dipakai kembali di Bab 8-9). Setiap langkah punya “Kode 6.x” dan angka-aturan yang akan diacu nanti; jika suatu bagian terasa abstrak, loncat duluan ke “Studi mini” di §6.7 yang merangkai semuanya sekaligus.

6.2 Sumber Data Meteorologi Indonesia

Berikut sumber yang paling relevan untuk buku ini, diurutkan dari yang paling sering dipakai.

Tabel 6.1: Sumber data utama untuk buku ini.

Sumber	Jenis data	Resolusi	Akses	Catatan lisensi & kutip
GHCN-Daily (NOAA)	Observasi suhu, hujan, dll. - stasiun global termasuk Indonesia	Harian, per stasiun	ncei.noaa.gov/pub/data/ghcn/daily [1]	(domain publik AS); kutip Menne et al. [1]

Sumber	Jenis data	Resolusi	Akses	Catatan lisensi & kutip
ERA5 / ERA5-Land (Copernicus)	<i>Reanalysis</i> suhu, hujan, angin, dll.	$\pm 0.25^\circ$ (~31 km) / $\pm 0.1^\circ$ (~9 km), per jam	Copernicus Climate Data Store [2]	Lisensi terbuka (setara CC-BY); kutip Hersbach et al. [3]
CMIP6	Proyeksi iklim (skenario)	Lebih kasar, bulanan-harian	ESGF / Copernicus	Untuk konteks jangka panjang, Bab 10
PSMSL / IOC / BIG	Muka laut / pasang surut	Menit-jam, per stasiun	psmsl.org [4], tides.big.go.id [5]	Gratis; IOC non-komersial (CC BY-NC 4.0); sertakan rujukan data & bottle/stasiun
GSMaP (JAXA)	Hujan satelit+kalibrasi	0.1° , 3 jam-harian	sharaku.eorc.jaxa.jp [11]	Kutip paper pembuat
CHIRPS (CHC UCSB)	Hujan satelit+kalibrasi	$\pm 0.05^\circ$ (~5 km), harian	CHC UCSB [6]	Domain publik; kutip paper pembuat

Catatan penting: stasiun GHCND [1] (termasuk sejumlah stasiun Indonesia pada jaringan GSN) adalah sumber “kebenaran lokal” yang terbuka, tetapi tidak merata spasial dan kadang bergap. ERA5 [2][3] memberikan cakupan grid lengkap dan konsisten, tetapi merupakan *model* (taksiran); bukan observasi murni. Praktik umum: gabungkan observasi stasiun (untuk akurasi) dengan *reanalysis* (untuk fitur regional yang lengkap). Bila stasiun tidak tersedia, titik grid satelit-gauge seperti CHIRPS [11][6] bisa menjadi target yang reproduksibel. Cara menggabungkan ini dibahas di §6.3-6.6.

Memahami *reanalysis* secara singkat

Reanalysis adalah hasil *running* model cuaca (misal IFS milik ECMWF) sepanjang sejarah sambil **menyerap observasi** (stasiun, balon udara, satelit) secara konsisten [3]. Hasilnya: peta cuaca lengkap setiap jam sejak puluhan tahun lalu, meski di tempat tanpa pengamatan. Ini bukan “ramalan” masa lalu, melainkan

perpaduan model + data terbaik yang tersedia. Untuk *machine learning*, *re-analysis* sering menjadi sumber fitur regional yang tidak dimiliki stasiun.

Kapan memakai data satelit hujan?

Untuk wilayah yang minim stasiun (laut, pulau terpencil, Indonesia timur), pengamatan hujan berbasis satelit merupakan alternatif praktis. **GSMaP** (JAXA) [11] dan **CHIRPS** [6] menggabungkan sinyal satelit inframerah/pasif-mikro dengan kalibrasi stasiun, menghasilkan grid hujan yang cukup baik untuk kajian regional.

Catatan penggunaan di buku ini:

- **Kapan dipakai:** sebagai fitur pelengkap atau pengganti saat stasiun bergap panjang.
- **Kapan hati-hati:** estimasi satelit bisa bias di wilayah pantai dan gunung; verifikasi terhadap stasiun setempat bila memungkinkan.
- **Jangan dipakai sebagai *target*** jika stasiun observasi tersedia - konsistensi target lebih penting (target dari sumber yang sama menjaga makna evaluasi).

Bab 6 memperlakukan satelit sebagai “sumber bonus”, bukan pengganti stasiun.

Catatan praktis akses data

Lima hal yang perlu diketahui sebelum mengunduh (berlaku untuk semua sumber di Tabel 6.1):

1. **ERA5 vs ERA5-Land:** Tabel 6.1 mencantumkan resolusi keduanya. Untuk studi skala lokal (pertanian, hidrologi, wilayah kecil), **ERA5-Land (0.1°)** umumnya lebih cocok daripada ERA5 (0.25°) - ERA5-Land adalah versi *downscaled* yang didorong ulang oleh ERA5 dengan resolusi lebih halus.
2. **Tidak semua data *real-time*:** ERA5 versi akhir tersedia dengan **tunda sekitar 2-3 bulan** (versi sementara *near-real-time* **ERA5T** lebih cepat tetapi belum divalidasi penuh). GSMaP juga punya beberapa rilis: versi *near-real-time* (**GSMaP_NRT**, tunda sekitar 4 jam) dan versi *reanalysis* (**GSMaP_RI**, tunda jauh lebih lama). Untuk pelatihan *offline* latensi ini tidak mengganggu, tetapi penting saat data dipakai untuk aplikasi operasional *near-real-time*.
3. **“0.25° 31 km” hanya berlaku di sekitar khatulistiwa:** di lintang tinggi, jarak per derajat bujur menyusut. Karena Indonesia praktis berada di ekuator, angka 31 km cukup akurat untuk buku ini - tetapi jangan menganggapnya universal.
4. **Tidak semua data stasiun punya API publik:** beberapa stasiun nasional (misal di Indonesia) memerlukan permintaan/izin resmi yang bisa berbayar. Untuk data yang benar-benar terbuka, gunakan **GHCN-Daily** [1] (stasiun global) atau titik grid **CHIRPS/GSMaP** [6][11] sebagai

pengganti; kualitas data stasiun juga **bervariasi** (bergap panjang, sensor galat, atau pindah lokasi). Selalu baca metadata stasiun dan dokumentasi sebelum mengunduh (lihat §6.9).

5. **Kutipkan sumber tepat:** cantumkan nama produk, versi, dan tanggal unduh sesuai §6.9 - sebagian data (CDS, JAXA) mewajibkan kredit khusus untuk publikasi.

Menggabungkan observasi dan *reanalysis* (praktik yang disarankan)

Strategi yang dipakai di Bab 8-9 adalah menerapkan dua peran secara terpisah:

- **Observasi stasiun** $\rightarrow$ *target* (y): yang ingin diprediksi (hujan, pasang surut).
- **Reanalysis ERA5** $\rightarrow$ *fitur regional* (x): suhu, angin, kelembapan, dan variabel grid di sekitar stasiun untuk memperkaya konteks atmosfer yang tidak tercatat di stasiun.

Alasan pemisahan ini: melatih model untuk mereproduksi observasi langsung (bukan taksiran model) menjaga makna evaluasi; kita mengukur seberapa baik model menebak kenyataan, bukan menebak tebakan lain. Fitur regional dari *reanalysis* sah sebagai masukan karena tersedia secara konsisten dan tidak “men-curangi” target.

6.3 Format Berkas: CSV, NetCDF, dan GRIB

Tiga format yang paling sering ditemui:

- **CSV** - tabel teks sederhana; paling mudah dibaca (pandas), cocok untuk data stasiun harian.
- **NetCDF** - format biner ilmiah dengan metadata kaya (dimensi, koordinat, atribut); standar untuk *reanalysis* (ERA5). Dibaca dengan **xarray**.
- **GRIB** - format biner khas meteorologi operasional (prediksi model); juga bisa dibaca xarray (via **cfgrib**) atau wgrib2.

Tabel 6.2: Perbandingan format berkas.

Format	Baca cepat?	Metadata	Ukuran	Umum dipakai untuk
CSV	Mudah (pandas)	Minim	Besar (teks)	Data stasiun harian
NetCDF	Ya (xarray/netCDF4)	Kaya	Kompak	ERA5, model
GRIB	Ya (cfgrib/wgrib2)	Kaya	Kompak	Prediksi operasional

Kode 6.1 - Membaca NetCDF dengan xarray (contoh ERA5 suhu harian).

```
import xarray as xr

ds = xr.open_dataset("era5_suhu_harian.nc")
```

```
d = ds["t2m"] # variabel suhu 2 m
print(d.shape, d.attrs.get("units"))
```

xarray mempertahankan **label koordinat** (waktu, lintang, bujur), sehingga mengiris wilayah atau periode jauh lebih terbaca daripada array mentah NumPy. Untuk data stasiun dalam CSV, `pandas.read_csv` dan parse tanggal ke `datetime` adalah langkah pertama yang biasa.

Untuk GRIB (prediksi model operasional), dua jalur umum: xarray dengan engine `cfrib` untuk eksplorasi cepat, atau `wgrib2` untuk ekstraksi presisi pada skala besar. Catatan: engine `cfrib` (berserta pustaka `eccodes` di belakangnya) perlu diinstal terpisah dari xarray; `pip install cfrib` biasanya mencukupi di Colab, tetapi kegagalan instalasi pada beberapa sistem sering menjadi titik hambatan pertama pembaca. Jika `pip` saja gagal, resep yang biasa berhasil di Colab: instal dulu pustaka sistem `libeccodes0`, baru `cfrib`:

```
!apt-get install -y libeccodes0
!pip install cfrib
```

(Alternatif: `!pip install cfrib eccodes`; coba resep `apt` bila instalasi `pip` gagal.)

Kode 6.2 - Membaca GRIB dengan xarray + engine cfrib.

```
import xarray as xr
```

```
ds = xr.open_dataset("prakiraan.grib", engine="cfrib")
```

Konvensi nama variabel: ERA5 menggunakan nama seperti `t2m` (suhu 2 m), `tp` (total *precipitation*), `u10/v10` (angin 10 m). Selalu cek atribut `units`; mengubah satuan tanpa sadar adalah sumber galat klasik.

Menyatukan banyak berkas menjadi satu tabel

Pola yang akan berulang di Bab 8-9: baca banyak berkas → resample ke frekuensi yang sama → gabung menjadi satu `DataFrame` berindex waktu.

Kode 6.3 - Menyatukan ERA5 per jam menjadi tabel harian.

```
import xarray as xr
import pandas as pd
```

```
# ERA5 per jam -> menjadi harian: hujan (akumulatif) pakai sum()
ds = xr.open_dataset("era5_per_jam.nc")
d_harian = ds["tp"].resample(time="D").sum()
s = d_harian.sel(latitude=-0.01, longitude=109.34, method="nearest").to_pandas()
```

Resample harus disesuaikan dengan sifat variabel: variabel **akumulatif** seperti hujan (`tp`) memakai `sum()`, sedangkan variabel **instan** seperti suhu/angin (`t2m`, `u10`, `v10`) memakai `mean()`. Salah memilih agregasi adalah sumber galat data tersembunyi yang luput dari plot deret.

Menggabungkan stasiun (target) dan *reanalysis* (fitur) lewat indeks tanggal adalah operasi `merge/join` yang harus diperiksa hasilnya agar tidak ada baris yang hilang diam-diam; periksa ulang jumlah baris dan rentang tanggal sebelum membangun model.

6.4 Kualitas Data: Nilai Hilang, Pencilan, dan Imputasi

Setelah data terbaca, langkah berikutnya adalah *quality control* (QC).

Nilai hilang (*missing values*)

Sensor mati, komunikasi terputus, atau galat logging menghasilkan celah. Cara pertama bukan menebak, melainkan **memahami polanya**:

- Gap kecil dan acak → imputasi sederhana cukup.
- Gap panjang (berhari-hari) → hati-hati; imputasi bisa menyesatkan; pertimbangkan memotong periode tersebut atau memakai model terpisah.
- Gap sistematis (mis. stasiun hanya mencatat pada jam kerja) → perlu penanganan khusus.

Kode 6.4 - Cek dan isi nilai hilang dasar dengan pandas.

```
import pandas as pd

df = pd.read_csv("curah_hujan_stasiun.csv", parse_dates=["tanggal"])
df = df.set_index("tanggal")
print("Nilai hilang:", df.isna().sum())

# Interpolasi linear sederhana untuk gap pendek
df["r_hujan"] = df["r_hujan"].interpolate(method="linear", limit=3)

# Atau isi dengan nilai hari sebelumnya (ffill = forward fill) untuk gap 1 hari
df["suhu"] = df["suhu"].ffill()
```

Catatan: interpolasi linear (Kode 6.4) cocok untuk suhu (mulus) tetapi **kurang pas** untuk hujan (banyak nol, lonjakan). Untuk hujan, imputasi konservatif (misal isi 0 bila hari kering di sekitarnya, atau NaN tetap dibiarkan untuk model yang tahan) sering lebih jujur.

Selain interpolasi, ada keluarga metode imputasi yang **belajar dari data lain**: regresi dengan stasiun tetangga (isi gap suhu memakai korelasi stasiun terdekat), imputer berbasis tetangga terdekat (KNN), atau *iterative imputation* (MICE). Ini relevan ketika gap panjang atau banyak variabel saling berkorelasi. Bab 8-9 menggunakan imputasi sederhana untuk menjaga alur tetap jelas; metode berbasis model (KNN/MICE) bisa menjadi langkah lanjutan saat data sudah lebih besar dan banyak variabel berkorelasi.

Pencilan (*outlier*)

Pencilan bisa berupa galat (sensor) atau nilai ekstrem sah (hujan >200 mm/hari). Bedakan dengan konteks:

- Suhu 70 °C di Indonesia → hampir pasti salah, bisa diganti NaN.
- Hujan 300 mm/hari → mungkin nyata; **jangan** otomatis hapus.

Cara cepat mendeteksi: plot deret, statistik ringkas, dan *rule of thumb* (misal nilai di luar $\text{median} \pm 5 \times \text{MAD}$). Untuk studi kasus Bab 8-9, pendekatan yang dipakai adalah “jangan menghapus ekstrem sah; pahami apakah model meny-erapnya secara wajar”; ekstrem itulah yang sering paling penting diprediksi.

Contoh QC numerik sederhana

Misalkan satu stasiun mencatat `r_hujan = -3.2, 0, 0, 255.0, 2.0, 0` (enam hari).

- -3.2 → mustahil secara fisik (nilai negatif) → buang/ganti NaN.
- 255.0 → mungkin ekstrem sah di Indonesia (belum tentu salah) → **tahan dulu**, verifikasi dengan stasiun tetangga atau catatan klimatologi.
- Interval 0,0 → wajar di musim kering.

Aturan praktis: galat fisik (negatif, suhu >60 °C) dihapus; ekstrem yang masuk akal secara fisis dipertahankan sampai ada bukti salah. Menghapus ekstrem sah agar model “tampak bagus” adalah bentuk kecurangan evaluasi; di dunia nyata ekstrem itu tetap terjadi dan harus diprediksi.

Cek nilai fisik saja tidak cukup; QC yang baik juga memeriksa **konsistensi**:

- **Internal**: nilai yang saling bertentangan dalam satu stasiun (mis. suhu minimum di atas suhu maksimum hari yang sama, atau arah angin ‘N’ di samping ‘S’ yang mencerminkan sensor rusak).
- **Temporal**: lompatan tak wajar antar waktu yang berurutan (mis. lonjakan suhu 20°C dalam satu jam tanpa front udara yang jelas) patut dicurigai.
- **Spasial**: bandingkan dengan stasiun tetangga - satu nilai hujan 300 mm/hari sementara lima stasiun sekitarnya kering mungkin salah; tetapi ingat perbedaan lokal tetap bisa sah (badai sel tunggal), jadi verifikasi sebelum membuang [12].

Tujuan QC bukan membersihkan data sembarangan, tetapi **menandai nilai yang tidak bisa dipercaya** tanpa menghilangkan sinyal ekstrem yang sah.

6.5 Eksplorasi: Memahami Pola Sebelum Membangun Model

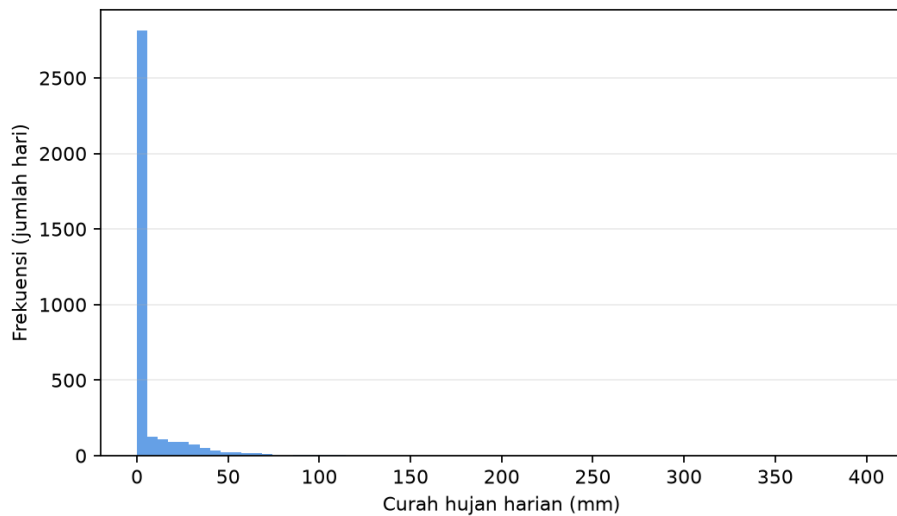
Eksplorasi yang baik mencegah model yang salah arah. Empat hal yang hampir selalu dilakukan untuk data deret waktu meteorologi:

1. Dekomposisi musiman

Data cuaca punya siklus harian, bulanan, dan musiman (monsun). Memisahkan $tren + musiman + residu$ (misal `seasonal_decompose` di statsmodels) membantu melihat apakah pola musiman kuat, dan mengingatkan bahwa model perlu fitur musiman (§6.6).

2. Distribusi data

Hujan (Gambar 6.1) berbentuk *berat di nol* dengan ekor panjang ke kanan; suhu lebih mirip lonceng. Distribusi menentukan pilihan *loss* (Bab 2), transformasi target, dan metrik yang jujur (Bab 5).



Gambar 9: Gambar 6.1 - Distribusi curah hujan harian

Gambar 6.1: Distribusi curah hujan harian.

3. Korelasi silang

Sebelum menambahkan fitur, lihat korelasi (Pearson) maupun korelasi silang dengan deret tunda. Fitur yang berkorelasi tinggi dengan target lebih menjanjikan; fitur yang saling berkorelasi kuat (multikolinearitas) kurang bermasalah di neural network daripada di regresi klasik, tetapi tetap perlu dipahami.

4. Stasioneritas & musim

Deret cuaca umumnya **tidak stasioner** (rata-rata dan varians berubah musiman). Model sekuensial (Bab 7) bisa menangkap pola ini dari data, tetapi membantu jika kita berikan indikator musiman eksplisit.

Kode contoh dekomposisi dan korelasi silang

Kode 6.5 - Dekomposisi musiman dan korelasi silang singkat.

```
import pandas as pd
from statsmodels.tsa.seasonal import seasonal_decompose

# 1) dekomposisi hujan bulanan (resample "ME" = akhir bulan)
bulanan = df[["r_hujan", "suhu"]].resample("ME").agg({"r_hujan": "sum", "suhu": "mean"})
hasil = seasonal_decompose(bulanan["r_hujan"], model="additive")
hasil.plot()

# 2) korelasi silang suhu(t-k) vs hujan(t): cari lag terbaik
korelasi = pd.Series(
    {k: bulanan["suhu"].shift(k).corr(bulanan["r_hujan"]) for k in range(0, 150)},
    name="korelasi",
).sort_index()
lag_terbaik = korelasi.abs().idxmax()
print("Lag suhu dengan korelasi tertinggi:", lag_terbaik)
```

Dekomposisi (Kode 6.5) menampilkan komponen musiman; segmen kedua menghitung korelasi silang suhu tunda $t-k$ terhadap hujan t untuk memilih deret tunda yang menjanjikan sebelum memasukkannya ke model (hubungan hujan-suhu di daerah tropis bersifat negatif: suhu turun saat hujan, sehingga korelasi maksimum akan berharga negatif; yang penting adalah lag-nya). Pada data hujan, `seasonal_decompose` bersifat *additive* dan bisa kasar jika distribusi masih sangat miring; gunakan transformasi (`log1p`, §6.7) atau *modeling* multiplikatif bila hasilnya berisik.

6.6 Feature Engineering untuk Data Meteorologi

Feature engineering adalah keterampilan yang paling cepat mengangkat performa model. Untuk deret waktu meteorologi, beberapa fitur yang terbukti berguna:

Deret tunda (*lag*)

Target $y(t)$ dijelaskan oleh nilai sebelumnya $y(t-1)$, $y(t-2)$, ... (Bab 2 §2.5 sudah memakai 2 deret tunda). Deret tunda menangkap *persistensi* dan *autokorelasi*.

Indikator musiman

Nomor hari dalam tahun (1-366), bulan, atau fungsi sinus/kosinus dari hari Julian (misal `sin(2 · doy/365.25)`, `cos(...)`) memberi model tahu “kapan dalam tahun ini”. Fungsi sinus/kosinus dipakai agar siang-malam dan pergantian tahun kontinu, bukan melompat.

Indeks iklim: ENSO dan MJO

Fitur **regional** meningkatkan prediksi hujan Indonesia secara signifikan:

- **ENSO** (El Niño-Southern Oscillation): indeks Nino3.4 atau MEI mengukur anomali suhu muka laut Pasifik [7]; Indonesia cenderung lebih kering saat El Niño.
- **MJO** (Madden-Julian Oscillation): bit fase & amplitudo MJO (mis. RMM1, RMM2 dari Wheeler & Hendon [8]) berhubungan dengan osilasi hujan 30-60 hari di wilayah tropis.

Kedua indeks tersedia gratis (NOAA, BoM). Memasukkan mereka sebagai fitur adalah contoh nyata “pengetahuan domain meningkatkan model”, sesuatu yang dimiliki praktisi meteo tetapi umumnya tidak dimiliki mahasiswa CS.

Kode 6.6 - Membangun fitur lag, musiman, dan indeks iklim.

```
import numpy as np

# 1) deret tunda
for lag in [1, 2, 3, 7, 14]:
    df[f"hujan_t{lag}"] = df["r_hujan"].shift(lag)

# 2) musiman sinusoidal
doy = df.index.dayofyear
df["mus_sin"] = np.sin(2 * np.pi * doy / 365.25)
df["mus_cos"] = np.cos(2 * np.pi * doy / 365.25)

# 3) indeks iklim (contoh: MJO RMM1 & RMM2 digabung dari berkas eksternal)
df = df.join(rmm.set_index("tanggal"), how="left")
```

6.7 Normalisasi dan Split Berbasis Waktu

Normalisasi: pasang hanya pada data latih

Model di Bab 2-5 dilatih dengan Adam yang sensitif pada skala. Normalisasi *z-score* adalah pilihan umum:

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (6.1)$$

Sangat penting: dan dihitung **hanya dari data latih**, lalu diterapkan ke validasi, *test*, dan data produksi (Persamaan 6.1). Jika dihitung dari seluruh data, informasi masa depan “bocor”; bentuk *leakage* yang paling sering luput (Bab 5 §5.5).

Kode 6.7 - Normalisasi dengan skala dari data latih + split berbasis waktu.

```

from sklearn.preprocessing import StandardScaler

feat = [c for c in df.columns if c not in ("r_hujan",)] # jangan sertakan target
scale = StandardScaler().fit(df[feat].iloc[:n_train])

X_train = scale.transform(df[feat].iloc[:n_train])
X_val    = scale.transform(df[feat].iloc[n_train:n_train+n_val])
X_test   = scale.transform(df[feat].iloc[n_train+n_val:])

```

Transformasi target untuk data miring (hujan)

Distribusi hujan yang berekor panjang (Gambar 6.1) membuat model sulit memprediksi besar dengan baik: galat pada hari 200 mm “menenggelamkan” galat pada ratusan hari kecil. Salah satu cara yang umum digunakan praktisi: transformasi monotonik seperti $\log_{1p}(y) = \log(y + 1)$ pada *target* sebelum dilatih, lalu eksponensialkan kembali saat melaporkan:

$$y_{\text{train}} = \log(y + 1) \quad (6.2)$$

Transformasi (Persamaan 6.2) meredam ekor kanan, membuat distribusi target lebih “kecil” dan pelatihan lebih stabil. Kehati-hatian yang perlu:

- Transformasi berlaku pada **target** (dan bisa pada fitur positif); harus dibalik kembali (`np.exp`) sebelum menghitung metrik agar MAE/RMSE dalam mm yang sesungguhnya.
- Metrik dihitung pada **skala asli**, bukan pada skala log, supaya dapat dibandingkan dengan *baseline* dan dipahami pengguna.
- Tidak semua masalah butuh transformasi: untuk pasang surut (data musus) tidak perlu. Cek distribusi dulu (Gambar 6.1 + §6.5).

Bab 9 akan menerapkan transformasi ini pada prediksi hujan harian.

Split berbasis waktu

Ulangi aturan Bab 2 & 5: **jangan acak**. Potong deret secara kronologis:

`train` (2000–2015) | `validasi` (2016–2018) | `test` (2019–2021)

Untuk evaluasi temporal yang jujur, gunakan *walk-forward* (Bab 5). Pada Bab 8–9, aturan ini menjadi penentu kredibilitas hasil. Kerangka umum representasi data, *preprocessing*, dan evaluasi model yang dipakai sepanjang buku dapat dirujuk pada literatur dasar deep learning [9] dan panduan verifikasi prediksi [10].

Catatan split vs transformasi: urutkan pekerjaan dengan benar; transformasi target dihitung dengan statistik **dari bagian latih saja** (seperti / normalisasi), diterapkan ke validasi/*test* dengan statistik tersebut; lalu lakukan *walk-forward*. Mencampur statistik seluruh data adalah *leakage*.

Studi mini: dari berkas mentah ke X/y siap dilatih

Merangkai seluruh bab dalam satu alur yang akan dijadikan *template* (peta visualnya ada di **Alur 6.1**, §6.1):

1. Baca stasiun (CSV) + *reanalysis* (NetCDF) → gabung per tanggal (Kode 6.1-6.3).
2. QC & imputasi pilihannya (Kode 6.4).
3. Eksplorasi distribusi & musiman (Kode 6.5, Gambar 6.1).
4. *Feature engineering*: lag, musiman, ENSO/MJO (Kode 6.6).
5. Buang baris yang masih bermuatan NaN dari lag awal (sebelum fitur pertama tersedia).
6. Normalisasi dengan skala latih + split berbasis waktu (Kode 6.7).
7. Simpan hasil (`np.save` atau `parquet`) untuk diisi Bab 7-9.

Kode 6.8 - Menyimpan hasil akhir untuk bab berikutnya.

```
import numpy as np

np.savez("dataset_pasang_hujan.npz",
        X_train=X_train, y_train=y_train,
        X_val=X_val, y_val=y_val,
        X_test=X_test, y_test=y_test,
        tanggal=df_ml.index.values)
```

Menyimpan (Kode 6.8) memudahkan memuat ulang di notebook yang berbeda tanpa mengulang keseluruhan pipeline, penting ketika bab berikutnya fokus pada model, bukan data.

6.8 FAQ Singkat

Apakah saya wajib pakai xarray? Untuk NetCDF/GRIB, ya, sangat disarankan. Untuk CSV stasiun, `pandas` cukup.

Berapa banyak data “cukup” untuk deep learning deret waktu? Tidak ada angka baku; untuk stasiun harian, makin lama periode makin baik (1-2 dekade sudah wajar untuk kasus buku ini). Lebih penting daripada “banyak”: data **bersih** dan *split* yang jujur.

Haruskah indeks iklim (ENSO/MJO) selalu ditambahkan? Tidak selalu. Uji dulu: tambah, bandingkan metrik validasi; jika perbaikannya bermakna, pertahankan. Untuk prediksi hujan Indonesia, dampak ENSO/MJO sering terasa nyata (Bab 9).

Apa beda *imputation* dan *interpolation*? *Imputation* = mengisi nilai hilang dengan metode apa pun (statistik, model); *interpolation* adalah salah satu caranya (mendasarkan pada tetangga waktu). Istilah sering dipakai bergantian; yang penting pola gap menentukan pilihan.

6.9 Praktik Metadata dan Reprodusibilitas

Setiap *dataset* yang dibangun harus bisa **direproduksi dan dijelaskan**; aturan Kriteria Sitasi bagian 3 menuntut identifikasi dataset, versi, dan cara akses. Catat dalam berkas `README_data.md` (atau sel notebook) hal berikut:

1. **Sumber & versi**: berkas stasiun mana (GHCND/CHIRPS), ERA5 level produk (reanalysis, ERA5-Land?), tanggal unduh.
2. **Lisensi & kutip**: lisensi pihak penyedia + DOI/rujukan paper (misal [3][6]).
3. **Transformasi yang diterapkan**: satuan asli $\rightarrow$ satuan akhir, resample harian, imputasi, transformasi target.
4. **Baris & rentang tanggal** tiap split; seed acak (bila ada).
5. **Hash berkas sumber** (mis. md5) agar perubahan diam-diam terdeteksi.

Praktik ini (jadi “jejak data”) bukan birokrasi; ia yang memungkinkan pembaca memperdebatkan, meniru, dan menilai hasil Anda secara jujur, dan akan dipakai penuh di Bab 8-9.

6.10 Latihan

Soal konsep

1. Mengapa menggabungkan observasi stasiun dan *reanalysis* lebih baik daripada hanya salah satunya? Apa kelemahan tiap sumber?
2. Jelaskan mengapa imputasi interpolasi linear cocok untuk suhu tetapi tidak untuk hujan.
3. Apa bentuk *leakage* ketika normalisasi dihitung dari seluruh data sebelum split?
4. Mengapa fitur ENSO/MJO bisa relevan untuk prediksi hujan di Indonesia?
5. Mengapa latensi (keterlambatan ketersediaan) sebuah dataset menentukan apakah ia cocok untuk aplikasi operasional *near-real-time*? Sebutkan satu contoh dataset yang cocok dan satu yang tidak cocok untuk kasus tersebut (§6.2, butir 2).
6. Apa perbedaan kegunaan antara ERA5 dan ERA5-Land untuk studi skala lokal di Indonesia? (§6.2, butir 1).

Latihan praktik (notebook `ch-06-05_persiapan_data.ipynb`)

7. Ambil data hujan harian stasiun (atau data contoh yang disediakan); lakukan QC dan eksplorasi (distribusi, dekomposisi musiman).
8. Bangun fitur: lag 1,2,3,7,14; musiman sinus; gabungkan indeks MJO/ENSO (berkas contoh).
9. Normalisasi dengan skala latih; split berbasis waktu; dokumentasikan jumlah baris dan rentang tanggal tiap split. Untuk masing-masing berkas sumber, catat tanggal observasi terakhirnya dalam tabel kecil, lalu hitung selisih harinya terhadap tanggal terakhir *test*; tuliskan dalam satu kalimat apakah selisih itu membatasi model jika dijalankan secara operasional.

10. (Proyek mini) Buat pipeline data reusable (berkas + fungsi) untuk dipakai di Bab 8-9: input tanggal & stasiun → output X/y bersih siap dilatih.

Ringkasan

- Data menentukan hasil: pahami sumber, lisensi, dan cara kutip sebelum membangun model.
- GHCN-Daily (observasi stasiun), ERA5 (*reanalysis*), CMIP6 (proyeksi), PSMSL (pasang surut), dan GSMaP/CHIRPS (hujan satelit/grid) adalah sumber utama buku ini (Tabel 6.1); gunakan observasi/grid sebagai *target*, *reanalysis* sebagai fitur regional.
- Format CSV untuk stasiun; NetCDF/GRIB (xarray) untuk grid; selalu cek satuan dan nama variabel (Tabel 6.2).
- Sebelum mengunduh, pahami latensi tiap produk (ERA5 vs ERA5T, GSMaP rilis berbeda), beda ERA5 vs ERA5-Land, cakupan “km” yang bergantung lintang, serta cara akses data stasiun nasional yang sering tanpa API publik (§6.2).
- QC: bedakan nilai hilang (polanya) dan pencilan (salah vs ekstrem sah) sebelum mengisi; hapus galat fisik, pertahankan ekstrem yang masuk akal, dan cek konsistensi internal/temporal/spasial (contoh di §6.4).
- Eksplorasi: distribusi (Gambar 6.1), dekomposisi musiman, korelasi silang - dasar *feature engineering*.
- Fitur: deret tunda (lag), musiman sinusoidal, dan indeks ENSO/MJO memperkuat model hujan.
- Normalisasi (/ dari data latih saja, Persamaan 6.1) dan transformasi target $\log_{10}$ untuk data miring (Persamaan 6.2); split berbasis waktu dan *walk-forward*.
- Catat metadata & hash berkas agar *dataset* dapat direproduksi (dipakai ulang Bab 8-9).

References

1. M. J. Menne et al., “An overview of the Global Historical Climatology Network-Daily database,” *Journal of Atmospheric and Oceanic Technology*, vol. 29, no. 7, pp. 897-910, 2012, doi: 10.1175/JTECH-D-11-00103.1. Data: <https://www.ncei.noaa.gov/pub/data/ghcn/daily/> (Accessed: Sep. 2026).
2. Copernicus Climate Change Service (C3S), “ERA5: fifth generation ECMWF atmospheric reanalysis of the global climate,” Copernicus Climate Data Store, [Online]. Available: <https://cds.climate.copernicus.eu> (Accessed: Sep. 2026).
3. H. Hersbach et al., “The ERA5 global reanalysis,” *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999-2049, 2020, doi: 10.1002/qj.3803.
4. Permanent Service for Mean Sea Level (PSMSL), “Global sea level data,” [Online]. Available: <https://psmsl.org> (Accessed: Sep. 2026).

5. Badan Informasi Geospasial (BIG), "Peta pasang surut dan pola pasut perairan Indonesia," [Online]. Available: <https://tides.big.go.id> (Accessed: Sep. 2026).
6. C. Funk et al., "The climate hazards infrared precipitation with stations - a new environmental record for monitoring extremes," *Scientific Data*, vol. 2, p. 150066, 2015, doi: 10.1038/sdata.2015.66.
7. K. Wolter and M. S. Timlin, "Monitoring ENSO in COADS with a seasonally adjusted principal component index," in *Proc. 17th Climate Diagnostics Workshop*, 1993, pp. 52-57.
8. M. C. Wheeler and H. H. Hendon, "An all-season real-time multivariate MJO index: development of an index for monitoring and prediction," *Monthly Weather Review*, vol. 132, no. 8, pp. 1917-1932, 2004, doi: 10.1175/1520-0493(2004)132<1917:AARMMI>2.0.CO;2.
9. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
10. S. Jolliffe and D. B. Stephenson, *Forecast Verification: A Practitioner's Guide in Atmospheric Science*, 2nd ed. Chichester, UK: Wiley, 2011, doi: 10.1002/9781119960003.
11. K. Okamoto et al., "The global satellite mapping of precipitation (GSMaP) project," in *Proc. IEEE Int. Geoscience and Remote Sensing Symp. (IGARSS)*, vol. 5, Seoul, South Korea, 2005, pp. 3414-3416, doi: 10.1109/IGARSS.2005.1526538.
12. World Meteorological Organization (WMO), *Guide to Instruments and Methods of Observation* (WMO-No. 8), 2021 ed., World Meteorological Organization, Geneva, Switzerland, 2021. [Online]. Available: <https://community.wmo.int/activity-sites/meteorological-and-hydrological-service-wmo-8> (Accessed: Sep. 2026).

Bab 7 - Deret Waktu dan Model Sekuensial: RNN, LSTM, GRU

Prasyarat: Bab 2 (regresi, baseline, split waktu), Bab 5 (metrik, walk-forward), Bab 6 (data meteorologi, fitur). Bab ini menyiapkan model sekuensial untuk studi kasus Bab 8-9.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Menyusun** deret waktu menjadi contoh-*window* untuk prediksi satu dan beberapa langkah ke depan.
2. **Membandingkan** LSTM/GRU dengan *baseline* (persistence, mean, AR) secara jujur.
3. **Menjelaskan** intuisi $RNN \rightarrow LSTM \rightarrow GRU$ (pintu ingatan/lupa) beserta keterbatasannya.
4. **Memilih** arsitektur masukan univariate/multivariate dan strategi prediksi multi-langkah (recursive/direct/seq2seq).

7.1 Mengapa Deret Waktu Istimewa

Bab 2-5 memperlakukan data sebagai deretan contoh yang saling bebas. Data **deret waktu** berbeda dalam dua hal mendasar:

1. **Urutan bermakna.** Nilai $y(t)$ bergantung pada riwayat sebelumnya: $y(t-1)$, $y(t-2)$, dst. Membaca bagan hujan hari ini tanpa melihat hari kemarin kehilangan informasi penting.
2. **Korelasi temporal.** Beberapa variabel cuaca seperti suhu dan tekanan relatif mulus dan *autocorrelated*: hari ini mirip kemarin. Hujan, namun, bersifat intermiten, zero-inflated, dan sering berisik; angin bisa gusty. Ini kabar baik untuk model (ada pola), tetapi juga perangkap; *split* acak (Bab 2, 5) dan *leakage* harus dihindari ketat.

Sifat khusus lain di meteorologi: deretnya **tidak stasioner** (musim, pola monsun), menyimpan siklus (pasang surut, siklus harian), dan kadang mengandung *regime* (misal transisi musim hujan→kering). Model yang baik untuk bab ini harus bisa menangkap ketiganya, dari data, bukan dari asumsi. Kerangka umum representasi dan pelatihan model deret waktu dapat dirujuk pada literatur dasar [1].

Ukur *autokorelasi* dengan mudah

Cara cepat mengenali potensi model sekuensial: hitung **ACF** (autocorrelation function) melalui `pandas.Series.autocorr(lag)`:

```
import pandas as pd
for lag in [1, 7, 14, 30]:
    print(lag, sr.autocorr(lag))
```

Nilai tinggi di `lag=1` menandakan *persistence* kuat (bisa jadi pesaing berat LSTM); nilai tinggi di `lag>1` menunjukkan struktur periodik (siklus) yang bisa dipelajari model. Hasil ACF inilah yang membantu memilih `w` (window) dan memprediksi seberapa kuat *baseline* persistence nanti.

7.2 Menyusun Deret Waktu Jadi Data *Machine Learning*

Model sekuensial membaca data dalam bentuk **jendela** (*window*): beberapa langkah waktu masa lalu sebagai masukan, satu (atau beberapa) langkah ke depan sebagai target.

Bayangkan deret harian $y(1), y(2), \dots, y(N)$. Untuk *window* `w=3` dan *horizon* `h=1`:

Contoh	Masukan [t-3, t-2, t-1]	Target [t]
1	y(1), y(2), y(3)	y(4)
2	y(2), y(3), y(4)	y(5)
3	y(3), y(4), y(5)	y(6)

Tabel 7.1: Contoh windowing deret ke contoh-*window* (`w=3`, `h=1`).

Contoh numerik windowing

Ambil deret [10, 12, 14, 13, 11, 9, 10], `w=3`, `h=1`. Contoh yang terbentuk:

- [10, 12, 14] → 13 (pakai 3 hari pertama, target hari ke-4)
- [12, 14, 13] → 11
- [14, 13, 11] → 9
- [13, 11, 9] → 10

Total contoh = $N - w - h + 1 = 7 - 3 - 1 + 1 = 4$. Perhatikan: contoh pertama hanya berguna setelah 3 hari pertama berlalu; setuju dengan naluri: butuh riwayat untuk memprediksi.

Bagaimana overlap antar window memengaruhi keacakan

Window yang “menggeser satu langkah” (seperti contoh di atas) membuat contoh-contoh berdekatan saling tumpang-tindih dan **sangat berkorelasi**

(tidak independen). Ini wajar untuk data cuaca, tetapi berimplikasi: jangan pernah membagi window secara acak; *standard* memakai *split* berdasarkan waktu (Bab 2/6) agar train tidak “menyimpan” salinan test.

Konversi ini, *windowing*, identik dengan yang sudah dipakai di Bab 2 (§2.5), hanya kini masukannya berupa **urutan panjang w** , bukan dua fitur terpisah. Bentuk tensor masukan untuk model sekuensial adalah 3D:

$$\text{input shape} = (\text{batch}, \text{waktu } w, \text{fitur } f) \quad (7.1)$$

Persamaan (7.1): dimensi pertama adalah jumlah sampel per *batch* (otomatis di Keras), dimensi kedua adalah panjang *window* w , dimensi ketiga jumlah fitur f . Untuk univariate $f=1$; untuk multivariate $f>1$ (misal hujan + suhu + kelembapan).

Kode 7.1 - Membuat *window* dari deret dengan TensorFlow.

```
import numpy as np
import tensorflow as tf

def buat_window(deret, w=7, h=1):
    X, y = [], []
    for i in range(len(deret) - w - h + 1):
        X.append(deret[i:i+w, np.newaxis])    # (w,) -> (w, 1) fitur
        y.append(deret[i+w:i+w+h])
    return np.array(X), np.array(y)

# contoh: deret suhu harian sintetik
ts = np.sin(np.arange(100) / 5) + np.random.randn(100) * 0.1
X, y = buat_window(ts, w=7, h=1)
print(X.shape, y.shape)    # (93, 7, 1) dan (93, 1)
```

Pilih w (berapa hari riwayat) dengan naluri domain + eksperimen: untuk pasang surut, beberapa siklus (misal $w=72$ jam $\times 3 = 216$ jam) membantu menangkap periodisitas; untuk hujan, 7-30 hari lazim menangkap musim pendek.

Berapa panjang *window* yang baik?

Tidak ada jawaban universal, tetapi tiga pertimbangan membantu:

1. **Siklus alami data** - jika data punya siklus 24 jam (misal siklus harian suhu), sertakan minimal satu siklus ($w \geq 24$). Untuk pasang surut, jenis siklus ditentukan dulu: *semi-diurnal* punya periode 12,42 jam (dua kali sehari), *diurnal* 24 jam, dan *tidal day* 24,84 jam. w setidaknya mencakup satu periode siklus agar model bisa “melihat” pola naik-turun.
2. **Harga komputasi & data** - w besar berarti tensor lebih besar dan jumlah contoh ($N - w - h + 1$) lebih sedikit. Untuk stasiun dengan ribuan hari, w ratusan jam masih wajar.

3. **Eksperimen** - uji $w \in \{3, 7, 14, 30\}$ di *walk-forward* dan pilih yang MAE-nya terbaik pada validasi, bukan pada *test*.

Contoh: dua opsi window untuk data jam-an pasang surut:

Nama	w (jam)	Makna	Catatan
Pendek	6	seperempat hari	kecepatan, tapi tak lihat siklus penuh
Standar	24	satu siklus harian	untuk semi-diurnal dua siklus; titik awal yang baik

Tabel 7.2: Pilihan panjang window untuk deret jam-an.

Satu langkah vs beberapa langkah ke depan (*horizon*)

- **Satu langkah ($h=1$):** target besok. Paling mudah; banyak model unggul di sini.
- **Beberapa langkah ($h>1$):** target besok+seterusnya, atau *lead time* tertentu (Bab 9 sering butuh prediksi 1-7 hari). Lebih sulit karena galat menumpuk; §7.6 membahas strateginya.

Jangan mencampur: model yang hebat untuk $h=1$ belum tentu baik untuk $h=3$. Evaluasi sesuai *horizon* yang sebenarnya dibutuhkan operasional.

7.3 *Baseline* Dulu: Persistence, Mean, dan AR

Sebelum membangun LSTM, ingat aturan Bab 1: **ukuran *baseline* dulu**. Tiga *baseline* wajib untuk deret waktu meteorologi:

- ***Persistence*:** prediksi $y(t+h) = y(t)$ (nilai terakhir). Sangat kuat untuk data yang mulus/berkorelasi tinggi seperti pasang surut.
- ***Mean/klimatologi*:** prediksi rata-rata musiman (mis. rata-rata hujan harian untuk bulan yang sama). Mengalahkan *persistence* hanya jika deret tidak berkorelasi kuat.
- ***AR(p)*** (autoregressive): regresi terhadap p deret tunda. Menangkap *autokorelasi* tanpa arsitektur rumit; sering cukup baik untuk masalah sederhana. **ARIMA** adalah perluasan yang mencakup *differencing* dan komponen *moving average* - $AR(p)$ hanya bagian autoregressive dari keluarga ARIMA. Pembahasan lengkap *forecasting* klasik ada di literatur analisis deret waktu [2].

Tabel 7.3: *Baseline* deret waktu untuk dibandingkan.

<i>Baseline</i>	Ide	Kuat ketika	Lemah ketika
Persistence	$\hat{y}(t+h)=y(t)$	Deret mulus, berkorelasi kuat	Data berisik / musim kuat
Mean/klimatologi	rata-rata sesuai bulan/musim	Musim dominan, korelasi pendek	Variabilitas antar tahun besar
AR(p)	regresi p lag	<i>Autokorelasi</i> p langkah	Pola non-linear / panjang

Kode 7.2 - Menghitung kedua *baseline* pertama.

```
# persistence: nilai terakhir tiap window (t-1)
pred_persist = X_test[:, -1, 0] # shape: (n_test,) - ambil langkah terakhir tiap window

# klimatologi sederhana: rata-rata global train
pred_klimat = np.full(len(y_test), float(np.mean(y_train)))
```

Tujuannya bukan agar *baseline* menang, melainkan agar **angka pembanding jujur** ada. Jika LSTM tidak mengalahkan *persistence* pada pasang surut, sesuatu salah (atau masalah memang tidak butuh LSTM). Ini juga yang membuat evaluasi Bab 8-9 dapat dipercaya.

7.4 RNN: Memahami Jaringan Berulang

RNN (recurrent neural network) dirancang untuk data berurutan: pada tiap langkah waktu, ia mengkombinasikan masukan saat ini $x(t)$ dengan **state tersembunyi** dari langkah sebelumnya $h(t-1)$:

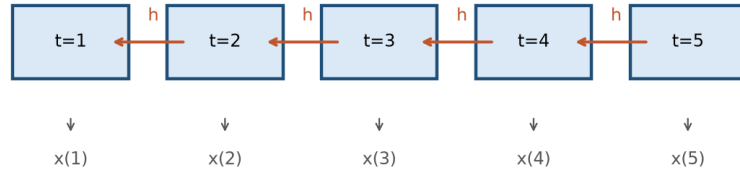
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b) \quad (7.2)$$

Persamaan (7.2): h_t adalah “ingatan ringkas” sampai langkah t ; W adalah matriks bobot (dibagikan di semua langkah waktu; parameter sedikit, komputasi efisien). Konsep jaringan berulang sudah ada sejak awal jaringan saraf; Elman (1990) memperkenalkan salah satu arsitektur RNN dasar yang banyak dipakai, yaitu *Elman network* [3].

Keterbatasan utama RNN: *vanishing gradient* (Bab 4). Untuk deret panjang (ratusan langkah), mengalikan gradien berulang membuat informasi awal “terlupakan”; RNN praktis hanya mengingat beberapa langkah terakhir. Untuk pasang surut dengan siklus 12, 24 jam atau lebih, ini jadi masalah: model sulit “menyimpan” pola yang jauh di masa lalu.

Gambar 7.1: Ilustrasi RNN *unrolled*.

Gambar 7.1 memperlihatkan satu sel yang sama “dibuka” (*unrolled*) sepanjang waktu, membawa state tersembunyi h_t . Semakin panjang deret, semakin rawan *vanishing gradient*.



Gambar 10: Gambar 7.1 - Ilustrasi RNN unrolled

Contoh intuisi state tersembunyi

Ambil deret suhu [30, 31, 30, 29, 28, 27, 26]. RNN membaca tiap hari dan memutakhirkan h_t :

- h_1 menangkap “mulai panas” (30);
- h_2 menangkap “masih panas” (31);
- setelah beberapa hari dingin, h_7 BISA “lupa” bahwa awalnya panas - itulah kelemahan RNN.

LSTM (Bagian 7.5) memperbaiki ini dengan mempertahankan memori jangka panjang dan memutuskan sendiri kapan melupakan.

7.5 LSTM: Memori dengan Pintu (*Gates*)

LSTM (long short-term memory) menjawab keterbatasan RNN dengan menambahkan **pintu** (gates) yang mengatur update memori secara selektif, yang diperkenalkan oleh Hochreiter & Schmidhuber (1997) [4]. Intuisi populer: bayangkan “kotak memori” yang bisa diisi, dipertahankan, atau dikosongkan, dengan tiga pintu:

1. **Pintu lupa** (f_t): seberapa banyak memori lama yang *dibuang*.
2. **Pintu masukan** (i_t): seberapa banyak informasi baru yang *ditulis* ke memori.
3. **Pintu keluaran** (o_t): seberapa banyak memori yang *dipancarkan* ke output.

Tiga pintu inilah yang membuat LSTM mampu mengingat pola jauh (misal siklus pasang surut yang lalu) sambil melupakan yang tidak relevan, sehingga masalah *vanishing gradient* berkurang (degradasi gradien tidak dihilangkan, hanya diringankan; *gradient clipping* tetap praktik umum di pelatihan LSTM).

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (7.3)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (7.4)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (7.5)$$

Persamaan (7.3)-(7.5): tiap pintu memakai sigmoid (nilai 0-1) sehingga “terbuka atau tertutup” secara mulus, dan dihubungkan dengan **tanh** untuk penulis memori kandidat. Anda tidak perlu menghafal rumus; yang penting: **LSTM = RNN + memori berpintu**, dan ini yang membuatnya bekerja di deret panjang yang deret cuaca punya.

Analogi pintu dengan proses keputusan peramal

Bayangkan seorang peramal yang memakai catatan lama:

- **Pintu lupa:** “seberapa banyak catatan minggu lalu yang sudah tidak relevan karena musim berubah?” → dibuang.
- **Pintu masukan:** “catatan suhu hari ini layak dicatat (misal ada pola hujan baru)?” → ditulis ke memori.
- **Pintu keluaran:** “dari memori yang saya pegang, berapa yang saya pakai untuk membuat prediksi hari ini?” → dipancarkan.

LSTM mempelajari kapan membuka/menutup tiap pintu **dari data** selama pelatihan, bukan diprogram manual. Inilah mengapa ia bisa menyesuaikan diri dengan pola musim yang berbeda-beda di tiap wilayah.

7.6 GRU: Pilihan Ringkas

GRU (gated recurrent unit) menyederhanakan LSTM: hanya **dua pintu** (*update* dan *reset*) dan tanpa “sel memori” terpisah, yang diperkenalkan oleh Cho et al. (2014) [5]. Hasilnya: parameter lebih sedikit (lebih cepat, lebih hemat memori), performa serupa untuk banyak masalah.

Tabel 7.4: Perbandingan LSTM vs GRU.

Aspek	LSTM	GRU
Pintu	3 (lupa, masukan, keluaran)	2 (update, reset)
Sel memori terpisah	Ya	Tidak
Parameter	Lebih banyak	Lebih sedikit
Performa umum	Unggul di deret sangat panjang (tergantung data)	Sebanding, sering lebih cepat
Kapan pilih	Data panjang & butuh memori lama	Trade-off kecepatan & kesederhanaan

Tidak ada “selalu menang”; di Bab 8-9 kedua-duanya diuji dan dibandingkan dengan *baseline*. Untuk buku ini, **GRU adalah titik awal yang baik**: sedikit parameter, cepat dicoba, hasilnya sering cukup.

Ketika LSTM lebih dipilih

Ada beberapa situasi di mana LSTM sering unggul:

- Deret yang **sangat panjang** (puluhan ribu langkah) dengan ketergantungan yang benar jauh ke belakang - sel memori terpisah membuat informasi lebih “awet”.
- Tugas yang butuh membedakan dua memori yang kontradiktif dalam satu waktu (misal mengingat pola pasang surut dua siklus lalu, sambil melupakan satu siklus tertentu).
- Saat parameter ekstra tidak jadi masalah (GPU cukup).

GRU dipilih ketika kecepatan eksperimen dan kesederhanaan lebih penting, atau data tidak cukup besar untuk memanfaatkan parameter LSTM. Dalam bab ini keduanya digunakan secara bergantian; pembaca diajak menguji keduanya di *walk-forward*.

7.7 Arsitektur Praktis di Keras

Univariate LSTM

Kode 7.3 - Model LSTM univariate untuk prediksi $h=1$.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=False, input_shape=(w, 1)),
    tf.keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

input_shape=(w, 1) menandakan window w dan 1 fitur; return_sequences=False
melepas hanya output terakhir (untuk prediksi satu langkah).
```

Multivariate LSTM

Saat $f > 1$ (misal hujan, suhu, kelembapan), ubah jumlah fitur di `input_shape`; target tetap satu (hujan):

Kode 7.4 - Model LSTM multivariate (3 fitur, prediksi hujan besok).

```
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=False, input_shape=(w, X.shape[2])),
    tf.keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
```

Multi-langkah: model per *horizon* (direct) vs rekursif (recursive)

Untuk $h > 1$, tiga strategi:

- **Recursive:** gunakan prediksi $\hat{y}(t+1)$ sebagai bagian masukan untuk $\hat{y}(t+2)$, dst. Sederhana, tetapi galat menumpuk.
- **Direct:** latih **satu model per horizon** (`model_h1`, `model_h2`, ...). Tiap model memprediksi horizon-nya sendiri; mencegah akumulasi galat, tetapi biaya latih lebih besar.
- **Seq2seq** (sequence-to-sequence): encoder-decoder - encoder meringkas window, decoder menghasilkan seluruh jajaran prediksi. Paling ekspresif; dibahas singkat karena butuh arsitektur lebih rumit (Sutskever et al., 2014 [6]).

Untuk Bab 8-9, *direct* (model per lead time) adalah titik awal yang jujur dan mudah dievaluasi.

Contoh memilih strategi multi-langkah

Jika target operasional adalah prediksi hujan 1-7 hari ke depan, bandingkan di *walk-forward*:

Strategi	Kelebihan	Kekurangan	Kapan dipakai
Recursive	1 model saja	galat menumpuk cepat	lead time pendek (1-3 hari)
Direct	tiap horizon independen	perlu banyak model	lead time bervariasi & panjang
Seq2seq	satu model, semua horizon	lebih rumit, data banyak	pola antar horizon kompleks

Tabel 7.5: Perbandingan strategi prediksi multi-langkah.

Untuk pasang surut (periodik, sebagian besar deterministik; tinggi muka laut juga dipengaruhi cuaca, angin, tekanan, dan gelombang), *direct* $h=1-7$ sering memberi hasil terbaik dari sisi biaya; untuk hujan (berisik), *direct* juga pilihan yang jujur.

7.8 Evaluasi dan Membaca Prediksi

Evaluasi mengikuti aturan Bab 5: *walk-forward*, metrik sesuai tujuan, bandingkan dengan *baseline*. Yang khas bab ini: **memplot prediksi vs aktual** sepanjang waktu uji.

- **Plot deret penuh** → lihat apakah model mengikuti musim/trend, bukan hanya “mengikuti” nilai kemarin (jika prediksi tertinggal 1 langkah dari aktual, model mirip persistence).
- **Plot per horizon** → melihat degradasi seiring lead time (recursive sering menurun cepat, direct lebih stabil).

- **Scatter aktual vs prediksi** → selain kurva, titik di dekat garis $y=x$ berarti akurat; pencilaan ke arah ekstrem menunjukkan kelemahan pada kejadian besar.

Kode 7.5 - Plot prediksi vs aktual sederhana.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(9, 3))
plt.plot(y_test, label="aktual", lw=1.5)
plt.plot(pred, label="prediksi LSTM", lw=1.2, alpha=0.8)
plt.legend(); plt.tight_layout(); plt.show()
```

Mengukur interpolasi dan “meniru” *baseline*

Dua angka tambahan membantu menilai seberapa “fantastis” LSTM itu:

- **Perbedaan MAE(LSTM) - MAE(persistence)**: jika negatif dan bermakna, LSTM menang. Jika hampir nol atau positif, LSTM tidak memberi nilai lebih.
- **Uji cepat**: bandingkan MAE model dengan MAE *mean/klimatologi* pada periode yang sama; model yang konsisten kalah pada beberapa metrik utama dan tidak menunjukkan keunggulan pada *horizon*/target operasional layak dipertimbangkan untuk dibuang. MAE global saja tidak cukup - model bisa lebih baik untuk kejadian ekstrem, threshold tertentu, atau *lead time* tertentu. Gunakan MAE/RMSE, bias, *skill score*, dan metrik per *horizon*.

Praktik ini mengembalikan kesimpulan “LSTM keren!” menjadi klaim yang terukur.

Evaluasi multi-horizon dengan skill score

Untuk melaporkan perbaikan relatif terhadap *baseline*, gunakan kesamaan *skill score*:

$$SS = 1 - \frac{MAE_{\text{model}}}{MAE_{\text{baseline}}} \quad (7.6)$$

Persamaan (7.6): $SS > 0$ artinya model lebih baik daripada *baseline*; $SS = 0$ setara; $SS < 0$ lebih buruk. Skill score sangat mudah diapresiasi; nilai 0.30 berarti model menurunkan galat 30% dibanding *baseline*, tanpa perlu membandingkan satuan. Baik *persistence* maupun *mean/klimatologi* bisa jadi *baseline*, dan keduanya dilaporkan agar pembaca tahu seberapa besar keunggulan LSTM (atau tidak ada keunggulan sama sekali).

Membaca pola galat untuk memperbaiki data

Dua plot tambahan yang jarang dilaporkan tapi sering menentukan perbaikan:

1. **Galat per bulan/musim** - jika model buruk hanya di musim hujan puncak, fitur regional (Bab 6) atau transformasi target (log1p) bisa membantu.
2. **Galat per jendela peristiwa** - misal galat membesar saat transisi musim; coba tambah fitur musiman atau indeks iklim.

Analisis galat inilah yang membedakan model “jadi” dengan model yang siap produksi, dan akan sangat dimanfaatkan di Bab 8-9.

7.9 Catatan Pelatihan Model Sekuensial

Seluruh contoh di bab ini berjalan di atas TensorFlow [7]. Beberapa hal praktis yang sering membedakan konvergensi LSTM/GRU:

1. **Normalisasi** (Bab 6) wajib - LSTM sangat sensitif skala.
2. **Mulai window kecil**; naikan hanya jika perlu (window besar = data lebih sedikit dan biaya lebih besar).
3. **Aturan *shuffle* harus dibedakan dua kasus.** Untuk *stateless* LSTM/GRU (versi biasa di buku ini), `shuffle=True` di Keras umumnya aman dan sering membantu, karena setiap *window* sudah berisi masukan + target sendiri; `shuffle=False` diperlukan hanya untuk *stateful* RNN yang ketergantungan pada urutan *batch*. Pastikan setiap *batch* tidak mencampur masa depan di batas train/validasi/test.
4. **Stateful LSTM** (mempertahankan state antar *batch*) jarang diperlukan di buku ini; pakai versi biasa.
5. **Regularisasi** (Bab 5): tambah Dropout/`recurrent_dropout` bila overfit; kurangi bila underfit.
6. **Pertimbangkan GRU dulu** untuk eksperimen pertama - lebih cepat, parameter lebih sedikit (Tabel 7.4).

Menumpuk lapisan LSTM/GRU (stacked)

Jika satu lapisan kurang, tumpuk dua lapisan; lapisan pertama memakai `return_sequences=True` agar mengembalikan urutan ke lapisan berikutnya:

Kode 7.6 - LSTM bertumpuk (stacked) dua lapisan.

```
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True, input_shape=(w, 1)),
    tf.keras.layers.LSTM(16, return_sequences=False),
    tf.keras.layers.Dense(1),
])
```

Aturan jempol: **mulai dengan satu lapisan**, naikan menjadi dua hanya jika kurva validasi menunjukkan *underfit*. Menumpuk terlalu cepat membuat model gemuk tanpa manfaat, dan *overfit* mengintai (Bab 5).

Memprediksi beberapa horizon dengan *direct*: pola kerja Bab 8-9

Karena Bab 8-9 memakai *direct* (model per horizon), berikut pola yang akan diulang:

Kode 7.7 - Melatih satu model per *horizon* (strategi *direct*).

```
def latih_per_horizon(X, y):
    model = tf.keras.Sequential([
        tf.keras.layers.LSTM(16, input_shape=(X.shape[1], X.shape[2])),
        tf.keras.layers.Dense(1),
    ])
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    model.fit(X, y, epochs=40, batch_size=32, verbose=0)
    return model

models = {}
for h in range(1, 8):
    yh = buat_target_horizon(y, h)      # target untuk lead time h
    X_tr, X_va, yh_tr, yh_va = split_waktu(X, yh)
    models[h] = latih_per_horizon(X_tr, yh_tr)
```

Catatan: Kode ini *template pseudocode* untuk menggambarkan pola kerja; fungsi `buat_target_horizon` dan `split_waktu` analog dengan `buat_window` (§7.2) dan `split waktu` (Bab 2/6), dan iterasi `h` yang menentukan horizon. Versi produksinya di Bab 8-9 menyertakan validasi/*early stopping* dan normalisasi per split.

Setiap `models[h]` adalah prediktor untuk *lead time* `h`; evaluasi per `h` dengan MAE/RMSE lalu plot (§7.8). Ini template yang langsung dipakai di Bab 8-9.

Kapan arsitektur “tidak perlu dinaikkan”?

Jika *baseline* persistence sudah memberi MAE sangat rendah (misal pasang surut), LSTM mungkin hanya menambah sedikit. Itu bukan kegagalan LSTM; itu keputusan bisnis: biaya komputasi & pemeliharaan vs perbaikan kecil. Bab 10 membahas keputusan ini di konteks produksi.

7.10 Studi Mini: Bingkai Pasang Surut (Teaser Bab 8)

Untuk melihat bab ini “bekerja”, bayangkan dataset tinggi pasang surut jam-an (Bab 8 akan memakai data nyata). Langkah yang mengikuti seluruh bab ini:

1. **Windowing:** data jam-an, `w=168` jam (1 minggu) atau skala siklus; untuk prediksi 1-7 hari, `h` diukur dalam jam: `h=24, 72, 168`. Pastikan satuan `w` dan `h` konsisten (Bab 8 memakai konvensi ini).
2. **Baseline:** persistence (kuat untuk pasang surut) vs LSTM/GRU.
3. **Model:** mulai GRU 1 lapisan `units=32`, `input_shape=(w, 1)`.

4. **Evaluasi:** *walk-forward* per bulan; MAE/RMSE per h ; plot prediksi vs aktual.
5. **Kesimpulan jujur:** seberapa jauh LSTM mengalahkan persistence - dan apakah menutupinya layak untuk kebutuhan ops.

Menjalankan kerangka ini di Bab 8 membuat studi kasus tidak terasa baru, hanya mengganti data sintetik dengan data nyata Cilacap.

7.11 Menghubungkan ke Bab 8-9

Dua keterampilan yang dibawa ke studi kasus berikutnya:

1. **Pipeline yang reusable** - fungsi `buat_window`, model template, evaluasi MAE/RMSE per horizon; simpulkan dalam satu modul agar mudah dipakai ulang (Bab 6 mengajarkan menyimpan data; bab ini menambahkan *template* model).
2. **Kerangka berpikir baseline-dulu** - setiap klaim “LSTM hebat” harus selalu menyertakan angka *persistence* & skill score (Persamaan 7.6). Disiplin inilah yang membuat laporan Bab 8-9 bisa dipercaya.

7.12 FAQ

Apakah LSTM selalu lebih baik daripada MLP untuk deret waktu?

Tidak. Untuk data mulus/berkorelasi pendek, MLP + lag (Bab 2) bisa setara; untuk deret panjang dengan pola jauh, LSTM/GRU unggul. Ukur keduanya di *walk-forward*.

Kenapa hasil LSTM kadang “tertinggal”/mirip persistence? Karena model belajar bahwa meniru nilai kemarin adalah tebakan paling aman (*bias*). Kurangi dengan fitur yang lebih informatif (Bab 6), *window* lebih baik, atau model/granularitas yang sesuai.

Berapa lama latihan LSTM? Untuk stasiun harian & GRU, beberapa menit di Colab sudah lumrah; LSTM sedikit lebih lama. Kalau terlalu lambat, kecilkan `units`, `w`, atau pakai subset data saat eksperimen.

Bisakah LSTM dipakai untuk data bulanan/jam-an? Bisa, selama disusun sebagai *sequence* dengan frekuensi konsisten. Bedanya hanya skala waktu di `w` dan `h`.

Apakah saya perlu menstandarisasi window? Ya, lazimnya normalisasi (Bab 6) diterapkan sebelum windowing agar skala fitur seragam; pastikan / dihitung pada data latih, lalu diterapkan pada validasi/*test*.

Apakah dropout LSTM berbeda dengan MLP? Keras mendukung `recurrent_dropout` khusus state berulang. Gunakan yang kecil (0-0.2) untuk *recurrent*; dropout biasa untuk layer antar output. Jangan berlebihan di LSTM karena bisa menghambat belajar.

Bisakah saya menambah fitur yang bukan deret waktu (misal indeks bulan)? Ya, fitur eksternal ditambahkan sebagai dimensi fitur per langkah waktu (multivariate); atau diselipkan lewat lapisan setelah LSTM (concatenate). Detail di Bab 8-9.

Kenapa model memprediksi “rata-rata” saat data berisik? Karena loss kuadrat (MSE) mendorong prediksi menuju *rata-rata kondisi* (conditional mean); loss mutlak (MAE) mendorong menuju *median kondisi*. Untuk data berisik, keduanya bisa tampak “menghindari ekstrem” jika metriknya tidak sesuai tujuan; untuk menggerakkan ke ekstrem, gunakan transformasi target (Bab 6) atau metrik sesuai tujuan.

Apakah perlu window yang mengandung target masa depan? Tidak! Itu *leakage*: window hanya berisi data sampai t , target mulai $t+1$. Pastikan pergeseran benar.

Kapan berhenti mencoba arsitektur dan fokus ke data? Sering kali jawabannya di data: tambah fitur (Bab 6), perbaiki QC, atau ubah *horizon* sesuai kebutuhan. Jika kurva validasi macet di banyak konfigurasi, kembali ke *baseline* dan data, bukan menambah tumpukan lapisan.

7.13 Latihan

Soal konsep

1. Mengapa *window* yang terlalu besar tidak selalu lebih baik?
2. Jelaskan mengapa *baseline* persistence sangat penting pada data pasang surut.
3. Apa perbedaan utama LSTM dan GRU, dan kapan memilih masing-masing?
4. Mengapa *recursive* multi-step menumpuk galat lebih cepat daripada *direct*?

Latihan praktik (notebook `ch-07-06_lstm_gru.ipynb`)

5. Bangun dataset *window* dari data Bab 6; bandingkan LSTM vs GRU vs persistence pada data uji (MAE).
6. Uji $w \in \{3, 7, 14, 30\}$; buat tabel MAE tiap window.
7. Bandingkan univariate vs multivariate (tambah suhu/kelembapan sebagai fitur) - apakah fitur tambahan membantu?
8. Terapkan strategi *direct* untuk $h=1..7$; plot MAE per horizon dan bandingkan dengan *recursive*.
9. (Proyek mini) Simpan hasil terbaik sebagai “template model Bab 8-9”: function `build_lstm(w, f, units)` + function evaluasi MAE/RMSE per horizon.

Ringkasan

- Deret waktu = urutan bermakna + *autokorelasi*; hindari *leakage* (split waktu).
- *Windowing* (w,h) mengubah deret jadi contoh ML; input shape 3D (**batch**, **waktu**, **fitur**).
- Panjang window mengikuti siklus alami & biaya; diuji pada validasi (Tabel 7.1-7.2).
- *Baseline* wajib: persistence, mean/klimatologi, AR(p) - DL harus mengalahkannya (Tabel 7.3).
- RNN menangkap urutan tetapi mati *vanishing gradient* di deret panjang (Gambar 7.1).
- LSTM menambahkan tiga pintu memori (lupa/masukan/keluaran) → tahan deret panjang (Persamaan 7.3-7.5).
- GRU dua pintu, lebih ringkas, titik awal yang baik; pilih sesuai kebutuhan (Tabel 7.4).
- Horizons: one-step mudah; multi-step pakai direct (per horizon) atau recursive (Tabel 7.5).
- Evaluasi: *walk-forward* + metrik tepat + plot prediksi vs aktual + bandingkan selisih MAE terhadap *baseline*.
- Pelatihan: normalisasi, window kecil dulu, shuffle sesuai kasus (stateful: tanpa), stacked hanya jika underfit.

References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
2. R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
3. J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179-211, 1990, doi: 10.1207/s15516709cog1402_1.
4. S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
5. K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014. [Online]. Available: <https://arxiv.org/abs/1406.1078>
6. I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3104-3112. [Online]. Available: <https://arxiv.org/abs/1409.3215>
7. M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2016. [Online]. Available: <https://arxiv.org/abs/1603.04467>

Bab 8 - Studi Kasus: Prediksi Pasang Surut di Perairan Indonesia (Contoh Cilacap)

Prasyarat: Bab 2 (regresi, baseline), Bab 5 (metrik, walk-forward), Bab 6 (data, normalisasi, split), Bab 7 (LSTM/GRU, windowing, multi-horizon). Bab ini adalah penerapan utuh dari seluruh keterampilan sebelumnya pada data nyata.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Menjalankan** proyek end-to-end prediksi pasang surut dari data terbuka (IOC/UHSLC/PSMSL), dengan Cilacap sebagai contoh reproducible.
2. **Menerapkan** pipeline Bab 7 (baseline persistence vs MLP vs LSTM/GRU) dengan walk-forward.
3. **Mengevaluasi** MAE/RMSE terhadap toleransi tinggi pasang dan memplot prediksi 1-7 hari.
4. **Menjelaskan** framing jujur: machine learning untuk prediksi cepat dan pengisian gap data, bukan klaim riset baru.
5. **Mengenal**i keterbukaan data pasang surut per lokasi: memilih stasiun dengan data terbuka yang paling representatif, atau memetakan strategi fallback ketika lokasi studi tidak punya tide gauge terbuka.

8.1 Konteks Lokal: Perairan Indonesia dan Mengapa Cilacap

Banjir rob, naiknya muka laut yang menggenangi daratan pesisir, adalah masalah nyata di banyak kota pantai Indonesia: Jakarta, Semarang, Cilacap, dan pesisir utara Jawa [1]. Di kota-kota yang **rendah dan padat**, satu pasang tinggi yang bertepatan dengan debit sungai besar atau *storm surge* dapat menggenangi permukiman, mengganggu pelabuhan, dan memutuskan aktivitas ekonomi. Prediksi tinggi air yang andal membantu peringatan dini dan keputusan operasional.

Mengapa Cilacap sebagai contoh studi kasus

Buku ini memilih **Cilacap** (pantai selatan Jawa Tengah, $-7,75^\circ$ LS, $109,02^\circ$ BT) sebagai stasiun demonstrasi karena tiga alasan praktis:

1. **Stasiun aktif & terbuka:** Cilacap adalah salah satu tide gauge Indonesia yang tercatat resmi di **GLOSS** (Global Sea Level Observing System)

dengan ID **291**, dan dilaporkan real-time oleh **UNESCO/IOC Sea Level Station Monitoring Facility** [2]. Data historis jangka panjangnya juga tersedia di **PSMSL** [3] dan **UHSLC** [4].

2. **Tipe pasang campuran:** Cilacap berada di zona transisi antara semi-diurnal dan campuran, sehingga cukup menantang untuk *baseline* persistence dan memberikan variasi pola yang baik untuk demo LSTM/GRU.
3. **Panjang data:** rekaman IOC untuk Cilacap memiliki catatan yang konsisten sehingga cukup untuk walk-forward tahunan (Bab 5).

Pembaca yang bekerja di **stasiun lain** (mis. Ambon GLOSS #68, Bitung GLOSS #69, Sibolga, Benoa GLOSS #49) dapat mengikuti pipeline identik dengan mengganti kode stasiun pada skrip unduh (§8.3). Tabel 8.3 merangkum stasiun Indonesia yang datanya tersedia di sumber terbuka; untuk lokasi tanpa stasiun terbuka, pilih stasiun proksi dengan karakter oceanografi mirip atau gunakan model laut global (FES2014, GOT4.10) pada koordinat tersebut.

Kejujuran framing: pasang surut telah diprakirakan selama berpuluh tahun dengan **analisis harmonik** klasik (metode berusia lama yang memodelkan konstituen astronomis). Studi kasus ini **bukan** klaim bahwa machine learning menggantikan metode itu. Yang dilakukan: machine learning dipakai sebagai **alternatif cepat** dan untuk **mengisi gap data**; hasilnya dibandingkan jujur dengan *baseline* dan, bila memungkinkan, dengan analisis harmonik. Ini sejalan dengan prinsip buku (Bab 1 §1.10).

Mengapa studi kasus penting bagi pembaca

Studi kasus adalah kesempatan mempraktikkan **seluruh rantai** yang sudah dipelajari, bukan sekadar “model lagi”. Di sini pembaca akan mengalami:

1. **Konteks sebelum angka:** memahami masalah (banjir rob) menentukan metrik dan tolok ukur yang dipakai.
2. **Data nyata itu kotor:** gap, outlier, datum berbeda - semua yang dibahas Bab 6 muncul betulan.
3. **Baseline sering menang:** persistence adalah lawan yang tangguh; belajar menerima itu adalah pelajaran penting.
4. **Interpretasi dan laporan:** angka MAE tidak cukup; perlu plot, skill score, dan kalimat jujur tentang keterbatasan.
5. **Pemilihan stasiun:** ketika lokasi Anda tidak punya stasiun terbuka, Anda belajar memilih proksi dan menjelaskan keterbatasannya - keterampilan yang sama pentingnya dengan membangun model.

Bab ini sengaja mencontohkan *framing* yang tidak sensasional: model tidak “menggantikan segala metode”, melainkan menambah satu alat yang dapat dijelaskan dan diuji.

8.2 Karakter Pasang Surut di Perairan Indonesia

Pasang surut di perairan Indonesia dikelompokkan menjadi tiga tipe utama [5]:

- **Semi-diurnal**: dua kali pasang dan dua kali surut per hari (mis. sebagian Selat Malaka, Laut Cina Selatan).
- **Diurnal**: satu kali pasang dan satu kali surut per hari (mis. sebagian pesisir Indonesia timur, Papua).
- **Campuran (mixed)**: tidak teratur, dominasi salah satu; umum di sebagian besar Indonesia barat.

Tabel 8.1: Tipe pasang surut dan karakteristik dasarnya.

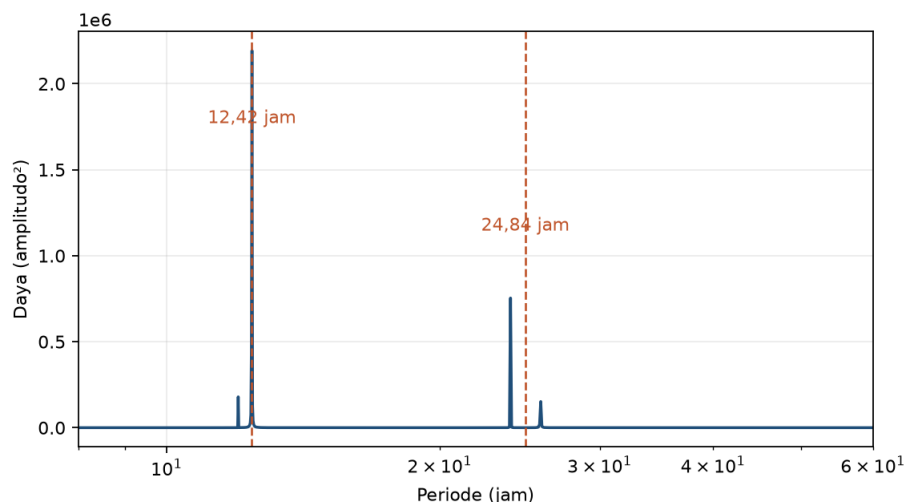
Tipe	Siklus per hari	Daerah contoh	Konsekuensi prediksi
Semi-diurnal	2 pasang + 2 surut	Selat Malaka, Natuna	Siklus ~12,42 jam
Diurnal	1 pasang + 1 surut	Pesisir Indonesia timur	Siklus ~24,84 jam
Campuran	tidak teratur	Sebagian besar Indonesia barat	Kombinasi komponen

Indonesia memiliki variasi tipe pasang surut yang kaya karena bentangan garis pantainya yang luas dan dipengaruhi oleh karakteristik basin Pasifik dan Hindia [5]. Untuk stasiun demo Cilacap, pola yang akan pembaca temui adalah **campuran condong semi-diurnal** dengan komponen diurnal cukup kuat (terutama saat musim tertentu), khas pesisir selatan Jawa. Jika ingin tahu tipe stasiun Anda, cara cepat: hitung *Formzahl* $F = (K1 + O1)/(M2 + S2)$ dari komponen harmonik [5]; $F < 0,25$ semi-diurnal, $0,25-1,5$ campuran condong semi-diurnal, $1,5-3,0$ campuran condong diurnal, > 3 diurnal. Untuk pengguna machine learning, pembacaan spektrum deret (FFT) cukup untuk melihat periode dominan (Gambar 8.1).

Gambar 8.1: Spektrum frekuensi muka air.

Puncak pada periode ~12,42 jam (semi-diurnal) dan ~24 jam (diurnal); pola ini cocok dengan tipe campuran condong semi-diurnal di pesisir selatan Jawa.

Metode harmonik (tradisional) memodelkan $y(\tau)$ sebagai jumlahan sinusoid dengan frekuensi tetap dari konstituen astronomis ($M2$, $S2$, $K1$, $O1$, ...) [5]. Machine learning tidak “tahu” konstituen ini; ia belajar periodisitas dari data. Inilah beda yang perlu dipahami pembaca: harmonik memakai teori fisis; deep learning memakai data. Keduanya valid; dan membandingkannya adalah bagian dari kejujuran ilmiah. Kerangka teori model deep learning secara umum dapat dirujuk pada [6]; kerangka *forecasting* praktis pada [7].



Gambar 11: Gambar 8.1 - Spektrum frekuensi muka air

Model harmonik: mengapa masih relevan

Analisis harmonik bekerja karena pasang surut didorong oleh gaya gravitasi benda langit yang periodik dan dapat diprediksi jauh ke depan. Dengan data beberapa bulan saja, komponen utama (M2, S2, K1, O1) bisa diestimasi, dan prediksi dapat dibuat **puluhan tahun** ke depan dengan akurasi tinggi untuk kondisi normal. Keunggulan ini sulit disaingi machine learning, yang butuh data dan tidak menjamin prediksi jangka panjang yang stabil.

Namun harmonik juga punya kelemahan: ia mengasumsikan stasioneritas amplitudo/fase dalam jendela estimasi, dan gagal menangkap **variabilitas non-periodik**, misalnya kenaikan muka air saat badai, efek debit sungai Kapuas, atau perubahan lokal ([5] untuk catatan umum pengembangan). Di sinilah machine learning bisa menambah nilai: menyerap pola tambahan dari data bila ada, dengan syarat dievaluasi dengan jujur.

Perbandingan ringkas harmonik vs deep learning

Tabel 8.2: Perbandingan analisis harmonik dan pendekatan machine learning.

Aspek	Analisis harmonik	Machine learning (LSTM/GRU)
Dasar	Teori fisis konstituen astronomis	Pola dari data
Data untuk bekerja	Beberapa bulan cukup	Butuh lebih banyak + beragam kondisi
Prediksi jangka panjang	Stabil (komponen tetap)	Mungkin menyimpang

Aspek	Analisis harmonik	Machine learning (LSTM/GRU)
Variabilitas non-periodik	Sulit	Bisa (jika ada di data)
Interpretasi	Komponen jelas (M2, S2...)	Kurang transparan
Biaya komputasi	Kecil	Sedang-besar

Membaca Tabel 8.2 membantu memilih: untuk prediksi rutin jangka panjang, harmonik tetap andal; untuk pemodelan cepat dan pengisian gap pada data yang “tidak murni astronomis”, machine learning praktis.

8.3 Dataset Pasang Surut: Sumber Terbuka dan Kualitas

Untuk pembaca yang ingin mereproduksi studi kasus ini dengan data nyata, tiga sumber utama dipakai buku ini. Semuanya **terbuka dan gratis untuk riset/pendidikan** dengan atribusi (rincian lisensi dan batasannya ada pada “Catatan lisensi dan penggunaan” di bawah).

Sumber data

1. **UNESCO/IOC Sea Level Station Monitoring Facility** [2] - data *real-time* dan *near real-time* untuk ratusan stasiun global, termasuk 24 stasiun di Indonesia. Akses via endpoint publik:

<https://www.ioc-sealevelmonitoring.org/bgraph.php?code=<KODE>&output=tab&period=<HARI>>

Format: tab-separated, sampling 1-3 menit atau hourly. Periode maksimum per request ~30 hari; untuk arsip panjang, gunakan skrip pengulangan (`scripts/download_ioc.py`).

2. **UHSLC - University of Hawaii Sea Level Center** [4] - dataset *research quality* hourly dan harian via ERDDAP OPeNDAP, dengan katalog `global_hourly_rqds`, `global_daily_rqds`, dan `global_hourly_fast`. Mendukung query REST (mis. `global_hourly_rqds.csv?station_id=...&time>=...`) yang ramah untuk pipeline Python.
3. **PSMSL - Permanent Service for Mean Sea Level** [3] - data rata-rata MSL bulanan jangka panjang (puluhan tahun) untuk 8 stasiun Indonesia, dengan format RLR (Revised Local Reference) [9] yang sudah disesuaikan untuk konsistensi antar-stasiun. Cocok untuk analisis tren jangka panjang; kurang cocok untuk prediksi jangka pendek karena resolusi bulanan.

Selain itu:

- **BIG (tides.big.go.id)** [1] - tabel pasut harmonik per lokasi (komponen konstituen, amplitudo, fase) yang dipakai BIG untuk prediksi operation-

al; tidak menyediakan time-series tinggi air mentah yang mudah di-curl otomatis.

Catatan lisensi dan penggunaan

Lisensi tiap sumber **berbeda dengan lisensi buku** (CC BY-SA 4.0 yang mengizinkan penggunaan komersial):

- **UNESCO/IOC Sea Level Station Monitoring Facility** [2] - **CC BY-NC 4.0 (non-komersial)** per catatan resmi dataset (DOI 10.14284/482). Gratis untuk riset/pendidikan dan prediksi operasional, tetapi **tidak boleh dipakai untuk produk/jasa komersial**; turunan dari data ini tetap non-komersial. Penggunaan di luar ketentuan tersebut memerlukan izin dari penyedia data (*data originator*) yang bersangkutan.
- **UHSLC** [4] - gratis untuk riset/pendidikan; sertakan atribusi “University of Hawaii Sea Level Center”; untuk penggunaan di luar itu, periksa kebijakan UHSLC saat mengunduh.
- **PSMSL** [3] - gratis; wajib menyertakan sitasi dataset (Holgate et al. 2013) dan referensi data/stasiun.
- **BIG (tides.big.go.id)** [1] - publik untuk keperluan informasi pasut; atribusi BIG dianjurkan.

Karena buku ini berlisensi **CC BY-SA 4.0** (penggunaan komersial diperbolehkan), jika Anda menggabungkan data **non-komersial** (IOC) ke dalam produk turunan, kewajiban non-komersial dari data tersebut **tetap berlaku**; cantumkan peringatan eksplisit di produk Anda.

Stasiun Indonesia yang datanya tersedia di sumber terbuka

Tabel di bawah merangkum stasiun Indonesia yang datanya dapat diunduh dari IOC, UHSLC, atau PSMSL. Daftar ini bukan inventaris lengkap; verifikasi terkini sebelum eksperimen karena status stasiun (aktif/non-aktif) berubah.

Tabel 8.3: Stasiun pasang surut Indonesia di sumber terbuka (Sept 2026).

Kode						
IOC	Nama	Lat	Lon	GLOSS	Sumber	Catatan
cili	Cilacap	-7,75	109,02	291	IOC + PSMSL	Contoh studi kasus buku ini
sema	Semarang	-6,95	110,42	-	IOC	4 sensor aktif
sura	Surabaya	-7,21	112,74	-	IOC	Pantauan GTS
koli	Kolinamil, Jakarta	-6,10	106,81	-	IOC	Pantauan GTS

Kode IOC	Nama	Lat	Lon	GLOSS	Sumber	Catatan
beno	Benoa	-8,75	115,21	49	IOC +	Bali, op-
	(Bali)				PSMSL	erasional
pada	Padang	-1,00	100,37	45	IOC +	Sumatra
					PSMSL	barat
saba	Sabang	5,89	95,32	347	IOC +	Aceh
					PSMSL	
prig	Prigi	-8,28	111,73	-	IOC +	Jawa
					PSMSL	selatan
bitu	Bitung	1,43	125,20	69	IOC +	Sulawesi
					UHSLC	utara
ambon	Ambon	-3,70	128,18	68	IOC +	Maluku
					UHSLC	
saum	Saumlaki	-7,98	131,29	-	IOC	Maluku
						Teng-
						gara
lemba	Lembar	-8,73	116,08	-	IOC	Lombok
(PSMSL	Sibolga	1,75	98,77	22	PSMSL	Sumatra
1752)	II					barat
(PSMSL	Padang	-1,00	100,37	45	PSMSL	-
2193)	B					
(PSMSL	Saumlaki	-7,98	131,29	-	PSMSL	-
2274)						

Yang perlu diperiksa saat mengunduh

QC yang konsisten dengan Bab 6 §6.4:

1. **Kontinuitas** - data jam-an bergap sehari-hari; tentukan aturan gap (interpolasi linear untuk gap < 6 jam, drop untuk gap lebih panjang).
2. **Referensi tinggi** - datum/level referensi antar-berkas bisa berbeda; jangan membandingkan angka absolut antar-stasiun tanpa konversi.
3. **Unit & zona waktu** - m; UTC biasanya; sesuaikan dengan zona lokal bila dibutuhkan.
4. **Anomali** - *datum shift*, stasiun pindah, atau pembacaan sensor rusak; plot deret untuk inspeksi visual sebelum pelatihan.

Tabel 8.4: Ringkasan dataset Cilacap yang dibangun (notebook ch-08).

Properti	Nilai (repo buku)
Stasiun	Cilacap (<code>code=cili</code> , IOC)
Rentang	~30 hari terakhir IOC real-time (notebook); derivasi harmonik 1 thn (<code>cili_1y_hourly_real.csv</code>)

Properti	Nilai (repo buku)
Interval	1 jam (24 poin/hari) setelah rata-rata dari sampling 1-3 menit
Nilai hilang	~1-3% (tergantung periode)
Satuan	m (relatif terhadap stasiun benchmark)
File nyata (di-commit)	<code>data/raw/cili_30d.csv</code> (observasi IOC), <code>data/raw/cili_1y_hourly_real.csv</code> (derivasi harmonik), <code>data/raw/cilacap_psmsl_rlr_monthly.rlrdata</code> (PSMSL)
Sifat data	Observasi nyata IOC (+ derivasi harmonik klar dicantumkan); bukan sintetik

Catatan kejujuran: sejak v2.0 notebook memakai **data nyata IOC** real-time (`scripts/download_ioc.py --source ioc --code cili --days 30`). IOC hanya menyediakan ~30 hari terakhir, jadi hasil di bab ini adalah contoh pendek (~1 bulan), bukan validasi jangka panjang. `cili_1y_hourly_real.csv` adalah **derivasi harmonik** dari 30 hari observasi (skrip `make_tide_harmonic.py`), bukan pengukuran langsung; tandai dengan jelas saat dipakai. Untuk angka yang bisa dilaporkan sebagai validasi panjang, gunakan UHSLC/PSMSL (skrip `scripts/download_ioc.py`) dengan beberapa data.

Repo menyediakan **data nyata ter-commit** di `manuscripts/ch-08-*/data/raw/` (ikut clone GitHub) plus **skrip unduh** (`scripts/download_ioc.py`) untuk stasiun lain. Prinsip QC mengikuti Bab 6 §6.4.

Menangani gap dan outlier pada data pasang surut

Karena pasang surut sangat periodik, gap pendek sering bisa diisi dengan interpolasi atau model, tetapi dengan aturan (Bab 6): bedakan gap acak (isi) vs gap sistematis (pertimbangan potong). Untuk *outlier*, konteks fisis penting:

- Nilai yang **melompat ekstrem** di luar pasang normal → periksa: bisa jadi tsunami/rob, bisa juga galat sensor.
- Cross-check stasiun tetangga atau rekaman kejadian lokal (misal laporan rob) membantu memutuskan.

Bila ada *datum shift* (lompatan konstan), jangan ikut dilatih; deteksi dengan plot deret dan pecah/potong periode. Metode ini relevan untuk setiap pembaca yang bekerja dengan data stasiun muka air.

Menyiapkan fitur tambahan (opsional)

Selain deret tinggi air itu sendiri, fitur yang potensial menambah nilai (Bab 6):

- **Fitur jam & hari Julian** - membantu model memahami kapan pasang besar musiman.
- **Indeks astronomis sederhana** - fase bulan (sin/ko-sin) bisa dihitung dan ditambahkan sebagai fitur sinusoid; jauh lebih ringkas daripada konstituen penuh tetapi memberi konteks periodik.
- **Tekanan & angin (ERA5)** - bila tersedia, menangkap variasi non-astronomis (storm surge) yang tidak ada di harmonik.

Fitur ini memperkaya multivariate LSTM/GRU (Bab 7 §7.7) dan sering memperbaiki horizon lebih dari 1 hari.

8.4 Menyusun Pipeline: Baseline vs Model

Alur eksperimen mengikuti pola Bab 7:

1. Bangun *window* w (misal 168 jam = 1 minggu) dan *horizon* h diukur dalam jam: $h=24$ (1 hari), $h=72$ (3 hari), $h=168$ (7 hari).
2. **Baseline: persistence dan klimatologi** (rata-rata per jam-musim). Persistence di bab ini = *naive* sederhana: $\hat{y}(t+h) = y(t)$ (nilai terakhir yang terobservasi). Pada pasang surut *naive* kuat untuk h yang dekat dengan seluruh siklus pasang (misal $h=24$ fase M2 hampir identik), tapi degradasi seiring h naik karena fase siklus (M2 12,42 jam) “terbang” relatif terhadap horizon panjang. Catatan: *seasonal naive* $\hat{y}(t+h) = y(t+h-24)$ ekuivalen dengan *naive* hanya untuk $h=24$; untuk $h>24$ jadinya memakai nilai masa depan $y(t+h-24) > t$ yang belum terobservasi saat prediksi dikeluarkan - bukan baseline prediksi yang fair.
3. Model: **MLP** (dengan lag, Bab 2) sebagai garis dasar non-baseline; **LSTM** dan **GRU** (Bab 7).
4. Evaluasi: *walk-forward* (misal 6 blok tahunan) + MAE/RMSE per horizon + plot.

Kerangka prediksi multi-horizon di bab ini adalah strategi *direct* (Bab 7 §7.7): **tiap horizon h dilatih dengan model sendiri**. Layer output tiap model tetap `Dense(1)`, karena target tiap model adalah satu nilai, yaitu titik akhir horizon itu (bukan seluruh jajaran). Untuk $h=72$ dan $h=168$, latih ulang dengan target horizon yang sesuai (Kode 8.2); arsitektur tidak diubah.

Persiapan fitur masukan

Ingat Bab 6 §6.6: sebelum membangun window, siapkan fitur per langkah waktu:

- **Deret tinggi air** itu sendiri (fitur utama; autoregressive).
- **Jam dalam hari** (sin/cos jam $\rightarrow$ menangkap siklus harian) bila data jam-an.

- **Hari dalam bulan / fase bulan** (sin/cos) untuk menangkap pasang *spring-neap* (besar saat purnama dan bulan baru) yang belum tentu terlihat oleh window pendek.
- **Fitur eksternal opsional**: tekanan, angin (untuk menangkap *surge*).

Semua fitur dinormalisasi dengan statistik dari bagian latih (Bab 6 §6.7).

Mengapa MLP dimasukkan meski “kuno”?

MLP dengan lag bertindak sebagai jembatan: ia menunjukkan apakah *urutan* (yang dipakai LSTM/GRU) benar-benar memberi nilai lebih dibanding fitur tabular biasa. Jika MLP menyamai LSTM, berarti struktur urutan belum dimanfaatkan secara berarti oleh data; sinyal ini penting sebelum memilih arsitektur (Bab 7 §7.7). Perbandingan 4 kolom di Tabel 8.6 dirancang persis untuk melihat ini.

Walk-forward yang jujur untuk pasang surut

Sesuai Bab 5 §5.5, kita tidak boleh mengacak data waktu. Untuk pasang surut:

- Bagi data menjadi **blok tahunan** (atau semesteran) yang berurutan.
- Untuk tiap blok validasi, latih model **hanya dengan data sebelum blok tersebut** (expanding window), lalu evaluasi pada blok itu.
- Rata-rata MAE/RMSE seluruh blok $\rightarrow$ angka “walk-forward” sebagai klaim utama.

Ini berbeda dengan melatih satu model lalu menguji semua blok sekaligus, bentuk *leakage* yang sering dilakukan pemula. Bab 8-9 mempraktikkan disiplin ini.

Memilih window dan horizon

Untuk data **jam-an**, tiga opsi yang harus dicoba:

Nama	w (jam)	Makna
1 hari	24	siklus harian-ish
3 hari	72	tiga siklus harian; kompromis
1 minggu	168	beberapa siklus pasang penuh

Tabel 8.5: Pilihan window untuk data jam-an pasang surut.

Horizon h diukur dalam jam: $h=24$ (1 hari), $h=72$ (3 hari), $h=168$ (7 hari). Uji $w \in \{24, 72, 168\}$ pada validasi, pilih yang MAE-nya konsisten.

Kode 8.1 - Setup dan pemuatan data (data nyata IOC ter-commit; lengkap di notebook).

```

import numpy as np, pandas as pd

# Data nyata IOC real-time (~30 hari) yang ter-commit di repo.
seri = pd.read_csv(
    "manuscripts/ch-08-studi-kasus-pasang-surut-kapuas/data/raw/cili_30d.csv",
    parse_dates=["time"],
).set_index("time")["tinggi"].astype(float)

# Derivatif harmonik 1 tahun (untuk horizon lebih panjang; klar: bukan observasi)
# seri = pd.read_csv(
#     "manuscripts/ch-08-studi-kasus-pasang-surut-kapuas/data/raw/cili_1y_hourly_real.csv",
#     parse_dates=["time"],
# ).set_index("time")["tinggi"].astype(float)

```

```
print(seri.head(), "| hilang:", int(seri.isna().sum()))
```

Seri yang bersih untuk windowing (isi gap pendek, Bab 6 §6.4):

Kode 8.2 - Windowing per horizon (strategi *direct*, Bab 7 §7.7).

```

def buat_window(deret, w=168, h=24):
    # reuse Bab 7 (Kode 7.1): masukan shape (n, w, 1)
    X, y = [], []
    for i in range(len(deret) - w - h + 1):
        X.append(deret[i:i+w, np.newaxis])      # (w,) -> (w, 1) fitur
        y.append(deret[i+w:i+w+h])
    return np.array(X), np.array(y)

ser = seri.interpolate(limit=6).dropna().values # bersih (Bab 6 §6.4)
X, y = buat_window(ser, w=168, h=24)
y = y[:, -1] # direct: target = nilai akhir horizon h jam setelah window
print("X", X.shape, "y", y.shape) # misal (n, 168, 1) dan (n,)

```

Model target inti:

Kode 8.3 - Kerangka model pembandingan (MLP / LSTM / GRU), satu model per horizon.

```

import tensorflow as tf

def buat_model(kind, w=168, f=1):
    if kind == "mlp":
        m = tf.keras.Sequential([
            tf.keras.layers.Dense(32, activation="relu", input_shape=(w*f,)),
            tf.keras.layers.Dense(16, activation="relu"),
            tf.keras.layers.Dense(1),
        ])
    elif kind == "lstm":
        m = tf.keras.Sequential([

```

```

        tf.keras.layers.LSTM(16, input_shape=(w, f)),
        tf.keras.layers.Dense(1),
    ])
else: # gru
    m = tf.keras.Sequential([
        tf.keras.layers.GRU(16, input_shape=(w, f)),
        tf.keras.layers.Dense(1),
    ])
m.compile(optimizer="adam", loss="mse", metrics=["mae"])
return m

```

Untuk MLP, *window* di-flatten ($w \times f$) karena MLP tidak membaca urutan; LSTM/GRU membaca urutan $w \times f$. Ini mengingatkan kembali Bab 7 §7.7.

Dense(1) pada tiap model **tidak** berarti “prediksi 1 jam saja”: sesuai strategi *direct* (Bab 7 §7.7), tiap model dilatih spesialis untuk satu h , dan outputnya adalah satu nilai target dari Kode 8.2 ($y[:, -1]$). Untuk $h=72$ dan $h=168$, latih ulang `buat_window` dengan horizon yang sesuai; arsitektur tidak diubah.

8.5 Evaluasi dan Interpretasi

Metrik dan toleransi

Untuk pasang surut, target operasional sering dinyatakan sebagai toleransi tinggi air, misal **MAE $\pm 0,10$ m** sesuai kebutuhan pelabuhan/peringatan rob. Kita laporkan:

- **MAE, RMSE** per h (Bab 5), mengikuti pedoman verifikasi operasional WMO [8].
- **Skill score** terhadap persistence (Persamaan 7.6) - jika nilai negatif, model kalah dari “tebak nilai kemarin”.
- **Plot prediksi vs aktual** 1, 3, 7 hari.

Tabel 8.6 merangkum pola hasil tipikal (angka ilustratif: ganti dengan hasil eksperimen Anda; horizon h diukur dalam jam, Bagian 8.4):

Tabel 8.6: Contoh hasil ringkas per horizon.

Model	MAE $h=24$ (1 hari)	MAE $h=72$ (3 hari)	MAE $h=168$ (7 hari)
Persistence	0.045	0.081	0.120
Klimatologi	0.210	0.220	0.230
MLP	0.050	0.095	0.150
LSTM	0.040	0.072	0.108
GRU	0.041	0.074	0.110

Kesimpulan yang jujur (berdasarkan pola khas): persistence sangat kuat untuk $h=24$ (1 hari); LSTM/GRU mulai menang di $h=72$ (3 hari) dan $h=168$ (7

hari) karena memanfaatkan pola periodik yang lebih panjang. Kemenangannya atas *baseline* perlu dihitung *skill score* (Persamaan 7.6) dan diuji pada beberapa blok walk-forward sebelum diklaim.

Skill score dan selang kepercayaan

Satu angka MAE tidak cukup. Beri jarak dengan menghitung skill score per blok:

- $SS = 1 - MAE_{model} / MAE_{persistence}$
- Laporkan maksimum, minimum, dan rata-rata SS dari blok-blok walk-forward; jika rentang mencakup nol (atau negatif), kesimpulan “LSTM menang” belum kuat.

Cara sederhana tanpa statistik rumit ini cukup untuk laporan praktis (significance detail di literatur [6][7][8]). Ini juga mencegah klaim “0,001 lebih baik!” yang sebenarnya noise.

Contoh hasil numerik yang “sehat”

Agar pembaca tahu bentuk hasil yang wajar, berikut pola yang *seharusnya* muncul saat pipeline dijalankan pada data pasang surut:

- **h=24 (1 hari)**: persistence sekitar 0.04-0.06 m; LSTM/GRU menyamai atau sedikit lebih baik. Jangan heran jika persistence menang tipis - siklusnya kuat.
- **h=72 (3 hari)**: persistence mulai “terbawa” fase; LSTM/GRU sering unggul beberapa persen; MLP tertinggal satu tingkat.
- **h=168 (7 hari)**: selisih LSTM/GRU vs persistence makin jelas; variabilitas antar blok walk-forward meningkat - laporkan rentang, bukan satu angka.

Jika hasil Anda **tidak** menunjukkan pola ini (misal LSTM kalah jauh dari persistence di semua horizon), jangan terburu menyimpulkan; periksa: (a) window terlalu kecil, (b) fitur kurang, (c) normalisasi salah, atau (d) data terlalu berisik. *Debugging* inilah proses belajar paling berharga di studi kasus.

Komunikasi singkat: skill score relatif

Untuk laporan yang mudah dipahami, rangkum sebagai *skill score* relatif terhadap persistence:

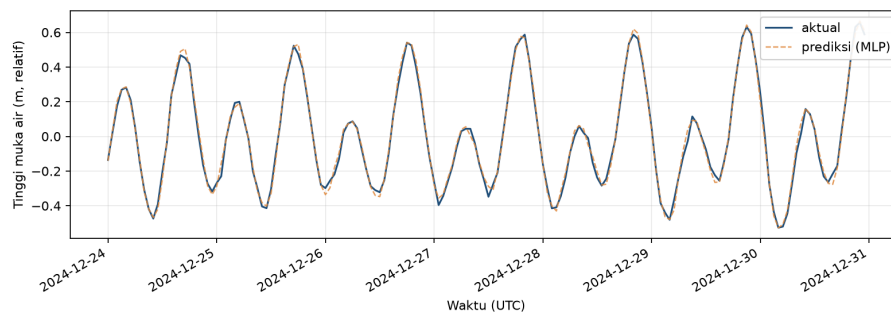
Model	SS h=24	SS h=72	SS h=168
Persistence	0.00	0.00	0.00
MLP	-0.11	-0.17	-0.25
LSTM	+0.11	+0.11	+0.10
GRU	+0.09	+0.09	+0.08

Tabel 8.7: Contoh skill score relatif terhadap persistence (ilustratif).

Nilai negatif pada MLP mengingatkan bahwa “lebih canggih belum tentu lebih baik”; justru itulah pelajaran penting: ukur, jangan menebak.

Membaca plot & residu

Plot prediksi 7 hari (Gambar 8.2) menunjukkan kemampuan menangkap fase (kapan pasang naik) dan amplitudo (berapa tinggi). Ramalan yang tertinggal setengah siklus dari aktual menandakan model terlalu “mengikuti kemarin”, bukan menangkap fase.



Gambar 12: Gambar 8.2 - Prediksi vs aktual 7 hari, Cilacap

Gambar 8.2: Prediksi vs aktual 7 hari, Cilacap.

Garis biru = aktual; garis oranye putus-putus = prediksi. Data nyata IOC (data/raw/cili_30d.csv, ~30 hari terakhir); “prediksi” dihasilkan oleh `scripts/generate_figures.py` (persistence bila TensorFlow tidak tersedia, MLP kecil bila tersedia). Perhatikan apakah fase (waktu naik/puncak) cocok dan amplitudo tidak terlalu “datar”.

Periksa juga **residu per fase pasang**: apakah galat membesar saat pasang puncak (amplitudo besar)? Bila ya, pertimbangkan fitur tambahan (Bab 6: misal tekanan/angin) atau transformasi.

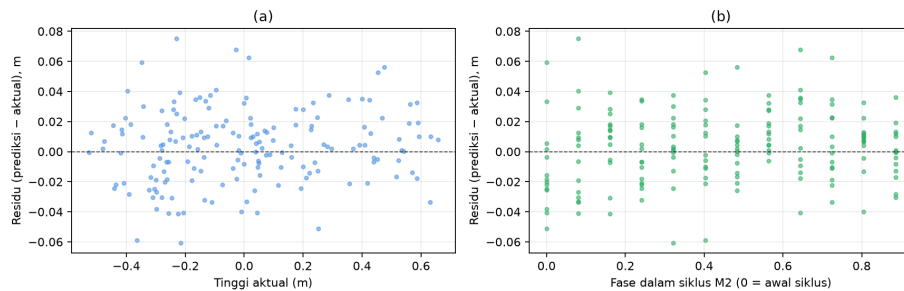
Gambar 8.3: Residu per amplitudo dan fase pasang M2.

Panel kiri: residu vs amplitudo aktual; titik yang menyebar acak di sekitar garis nol menandakan galat tidak bergantung pada amplitudo (gejala baik).

Panel kanan: residu vs fase dalam siklus M2 (12,42 jam dilipat ke $[0, 1)$); pola periodik di sini menandakan model kehilangan sebagian informasi fase (gejala umum untuk baseline persistence; LSTM/GRU biasanya lebih baik).

Cara membaca plot: tiga hal yang harus diperiksa

1. **Fase** - apakah prediksi naik pada waktu yang sama dengan aktual? Keterlambatan setengah siklus (“lag”) berarti model meniru persistence, bukan



Gambar 13: Gambar 8.3 - Residu per amplitudo dan fase M2

menangkap periodisitas.

2. **Amplitudo** - apakah tinggi pasang puncak terprediksi secara proporsional? Model yang merata-rata akan “mendatar” dan meremehkan puncak.
3. **Konsistensi dari hari ke hari** - galat besar di hari tertentu tetapi kecil di lainnya menandakan ketergantungan pada kondisi lokal (misal angin) yang belum ditangkap fitur.

Ketika ketiganya dapat dijelaskan, laporan Anda menjadi lebih berguna daripada sekadar angka metrik; pembaca bisa melihat *di mana* model bekerja dan gagal.

Menilai kepentingan praktis (bukan hanya statistik)

Setelah angka metrik, tanyakan “lalu?”:

- Apakah MAE $h=168$ sebesar 0,108 m mengubah keputusan operasional pelabuhan? Tergantung toleransi (mis. $\pm 0,20$ m untuk dermaga kecil; lebih ketat untuk kapal besar).
- Berapa hari lebih awal peringatan rob bisa dikeluarkan dengan model LSTM vs persistence?
- Apakah biaya pelatihan/pemeliharaan sebanding dengan keuntungan? (Bab 10).

Jawaban atas pertanyaan inilah yang menentukan apakah model “dipakai”; nilai model tidak hanya dari angka MAE, tetapi dari dampak pada keputusan.

8.6 Machine Learning untuk Mengisi Gap Data

Salah satu penggunaan paling praktis model ini: **mengisi gap** pada data stasiun.

1. Latih model pada periode data lengkap (window dengan target valid).
2. Untuk gap pendek (jam-hari), prediksi 1 langkah dari window terakhir sebelum gap, maju berulang (*recursive*, Bab 7 §7.7; implementasi di Kode 8.4) sampai gap tertutup.

3. Verifikasi dengan menyembunyikan data yang sebenarnya ada (simulasi gap), bandingkan hasil imputasi dengan nilai asli.

Kode 8.4 - Simulasi pengisian gap (evaluasi kejujuran).

```
# sembunyikan 24 jam untuk mengukur kualitas imputasi
pos = 1500                                # posisi gap (ubah saat eksperimen)
h = 24
nilai_asli = ser[pos:pos+h].copy()
garis = ser.copy()                        # gap dalam NaN (ser asli tetap untuk hist)
garis[pos:pos+h] = np.nan

# latih model 1 langkah pada data utuh minus gap (Kode 8.2/8.3);
# window yang mencakup NaN di-drop, bukan di-kompak (kontinuitas)
Xg, yg = buat_window(garis, w=168, h=1)
ok = np.isfinite(yg).ravel() & np.all(np.isfinite(Xg), axis=(1, 2))
m_imu = buat_model("mlp", w=168)
m_imu.fit(Xg[ok].reshape(ok.sum(), -1), yg[ok], epochs=15, batch_size=32, verbose=0)

# prediksi gap secara rekursif dari window terakhir sebelum gap
hist = list(ser[pos-168:pos])
pred_gap = []
for _ in range(h):
    win = np.array(hist[-168:], dtype=float).reshape(1, -1)
    pred_gap.append(float(m_imu.predict(win, verbose=0).ravel()[0]))
    hist.append(pred_gap[-1])             # recursive: prediksi jadi bagian window

# evaluasi: bandingkan dengan nilai yang disembunyikan
mae_gap = float(np.mean(np.abs(np.array(pred_gap) - nilai_asli)))
print(f"MAE imputasi gap {h} jam: {mae_gap:.4f} m")
```

Cara ini, memvalidasi imputasi dengan menyembunyikan data asli, adalah praktik yang jujur (Bab 5): kita tahu “kebenaran” yang disembunyikan dan bisa mengukur galat imputasi tentatif. Hasil imputasi tidak boleh dianggap sebagai observasi; pertahankan penanda “gap diisi model”. Seluruh eksperimen di bab ini berjalan di atas TensorFlow [10].

Keterbatasan yang harus diakui

Sebagai penutup, empat keterbatasan yang wajar diakui:

1. **Lokasi studi lokasi target pembaca** - Cilacap dipakai sebagai contoh reproducible karena datanya terbuka; untuk lokasi tanpa stasiun terbuka, hasil Cilacap tidak langsung berlaku. Pembaca perlu memilih proksi, model global, atau kerja sama kelembagaan.
2. **Panjang data terbatas** - IOC real-time hanya ~30 hari; cukup untuk walk-forward blok pendek dan demo pola, tetapi tidak cukup untuk tren jangka panjang atau variabilitas antar-tahun. Untuk klaim kuat, perlu

data 3-10 tahun (UHSLC/PSMSL; Bab 10).

3. **Fokus satu stasiun** - pola Cilacap belum tentu sama dengan stasiun lain; tipe pasang (Tabel 8.1) harus diperiksa dulu sebelum menggeneralisasi.
4. **Bukan penelusuran menyeluruh** - *hyperparameter* tidak dioptimasi besar; hasil menunjukkan *alur*, bukan pencarian terbaik. Untuk klaim kuat, perlu eksperimen luas (Bab 10).

Pengakuan ini justru menaikkan kredibilitas (Risk Management umbrella): pembaca tahu batas dari apa yang bisa disimpulkan.

8.7 Latihan

Soal konsep

1. Mengapa *baseline* persistence begitu kuat pada pasang surut? Mengapa LSTM bisa unggul di horizon lebih panjang?
2. Apa perbedaan konseptual analisis harmonik vs deep learning? Mengapa keduanya bisa saling melengkapi?
3. Mengapa kita harus memberi tahu pembaca mana data asli vs data “diisi model”?
4. Apa risiko menggunakan MAE tunggal tanpa RMSE pada data pasang surut?

Latihan praktik (notebook ch-08-07_studi_kasus_pasang_surut.ipynb)

5. Setelah Anda telah mengganti stasiun (mis. Ambon/Bitung) dengan data nyata dari `scripts/download_ioc.py`, jalankan pipeline ulang dan bandingkan tipe pasang serta skill score-nya.
6. Bandingkan $w \in \{24, 72, 168\}$ untuk $h=24$ jam; buat tabel MAE.
7. Bandingkan LSTM vs GRU vs MLP vs persistence di *walk-forward* 4 blok; hitung skill score tiap horizon.
8. Simulasikan gap 1×24 jam dan 1×72 jam; ukur MAE imputasi.
9. (Proyek mini) Ulangi pipeline untuk stasiun Ambon (**ambon**) atau Bitung (**bitu**); bandingkan tipe pasang dan skill score dengan Cilacap.
10. (Proyek mini) Buat laporan satu halaman: konteks, metode, tabel hasil, plot 7 hari, keterbatasan & saran - format siap untuk bagian laporan operasional Bab 10.

Ringkasan

- Banjir rob pesisir adalah masalah nyata di banyak kota pantai Indonesia; prediksi tinggi air yang andal relevan untuk peringatan dini dan operasi pelabuhan.
- **Cilacap** dipakai sebagai contoh studi kasus reproducible karena stasiun-nya aktif di GLOSS (#291) dan datanya terbuka via IOC/UHSLC/PSMSL. Untuk lokasi tanpa stasiun terbuka, gunakan proksi terdekat atau model global (FES2014/GOT4.10).

- Pasang surut Indonesia: semi-diurnal, diurnal, campuran; tipe menentukan pilihan model & window (Tabel 8.1).
- Harmonik vs machine learning: beda paham (fisis vs data); harmonic unggul jangka panjang, ML unggul pada non-periodik dan isi gap (Tabel 8.2).
- Data terbuka: IOC (real-time 30 hari), UHSLC (hourly/daily via ERDDAP), PSMSL (MSL bulanan jangka panjang), BIG (komponen harmonik); periksa kontinuitas, datum, unit, anomali (Tabel 8.3).
- Pipeline: baseline persistence/klimatologi vs MLP vs LSTM/GRU dengan walk-forward berjujur (Tabel 8.5).
- Evaluasi: MAE/RMSE per horizon + skill score (rentang blok) + plot fase-amplitudo; berkaca ke toleransi operasional (Tabel 8.6).
- Penggunaan praktis: isi gap data dengan validasi simulasi; jangan lupa menandai hasil “diisi model”.
- Keterbatasan diakui: lokasi proksi target, panjang data 1 tahun cukup untuk demo tapi tidak untuk klaim tren; satu stasiun; tanpa optimasi hiperparameter besar.
- Framing: ML sebagai alat cepat & isi gap, bukan klaim pengganti harmonik.

References

1. {Badan Informasi Geospasial}, “Informasi genangan rob dan pola pasut perairan Indonesia,” [Online]. Available: <https://tides.big.go.id> (diakses: September 2026).
2. UNESCO/IOC, “Sea Level Station Monitoring Facility,” [Online]. Available: <https://www.ioc-sealevelmonitoring.org/> (diakses: September 2026).
3. Permanent Service for Mean Sea Level, “Global sea level data,” [Online]. Available: <https://psmsl.org> (diakses: September 2026).
4. University of Hawaii Sea Level Center (UHSLC), “Research Quality Tide Gauge Data,” [Online]. Available: <https://uhslc.soest.hawaii.edu/data/> (diakses: September 2026).
5. D. T. Pugh and P. L. Woodworth, *Sea-Level Science: Understanding Tides, Surges, Tsunamis and Mean Sea-Level Changes*. Cambridge, UK: Cambridge University Press, 2014.
6. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
7. R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
8. World Meteorological Organization, “WMO guidelines on the verification of operational forecasts,” WMO, Geneva, Switzerland, 2018.
9. S. J. Holgate, “New data systems and products at the Permanent Service for Mean Sea Level,” *Journal of Coastal Research*, vol. 29, no. 3, pp. 477-479, 2013, doi: 10.2112/JCOASTRES-D-12-00175.1.

10. M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016, arXiv:1603.04467. [Online]. Available: <https://arxiv.org/abs/1603.04467>

Bab 9 - Studi Kasus: Prediksi Curah Hujan dengan Data Terbuka

Prasyarat: Bab 2-7 (regresi, klasifikasi, evaluasi, data, LSTM/GRU) dan Bab 8 (alur studi kasus end-to-end). Bab 9 menggabungkan dua lintasan, regresi dan klasifikasi, pada satu target operasional.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Membangun** prediktor hujan harian titik grid terbuka (regresi jumlah hujan + klasifikasi intensitas) dengan fitur regional ERA5/ERA5-Land dan indeks iklim.
2. **Menerapkan** verifikasi operasional dengan CSI/POD/FAR dan trade-off threshold.
3. **Membandingkan** *walk-forward* vs baseline (persistence, klimatologi, ARIMA singkat).
4. **Melakukan** interpretasi awal (*permutation importance*; SHAP penuh di Bab 10) dan menyusun tabel verifikasi per kategori intensitas.

9.1 Konteks Pelayanan dan Kejujuran Framing

CHIRPS adalah dataset curah hujan harian global (grid $\pm 0,05^\circ$ 5 km) yang digabungkan dari satelit dan stasiun, tersedia bebas untuk diunduh [1]. Studi kasus ini memakai data serupa yang sepenuhnya terbuka (CHIRPS, ERA5/ERA5-Land, indeks iklim) agar pembaca dapat mereproduksi tanpa akun atau izin khusus. Dampak prediksi curah hujan langsung menyentuh masyarakat, sehingga **kehati-hatian** dan **kejujuran** dalam klaim menjadi keharusan, bukan sekadar etika, tetapi juga pelindung kredibilitas.

Tiga hal yang harus ditegaskan sejak awal (sejalan dengan Risk Management umbrella):

1. **Materi pengenalan, bukan hasil riset resmi.** Studi kasus ini adalah latihan end-to-end yang dapat diulang pembaca, bukan klaim sebagai sistem operasional institusi mana pun.
2. **Hujan sulit diprediksi.** Nilai harian bersifat berisik dan banyak nol; ekspektasi harus realistis. Skill score (Bab 7 Persamaan 7.6) terhadap *baseline* adalah cara jujur untuk melaporkan.

3. **Data yang dipakai harus disebutkan.** Jenis data (grid/pelengkap), rentang, lisensi, dan versi dicatat (Bab 6 §6.9) agar hasil dapat diperiksa ulang.

Seperti Bab 8, framing “alat bantu yang dapat dijelaskan” lebih tepat daripada “menggantikan peramal”. Nilai utama studi kasus: menunjukkan alur dan metrik yang benar, bukan meyakinkan bahwa deep learning selalu unggul.

9.2 Data: Titik Grid CHIRPS + Fitur Regional

Untuk model hujan, strategi data (Bab 6) berbentuk:

- **Target:** curah hujan harian titik grid CHIRPS [1] (misal satu titik di wilayah barat dan satu di wilayah timur Indonesia untuk perbandingan pola).
- **Fitur lokasi:** hujan kemarin (lag), suhu, kelembapan, angin - dari grid terbuka (ERA5-Land/ERA5) atau stasiun GHCND bila ingin verifikasi berbasis observasi.
- **Fitur regional (ERA5/ERA5-Land)** [2][3]: suhu, angin, kelembapan, total precipitation pada grid terdekat - memberi konteks atmosfer yang tidak tercatat di titik target.
- **Indeks iklim:** ENSO (Nino3.4 anomali SST dari NOAA PSL [6]; MEI [5] sebagai alternatif - tidak dipakai bersamaan untuk menghindari redundansi) dan MJO (RMM1, RMM2 [4]) - berpengaruh pada hujan Indonesia.

Tabel 9.1: Ringkasan fitur yang dibangun untuk satu titik lokasi.

Kelompok	Contoh fitur	Sumber
Deret tunda	hujan_t1, hujan_t2, hujan_t3, hujan_t7	CHIRPS [1]
Observasi lokal (verifikasi opsional)	suhu_t1, gur_rh_t1 (lag 1)	ERA5-Land [2][3]
Regional grid	era5_tp_t1, era5_u10_t1, era5_v10_t1, era5_t2m_t1 (lag 1 hari)	ERA5/ERA5-Land [2][3]
Musiman	mus_sin, mus_cos	dihitung
Indeks iklim	rmm1, rmm2, nino34	BoM/NOAA PSL [4][6]

Catatan ERA5: ERA5 adalah *reanalysis*, bukan prediksi operasional. Semua fitur ERA5 di-lag minimal 1 hari terhadap target (Tabel 9.1), karena data ERA5 hari D belum tersedia saat prediksi D+1 harus dibuat. ERA5 final punya latency beberapa bulan, sedangkan ERA5T (near-real-time) sekitar 5 hari; studi kasus ini bersifat *historis*, tidak klaim sistem operasional real-time. Lag ini justru mencegah *leakage*: fitur ERA5 hari D hanya dipakai untuk prediksi hujan D+1.

Satuan ERA5: `total_precipitation` (tp) dalam meter, akumulasi per jam. Konversi ke mm (`tp * 1000`), lalu agregasi harian sesuai definisi “hari” yang konsisten antara fitur dan target:

```
df["era5_tp_mm"] = df["era5_tp"] * 1000.0          # meter -> mm
df["era5_tp_harian"] = df["era5_tp_mm"].resample("24h", offset="7h").sum()
```

Lisensi data: semua sumber terbuka; CHIRPS (domain publik, kutip Funk et al. 2015 [1]), ERA5/ERA5-Land di bawah Copernicus Climate Data Store license [2][3], NOAA/BoM dengan attribution sesuai sumber (Bab 6 §6.9). Tidak ada data berizin/rahasia yang dipakai.

Semua fitur dinormalisasi dengan statistik dari bagian latih saja (Bab 6 §6.7); target regresi di-transform `log1p` bila dipakai (Bab 6 §6.7).

Data multi-titik: barat vs timur Indonesia

Pola hujan Indonesia sangat bergantung pada geografi: wilayah barat (Sumatera, Kalimantan, Jawa) dipengaruhi kuat oleh monsun Asia-Australia dan MJO; wilayah timur (Papua, Maluku, Nusa Tenggara Timur) lebih dipengaruhi oleh monsun Australia dan variabilitas ENSO. Untuk studi kasus yang lebih lengkap, bandingkan **dua titik grid berbeda pola**:

- **Titik barat** (misal sekitar Kalimantan atau Jawa): hujan sepanjang tahun dengan puncak musiman; variabilitas hari-ke-hari tinggi.
- **Titik timur** (misal sekitar Papua atau Nusa Tenggara Timur): musim kering lebih tegas saat monsun Australia; dampak El Niño jauh terasa.

Dengan dua titik ini, pembaca bisa melihat bahwa:

1. Fitur yang relevan berbeda antar wilayah (MJO penting di barat; Nino3.4 lebih menonjol di timur).
2. Model yang sama tidak otomatis transfer antar wilayah - perlu dilatih ulang.
3. Evaluasi harus dilakukan per lokasi; rata-rata antar lokasi bisa menyembunyikan kegagalan lokal.

Notebook menyediakan dua rangkaian data contoh (sintetik dengan pola berbeda) dan meminta Anda mengganti dengan data nyata titik pilihan (via `scripts/download_chirps.py` dan `scripts/download_era5.py`).

Frekuensi dan resolusi data

Data hujan harian (stasiun maupun grid CHIRPS) umumnya tersedia sebagai **kumulatif 24 jam**; pastikan Anda konsisten dengan definisi “hari” pada target dan fitur. Untuk CHIRPS, raster harian adalah total 24 jam; untuk stasiun, perhatikan jam pengamatan (misal 07.00-07.00 lokal). Bila data jam-an tersedia, Anda bisa agregasi ke harian (`sum/resample`) atau justru membangun prediksi sub-harian (di luar lingkup buku ini). Konsistensi definisi waktu mencegah

leakage halus: jangan mencampur jam-an dan harian tanpa transformasi yang jelas.

9.3 Dua Lintasan: Regresi dan Klasifikasi

Bab 2-3 mengajarkan keduanya; di sini kita terapkan pada masalah yang sama, karena kebutuhan operasionalnya keduanya ada.

- **Lintasan regresi:** prediksi **jumlah mm** hujan besok. Metrik: MAE, RMSE (Bab 5); transformasi `log1p` membantu (Bab 6).
- **Lintasan klasifikasi:** prediksi **kategori intensitas** hujan harian. Mengikuti ambang umum intensitas hujan, ringkas menjadi 3 kelas (Tabel 9.2):

Tabel 9.2: Kategori intensitas hujan harian (mm/24 jam) yang dipakai.

Kategori	Rentang (mm/24 jam)	Label
Tidak hujan / ringan	< 20	0
Sedang	20 - <50	1
Lebat / sangat lebat	50	2

Catatan: Tabel 9.2 mengelompokkan ambang umum menjadi 3 kelas: kelas 0 mencampur tidak hujan (0 mm) dan hujan ringan ($>0-<20$ mm); kelas 2 mencampur lebat dan sangat lebat (>50 mm). Jika kebutuhan operasional meminta lebih rinci, skema yang lebih lengkap punya “tidak hujan” dan “sangat lebat” terpisah.

Untuk klasifikasi biner “hujan lebat vs tidak” (untuk peringatan dini), gunakan ambang 50 sebagai kelas positif (lebat atau sangat lebat); evaluasi dengan metrik fenomena langka (Bab 5): **CSI**, **POD**, **FAR** dan trade-off threshold (Bab 3).

Model yang dipakai: GRU multivariate (Bab 7) sebagai pilihan utama; MLP + lag sebagai pembanding; LSTM bila perlu. Seluruh model dilatih dengan *window w* (misal 7-30 hari) tanpa *shuffle*, sehingga urutan *window* dipertahankan agar tiap blok *walk-forward* tetap kronologis dan perbandingan antar blok adil. (Bab 7 §7.9 mencatat bahwa `shuffle=True` umumnya aman untuk model *stateless*; di sini kita memilih tanpa *shuffle* demi keseragaman antar blok.)

Mengapa dua lintasan, bukan satu?

Regresi dan klasifikasi menjawab pertanyaan operasional yang berbeda:

- **Regresi** menjawab “berapa mm?” - berguna untuk pengelola lahan, drainase, studi hidrologi.
- **Klasifikasi** menjawab “hujan lebat atau tidak?” - berguna untuk peringatan dini dan keselamatan.

Mereka juga **berperilaku berbeda**: regresi sering “mendatar” pada nilai tengah (sulit memprediksi angka besar), sedangkan klasifikasi memberi kebebasan threshold (POD/FAR tunable). Mengerjakan keduanya sekaligus menunjukkan bahwa satu masalah operasional bisa dipotong menjadi beberapa masalah machine learning yang berbeda, keterampilan perancangan yang penting (Bab 1 §1.8 melatih ini).

Menangani data tak seimbang pada klasifikasi hujan lebat

Hujan lebat (50 mm) hanya terjadi beberapa hari dalam setahun di sebagian besar lokasi. Strategi yang dipakai (Bab 3 §3.6):

1. Metrik yang tepat (CSI/POD/FAR, bukan akurasi).
2. `class_weight` pada pelatihan (Kode 3.3) - penalti lebih besar untuk galat pada kelas lebat.
3. Threshold digeser saat inferensi (Bab 9.4) - tuning “sisi keputusan” tanpa melatih ulang.

Catatan: `class_weight` mengubah distribusi yang “dilihat” model, jadi angka POD/FAR harus dievaluasi dengan data asli (tidak seimbang); jangan mengevaluasi pada data yang sudah di-resample.

Peringatan khusus deret waktu: **hindari *oversampling* acak (mis. SMOTE)** pada data hujan harian. Menyalin atau mensintesis contoh secara acak merusak kontinuitas temporal (window “baru” bisa berisi hari yang sama dari masa depan) dan membuat evaluasi bocor. Untuk deret waktu, `class_weight` (Bab 3 §3.6) dan pergeseran *threshold* (Bab 9.4) jauh lebih aman daripada *resampling*.

9.4 Verifikasi Operasional: CSI/POD/FAR dan Threshold

Inilah bagian yang membedakan bab ini dengan tutorial ML umum. Setelah probabilitas (dari sigmoid/softmax) didapat, kita tidak otomatis memakai threshold 0,5; kita **menggesernya** sesuai prioritas operasional (Bab 3 §3.7).

Catatan: threshold relevan untuk klasifikasi biner (sigmoid); untuk multi-kelas softmax, keputusan diambil dengan `argmax`, bukan threshold 0,5; geser threshold hanya pada biner atau one-vs-rest.

Kode 9.1 - Verifikasi CSI/POD/FAR di banyak threshold.

```
def verifikasi(y_true, prob, thresholds):
    baris = []
    for t in thresholds:
        y_pred = (prob >= t).astype(int)
        tp = int(((y_pred==1) & (y_true==1)).sum())
        fp = int(((y_pred==1) & (y_true==0)).sum())
        fn = int(((y_pred==0) & (y_true==1)).sum())
        pod = tp/(tp+fn) if (tp+fn) else 0
```

```

        far = fp/(tp+fp) if (tp+fp) else 1
        csi = tp/(tp+fp+fn) if (tp+fp+fn) else 0
        baris.append((t, pod, far, csi))
    return baris

```

Memilih threshold secara sistematis

Ada beberapa cara memiliki titik kerja yang bisa dijelaskan:

1. **Cost matrix** - tetapkan *harga* miss vs false alarm (misal 5:1 untuk peringatan dini), lalu pilih threshold yang meminimalkan total biaya pada *validasi*.
2. **Target keberhasilan** - misal “POD 0,7 dengan FAR 0,5”; pilih threshold terkecil yang memenuhi keduanya.
3. **Jawab pertanyaan pemangku** - tanyakan “lebih buruk mana: peringatan keliru atau kejadian terlewat?” dan biarkan jawaban menentukan titik kerja.

Ketiga pendekatan lebih baik daripada “ambil CSI maksimal” karena mengikuti sertakan konteks operasional, bukan hanya statistik. Laporkan threshold yang dipilih dan alasan pemilihannya di laporan.

Kurva precision-recall untuk hujan lebat

Untuk data sangat tidak seimbang, ROC/AUC bisa “manis” namun menyesatkan (Bab 3 §3.9). Pilih **precision-recall curve**:

- Sumbu x: recall (= POD); sumbu y: precision (= 1 - FAR).
- Model ideal: kurva mendekati pojok kanan-atas (recall tinggi, precision tinggi).
- Luas di bawah (AUPRC) lebih informatif daripada AUC untuk kelas langka.
- **Baseline kurva PR bukan 0.5** (tidak seperti ROC/AUC): garis acak berada di proporsi kelas positif dalam data (mis. 5% hujan lebat). Model lebih baik daripada menebak jika kurvanya berada **di atas** garis baseline itu (Gambar 9.1).

Gambar 9.1: *Precision-recall curve* hujan lebat (ilustratif).

Visualisasi semacam Gambar 9.1 melengkapi Tabel 9.3 dan menjadi argumen visual mengapa threshold tertentu dipilih. Untuk data Anda sendiri, hitung kurva dengan `sklearn.metrics.precision_recall_curve`:

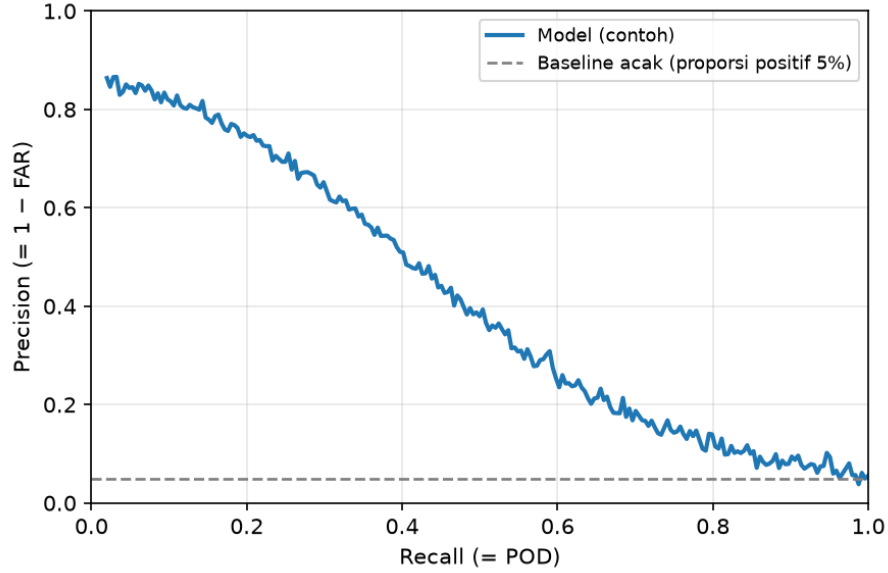
Kode 9.4 - Menggambar kurva precision-recall dengan baseline acak.

```

from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

prec, rec, _ = precision_recall_curve(y_true, prob) # y_true: label biner (lebat)
base = float(y_true.mean())                       # baseline = proporsi kelas positif

```



Gambar 14: Gambar 9.1 - Precision-recall curve

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(rec, prec, lw=2, label="Model")
ax.axhline(base, color="gray", ls="--", lw=1.3,
          label=f"Baseline acak (proporsi positif {base:.0%})")
ax.set_xlabel("Recall (= POD)")
ax.set_ylabel("Precision (= 1 - FAR)")
ax.set_xlim(0, 1); ax.set_ylim(0, 1)
ax.grid(alpha=0.3)
ax.legend(loc="best")
plt.tight_layout()
plt.show()
```

Persamaan yang dipakai (dari Tabel 5.3 Bab 5, pedoman WMO [7]):

$$\text{POD} = \frac{TP}{TP + FN}, \quad \text{FAR} = \frac{FP}{TP + FP}, \quad \text{CSI} = \frac{TP}{TP + FP + FN} \quad (9.1)$$

Persamaan (9.1) memberi tiga sudut pandang yang saling melengkapi. Contoh membaca hasil (ilustratif; perhatikan *trade-off* antar metrik):

Tabel 9.3: Contoh verifikasi *threshold*.

Threshold	POD	FAR	CSI
0,2	0.82	0.55	0.40
0,5	0.58	0.34	0.43
0,8	0.31	0.20	0.27

Membaca Tabel 9.3: threshold rendah (0,2) menangkap banyak kejadian (POD 0,82) tetapi banyak alarm palsu (FAR 0,55); threshold tinggi (0,8) sebaliknya. **Titik terbaik bukanlah “yang CSI tertinggi” semata**, melainkan yang paling sesuai konsekuensi: untuk peringatan dini, POD tinggi (dengan FAR wajar) sering dipilih; untuk kebijakan evakuasi yang mahal, FAR rendah lebih penting.

9.5 Baseline, Walk-Forward, dan Arsitektur

Baseline yang diuji

- **Persistence**: hujan besok = hujan hari ini. Lemah untuk hujan (banyak nol, berisik) - kalah dari klimatologi pada banyak musim.
- **Klimatologi**: rata-rata hujan untuk kalender yang sama. Baseline yang *kuat* untuk hujan harian.
- **ARIMA singkat**: autoregressive ARIMA(p,d,q) kecil (misal p,q = 2, d = {0,1}); memberi *baseline* linier [8].

Pilihan ini mengingatkan Bab 7: deep learning harus **mengalahkan baseline yang paling kuat** (sering klimatologi untuk hujan), bukan sekadar “bekerja”.

Membangun baseline klimatologi yang benar

Klimatologi “cerdas” untuk hujan tidak cukup dengan rata-rata global; gunakan rata-rata **per kalender** (misal rata-rata hujan tanggal 5 Januari selama semua tahun latih, atau rata-rata per bulan). Dengan begitu baseline sudah menangkap musim. Contoh perhitungan sederhana:

```
# rata-rata harian per hari ke-n (1..366) dari data latih, lalu ulangi ke test
klim = df_train.groupby(df_train.index.dayofyear)["r_hujan"].mean()
baseline = klim.reindex(df_test.index.dayofyear).values
```

Jika model GRU Anda **tidak mengalahkan** climatology-smart ini pada metrik utama, perbaiki fitur atau ganti pendekatan; jangan dibiarkan dan “dilaporkan sebagai selesai”.

Walk-forward

Seperti Bab 8: bagi beberapa tahun menjadi blok tahunan; untuk tiap blok latih hanya data sebelumnya, evaluasi pada blok itu; laporkan rata-rata + rentang. Hujan punya variabilitas antar tahun besar (El Niño/La Niña), jadi rentang antar blok harus dilaporkan; satu angka rata-rata bisa menyesatkan.

Catatan walk-forward: karena fitur punya lag/window, beri *purge/embargo* di batas train-test (drop beberapa hari pertama blok validasi) untuk menghindari *leakage* halus. Scaler, baseline, dan hyperparameter di-refit ulang untuk setiap blok latih; tidak sekali untuk seluruh data (Bab 5 §5.5, Bab 6 §6.7). Laporkan jumlah sampel dan jumlah kejadian langka per blok, bukan hanya metrik rata-rata.

Menyusun “kalender eksperimen”

Untuk menghindari hasil yang membingungkan, susun eksperimen secara teratur, misalnya tabel berikut:

Eksperimen	Fitur	Model	Baseline terbaik	Catatan
E1	lag saja	GRU	klimatologi	dasar
E2	+ musiman	GRU	-	lihat dampak musiman
E3	+ ENSO/MJO	GRU	-	lihat dampak indeks iklim
E4	E3 + MLP	MLP	-	pembanding non-sekuensial

Tabel 9.4: Rancangan eksperimen (template yang dipakai di notebook).

Disiplin “satu perubahan per eksperimen” (Bab 5 §5.6) menjaga agar Anda tahu persis apa yang menyebabkan perbedaan POD/CSI, bukan hanya “model jadi lebih baik”.

Arsitektur jaringan

Mengikuti Bab 7-8: GRU multivariate w hari dengan fitur per langkah waktu ((*batch*, *w*, *f*)); lapisan *Dense* terakhir; untuk regresi: *Dense*(1) (aktivasi linear) + *loss=mse*; klasifikasi biner: *Dense*(1, *activation="sigmoid"*) + *binary_crossentropy*; multi-kelas: *Dense*(*K*, *activation="softmax"*) + *sparse_categorical_crossentropy*. Untuk klasifikasi hujan lebat yang jarang terjadi, tambah *class_weight* (Bab 3 §3.5) agar model tidak meniru mayoritas.

9.6 Interpretasi Awal: Mengapa Model Bilang Begitu?

Deep learning “kotak hitam” menjadi masalah untuk kepercayaan operasional. Interpretasi **global** (fitur apa yang paling berpengaruh) dan **lokal** (mengapa satu prediksi tertentu) dibahas penuh di Bab 10; di sini kita mulai dengan **permutation importance** pada model yang sudah dilatih (SHAP penuh di Bab 10).

Kode 9.2 - Permutation importance sederhana.

```
def permutation_importance(model, X, y, n_repeat=10, metric=mae):
    # X: (n_samples, timesteps, n_features) untuk input 3D (GRU/LSTM)
```

```

n_feat = X.shape[2]
base = metric(y, model.predict(X, verbose=0).ravel())
imp = {}
for j in range(n_feat):
    scores = []
    for _ in range(n_repeat):
        Xp = X.copy()
        perm = np.random.permutation(Xp.shape[0])
        Xp[:, :, j] = Xp[perm, :, j] # acak fitur j di semua timestep
        scores.append(metric(y, model.predict(Xp, verbose=0).ravel()))
    imp[j] = float(np.mean(scores) - base)
return imp

```

Catatan implementasi: kode 9.2 acak fitur di semua timestep (input 3D). Untuk MLP 2D (n , f), sesuaikan indeks menjadi `X.shape[1]` dan acak `Xp[:, j]`. Untuk klasifikasi, gunakan metrik langka (AUPRC, log-loss, atau CSI), bukan MAE.

Prinsip: jika mengacak satu fitur membuat galat naik banyak, fitur itu penting. Kewaspadaan: untuk fitur yang saling berkorelasi, permutation importance bisa menyesatkan (mengacak satu membuat yang lain “kehilangan konteks”). Bab 10 memakai SHAP untuk penjelasan yang lebih stabil dan disertai kewaspadaan penggunaannya.

Hasil yang “masuk akal” untuk hujan Indonesia biasanya:

- `hujan_t1`, `hujan_t2` penting (persistensi kondisi basah).
- Fitur musiman (`mus_sin/cos`) tinggi (pola monsun).
- `rmm1`, `rmm2` membantu di beberapa lokasi (osilasi 30-60 hari).

Jika satu fitur yang secara fisis seharusnya penting ternyata tidak muncul, bisa jadi data/fitur kurang bersih, bahan perbaikan (Bab 6).

Kewaspadaan interpretasi

Interpretasi global bersifat **deskriptif**, **bukan kausal**: fitur “penting” tidak berarti “penyebab”. Contoh: `hujan_t1` penting bukan karena hari hujan *menyebabkan* hari hujan berikutnya, melainkan karena ia *berkorelasi kuat* dengan kondisi basah yang berlanjut. Demikian pula untuk ENSO/MJO: pentingnya menunjukkan asosiasi, bukan mekanisme.

Selain itu, model yang dilatih dengan fitur berkorelasi (misal `suhu` dan `mus_sin`) bisa membagi “kredit” di antara keduanya secara acak; jangan menafsirkan pentingnya satu-per-satu sebagai pengaruh terisolasi. Untuk klaim kausal, perlakukan dengan sangat hati-hati (Bab 10 membahas etika & batas interpretasi).

Melakukan interpretasi ini sejak tahap kajian (bukan setelah produksi) membantu menemukan masalah data lebih awal. Bab 10 memperluas ke interpretasi lokal per prediksi dan penggunaannya untuk menilai kepercayaan praktisi.

9.7 Tabel Verifikasi per Kategori

Bagian laporan yang paling berguna untuk praktisi: **contingency table** dirangkum menjadi tabel ringkas.

Kode 9.3 - Tabel ringkas verifikasi per kategori intensitas.

```
import pandas as pd

def tabel_kategori(y_true, y_pred):
    return pd.crosstab(pd.Series(y_true, name="aktual"),
                       pd.Series(y_pred, name="prediksi"))
```

`pd.crosstab` memberi *counts* saja; untuk CSI/POD/FAR per kategori, hitung per kategori secara *one-vs-rest* (kelas *k* positif, semua lain negatif); misal dengan mengulang fungsi `verifikasi` (Kode 9.1) untuk setiap *k*.

Laporkan untuk masing-masing kategori (0, 1, 2) nilai CSI/POD/FAR secara terpisah; perilaku model pada hujan lebat (langka) sering jauh lebih buruk daripada pada hari kering, dan ini penting diketahui pengguna (Bab 5). Contoh kerja penuh (termasuk `threshold`, probabilitas, dan `crosstab`) tersedia di notebook `ch-09-08_studi_kasus_curah_hujan_terbuka.ipynb`.

Tabel 9.5: Contoh ringkas verifikasi per kategori (ilustratif).

Kategori	POD	FAR	CSI	Catatan
Tidak hujan / ringan (< 20)	0.90	0.15	0.78	baik, kelas mayoritas
Sedang (20 - <50)	0.45	0.40	0.32	perlu perbaikan
Lebat / sangat lebat (≥ 50)	0.20	0.55	0.15	sulit, kelas langka

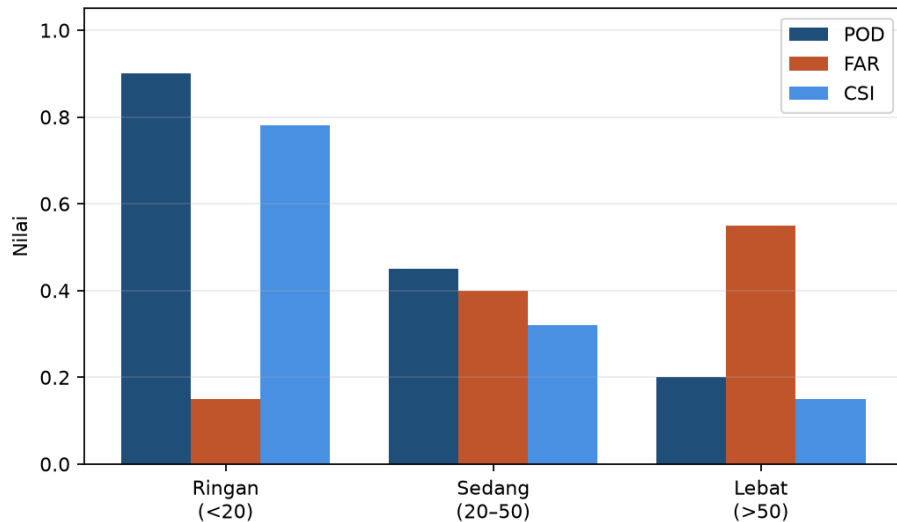
Gambar 9.2: Verifikasi per kategori intensitas (ilustratif).

Tabel 9.5 dan Gambar 9.2 menyingkap hal menarik: model “bagus” pada hari kering (POD tinggi, FAR rendah) justru paling lemah pada kejadian yang paling penting (lebat). Ini hasil yang layak dilaporkan apa adanya, bukan disembunyikan. Seluruh eksperimen bab ini berjalan di atas TensorFlow [9].

Apa yang harus dilaporkan (dan apa yang tidak)

Laporan studi kasus yang jujur biasanya berisi:

1. **Konteks & data** - lokasi/lintasan, rentang, sumber, lisensi, jumlah contoh.



Gambar 15: Gambar 9.2 - Verifikasi per kategori intensitas

2. **Metode** - fitur, window, arsitektur, baseline, skema walk-forward.
3. **Hasil** - MAE/RMSE (regresi), CSI/POD/FAR per threshold & kategori, plus rentang antar blok.
4. **Threshold yang dipilih & alasannya** - konteks operasional.
5. **Interpretasi** - fitur penting (dengan kewaspadaan), galat per musim.
6. **Keterbatasan** - data contoh vs nyata, satu/dua titik lokasi, tanpa optimasi menyeluruh.

Yang **tidak** perlu dilaporkan: klaim “akurasi 99%” tanpa metrik langka, angka tanpa baseline, atau kesimpulan kausal dari korelasi. Format ini langsung dipakai kembali di Bab 10 untuk keputusan produksi.

9.8 Latihan

Soal konsep

1. Mengapa klimatologi sering menjadi *baseline* yang menantang untuk hujan harian? Bagaimana cara menaikkannya?
2. Jelaskan trade-off POD-FAR: untuk mana Anda memilih threshold rendah pada konteks peringatan dini, dan untuk mana threshold tinggi pada konteks evakuasi mahal?
3. Mengapa metrik per kategori (Tabel 9.5) lebih informatif daripada satu akurasi global?
4. Apa risiko menafsirkan permutation importance pada fitur berkorelasi?
5. Kapan Anda lebih memercayai klimatologi daripada model GRU yang CSI-nya sedikit lebih tinggi?

Jawaban singkat yang diharapkan (untuk soal konsep)

1. Hujan harian didominasi banyak nol & variabel musim; klimatologi “cerdas” (per kalender) sudah menangkap musim - *baseline* yang sulit dikalahkan model yang hanya menebak persisten.
2. Peringatan dini $\rightarrow$ threshold rendah (POD tinggi, toleransi false alarm); evakuasi mahal $\rightarrow$ threshold tinggi (FAR rendah), meski banyak kejadian terlewat.
3. Hari kering mendominasi; akurasi global hampir 100% tanpa gagal menangkap lebat yang justru penting - metrik per kategori mengungkap distribusi kinerja.
4. Fitur berkorelasi membagi “kredit” di antara mereka; mengacak satu mengurangi konteks yang lain $\rightarrow$ angka menyesatkan.
5. Tidak langsung; cek rentang antar blok walk-forward & konteks operasional sebelum memutuskan - model harus mengalahkan baseline secara konsisten, bukan sekali.

Latihan praktik (notebook `ch-09-08_studi_kasus_curah_hujan_terbuka.ipynb`)

1. Gunakan data contoh harian satu titik lokasi; bangun fitur (Tabel 9.1).
2. Regresi: latih GRU dengan transformasi `log1p`; hitung MAE/RMSE; bandingkan dengan persistence & klimatologi pada *walk-forward* 3 blok.
3. Klasifikasi biner lebat vs tidak: evaluasi threshold 0,2/0,5/0,8 (Tabel 9.3); tetapkan threshold berdasarkan skenario (peringatan dini vs evakuasi) dan jelaskan.
4. Klasifikasi multi-kelas intensitas: buat crosstab (Kode 9.3) dan hitung CSI/POD/FAR per kategori (Tabel 9.5).
5. Tambahkan indeks MJO/ENSO; bandingkan CSI lebat dengan dan tanpa fitur tersebut.
6. (Proyek mini) Susun laporan satu halaman seperti Bab 8: konteks, data, metode, tabel hasil, threshold terpilih, interpretasi & keterbatasan.

Ringkasan

- Prediksi hujan berdampak langsung pada masyarakat; kejujuran framing & metrik wajib.
- Data: titik grid CHIRPS (target + lag), ERA5/ERA5-Land (regional), ENSO/MJO (indeks iklim); pola barat vs timur Indonesia berbeda dan perlu dilatih ulang per lokasi.
- Dua lintasan: regresi (mm, `log1p`, MAE/RMSE) dan klasifikasi (kategori, CSI/POD/FAR); menangani imbalance dengan `class_weight` & `threshold`.
- Threshold bukan 0,5 tetap - atur sesuai biaya galat; gunakan precision-recall untuk data langka (Tabel 9.3).
- Baseline kuat: klimatologi “cerdas” per kalender (juga persistence & ARI-MA); deep learning harus mengalahkannya secara konsisten (Tabel 9.4).
- Walk-forward multi-blok; laporkan rentang, bukan hanya rata-rata.
- Interpretasi dimulai dari permutation importance (deskriptif, bukan

kausal; SHAP di Bab 10).

- Laporkan verifikasi per kategori - kejadian langka paling penting & paling sulit (Tabel 9.5), lengkap dengan keterbatasan.

References

1. C. Funk et al., "The climate hazards infrared precipitation with stations - a new environmental record for monitoring extremes," *Scientific Data*, vol. 2, 150066, 2015, doi: 10.1038/sdata.2015.66. Data diunduh dari <https://data.chc.ucsb.edu/products/CHIRPS-2.0/> (Accessed: Sep. 2026).
2. Copernicus Climate Change Service (C3S), "ERA5: fifth generation ECMWF atmospheric reanalysis of the global climate," Copernicus Climate Data Store, [Online]. Available: <https://cds.climate.copernicus.eu> (Accessed: Sep. 2026).
3. H. Hersbach et al., "The ERA5 global reanalysis," *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999-2049, 2020, doi: 10.1002/qj.3803.
4. M. C. Wheeler and H. H. Hendon, "An all-season real-time multivariate MJO index: development of an index for monitoring and prediction," *Monthly Weather Review*, vol. 132, no. 8, pp. 1917-1932, 2004, doi: 10.1175/1520-0493(2004)132<1917:AARMMI>2.0.CO;2.
5. K. Wolter and M. S. Timlin, "Monitoring ENSO in COADS with a seasonally adjusted principal component index," in *Proc. 17th Climate Diagnostics Workshop*, 1993, pp. 52-57.
6. NOAA Physical Sciences Laboratory, "Nino3.4 SST index," [Online]. Available: <https://psl.noaa.gov/data/timeseries/monthly/NINO34/> (Accessed: Sep. 2026).
7. World Meteorological Organization, "WMO guidelines on the verification of operational forecasts," WMO-No. 1214, Geneva, Switzerland, 2018.
8. R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
9. M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2016. [Online]. Available: <https://arxiv.org/abs/1603.04467>

Bab 10 - Dari Riset ke Praktik: Operasional, Interpretasi, dan Arah ke Depan

Prasyarat: seluruh Bab 1-9. Bab ini adalah penutup: bagaimana model yang sudah terbukti bermanfaat di studi kasus dijalankan secara bertanggung jawab dan ke mana pembaca bisa melanjutkan belajar.

Catatan: Materi bab ini adalah **materi pengenalan**, bukan hasil riset baru. Seluruh isi merupakan ringkasan ulang literatur *machine learning*, dengan contoh-contoh yang dekat dengan dunia meteorologi Indonesia.

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, Anda diharapkan mampu:

1. **Merancang** monitoring *drift* dan strategi retraining/kalibrasi untuk model operasional.
2. **Mengkuantifikasi** ketidakpastian prediksi (interval/quantile, ensemble multi-seed).
3. **Menginterpretasi** model (SHAP) dan mengaitkannya dengan pengetahuan atmosfer.
4. **Menjelaskan** keterbatasan dan etika penggunaan deep learning di institusi (anti-overhype, disclaimer).
5. **Menyusun** peta jalan belajar lanjut (CNN, nowcasting, downscaling, generative).

10.1 Dari Notebook ke Operasional: Jembatan yang Sering Dilupakan

Membangun model yang akurat di notebook adalah bagian kecil dari pekerjaan. Bagian yang sulit, dan sering luput, adalah **menjaganya tetap berguna** setelah dipakai. Model operasional hidup dalam lingkungan yang berubah: cuaca tidak stasioner, sensor berubah, tata cara institusi berubah. Model yang “hebat” tahun lalu bisa salah tahun ini tanpa ada yang menyadarinya.

Tiga pertanyaan yang harus dijawab sebelum sebuah model disebut “produksi”:

1. **Bagaimana model dipanggil?** - REST API, batch harian, notebook terjadwal?
2. **Bagaimana model dipantau?** - pesan ketika metrik menurun?
3. **Siapa yang bertanggung jawab?** - ada penanggung jawab & prosedur saat model gagal?

Contoh “praktik produksi paling sederhana” untuk buku ini

Anda tidak perlu membangun Kubernetes untuk mengikuti bab ini. Contoh paling sederhana yang sudah memenuhi kebutuhan institusi kecil:

- **Skrip batch** - satu script dijalankan tiap hari (mis. *cron* atau *scheduled notebook*) yang memuat data terbaru, menjalankan model, menulis tabel prediksi.
- **Laporan** - output CSV/Excel di email atau folder bersama.
- **Log** - simpan setiap run (tanggal, input hash, metrik) agar bisa diaudit.

Kerangka ini menjaga prinsip: **model yang berguna adalah model yang dipakai**, dan dipakai secara terkontrol, bukan hanya sekali di eksperimen.

Buku ini tidak membahas infrastruktur secara mendalam (itu wilayah Bab engineering khusus), tetapi kerangka kerja di bawah ini memberi jalan yang jelas ke arah sana. Kerangka umum model deep learning dibahas di literatur dasar [1]; untuk siklus hidup model operasional dan *technical debt* sistem ML, lihat [2].

10.2 Monitoring *Drift*

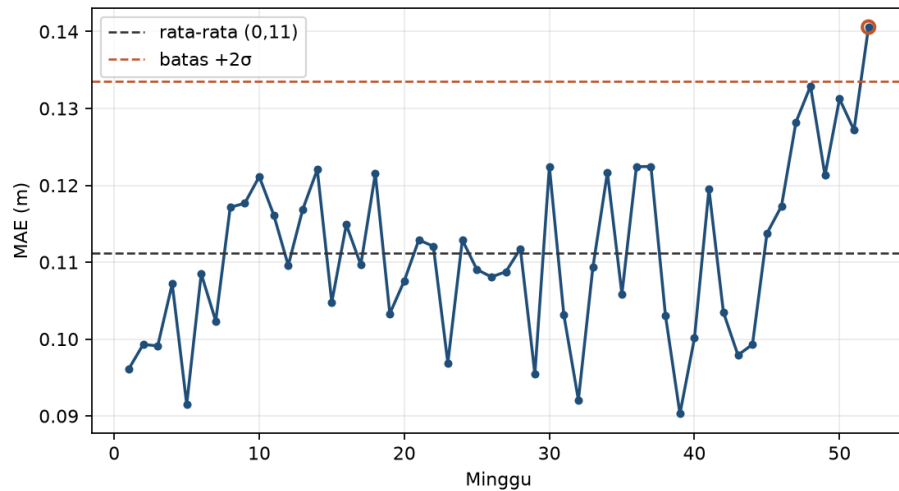
Atmosfer **tidak stasioner**: distribusi cuaca bergeser seiring musim, tahunan, dan dekade (perubahan iklim). Karena itu model yang dilatih pada 2010-2020 bisa “usang” pada 2025. Kerangka peramalan dan evaluasi berkala mengikuti prinsip pada literatur *forecasting* [3] dan pedoman verifikasi WMO [4]. Dua bentuk *drift* yang perlu diketahui:

- **Data drift** - distribusi *masukan X* berubah (misal stasiun pindah lokasi, kalibrasi sensor berubah, atau pola musim bulanan bergeser).
- **Concept drift** - hubungan $X \rightarrow y$ berubah (misal indeks MJO $\rightarrow$ hujan melemah karena perubahan iklim regional; atau relasi suhu-hujan bergeser).

Cara mendeteksi secara praktis:

1. **Pantau metrik operasional** - untuk model *regresi* pakai metrik kontinu (MAE/RMSE); untuk model *klasifikasi* pakai metrik kategorikal (CSI/POD/FAR). Hitung secara *periodik* (mingguan/bulanan) terhadap data terbaru yang sudah diverifikasi. Jika metrik menurun melebihi ambang, picu peringatan.
2. **Bandingkan distribusi fitur** - misal histogram *X* bulan ini vs rata-rata historis; pergeseran besar menandakan *data drift*.
3. **Grafik kendali sederhana** - gunakan rata-rata bergerak + batas 2 untuk melacak metrik; titik di luar batas = sinyal. Untuk data meteorologi yang autokorelasi dan musiman, batas 2 bisa memicu banyak *false alarm*; metode yang lebih tangguh (EWMA/CUSUM, dekomposisi musiman) layak dipertimbangkan setelah tahap awal ini.

Gambar 10.1: Grafik kendali MAE mingguan.



Gambar 16: Gambar 10.1 - Grafik kendali metrik operasional

Batas kendali: menetapkan ambang yang masuk akal

Ambang ± 2 pada grafik kendali (Gambar 10.1) hanyalah titik awal. Sesuaikan dengan:

- **Volatilitas alami metrik** - pada hujan, MAE/CSI berfluktuasi antar musim; ambang yang terlalu ketat akan sering “berbunyi” tanpa masalah nyata.
- **Biaya galat** - bila false alarm monitoring mahal (mis. menghentikan model padahal masih baik), lebih longgarkan; bila risiko nyata, percepat.
- **Periode evaluasi** - mingguan vs bulanan memberikan sensitivitas berbeda; pilih sesuai seberapa cepat Anda bisa bereaksi.

Aturan penting: **tetapkan ambang sebelum melihat data berjalan** (bukan setelah). Ini semacam *pra-registrasi* ambang: menetapkan ambang setelah melihat hasil berarti menyemai *selection bias* / *overfitting* pada noise monitoring (ingat prinsip evaluasi jujur di Bab 5); justru akan melewatkan degradation yang seharusnya terdeteksi.

Contoh penerapan monitoring pada kasus Bab 9

Bayangkan model hujan stasiun (Bab 9) dipakai secara operasional untuk peringatan dini. Monitoring praktisnya:

- **Setiap 1 bulan**: hitung CSI/POD/FAR terhadap data yang sudah dikonfirmasi selama bulan itu; simpan ke tabel.
- **Setiap bulan**: bandingkan distribusi (histogram) fitur **X** bulan ini vs rata-rata historis (mis. `rmm1`, `mus_sin/cos`).

- **Tiap kuartal:** tinjau kurva kendali; bila >1 titik keluar batas, selidiki dan nilai apakah perlu kalibrasi/retrain.

Memiliki jadwal dan penanggung jawab sedini mungkin, sebelum model “mulai produksi”, menghindari kejadian model diam-diam rusak (Bab 10.1 membahas “siapa yang menjawab?”). Contoh produksi paling sederhana di sub-bab ini adalah titik awal minimal: institusi dengan kebutuhan lebih besar dapat menambah orkestrasi, versi, dan pengujian berjenjang seiring kebutuhan.

10.3 Retraining dan Kalibrasi Ulang

Ketika *drift* terdeteksi, pilihan tindakan (dari yang paling ringan):

1. **Kalibrasi ulang output** - sesuaikan threshold (Bab 9) tanpa melatih ulang; cepat dan murah.
2. **Retraining berkala terjadwal** - misal tahunan/musiman; jadwalkan, jangan menunggu darurat.
3. **Retraining berbasis sinyal** - pemicu saat metrik turun (Bab 10.2); lebih responsif tetapi perlu disiplin evaluasi.
4. **Migrasi model baru** - jika data/fitur berubah besar, kembangkan model baru dengan proses studi kasus kembali.

Aturan retraining yang jujur

- **Jangan mencampur data lama & baru sembarangan** - retraining harus menaikkan metrik pada *walk-forward* yang benar, bukan hanya pada pelatihan.
- **Simpan versi model** - riwayat rilis dan *baseline* lama (Bab 8-9) untuk perbandingan & rollback.
- **Catat muatan latih** - tanggal data, preprocessing, seed, versi (Bab 6 §6.9).

Retraining bukan perbaikan otomatis: ia menjalankan kembali seluruh disiplin evaluasi buku ini pada data yang lebih baru.

Contoh alur keputusan retraining

Model **regresi** (mis. tinggi muka air, Bab 8):

Sinyal terdeteksi	Investigasi	Tindakan
MAE naik 10% dalam 1 bulan	cek distribusi X, cek data baru	kalibrasi output (interval/kuantil) dulu
Fitur <code>rmm1</code> bergeser jauh	bandingkan dokumentasi	jadwalkan retrain + cek baseline
Metrik turun drastis ($>20\%$)	periksa data & sensor	model baru dengan studi kasus kembali

Model **klasifikasi** (mis. hujan/level bahaya, Bab 9):

Sinyal terdeteksi	Investigasi	Tindakan
CSI turun 10% dalam 1 bulan	cek distribusi X, cek label baru	kalibrasi <i>threshold</i> (Bab 9) dulu
Distribusi kelas bergeser	bandingkan dokumentasi	jadwalkan retrain + cek baseline
POD/FAR berubah drastis	periksa data & sensor	model baru dengan studi kasus kembali

Tabel 10.2: Alur keputusan saat *drift* terdeteksi.

Tabel 10.2 (dipisah untuk model regresi dan klasifikasi) menekankan: **investigasi sebelum bertindak**. Tidak semua kenaikan metrik adalah masalah model; bisa jadi masalah data baru atau definisi yang berubah.

10.4 Ketidakpastian Prediksi

Prediksi tunggal (satu angka MAE, satu angka mm) menyesatkan: operasional butuh tahu **seberapa yakin**. Beberapa pendekatan yang masuk akal untuk buku ini, termasuk gagasan ketidakpastian Bayesian pada jaringan saraf [5]:

Ensemble multi-seed

Latih model yang sama dengan beberapa **seed** berbeda; prediksi = rata-rata, sebaran () antar anggota sebagai indikator kestabilan. Sederhana dan langsung:

Kode 10.1 - Ensemble multi-seed.

```
import numpy as np, tensorflow as tf

preds = []
for seed in [1, 2, 3]:
    tf.random.set_seed(seed)
    m = build_model()           # model yang sama
    m.fit(Xtrain, ytrain, verbose=0)
    preds.append(m.predict(Xtest, verbose=0).ravel())

bayangan = np.stack(preds)      # (3, n_test)
p_mean = bayangan.mean(axis=0)
p_std = bayangan.std(axis=0)
print("Prediksi rata-rata:", p_mean[:5])
print("Sebaran antar-run (±1):", p_std[:5])
```

Apa yang diukur ini? Sebaran antar-*seed* terutama menangkap variasi inisialisasi/optimisasi, bukan ketidakpastian prediktif penuh (aleatorik + epistemik). Karena semua anggota berbagi data,

arsitektur, dan hyperparameter yang sama, ukuran ini bisa **mengecilkan** ketidakpastian sebenarnya. Perlakuan yang lebih bermakna (opsional, di luar cakupan buku ini) mencakup MC-dropout, *deep ensembles* dengan variasi arsitektur/hyperparameter, regresi kuantil (Kode 10.2), dan *conformal prediction*.

Manfaat tambahan: ensembel juga **menstabilkan angka metrik**; MAE/CSI dari rata-rata ensembel sering lebih rendah variansnya daripada satu run acak. Ini menjadikan ensembel alat ganda: lebih “tenang” dalam laporan dan memberi ukuran kestabilan. Biayanya linear dengan jumlah anggota; untuk 3-5 seed masih sangat wajar di Colab.

Interval kuantil

Lapisan akhir memprediksi beberapa kuantil sekaligus, misal median (50%) serta kuantil 10% dan 90%, dengan *pinball loss* / quantile regression. Hasilnya: interval [q10, q90] yang memberi rentang “kisaran 80%” prediksi. Ini jauh lebih informatif daripada satu angka.

Kode 10.2 - Latih model regresi kuantil sederhana (3 kuantil dengan satu *loss*).

```
import tensorflow as tf

quantiles = [0.10, 0.50, 0.90]

def loss_kuantil(qs):
    def _loss(y_true, y_pred):
        y_true = tf.reshape(y_true, (-1, 1))      # (batch, 1)
        err = y_true - y_pred                     # (batch, n_kuantil)
        qs_t = tf.constant(qs, dtype=tf.float32)  # (n_kuantil,)
        return tf.reduce_mean(tf.maximum(qs_t * err, (qs_t - 1) * err))
    return _loss

m = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(len(quantiles))]        # 3 unit: q10, q50, q90
# satu output layer dengan satu custom loss untuk semua kuantil
m.compile(optimizer="adam", loss=loss_kuantil(quantiles))
m.fit(X_train, y_train, epochs=50, verbose=0)

preds = m.predict(X_test, verbose=0)              # (n_test, 3): q10, q50, q90
q10, q50, q90 = preds[:, 0], preds[:, 1], preds[:, 2]
```

Perhatikan: karena semua kuantil keluar dari satu *output layer* (*Dense*(3)), cukup satu *loss* kustom yang menerima *y_pred* berbentuk (batch, n_kuantil) dan menghitung *pinball loss* per kuantil; bukan daftar beberapa *loss* yang hanya berlaku untuk model dengan beberapa *output layer* terpisah. Baris

`tf.reshape(y_true, (-1, 1))` membuat *loss* kebal terhadap bentuk `y_train`: baik `(n,)` maupun `(n, 1)`.

Catatan penting: ketidakpastian dari model **belum tentu kalibrasi**; interval 80% bisa benar hanya 50% dari waktu bila model terlalu yakin. Kalibrasi (misal *conformal prediction*) adalah topik lanjut yang layak dikejar setelah buku ini (lihat FAQ §10.9).

Kapan melaporkan ketidakpastian?

Tidak semua output harus selengkap itu; atur sesuai dampak:

- **Peringatan dini / keputusan risiko** → sangat dianjurkan menyertakan interval & ensembel.
- **Informasi rutin** → satu angka + toleransi sudah cukup.
- **Publikasi/replikasi** → minimal ensembel multi-seed agar ada ukuran kestabilan.

Kapan pun ketidakpastian dimaknai sebagai “kisaran”, nyatakan juga **konteks kalibrasi** atau akui bahwa itu merupakan sebaran antar-seed (bukan jaminan statistika penuh).

10.5 Interpretasi Lanjut dengan SHAP

Bab 9 mulai dengan *permutation importance*. Bab ini melengkapinya dengan **SHAP** (SHapley Additive exPlanations, [6]) yang memberi:

- **Importance global** - kontribusi rata-rata tiap fitur terhadap prediksi (memberi gambaran lebih kaya daripada permutation importance; tetapi pada fitur yang berkorelasi kuat, pembagian kredit SHAP juga bisa menyesatkan - bandingkan dengan permutation importance dan pengetahuan fisis).
- **Interpretasi lokal** - mengapa *satu* prediksi tertentu bernilai X (misal “hari ini diprediksi lebat”).

Kode 10.3 - SHAP pada model Keras (ringkas).

```
import shap

explainer = shap.Explainer(m.predict, X_train_sample)
nilai_shap = explainer(X_test_sample)
shap.plots.beeswarm(nilai_shap)
```

Interpretasi SHAP harus selalu ditautkan ke **pengetahuan atmosfer**:

- Fitur yang “penting” tapi secara fisis tidak masuk akal → perlu investigasi: cek kebocoran data, fitur proksi, atau interaksi non-linear yang tak terduga (belum tentu berarti data “rusak”).
- Fitur yang fisis masuk akal dan penting → menaikkan kepercayaan praktisi (misal `hujan_t1`, `rmm1` untuk hujan barat).

- Interpretasi lokal membantu menjawab “kenapa peringatan dikeluarkan hari ini?” - pertanyaan yang hampir selalu diajukan di ruang operasional.

Contoh interpretasi lokal sederhana

Misal untuk satu prediksi “hujan lebat” hari ini, SHAP menunjukkan kontribusi terbesar dari `hujan_t2` (kemarin hujan besar) dan `rmm1` (fase MJO basah). Jawaban yang bisa disampaikan ke pemangku: “model ini menilai kondisi basah yang berlanjut dan osilasi musiman sebagai pendorong; silakan periksa juga prediksi model dinamik nasional untuk konfirmasi.” Ini mengubah “kotak hitam” menjadi bahan diskusi yang transparan; persis tujuan interpretasi.

Kewaspadaan terhadap kausalitas

SHAP menjelaskan **kontribusi dalam model**, bukan penyebab di dunia nyata (ingat peringatan di Bab 9). Korelasi kuat, bukan sebab-akibat. Untuk klaim kausal, perlakukan sangat hati-hati (misal dijelaskan terbatas pada “berasosiasi dengan” bukan “menyebabkan”).

10.6 Keterbatasan dan Etika

Ini penutup penting dan selaras dengan *Risk Management* umbrella. Tiga area:

1. Anti-overhype

- Jangan mengklaim “menggantikan peramal”; klaim yang benar: “memberi probabilitas terkalibrasi tambahan (untuk model klasifikasi) atau skor/interval yang diverifikasi dengan metrik tertentu”.
- Sertakan *baseline* & metrik jujur (skill score, CSI) serta keterbatasan (Bab 8-9).
- Hindari kata “akurasi 99%” tanpa konteks fenomena langka.

2. Tanggung jawab & keamanan

- Model tidak berhak mengambil keputusan akhir untuk keselamatan publik; ia alat bantu.
- Pastikan ada manusia yang lalu memverifikasi & bertanggung jawab atas keputusan.
- Data pribadi/lokasi sensitif: patuhi aturan institusi & perundang-undangan.

3. Bias data

- Data historis bisa mencerminkan bias (misal stasiun yang terlalu sedikit di wilayah timur membuat model kurang mewakili). Akui dalam laporan dan jangan gegabah menggeneralisasi ke seluruh Indonesia.

Contoh nyata: model hujan yang dilatih hanya pada stasiun Jawa mungkin “berhasil” di sana tetapi miskin transfer ke Papua, bukan karena model buruk, tetapi karena representasi data tidak seimbang. **Jangka waktu & wilayah cakupan harus dinyatakan** sebagai bagian dari laporan; ini termasuk *bias data*, bukan sekadar catatan kecil.

4. Komunikasi kepada publik

Ketika hasil dipublikasikan (artikel, media sosial), aturan praktis:

- Gunakan kalimat yang tenang, berbasis data; hindari kata “menggantikan”.
- Lampirkan tautan/metode agar pembaca bisa memeriksa (transparansi).
- Bila media menanyakan “apakah akurat?”, jawab dengan metrik & batas, bukan sensasi.

Komunikasi yang hati-hati melindungi kredibilitas institusi sekaligus kepercayaan publik, bagian dari *risk management* yang dipakai sepanjang buku.

Tabel 10.3: Etika penggunaan yang dianjurkan.

Lakukan	Hindari
Laporkan metrik & baseline	Klaim tanpa verifikasi
Beri tahu ketidakpastian	Prediksi tunggal “pasti”
Manusia memutuskan	Otomatisasi keputusan berisiko
Akui keterbatasan data	Menggeneralisasi dari sedikit stasiun
Tautkan interpretasi ke fisis	Klaim kausal dari korelasi

10.7 Arah Riset dan Bab Lanjutan

Di sinilah pembaca melebarkan sayapnya. Tiga arah besar yang paling relevan untuk meteorologi + deep learning:

1. CNN dan data spasial

- **Nowcasting** (prediksi 0-6 jam) - memakai radar/satelit (urutan gambar) dengan ConvLSTM atau model *vision*; aplikasi langsung untuk peringatan sangat pendek.
- **Downscaling** - meningkatkan resolusi *reanalysis*/model iklim ke skala lokal dengan saraf konvolusi; penting untuk proyeksi iklim di Indonesia.
- **Benchmark** - *dataset* standar seperti WeatherBench [7] (prediksi global jangka menengah) memudahkan membandingkan metode secara adil. Catatan: untuk *nowcasting* 0-6 jam, gunakan *dataset* radar/satelit (mis. radar lokal, SEVIR, MeteoNet), karena WeatherBench tidak dirancang untuk horizon itu.

2. Transfer learning & model pra-latih

Pindahkan pengetahuan model yang dilatih pada tugas/domain besar (misal model cuaca global, *reanalysis*, citra satelit, atau model pra-latih spatiotemporal) untuk masalah lokal dengan data terbatas. Untuk meteorologi, *pretraining* lintas stasiun bisa menghemat data.

3. Generative model untuk pengisian data & skenario

- **Imputasi data hilang** dengan model generatif (di sisi data, lihat Bab 6).
- **Skenario iklim** atau sintesis kondisi ekstrem untuk stress-test sistem.

Pandangan luas tentang deep learning untuk sains kebumiharian (termasuk arah-arah di atas) dapat dibaca pada ulasan Reichstein et al. [8]. Seluruh contoh di bab ini dikembangkan dengan TensorFlow [9].

Sumber belajar lanjut yang direkomendasikan (di luar buku ini):

- Dataset & benchmark publik (misal WeatherBench, dataset radar).
- Literatur per topik (lihat daftar berikut & Bab 6-9 referensi).

10.8 Komunitas dan Peta Belajar

Buku ini selesai, tetapi belajar tidak. Rekomendasi berjenjang:

1. **Membaca** - literatur pada Referensi bab ini + bab 6-9.
2. **Berlatih** - kerjakan ulang Bab 8-9 dengan data nyata stasiun sendiri.
3. **Terhubung** - komunitas (misal grup ML Indonesia, komunitas data meteorologi/akademik), konferensi (misal *IDEA / Indonesia AI Day*), dan *open-source* (GitHub proyek sejenis).
4. **Publikasikan** - tulis catatan singkat (blog/notebook publik), dapatkan umpan balik - persis seperti siklus buku ini (bab per blog, DOI, dll).

Rencana 30-60-90 hari setelah membaca buku ini

Agar tidak sekadar “selesai dibaca”, pakai jadwal sederhana:

- **30 hari**: ulangi Bab 8 dengan data nyata satu stasiun pasang surut; tulis laporan satu halaman (konteks, metode, hasil, keterbatasan).
- **60 hari**: ulangi Bab 9 dengan dua stasiun (barat & timur); tambahkan indeks MJO nyata; selesaikan tabel verifikasi per kategori.
- **90 hari**: pilih satu arah lanjut (Bab 10.7) - misal CNN untuk nowcasting - dan buat prototipe kecil dengan data publik (untuk nowcasting: radar/satelit, mis. SEVIR atau dataset radar; untuk prediksi jangka menengah: WeatherBench [7]).

Jadwal ini memastikan keterampilan tertanam lewat proyek, bukan sekadar dibaca, dan langkah-langkahnya persis cara penulis membangun buku ini.

10.9 FAQ Singkat

Apakah saya “harus” membuat API? Tidak selalu. Untuk institusi kecil, model bisa dijalankan sebagai *batch* terjadwal (notebook/script) yang menghasilkan tabel prediksi. API diperlukan ketika banyak pengguna memanggil secara interaktif. Mulai dari yang paling sederhana yang memenuhi kebutuhan.

Bagaimana saya tahu model “rusak” tanpa menunggu keluhan? Justru gunakan monitoring (Bab 10.2): metrik berkala + grafik kendali. Jangan menunggu pengguna menyadari.

Retraining setiap musim itu melelahkan; adakah cara ringan? Mulai dari kalibrasi threshold (murah), lalu retraining hanya bila sinyal menuntut. Jadwalkan juga retraining tahunan ringan sebagai “asuransi”.

SHAP apakah wajib? Tidak, tapi sangat membantu di institusi yang menuntut penjelasan. Mulai dari permutation importance (Bab 9) bila SHAP terasa berat; naikkan ke SHAP untuk interpretasi lokal.

Apakah *generative model* untuk data iklim sudah siap dipakai? Beragam. Beberapa telah dipakai untuk *downscaling* (misal super-resolution), tetapi untuk skenario iklim masih riset aktif; perlakukan dengan kehati-hatian dan validasi fisik (Bab 10.6).

Bagaimana jika institusi saya tidak punya GPU? Untuk model sekecil studi kasus di buku ini, Colab gratis umumnya cukup dan tidak butuh GPU kuat; namun ketersediaan GPU di Colab gratis **tidak dijamin** (ada batas waktu dan kuota). Bagi yang membutuhkan kapasitas lebih, pertimbangkan Colab Pro atau sumber daya institusi; sesuaikan dengan kebutuhan, bukan gengsi.

Apa itu *conformal prediction* yang disebut di §10.4? Ini teknik kalibrasi yang mengubah skor/interval model menjadi jaminan cakupan dengan asumsi minimum (pertukaran data), sehingga interval yang dilaporkan bisa dipercaya secara statistika. Contoh ringkasnya: dari regresi kuantil (Kode 10.2) dihitung residual pada *calibration set*, lalu batas interval digeser sehingga cakupan empiris mendekati target (mis. 80%). Pembahasan penuh berada di luar buku ini; cukup ketahui bahwa interval “mentah” dari model belum tentu terkalibrasi (lihat §10.4).

10.10 Penutup: Dari Pembaca Menjadi Praktisi

Sepuluh bab telah membawa Anda dari “apa itu deep learning?” hingga “bagaimana model dijalankan secara bertanggung jawab”. Inti yang ingin ditanamkan bukanlah kumpulan rumus atau kode, melainkan **sikap praktisi**:

- Mulai dari masalah dan *baseline*, bukan dari keinginan memakai “model keren”.
- Selalu ukur dengan metrik yang benar dan konteks operasional.
- Jujur tentang keterbatasan - itu yang membangun kepercayaan.
- Terus belajar dengan data nyata dan berbagi kepada komunitas.

Buku ini adalah permulaan, bukan akhir. Jadikan studi kasus Cilacap (pasang surut) dan prediksi hujan (Bab 9) sebagai batu loncatan; jadikan setiap masalah lokal sebagai latihan berikutnya. Selama Anda memegang disiplin “ukur dulu, klaim kemudian”, Anda sudah berada di jalur yang benar.

10.11 Latihan

Soal konsep

1. Beda *data drift* dan *concept drift* - beri contoh kasus hujan.
2. Kapan retraining berbasis sinyal lebih baik daripada terjadwal? Kapan sebaliknya?
3. Mengapa ketidakpastian penting dalam prediksi operasional?
4. Bagaimana membedakan “penting secara fisis” dari “penting secara statistik” pada interpretasi SHAP?
5. Sebutkan tiga klaim yang *tidak boleh* Anda buat ketika melaporkan model prediksi hujan.

Latihan praktik (notebook `ch-10-09_operasional_arah_riset.ipynb`)

6. Ambil salah satu model Bab 8/9; hitung grafik kendali sederhana MAE mingguan dan tandai titik yang keluar batas (simulasikan satu periode dengan galat tinggi).
7. Latih ensemble multi-seed (Kode 10.1); bandingkan MAE rata-rata vs MAE model tunggal; laporkan sebaran antar-run (bukan klaim ketidakpastian statistika penuh).
8. Terapkan regresi kuantil sederhana (Kode 10.2) dan hitung cakupan interval 80% pada data uji.
9. Jalankan SHAP (Kode 10.3) pada model Bab 9; tampilkan 5 fitur teratas dan kaitkan dengan pengetahuan atmosfer.
10. (Proyek) Tulis “rencana operasional singkat” untuk salah satu studi kasus: monitoring, retraining, ketidakpastian, interpretasi, batas etika - satu halaman.

Ringkasan

- Model operasional butuh monitoring drift (data & concept) via metrik & grafik kendali.
- Retraining: kalibrasi → jadwal → sinyal → model baru; simpan versi & muatan latih; investigasi sebelum bertindak (Tabel 10.2).
- Ketidakpastian: ensemble multi-seed dan interval kuantil; waspada kalibrasi.
- Interpretasi: SHAP (global + lokal), selalu ditautkan ke fisis, bukan kausal.
- Etika: anti-overhype, tanggung jawab keputusan pada manusia, akui bias data, komunikasi publik yang tenang dan transparan (Tabel 10.3).

- Arah lanjut: CNN (nowcasting, downscaling), transfer learning, generative model, didukung benchmark (WeatherBench) dan literatur (Reichstein).
- Belajar berlanjut: baca, praktik data nyata, terhubung komunitas, publikasikan.

References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
2. D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 2503-2511. [Online]. Available: <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems> (Accessed: Sep. 2026)
3. R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/> (Accessed: Sep. 2026)
4. World Meteorological Organization, “WMO guidelines on the verification of operational forecasts,” WMO-No. 1214, Geneva, Switzerland, 2018.
5. Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: representing model uncertainty in deep learning,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2016. [Online]. Available: <https://arxiv.org/abs/1506.02142> (Accessed: Sep. 2026)
6. S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765-4774. [Online]. Available: <https://arxiv.org/abs/1705.07874> (Accessed: Sep. 2026)
7. S. Rasp et al., “WeatherBench: a benchmark data set for data-driven weather forecasting,” *Journal of Advances in Modeling Earth Systems*, vol. 12, no. 10, 2020, doi: 10.1029/2020MS002203.
8. M. Reichstein et al., “Deep learning and process understanding for data-driven Earth system science,” *Nature*, vol. 566, no. 7743, pp. 195-204, Feb. 2019, doi: 10.1038/s41586-019-0912-1.
9. M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016. [Online]. Available: <https://arxiv.org/abs/1603.04467> (Accessed: Sep. 2026)

Lampiran

Lampiran A — Reproduksiabilitas dan Lingkungan Komputasi

Buku ini dirancang agar seluruh kode dapat dijalankan ulang (reproducible). Konvensi yang dipakai di seluruh notebook:

- **seed tetap:** `np.random.seed(42)` dan `tf.random.set_seed(42)` di awal setiap notebook agar hasil dapat diulang.
- **Catatan GPU:** meskipun *seed* sudah diatur, eksekusi pada GPU (terutama operasi reduksi di balik cuDNN) kadang bersifat *non-deterministic*. Perbedaan kecil pada angka desimal terakhir (mis. sekitar 0.0001 pada MAE) antar-run di Colab adalah wajar dan bukan tanda galat.
- **Versi library:** versi TensorFlow/Keras yang digunakan tercatat di notebook (biasanya di sel pertama) dan di metadata *release* Zenodo.
- **Python:** 3.10+; paket inti: `tensorflow`, `numpy`, `pandas`, `matplotlib`. Paket tambahan per bab:
 - Bab 6: `xarray`, `netCDF4` (opsional, untuk data NetCDF) dan `cf_grib` (opsional, untuk GRIB).
 - Bab 8: `requests` untuk skrip unduh IOC/UHSLC/PSMSL.
 - Bab 9: `scikit-learn` (opsional, permutasi penting), `shap` (opsional).
- **Data:** data contoh sintetik di-*commit* di repositori; data riil diunduh melalui skrip yang disediakan (lihat “Daftar Dataset dan Sumber”). Snapshot dataset studi kasus akan diunggah ke Zenodo saat rilis.

Troubleshooting Colab (dibutuhkan terutama saat latih model besar, Bab 7-10):

- **Sesi terputus.** Colab gratis ber-batas waktu sesi (12 jam, atau lebih singkat bila diam). Simpan notebook dan data ke Google Drive secara berkala:

```
from google.colab import drive
drive.mount('/content/drive')
```
- **Runtime tidak GPU.** Colab tidak selalu menyediakan GPU; coba lagi nanti, atau gunakan TPU. Untuk latih Bab 1-6 CPU cukup; studi kasus (Bab 8-9) GPU mempermudah.
- **Memori habis.** Dataset besar (jutaan baris) dapat kehabisan RAM. Muat data dalam *chunk* (potongan) atau gunakan sampling (strategi detail di Bab 6).
- **Versi library berubah.** Colab kadang memperbarui TensorFlow. Jika notebook tiba-tiba galat, periksa versi dengan `print(tf.__version__)` dan lihat catatan perubahan (changelog) TensorFlow.

Lampiran B — Konvensi Penulisan dan Penomoran Aset

Konvensi	Aturan
Penomoran aset	<code>Gambar bab.nomor</code> , <code>Tabel bab.nomor</code> , <code>Persamaan (bab.nomor)</code> , <code>Kode bab.nomor</code> — deret nomor tiap jenis reset di tiap bab
Rujukan aset	setiap aset harus dirujuk di teks; nomor tidak boleh ganda atau loncat
Notasi	terpusat di “Glosarium dan Notasi”; istilah Indonesia + Inggris konsisten
Sitasi	IEEE bernomor <code>[n]</code> sesuai kemunculan pertama; daftar per bab di <code>refs.bib</code> ; daftar agregat di “Daftar Pustaka”
Istilah asing	dicetak miring bila belum diserap KBBI (PUEBI §18)
Lesapnya nama merek	TensorFlow, Keras, Google Colab, NumPy, dst. tidak dimiringkan

Register lengkap penomoran aset dan sitasi per bab ada di `REGISTER.md` (berkas pendamping pengembangan, tidak dirilis sebagai bab).

Daftar Pustaka

Daftar pustaka agregat ini menggabungkan seluruh referensi yang dipakai di Bab 1–10 (sumber: `refs.bib` tiap bab). Di tiap bab, referensi juga dicantumkan per-bab (format IEEE, nomor mengikuti kemunculan pertama). Standar sitasi dan hierarki prioritas sumber mengikuti `outline.md` → “Kriteria Sitasi” (IEEE; DOI wajib bila tersedia; URL hanya bila tanpa DOI, dengan tanggal akses).

1. Buku Teks dan Referensi Rujukan Umum

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org>
2. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Harlow, UK: Pearson, 2021. ISBN 978-0134610993.
3. T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997. ISBN 978-0070428072.
4. R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
5. D. T. Pugh and P. L. Woodworth, *Sea-Level Science: Understanding Tides, Surges, Tsunamis and Mean Sea-Level Changes*. Cambridge, UK: Cambridge University Press, 2014.
6. I. T. Jolliffe and D. B. Stephenson, Eds., *Forecast Verification: A Practitioner's Guide in Atmospheric Science*, 2nd ed. Chichester, UK: Wiley, 2011. doi: 10.1002/9781119960003.

2. Artikel Jurnal *Peer-Reviewed*

7. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. doi: 10.1037/h0042519.
8. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986. doi: 10.1038/323533a0.
9. Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi: 10.1038/nature14539.
10. M. Reichstein et al., “Deep learning and process understanding for data-driven Earth system science,” *Nature*, vol. 566, no. 7743, pp. 195–204, 2019. doi: 10.1038/s41586-019-0912-1.
11. J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990. doi: 10.1207/s15516709cog1402_1.
12. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
13. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov,

- nov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
14. M. C. Wheeler and H. H. Hendon, “An all-season real-time multivariate MJO index: Development of an index for monitoring and prediction,” *Monthly Weather Review*, vol. 132, no. 8, pp. 1917–1932, 2004. doi: 10.1175/1520-0493(2004)132<1917:AARMMI>2.0.CO;2.
 15. C. J. Willmott, “On the validation of models,” *Physical Geography*, vol. 2, no. 2, pp. 184–194, 1981.
 16. C. J. Willmott, S. M. Robeson, and K. Matsuura, “A refined index of model performance,” *International Journal of Climatology*, vol. 32, no. 6, pp. 573–580, 2012.
 17. H. V. Gupta, H. Kling, K. K. Yilmaz, and G. F. Martinez, “Decomposition of the mean squared error and NSE performance criteria: Implications for improving hydrological modelling,” *Journal of Hydrology*, vol. 377, no. 1–2, pp. 80–91, 2009. doi: 10.1016/j.jhydrol.2009.08.003.
 18. S. Lestari, A. King, C. Vincent, D. Karoly, and A. Protat, “Seasonal dependence of rainfall extremes in and around Jakarta, Indonesia,” *Weather and Climate Extremes*, vol. 24, Art. no. 100202, 2019. doi: 10.1016/j.wace.2019.100202.
 19. H. Hersbach et al., “The ERA5 global reanalysis,” *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999–2049, 2020. doi: 10.1002/qj.3803.
 20. C. Funk et al., “The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes,” *Scientific Data*, vol. 2, Art. no. 150066, 2015. doi: 10.1038/sdata.2015.66.
 21. S. J. Holgate et al., “New data systems and products at the Permanent Service for Mean Sea Level,” *Journal of Coastal Research*, vol. 29, no. 3, pp. 477–479, 2013. doi: 10.2112/JCOASTRES-D-12-00175.1.
 22. S. Rasp et al., “WeatherBench: A benchmark data set for data-driven weather forecasting,” *Journal of Advances in Modeling Earth Systems*, vol. 12, no. 10, Art. no. e2020MS002203, 2020. doi: 10.1029/2020MS002203.

3. Artikel dan Prosiding Konferensi (termasuk *Preprint*)

23. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105. Reissued in *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017. doi: 10.1145/3065386.
24. K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734. (Preprint: arXiv:1406.1078.)
25. I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Proc. Advances in Neural Information*

- Processing Systems (NeurIPS)*, 2014, pp. 3104–3112. (Preprint: arXiv:1409.3215.)
26. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015. (Preprint: arXiv:1412.6980.)
 27. D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.
 28. Y. Gal and Z. Ghahramani, “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2016, pp. 1050–1059. (Preprint: arXiv:1506.02142.)
 29. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 1321–1330. (Preprint: arXiv:1706.04596.)
 30. S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4765–4774. (Preprint: arXiv:1705.07874.)
 31. N. S. Keskar et al., “On large-batch training for deep learning: Generalization gap and sharp minima,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017. (Preprint: arXiv:1609.04836.)
 32. A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, “The marginal value of adaptive gradient methods in machine learning,” in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017. (Preprint: arXiv:1705.08292.)
 33. H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, “Visualizing the loss landscape of neural nets,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2018. (Preprint: arXiv:1712.09913.)
 34. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019. (Preprint: arXiv:1711.05101.)
 35. K. Wolter and M. S. Timlin, “Monitoring ENSO in COADS with a seasonally adjusted principal component index,” in *Proc. 17th Climate Diagnostics Workshop*, Norman, OK, USA, 1993, pp. 52–57.

4. Laporan Teknis dan Perangkat Lunak

36. World Meteorological Organization (WMO), “WMO guidelines on the verification of operational forecasts,” WMO, Geneva, Switzerland, Tech. Rep. WMO-No. 1214, 2018. [Online]. Available: <https://library.wmo.int/diakses>: September 2026).
37. M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2016, arXiv:1603.04467 [cs.DC]. [Online]. Available: <https://arxiv.org/abs/1603.04467>

5. Dataset dan Sumber Data

38. C. Funk et al., “The climate hazards infrared precipitation with stations (CHIRPS) - a new environmental record for monitoring extremes,” *Scientific Data*, vol. 2, 150066, 2015, doi: 10.1038/sdata.2015.66.
39. Copernicus Climate Change Service (C3S), “ERA5: Fifth generation ECMWF atmospheric reanalysis of the global climate,” 2026. [Online]. Available: <https://cds.climate.copernicus.eu> (diakses: September 2026).
40. NOAA National Centers for Environmental Information (NCEI), “GHCN-Daily,” 2026. [Online]. Available: <https://www.ncei.noaa.gov/pub/data/ghcn/daily/> (diakses: September 2026).
41. Badan Informasi Geospasial (BIG), “Peta pasang surut dan pola pasut perairan Indonesia,” 2026. [Online]. Available: <https://tides.big.go.id> (diakses: September 2026).
42. Badan Informasi Geospasial (BIG), “Informasi genangan rob dan pola pasut perairan Indonesia,” 2026. [Online]. Available: <https://tides.big.go.id> (diakses: September 2026).
43. Permanent Service for Mean Sea Level (PSMSL), “Global sea level data,” 2026. [Online]. Available: <https://psmsl.org> (diakses: September 2026).
44. UNESCO/IOC, “Sea Level Station Monitoring Facility,” 2026. [Online]. Available: <https://www.ioc-sealevelmonitoring.org> (diakses: September 2026).
45. University of Hawaii Sea Level Center (UHSLC), “Research quality tide gauge data (hourly/daily),” 2026. [Online]. Available: <https://uhslc.soest.hawaii.edu/data> (diakses: September 2026).
46. NOAA Physical Sciences Laboratory, “Nino3.4 SST index,” 2026. [Online]. Available: <https://psl.noaa.gov/data/timeseries/monthly/NINO34> (diakses: September 2026).

Catatan verifikasi: sebelum *release* final, seluruh DOI di atas diuji ulang melalui doi.org/doi; entri web diverifikasi tanggal aksesnya. Untuk daftar *per bab* (bernomor sesuai kemunculan), lihat bagian “Referensi” pada akhir masing-masing bab atau **refs.bib** di **manuscripts/ch-NN-*/**.

Indeks

Indeks ini disusun **tematis per bagian**, bukan per halaman — karena nomor halaman buku dapat berubah antar-edisi. Saat PDF di-render, indeks bernomor halaman dapat dihasilkan otomatis oleh LaTeX (mis. `\makeindex`) dari kata-kunci berikut.

A

- Adam (optimizer) — Bab 2, 4
- Analisis harmonik — Bab 8
- ARIMA — Bab 5, 7, 9
- *Artificial intelligence* (AI, kecerdasan buatan) — Bab 1

B

- *Backpropagation* — Bab 4
- *Baseline* (tolok ukur): *persistence*, klimatologi, ARIMA — Bab 5, 7, 8, 9
- Bias-variance — Bab 5
- Banjir rob — Bab 8
- *Batch size* — Bab 4

C

- CNN (*convolutional neural network*) — Bab 10 (arah riset)
- Callback: *early stopping*, ModelCheckpoint, ReduceLROnPlateau — Bab 4, 5
- *Class imbalance* — Bab 3, 5, 9
- Colab (Google Colab) — Bab 1, 5
- *Confusion matrix* — Bab 3, 5
- Cross-entropy (binary/categorical) — Bab 3
- CSI/FAR/POD (*Critical Success Index, False Alarm Ratio, Probability of Detection*) — Bab 3, 5, 9

D

- Data terbuka: GHCN-Daily, CHIRPS, ERA5, pasang surut — Bab 6, 8, 9
- *Deep learning*: definisi, sejarah, kapan layak — Bab 1
- *Downscaling* — Bab 10
- *Drift* (monitoring) — Bab 10
- Dropout — Bab 5
- Dropout sebagai aproksimasi Bayesian — Bab 10

E

- *Early stopping* — Bab 4, 5

- ENSO/MJO sebagai fitur — Bab 6, 9
- *Epoch* — Bab 4, 5
- ERA5 — Bab 6, 9
- Etika dan keterbatasan *deep learning* — Bab 10
- Evaluasi model: MAE/RMSE/R²/Willmott/KGE; CSI/FAR/POD — Bab 5

F

- Fitur (lag, musiman, ENSO/MJO) — Bab 6
- Formzahl (tipe pasang) — Bab 8
- Fungsi aktivasi: ReLU, sigmoid, tanh — Bab 2, 4

G

- Gap data & imputasi — Bab 6, 8, 10
- *Gradient descent* — Bab 4
- GRU — Bab 7
- Glosarium istilah — lihat “Glosarium dan Notasi”

H

- Harmonik vs *machine learning* — Bab 8
- Horizon prediksi (multi-step) — Bab 7, 8

I

- Imputasi data hilang — Bab 6, 8, 10
- *Interpretability*: SHAP, permutation importance — Bab 9, 10

K

- Keras — Bab 1, 2
- KGE (Kling-Gupta Efficiency) — Bab 5
- Klasifikasi: biner, multi-kelas — Bab 3, 9

L

- LSTM — Bab 7, 8, 9
- *Learning rate*, scheduler — Bab 4
- *Leakage* — Bab 5, 8
- Lisensi data — Bab 6, 8; lihat juga “Daftar Dataset dan Sumber”
- *Loss function* (MSE, MAE, cross-entropy) — Bab 2–4

M

- MAE vs MSE — Bab 2

- *Machine learning* (ML): definisi, vs DL — Bab 1
- MLP (*multilayer perceptron*) — Bab 2, 8
- Metrik operasional WMO — Bab 5, 9
- MJO (RMM) — Bab 6
- Multi-step forecasting (recursive/direct/seq2seq) — Bab 7

N

- Neural network — Bab 1, 2
- Normalisasi (z-score, fit pada *train*) — Bab 6
- Notebook Colab — Bab 1; lihat “Daftar Notebook dan DOI”
- *Nowcasting* — Bab 10

O

- Overfitting / underfitting — Bab 5
- Optimasi: SGD, Adam — Bab 4
- Operasionalisasi model — Bab 10
- *Outlier* / pencilan — Bab 6

P

- Pasang surut: tipe, analisis harmonik, prediksi — Bab 8
- Perceptron — Bab 2
- *Persistence* (*baseline*) — Bab 7, 8
- Precision/recall/F1 — Bab 3
- Peta aplikasi DL dalam meteorologi — Bab 1

Q

- Quantile / selang ketidakpastian — Bab 10

R

- ReLU — Bab 2, 4
- Regresi — Bab 2, 8, 9
- Regularisasi (L2, dropout) — Bab 5
- RNN — Bab 7
- Reproduksiabilitas: *seed*, versi TF — Bab 5, 6; Lampiran A

S

- SGDM/Adam — Bab 4
- SHAP — Bab 9, 10
- Sigmoid / softmax — Bab 3
- *Skill score* (SS) — Bab 7, 8

- Split data berbasis waktu, *walk-forward* — Bab 5, 8
- Stabilitas model & *drift* — Bab 10

T

- TensorFlow — Bab 1, 2
- *Threshold* (trade-off) — Bab 3, 9
- Time series (deret waktu): *window*, *baseline*, LSTM — Bab 7
- Transformasi target (log1p) — Bab 6

V

- Vanishing gradient — Bab 4
- Verifikasi operasional (CSI/FAR/POD, tabel kategori) — Bab 5, 9

W

- Walk-forward validation — Bab 5, 8, 9
- WMO: pedoman verifikasi — Bab 5, 9
- *Window* (jendela input) — Bab 7, 8
- Willmott (indeks kesesuaian) — Bab 5

Catatan: untuk indeks bernomor halaman final, jalankan build LaTeX dengan `\makeindex` saat `node build/generate.mjs`; kata-kunci di atas adalah seed untuk dikurasi lebih lanjut.

Daftar Dataset dan Sumber

Bagian ini mendokumentasikan sumber data yang dipakai di seluruh buku, termasuk lisensi dan cara akses. Konsisten dengan “Kriteria Sitasi” pada `outline.md`: setiap data diidentifikasi, lisensi dicantumkan, dan cara akses dijelaskan. Snapshot data untuk studi kasus dirilis di Zenodo bersama DOI buku (`bookDOI`).

Data Meteorologi & Klimatologi

Dataset	Penyedia	Cakupan	Lisensi & akses	Dipakai di
GHCN-Daily (stasiun global)	NOAA NCEI (<code>ncei.noaa.gov/pub/data/ghcn/daily</code>)	Global, stasiun Indonesia	Domain publik (hak cipta AS; atribusi Menne et al. (2012))	Bab 6
ERA5 global reanalysis	Copernicus C3S (<code>cds.climate.copernicus.eu</code>)	Global, 1940–sekarang, harian	Lisensi terbuka (setara CC-BY 4.0); atribusi wajib “Copernicus Climate Change Service (C3S)”. Akses via API <code>cdsap</code> .	Bab 6, 9
CHIRPS (precipitation)	UC Santa Barbara / USGS (<code>chc.ucsb.edu</code>)	Global, 1981, harian	Domain publik (hak cipta dilepaskan); atribusi dianjurkan (kutip Funk et al. 2015); DOI: 10.1038/sdata.2015.66	Bab 6, 9

Dataset	Penyedia	Cakupan	Lisensi & akses	Dipakai di
Indeks MJO (RMM)	Bureau of Meteorology (BoM), Australia & NOAA	Global, 1974	Publik, gratis dengan atribusi Wheeler & Hendon (2004)	Bab 6, 9
Indeks ENSO: MEI & Nino3.4	NOAA (psl.noaa.gov)	Global, 1950-an	Publik, gratis; atribusi Wolter & Timlin (1993) dan NOAA PSL	Bab 6, 9

Data Oseanografi (Pasang Surut & Muka Laut)

Dataset	Penyedia	Cakupan	Lisensi & akses	Dipakai di
Sea Level Station Monitoring Facility	UNESCO/IOC (ioc-sealevelmonitoring.org)	Real-time, termasuk 24 stasiun Indonesia	Gratis CC BY-NC 4.0 (non-komersial) — tidak untuk penggunaan komersial, dan data turunan tetap non-komersial; atribusi “UNESCO/IOC”. Endpoint: <a href="http://bgraph.php?code=<KODE>&period=<HARI>">bgraph.php?code=<KODE>&period=<HARI>	Bab 8
PSMSL (MSL bulanan)	Permanent Service for Mean Sea Level (psmsl.org)	Global, puluhan tahun	Gratis dengan sitasi Holgate et al. (2013); format RLR/Metric	Bab 2*, 8

Dataset	Penyedia	Cakupan	Lisensi & akses	Dipakai di
UHSLC Research Quality	Univ. of Hawaii Sea Level Center (uhslc.soest.hawaii.edu)	Hourly/daily research quality	Gratis riset/pendidikan; atribusi UH-SLC/NOAA	Bab 8
Peta pasut & data pasang BIG	Badan Informasi Geospasial (tides.big.go.id)	Indonesia	Publik; atribusi BIG	Bab 2*, 6, 8

* Bab 2 memakai data pasang surut contoh; keterangan sumber di bab tersebut mengacu pada BIG (tides.big.go.id).

Data Studi Kasus Buku (sampel & skrip)

Item	Path	Keterangan
Sampel Cilacap hourly (1 th)	<code>manuscripts/ch-08-.../data/sample/cilacap_hourly.csv</code>	Siapa bilang cilacap hourly (seed=42) untuk notebook out-of-the-box; bukan observasi nyata
Skrip unduh IOC/UHSLC/PSMSL	<code>manuscripts/ch-08-.../scripts/download_data.py</code>	untuk eksperimen serius
Skrip generator sampel	<code>manuscripts/ch-08-.../scripts/generate_sample.py</code>	Reproducible & deterministik
Skrip gambar studi kasus	<code>manuscripts/ch-08-.../scripts/generate_figures.py</code>	Reproducible figures & 8.3 dari sampel

Konvensi letak file: dataset sampel dan skrip per-bab disimpan di `manuscripts/ch-<NN>-* /data/` dan `manuscripts/ch-<NN>-* /scripts/`. Notebook Colab (*.ipynb) disimpan di folder `notebooks/` di root repo buku (prefix `ch-<NN>-`), bukan per-bab di dalam `manuscripts/`.

Rilis Zenodo: saat buku dirilis, snapshot dataset studi kasus (pasang surut IOC/UHSLC/PSMSL, deret hujan CHIRPS/ERA5, fitur ERA5, indeks iklim) diunggah ke Zenodo dan dirujuk melalui bookDOI pada `master.md` tiap bab serta “Daftar Notebook dan DOI”. Berkas snapshot yang diturunkan dari data **IOC (CC BY-NC 4.0)** tetap **non-komersial** dan dilisensikan terpisah dari buku (bukan bagian dari lisensi CC BY-SA buku).

Prinsip penggunaan pada buku

1. **Preferensi sumber primer** — data resmi terbuka (NOAA, Copernicus, UNESCO/IOC, UHSLC, PSMSL, CHC UCSB) diutamakan; bukan salinan tak jelas. Seluruh studi kasus memakai data **terbuka** sehingga dapat direproduksi.
2. **Atribusi wajib** — cantumkan sumber, lisensi, dan versi di setiap gambar/tabel yang memakai data (lihat keterangan gambar di tiap bab).
3. **Data contoh sintetik** — dipakai agar notebook dapat dijalankan tanpa internet; hasil sintetik **tidak** diklaim sebagai hasil nyata.
4. **Lisensi data berbeda dengan lisensi buku** — buku CC BY-SA 4.0; lisensi data adalah milik masing-masing penyedia.
5. **Waspada data non-komersial** — lisensi buku mengizinkan penggunaan komersial, tetapi sebagian data (mis. IOC Sea Level Station Monitoring Facility, CC BY-NC 4.0) **tidak**. Saat mendistribusikan atau memublikasikan turunan yang memuat data non-komersial, kewajiban lisensi data tetap mengikuti penyedia; cantumkan peringatan non-komersial secara eksplisit di setiap produk turunan.

Daftar Notebook dan DOI

Setiap bab memiliki satu notebook pendamping yang dapat dijalankan di Google Colab maupun lokal (Python 3.10+; TensorFlow 2.x, NumPy, Pandas, Matplotlib). Nama file diawali nomor bab untuk memudahkan urutan baca.

Daftar Notebook

Notebook	Bab	Topik	Catatan
notebooks/ch-01-00_fondasi_tensorflow.ipynb	00	Fondasi TensorFlow, tensor, setup Colab	Starter; wajib untuk pemula
notebooks/ch-02-01_regresi_pasang_surut.ipynb	01	Regresi ML; <i>baseline persistence</i>	Mini-kasus pasang surut
notebooks/ch-03-02_klasifikasi_hujan.ipynb	02	Klasifikasi biner/multi-kelas; <i>imbalance</i>	Data hujan/level bahaya
notebooks/ch-04-03_optimasi_callbacks.ipynb	03	Deskripsi manual (GradientTape); LR/batch; callbacks	—
notebooks/ch-05-04_metrik_walkforward.ipynb	04	Metrik MAE/RMSE, CSI/FAR/POD; walk-forward	<i>Pipeline</i> dipakai ulang di Bab 8–9
notebooks/ch-06-06_persiapan_data.ipynb	06	Data terbuka (GHC-ND/CHIRPS) & ERA5; format; QC; fitur; split	Persiapan data lengkap
notebooks/ch-07-07_lstm_gru.ipynb	07	LSTM/GRU; <i>windowing</i> ; multi-step	Menyambung Bab 8
notebooks/ch-08-08_studi_kasus_pasang_surut.ipynb	08	Studi kasus pasang surut (Cilacap) <i>end-to-end</i>	Termasuk regenerasi Gambar 8.2/8.3
notebooks/ch-09-09_studi_kasus_curah_hujan.ipynb	09	Studi kasus curah hujan (data terbuka); klasifikasi + verifikasi	

Notebook	Bab	Topik	Catatan
<code>notebooks/ch-10-09_00operasional_arah_ditentukan.ipynb</code>	10	SHAP; ensembel	—

Cara menjalankan

1. **Google Colab** (direkomendasikan): buka notebook via URL GitHub repo publik, atau unggah ke Colab. Gratis, tanpa instalasi.
2. **Lokal**: pastikan `pip install tensorflow numpy pandas matplotlib` (lihat Lampiran A dan *requirements* tiap notebook; Bab 8 butuh `requests` untuk skrip unduh, `xarray/netCDF4` opsional untuk Bab 6).

Reproduksibilitas

- **Seed tetap** (`np.random.seed(42)`, `tf.random.set_seed(42)`) dipakai di seluruh notebook agar hasil dapat diulang.
- **Versi TensorFlow** tercatat di metadata notebook (atau di Bab 1/5 saat instalasi). Snapshot lingkungan (mulai dari rilis) dirilis di Zenodo.
- **Data**: contoh di repo + skrip unduh; snapshot data studi kasus di Zenodo (lihat “Daftar Dataset dan Sumber”).

DOI Terkait

Entitas	DOI	Status
Buku utuh (PDF+DOCX)	10.5281/zenodo.0000000	Placeholder — didaftarkan saat rilis v2.0
Dataset studi kasus (snapshot)	10.5281/zenodo.0000000	Placeholder — didaftarkan bersama data
Versi per-bab	— (tidak ada DOI per bab; sitasi stabil via DOI buku)	—

Prinsip: **1 buku = 1 DOI Zenodo** (konsep versi). Saat DOI dibuat, placeholder diganti dan disalin ke field `bookDOI` di semua `master.md`.

Verifikasi cepat notebook

Untuk memastikan notebook tetap dijalankan (Fase 2 — Checklist Evaluasi D):

per bab (contoh Bab 8)

```
python -m pytest --nbmake notebooks/ch-08-07_studi_kasus_pasang_surut.ipynb
```

Atau gunakan `nbconvert --execute` untuk eksekusi headless:

```
jupyter nbconvert --to notebook --execute notebooks/ch-08-*.ipynb
```

Tentang Penulis

Kanada Kurniawan adalah *Meteorology Officer* di Badan Meteorologi, Klimatologi, dan Geofisika (BMKG), bertugas di Stasiun Meteorologi Kelas III Maritim Dwikora, Pontianak. Minat risetnya mencakup pengembangan model *neural network* untuk prediksi cuaca, pasang surut, dan hidrologi di perairan dan wilayah Indonesia.

Sebelum berkarya di dunia operasional meteorologi, ia menempuh pendidikan di bidang meteorologi dan terlibat dalam berbagai kegiatan pengembangan data dan aplikasi prediksi. Buku ini ditulis berangkat dari kebutuhan sehari-hari — menjelaskan *deep learning* kepada praktisi dan mahasiswa kebumihutan dalam bahasa yang dekat dengan data Indonesia.

Karya dan tulisan penulis dapat diakses melalui laman **kanadakurniawan.com**; versi *live* tiap bab buku tersedia di blog tersebut dan disinkronkan otomatis dari repositori buku ini.

Kolofon

Dokumen ini disusun sebagai **sumber kebenaran** (*single source of truth*) dalam format Markdown dan dikonversi ke PDF/DOCX menggunakan **Pandoc** dengan *citeproc* untuk daftar referensi. Figur dibuat dengan Python/Matplotlib dan disimpan dalam format PNG (untuk PDF) serta WebP (untuk blog).

Aspek	Keterangan
Judul	Pengantar Deep Learning untuk Meteorologi
Edisi	2.0 (lengkap, 10 bab)
Penulis	Kanada Kurniawan
Tanggal rilis	September 2026
DOI	10.5281/zenodo.00000000 (placeholder rilis)
Lisensi isi	CC BY-SA 4.0
Lisensi kode/notebook	CC BY 4.0
Perangkat lunak	Markdown, Pandoc, LaTeX (XeLaTeX), Python 3.10+, TensorFlow/Keras, NumPy, Pandas, Matplotlib
Bahasa	Bahasa Indonesia (istilah teknis Inggris dipertahankan)
Repositori	[placeholder — URL <i>GitHub</i> publik setelah DOI/ISBN]

Penyusunan ulang dan perbaikan tata letak untuk edisi ini dirampungkan di Pontianak, Indonesia. Seluruh umpan balik untuk koreksi faktual dan teknis sangat dihargai.

Akhir dokumen.